Accenture Duck Creek

EXAMPLE Express® Reference Guide

EXAMPLE Express® Reference Guide

This document contains information about the features and functions of EXAMPLE Express® and instructions for configuring and customizing it.

The information contained in this guide is based on an EXAMPLE Express® implementation that uses Avant Garde–based skins. For differences that exist when running an EXAMPLE Express® configuration that uses Lila-based skins, see the appendix *Using Lila-Based Skins*.

EXAMPLE Express® Functionality

EXAMPLE Express® has the following important features:

- An architecture that supports integration with other EXAMPLE Platform® products and other systems
- An out-of-the-box policy administration interface
- Out-of-the-box consumer access screens and the ability to implement a custom consumer access system
- Database management tools (through EXAMPLE UserAdmin™)
- A portfolio feature that allows policies to share common information
- Flexible, role-based security (through EXAMPLE UserAdmin™)
- A framework that allows carriers to extend and customize the EXAMPLE Express® system as needed

Architecture

EXAMPLE Express® works with the EXAMPLE Platform® and other downstream systems to provide carriers with a complete policy administration system.

EXAMPLE Express® in the EXAMPLE Platform®

EXAMPLE Express® works with the following other EXAMPLE Platform® products to facilitate policy administration through the web:

- **EXAMPLE ManuScripts™** – EXAMPLE Express® allows users to view and interact with insurance products defined through EXAMPLE ManuScripts™ (built in EXAMPLE Author®). These ManuScripts contain the complete product definition that EXAMPLE Server® uses to collect data, run rules and rating, and control product workflow.
- **EXAMPLE Server®** – EXAMPLE Express® acts as a translator between the user and EXAMPLE Server®, allowing users to send and receive XML data (policy data) and the requests and responses required for rating, underwriting, and other policy administration actions through a user-friendly interface. If desired, EXAMPLE Server® can be extended to allow integration with third-party data services such as MVRs and credit checks, or with internal services that can perform tasks such as querying the claims system for number of losses, generating new policy numbers, or performing clearance.
- **EXAMPLE UserAdmin™** – EXAMPLE UserAdmin™ enables administrators to assign and regulate the privileges of EXAMPLE Express® users using role-based security. With EXAMPLE UserAdmin™, carriers can define security permissions according to their distinct organizational needs.
- **EXAMPLE TransACT®** – (Optional) EXAMPLE Express® can work with EXAMPLE TransACT® to extend its policy processing capabilities through policy issuance and beyond. EXAMPLE TransACT® allows users to perform endorsements, cancellations, and other transactions, and it supports out-of-sequence transactions, audit processing, and batch processing.

- **EXAMPLE Forms™** – (Optional) EXAMPLE Express® can work with EXAMPLE Forms™ to provide document generation capabilities. EXAMPLE Forms™ allows users to view, select, and generate policy forms.

EXAMPLE Express® and Other Systems

Through EXAMPLE Server®, data collected using EXAMPLE Express® can be passed to downstream systems. In addition, EXAMPLE Express® itself can be integrated with other systems using custom post actions and custom content pages.

For information about EXAMPLE Server® and options for integration, see the *Guide to EXAMPLE Server®*. For information about custom post actions and content pages, see *OnDemand Framework* and *Customizing EXAMPLE Express®*.

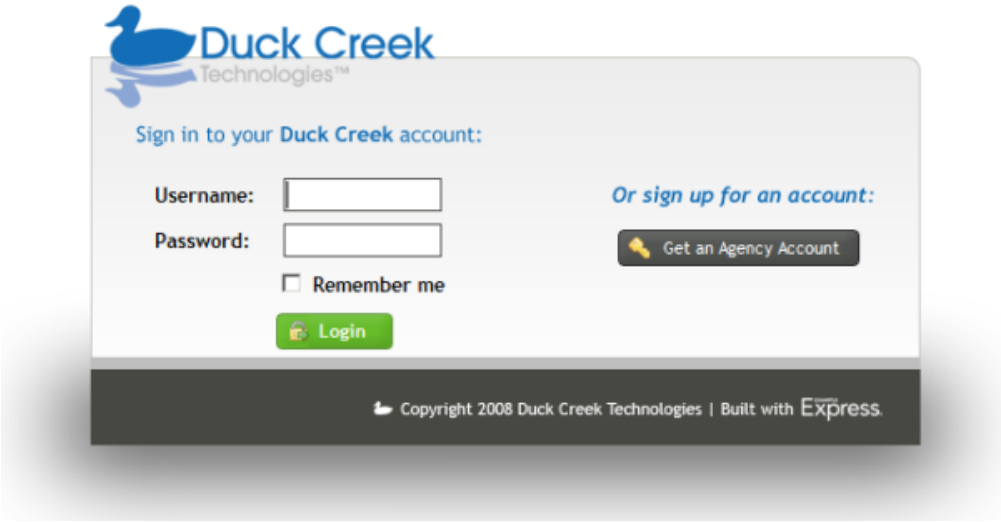
Policy Administration Interface

EXAMPLE Express® provides an online policy administration interface that allows users to view and work with policies, quotes, and clients. This interface includes the following:

- Login
- New Agency Signup
- Policy Administration Dashboard
- Notifications
- Tasks
- Batch Reports
- Search
- Policy pages
- Client pages
- New Quote
- Underwriter's Workbench Toolbar

Login

The EXAMPLE Express® Login page allows existing users to log in with a username and password, or new users to request an agency account. Out of the box, the EXAMPLE Express® Login page appears as follows:



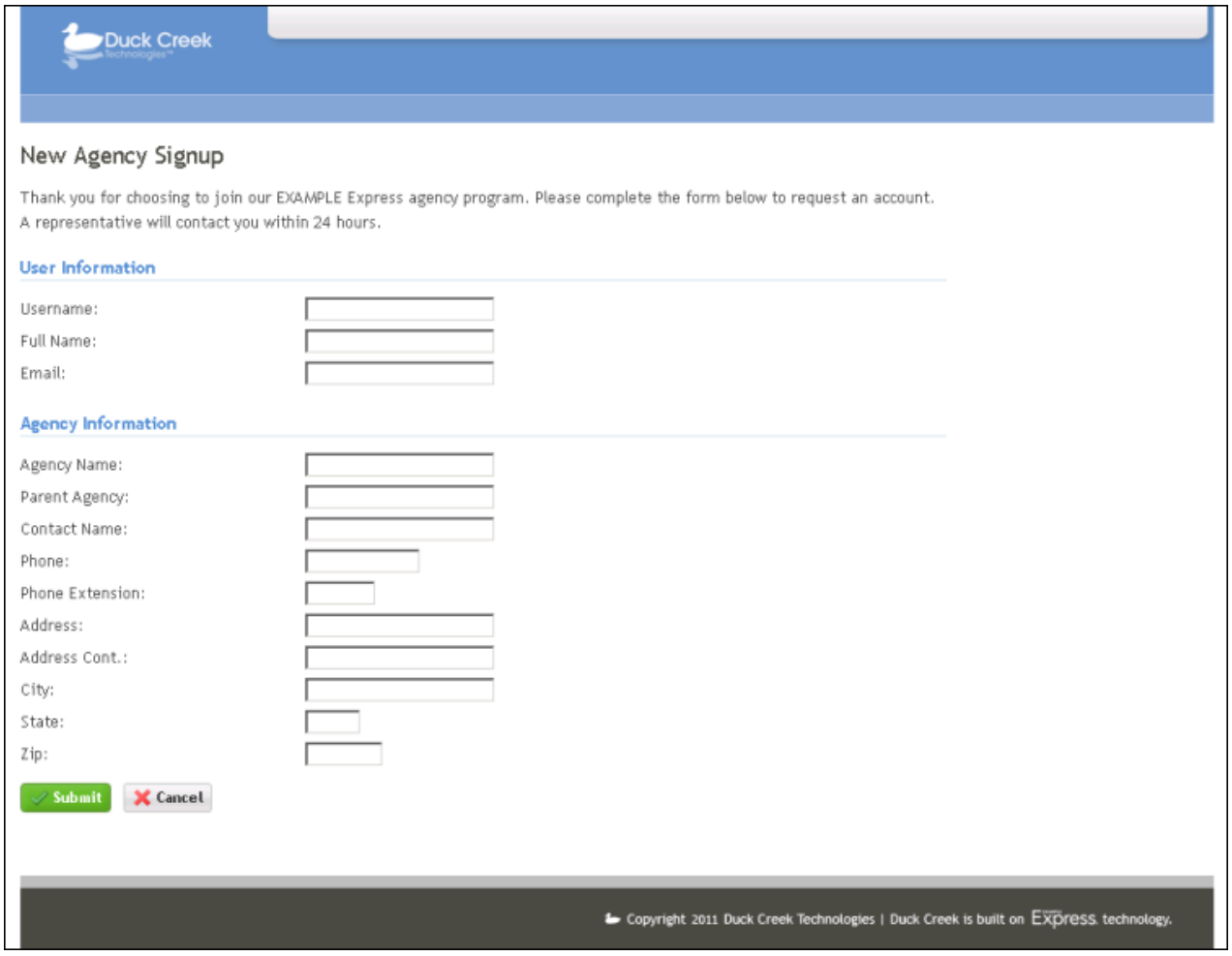
The Login page can be customized. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

If desired, EXAMPLE Express® can bypass the Login page using credentials passed from another system. For information on this capability and other authentication methods, see the *EXAMPLE Server ™ User Guide*.

New Agency Signup

The New Agency Signup page (accessed by clicking **Get an Agency Account** on the Login page) allows new agents to request an account. Requested accounts are not activated until they are approved by a system administrator in EXAMPLE UserAdmin™.

Out of the box, the New Agency Signup page appears as follows:



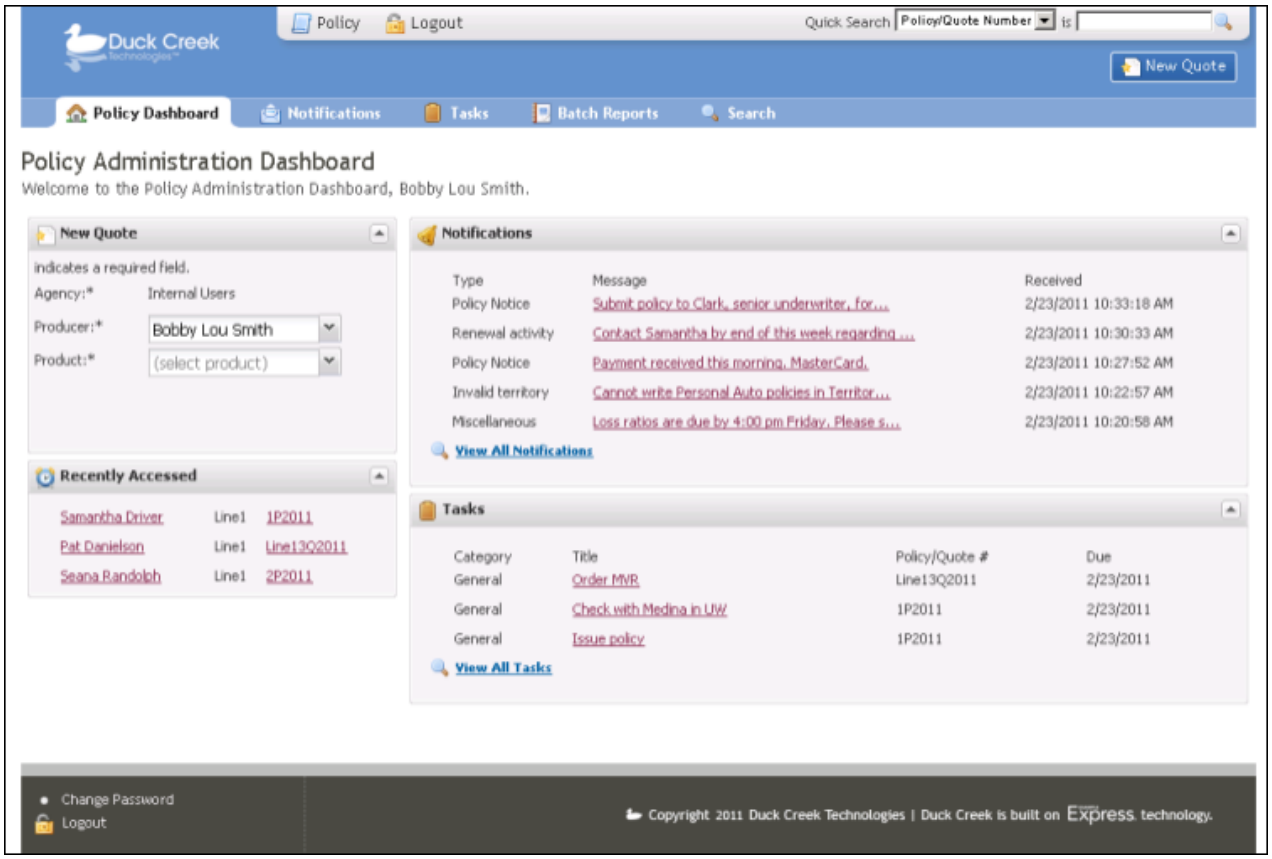
The New Agency Signup page can be customized. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Policy Administration Dashboard

The Policy Administration Dashboard provides a summary view of everything going on in the policy administration system and quick access to frequently performed activities (such as starting a new quote). Summary information can include:

- **Recently accessed quotes and policies** – The dashboard displays the last five policies and quotes that the logged-in user has accessed.
- **Recent notifications** – The dashboard displays the five most recent notifications that were sent to all users, sent to the logged-in user, or sent to any entities at which the user has been assigned *ViewNotifications* privileges.
- **Tasks** – The dashboard displays tasks assigned to the user that are due soon from all task queues at which the user has been assigned *ViewTasks* privileges.

Carriers can customize the Policy Administration Dashboard for each type of user (agent, underwriter, etc.), adding, removing, or customizing content to best meet each user's needs. Out of the box, the Policy Administration Dashboard appears as follows. (The example below shows the out-of-the-box configuration for Agents.)



For information on customizing the Policy Administration Dashboard for each type of user, see *Customizing the Policy Administration Dashboard*. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

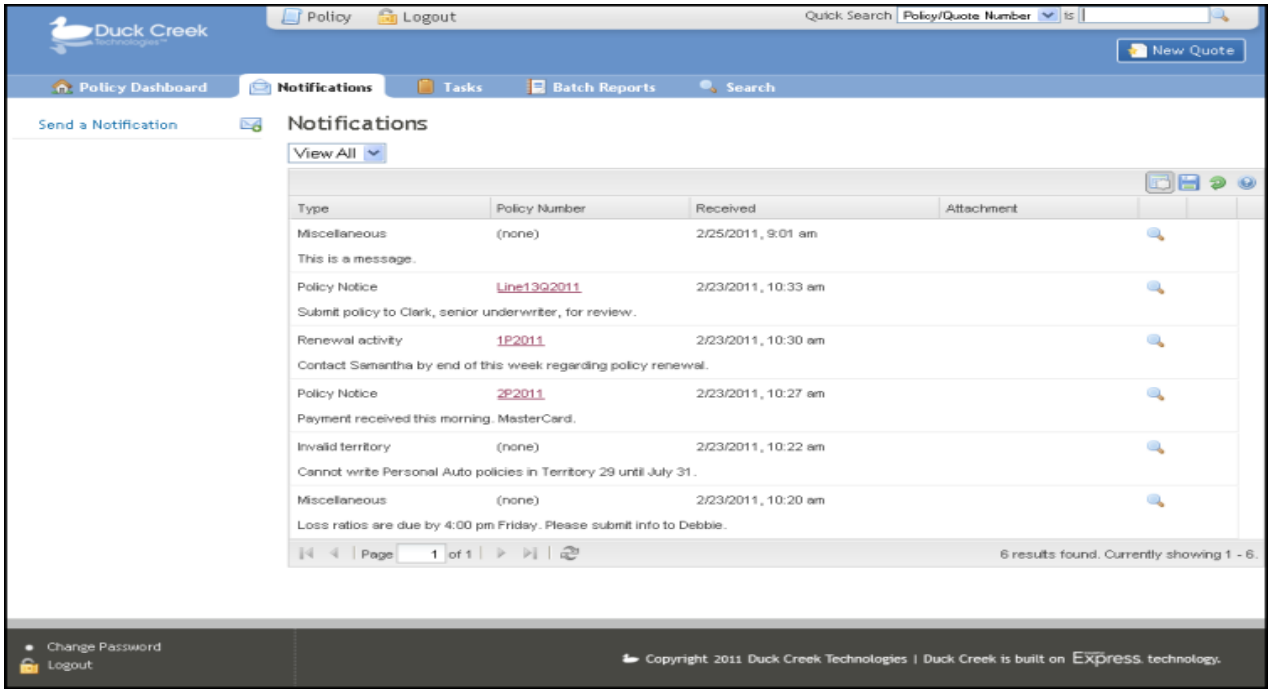
Notifications

The Notifications page allows users to send and receive important announcements or informations. Notifications can be generated by individual users, or by the system using ManuScript logic. Notifications can be associated with specific policies (for example, John Doe's policy has been approved), or can convey general announcements that are not policy-specific (for example, Agents in Territory 31 cannot write Personal Auto policies through 1/1/11).

Notifications can be sent to individual users or to groups of users. EXAMPLE Express® tracks the status of each notification per user, allowing notifications to be tracked as read or unread for each user. In addition, notifications can be managed through batch processing.

Out of the box, the notification system provides one-way communication from management (or the system) to users. If desired, the notification system can be modified to support two-way communication between users.

Out of the box, the Notifications page appears as follows:



From the Notifications page, users can:

- **View received notifications** – Users can view notifications sent to them, to all users, or to entities at which the user has been assigned the *ViewNotifications* privilege. Out of the box, most users can only view notifications sent to them or to the entities where they have the *ViewNotifications* privilege. Users with system administrator privileges in EXAMPLE UserAdmin™ (the Security Level CarrierAdmin privilege), however, can view all notifications in the database by selecting the **View Entire Database** check box. All users can open individual notifications and view details. For more information, see *Notification Details Page*.
- **Create and send new notifications** – Users can create new notifications using an EXAMPLE Express® system page or a customized EXAMPLE ManuScript™. For more information, see *New Notification Page*.
- **Delete old notifications** – Administrators can delete old notifications (if granted the appropriate privilege in EXAMPLE UserAdmin™. Deleting a notification removes the notification from the database and from all other users' lists of notifications.

Use caution when deleting notifications. Deleting a notification deletes the notification from the database. The notification will be removed from all users' lists of notifications.

In addition to viewing notifications from the Notifications page, users can also view notifications associated with a specific policy on that policy's Policy Details page. For more information about the Policy Details page and its notifications section, see *Policy Details*.

For information about customizing notifications, generating notifications automatically, or setting up batch processes for notifications, see *Customizing Notifications*. For general information about customizing EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

New Notification Page

Users can create and send new notifications using the New Notification page (accessed by clicking **Send a Notification** on the main Notifications page). This page can easily be customized to meet each Carrier's unique needs through one of two approaches:

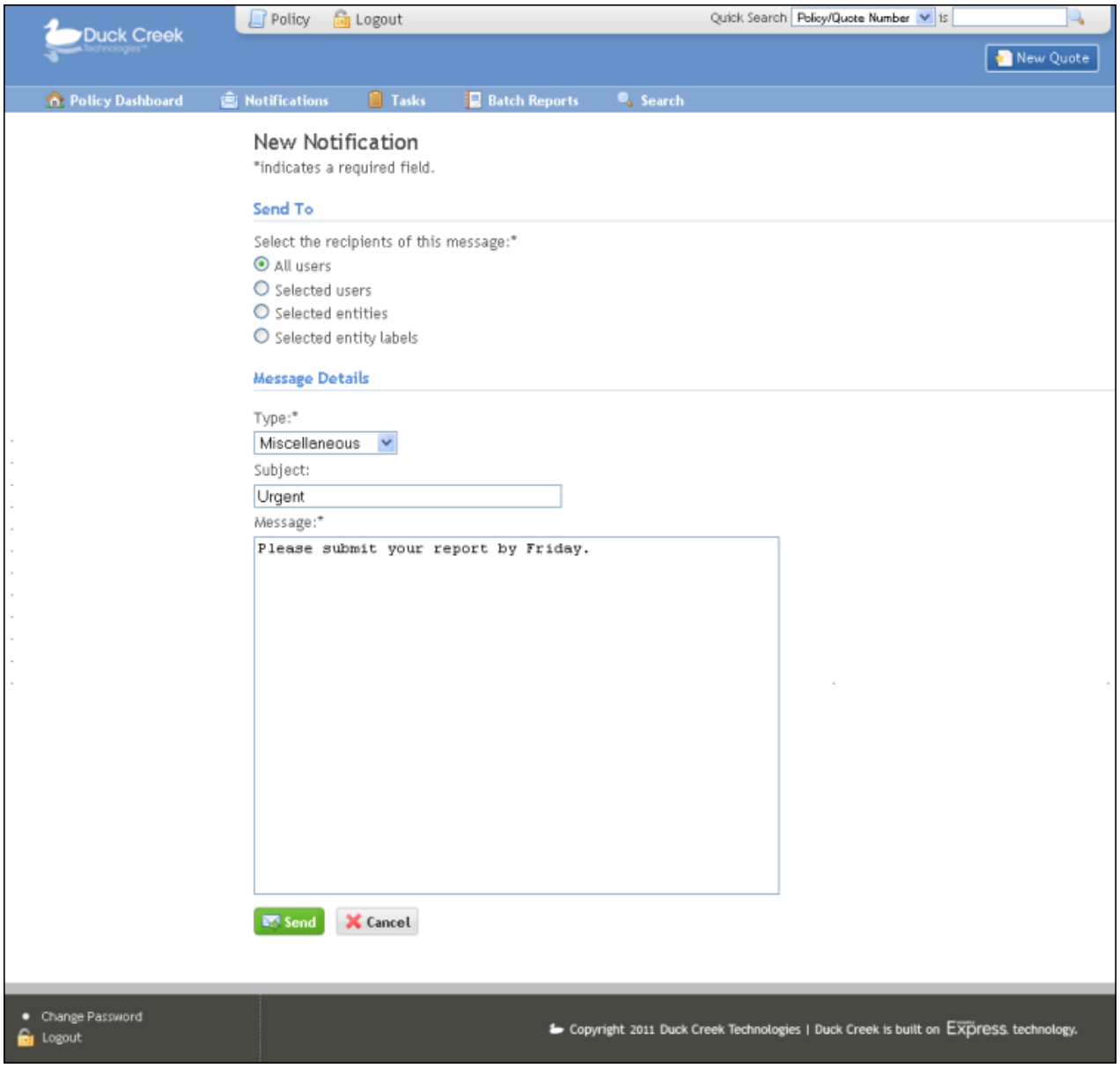
- A system page
- A system ManuScript

You can send a notification that will be associated with a specific policy from the Policy Details page. All users with the *ViewNotifications* privilege at the entity associated with that policy will be able to view the notification on the Policy Details page.

EXAMPLE Express® System Page

Carriers can use a built-in New Notification system page that allows users to specify recipients for the notification, the type of notification, a subject for the notification, and the text of the notification. This page provides a simple, straightforward method for creating notifications, and requires very little customization.

Out of the box, EXAMPLE Express® uses the New Notification system page, but Carriers can modify their implementation to use a Message Detail ManuScript.



When sending a notification, users can specify any one of the following four possible recipients:

- **All users** – All users will receive the notification.
- **Selected users** – The user can select from a list of all users who belong to the entities at which the logged-in user is a member and has the *ViewUserList* privilege.
- **Selected entities** – The user can select from a list of all entities where he or she has the *ViewEntity* privilege.

- **Selected entity labels** – The user can select from a list of entity labels. All users who belong to an entity with that entity label will receive the notification.

EXAMPLE Express® System ManuScript

Carriers can implement a custom Message Detail ManuScript. Using a custom ManuScript allows Carriers to define advanced editing and business rules, and to add features such as the ability to attach files to notifications.

The Message Detail ManuScript replaces the New Notification system page and the Notification Details system page. For more information about the Notification Details page, see *Notification Details Page*.

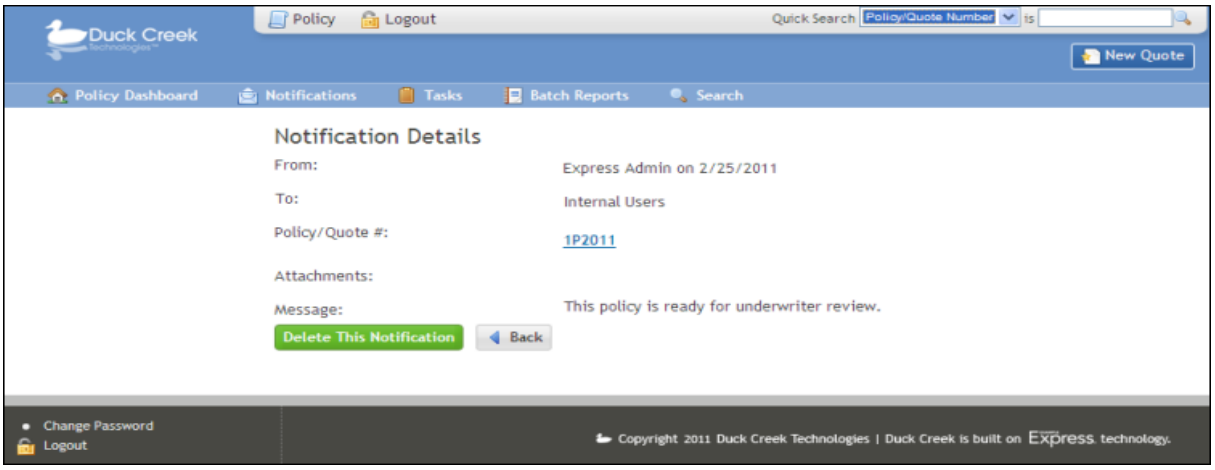
For instructions on customizing EXAMPLE Express® to use a Message Detail ManuScript, see *Configuring a Message Detail ManuScript*. For more information about system ManuScripts in general, see *System ManuScripts*.

Notification Details Page

Users can view details of a notification by clicking that notification in a list of notifications. Like the New Notification page, the Notification Details page can easily be customized to meet each Carrier's unique needs through one of two approaches:

- **EXAMPLE Express® system page** – Carriers can implement a built-in Notification Details system page that allows users to view the following details about each notification.
  - **From** – Who sent the notification. If the notification was system-generated, the value will be *System*.
  - **To** – Recipients of the notification. This may include individual users or groups of users.
  - **Policy/Quote #** – Number of the policy or quote with which the notification is associated and hyperlink to the policy or quote. If the notification is not associated with any policy or quote, this detail does not appear.
  - **Deleted** – Date the notification was deleted by the logged in user. This detail only appears if the notification has been deleted by the logged in user.
  - **Attachments** – Documents attached to the notification.
  - **Message** – Text of the notification.

  This page provides a simple, straightforward method for viewing notification details, and requires very little customization.



- **EXAMPLE Express® system ManuScript** — Carriers can implement a custom Message Detail ManuScript. Using a custom ManuScript allows Carriers to define advanced business rules, and to add any desired features.

Out of the box, EXAMPLE Express® uses the Notification Details system page, but Carriers can modify their implementation to use a Message Detail ManuScript.

The Message Detail ManuScript replaces the New Notification system page and the Notification Details system page. For more information about the New Notification page, see *New Notification Page*.

For instructions on customizing EXAMPLE Express® to use a Message Detail ManuScript, see *Configuring a Message Detail ManuScript*. For more information about system ManuScripts in general, see *System ManuScripts*.

Tasks

The Tasks section allows users to view and work with tasks. Tasks can be generated by individual users, or by the system using ManuScript logic. All tasks are associated with a policy or quote. In addition, all tasks are assigned to task queues (that are defined by privileges assigned in EXAMPLE UserAdmin™). Within queues, tasks can be assigned to individual users or remain available for any user with access to that queue to pick up. Like notifications, tasks can be managed through batch processing. The Tasks section displays all tasks associated with all task queues to which the user belongs.

In addition to viewing tasks from the Tasks section, users can also view open tasks associated with a specific policy on that policy's Policy Details page. Users can create new tasks for a policy from this page. For more information about the Policy Details page and its tasks section, see *Policy Details*. For information about the New Task page, see *New Task Page*.

Furthermore, Mass Tasks functionality lets a user reassign more than one task. Mass Tasks uses:

- ViewTasksOnly, which allows users to view task queues and task details to which they have permissions.
- ManageTasks, which allows users to view the task list, task details, pick up tasks, edit tasks, and complete tasks to which they have permission. Users can send and return a task to the task queue.

For information about configuring Mass Tasks, see *Setting Up Mass Tasks*.

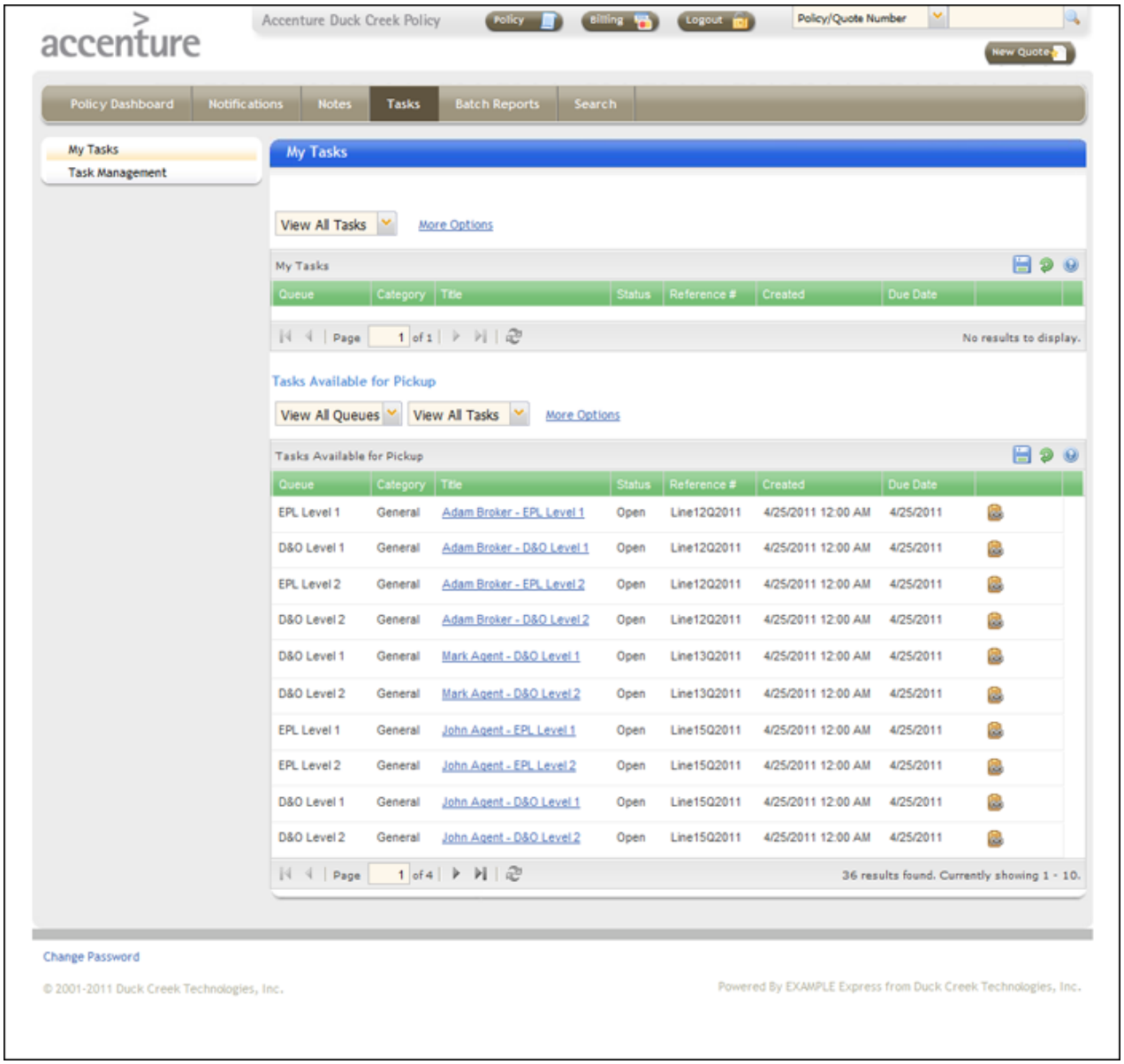
For information about customizing tasks, generating tasks automatically, setting up task queues, or setting up batch processes for tasks, see *Customizing Tasks*. For general information about customizing EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

My Tasks

The My Tasks page is divided into two sections:

- My Tasks displays all tasks assigned directly to the user
- Tasks Available for Pickup displays all tasks the user can pick up

Out of the box, this page is visible to all users, and it appears as follows:



From the My Tasks page, users can open individual tasks and view details or perform any of the following actions (based on the user's privileges):

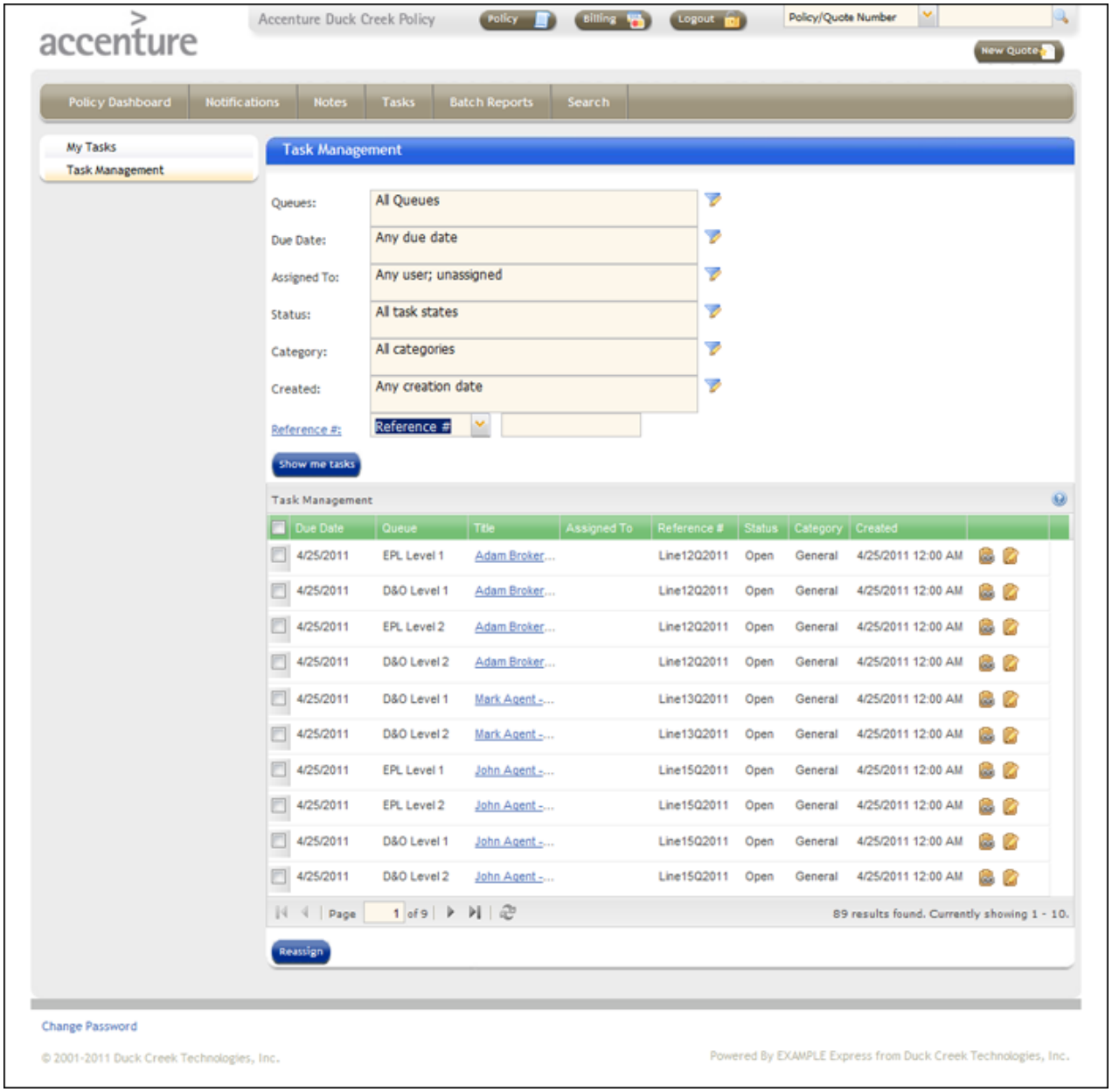
- Pick up the task (if not already assigned)
- Complete the task (if assigned to the user)
- Decline the task
- Edit the task (if the user created the task, or if the task is assigned to the user), or add notes to the task

- Attach items to the task (if the user has attachment privileges and created or has been assigned the task)
- Send the task back to the queue (if assigned to the user)
- Reassign the task
- Delete the task

For more information about the Task Details page and the actions available on that page, see *Task Details Page*.

Task Management

The Task Management page is intended for users who must be able to view and manage task workloads. This page displays all tasks in all queues the user has permission to work with. On this page, users can filter tasks by queue, due date, assignee, status, category, or the date created. Users can then use a shortcut to reassign tasks, or open task details to view or perform other task actions. Out of the box, this page is visible to Underwriter Managers only, and it appears as follows:



Using Task Management Filters

To filter the lists of tasks that display, select from the following list of filters and click **Show me tasks**:

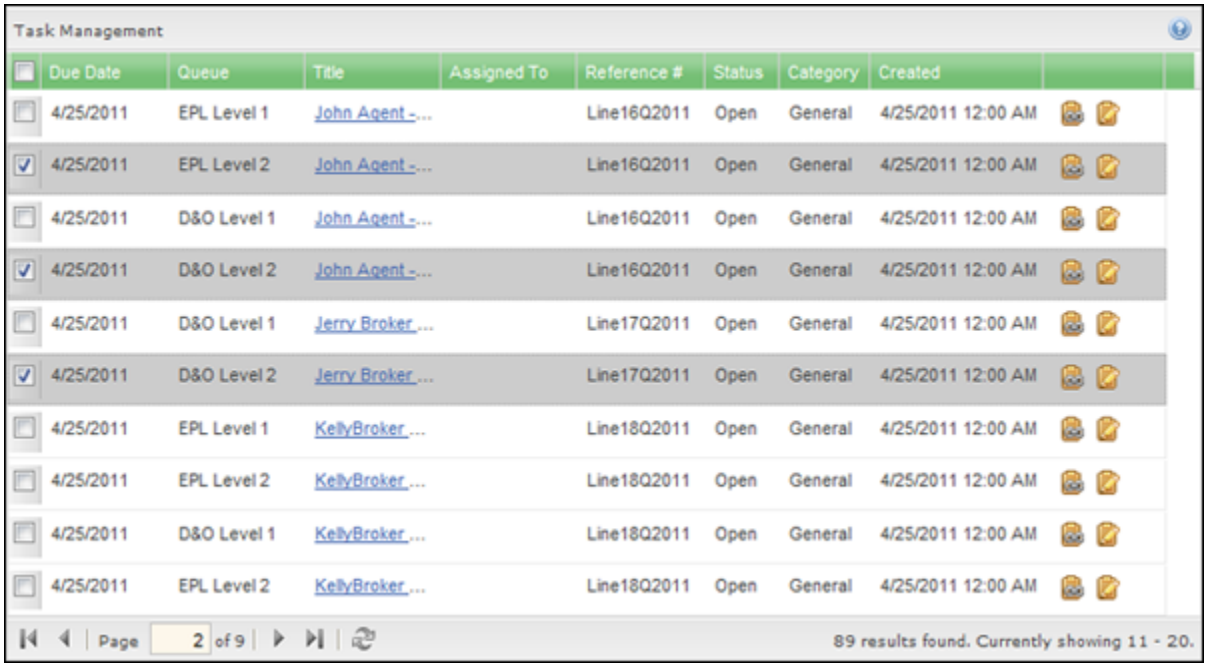
- **Queues** – Click the button to the right of the selection box to display a list of queues. Select all of the queues you want to search by selecting one or more checkboxes. Up to ten queues display on each page of the Queues selection box; page through the box to select additional queues not shown on page one.

- **Due Date** – Click the button to the right of the selection box to display the date selection box. Set both dates to the same to search on a specific date, or select the starting and end date on or between which to search. Click **Save** to set the selected dates.
- **Assigned To** – Click the button to the right of the selection box to display a list of people whose tasks the user can search. Select all of the people to search by selecting one or more checkboxes. Up to ten assignees display on each page of the Assigned To selection box. Page through the box to select additional people not shown on page one.
- **Status** – Click the button to the right of the selection box to display a list of statuses to search. Choose the statuses to search by selecting one or more checkboxes.
- **Category** – Click the button to the right of the selection box to display a list of categories (e.g., Billing or General) to search. Choose the statuses to search by selecting one or more checkboxes.
- **Created** – Click the button to the right of the selection box to display the date selection box. Set both dates to the same value to search on a specific date, or select the starting and end date for a search range. Click **Save** to set the selected dates.
- **Reference #** – Click the Reference Number list to choose Reference Number or Title/Desc (description) as a search parameter. Enter a policy, quote, bill, or account number, title, or description in the text box to the right of the dropdown.

Reassigning Multiple Selections

To reassign multiple selections:

1. In the Task Management Area, select the check boxes to the left of the listed tasks.
2. (Optional) To reassign multiple tasks from more than one page at the same time, select checkboxes on the current page, navigate to a new page, and select additional checkboxes.



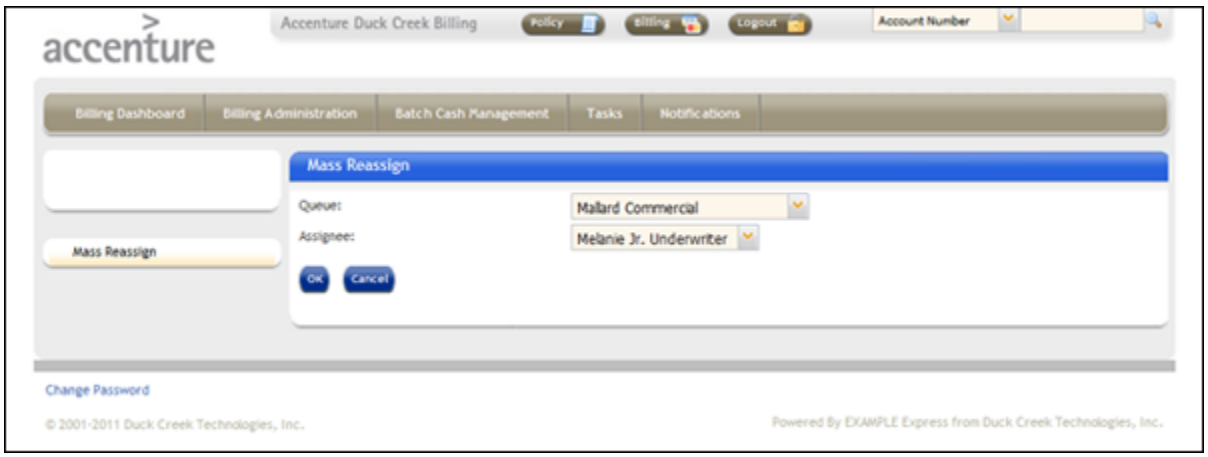
3. After tasks are selected, click **Reassign**.
4. Use the Mass Reassign page to reassign the selected tasks.

Mass Reassign Page

Users can reassign more than one task to more than one person at a time.

To reassign tasks:

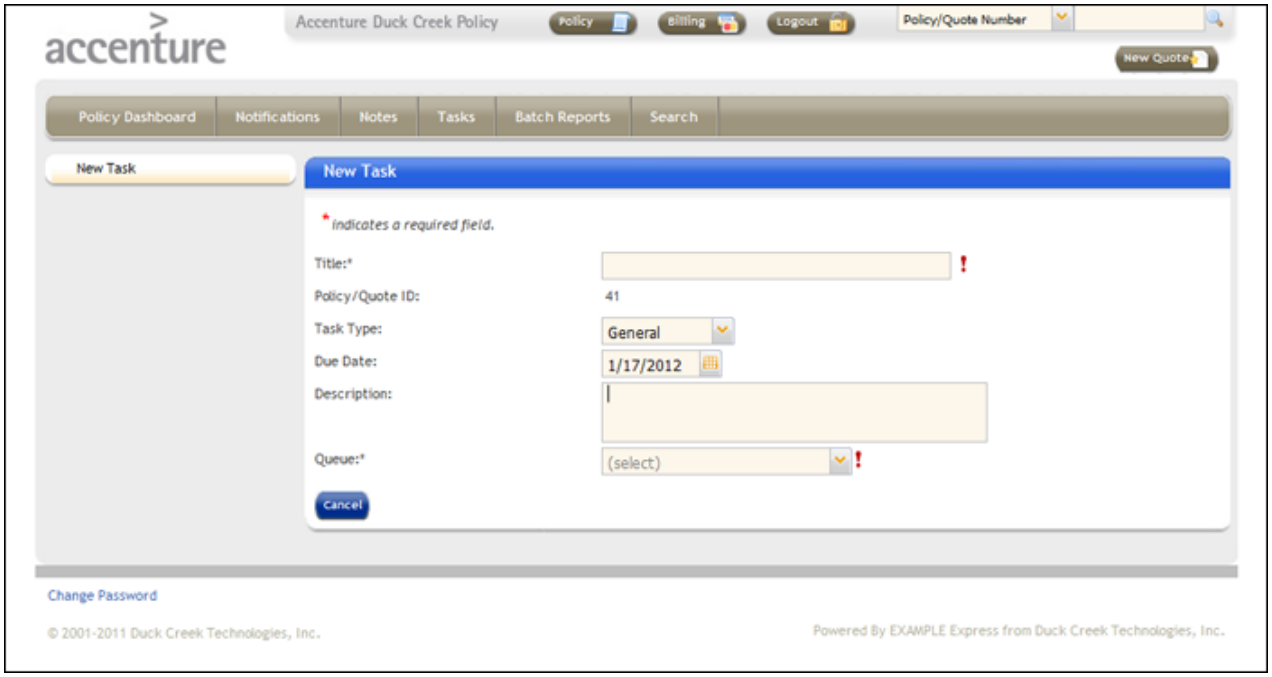
1. Use the Task Assignment page to select the tasks to reassign.
2. On the Mass Reassign page, select the queue where the reassigned tasks should be placed.
3. In the Assignee list, click the appropriate assignee.
4. Click **OK** to reassign all the previously selected tasks to the new queue and assignee, or Cancel to exit without reassigning the tasks.



New Task Page

Users can create new tasks using the New Task page. On this page, users can enter information for a new task, including the title, type of task, due date, and description. Users can then assign the task to the appropriate queue and, if they have the appropriate permissions, to an individual within the specified queue.

Out of the box, the New Task page appears as follows:



The New Task page is controlled by the Task Detail ManuScript. If desired, carriers can use the out-of-the-box Task Detail ManuScript, or they can modify their implementation to use a custom Task Detail ManuScript.

For instructions on using a custom Task Detail ManuScript or customizing the out-of-the-box Task Detail ManuScript, see *Customizing the Task Detail ManuScript*. For information about system ManuScripts in general, see *System ManuScripts*.

Task Details Page

Users can view details of a task by clicking that task in the list of tasks. Out-of-the-box, tasks have the following details.

Item	Description
Title	Title of the task.
Policy/ Quote #	Number of the policy or quote with which the task is associated and hyperlink to the policy or quote.
Category	Type of task. Options for the category are Carrier-defined. For instructions on defining task categories, see *Customizing Task Categories*.
Due Date	Date the task is due.
State	State of the task. Options are Open, Completed, Closed (Carrier-defined reason codes are available), or Deleted. Out of the box, EXAMPLE Express® comes with a reason code of "Declined" for the *Closed* state. For instructions on defining additional reason codes, see *Customizing Task Reason Codes*.

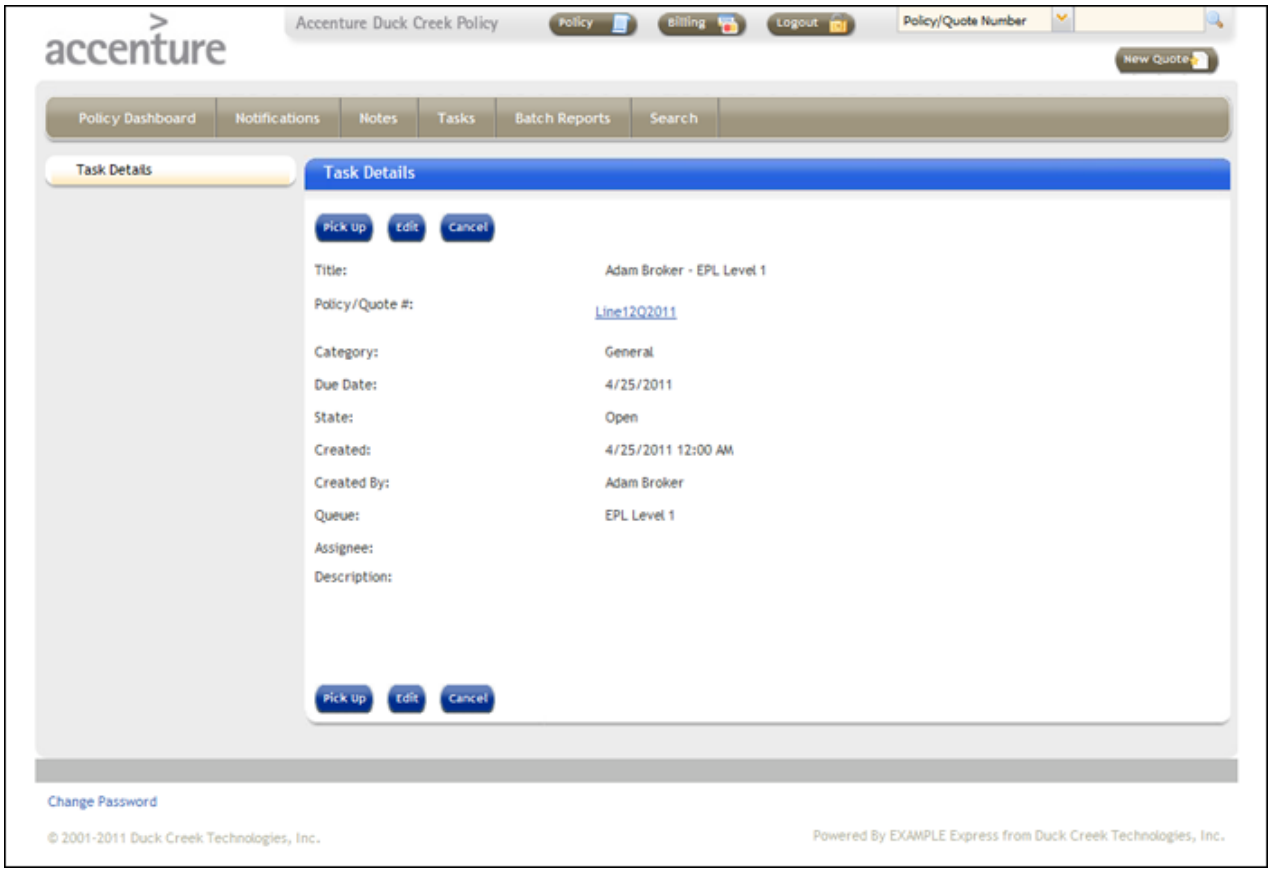
Created	Date task was created.
Created By	User who created the task. If the task was system-generated, the value will be *System*.
Queue	Queue to which the task is assigned.
Assignee	User (within the assigned queue) to which the task is assigned.
Description	Task details.
Notes	Any user-added notes for the task.
Task History	History of the task, including any changes to assignee, queue, or status.
Attachments	Documents attached to the task.

Actions available for tasks include:

- **Pick up** – Available only if a user has permission to view and work with tasks, as determined by privileges in EXAMPLE UserAdmin™. Available only when a task is not already assigned. Picking up a task assigns the task to the user who picks it up.
- **Complete** – Available only if a user has permission to view and work with tasks, as determined by privileges in EXAMPLE UserAdmin™. Available only when a task is assigned to the user. Marking a task as complete sets the state of the task to *Completed*.
- **Decline** – Available only if a user has permission to decline the task, as determined by privileges in EXAMPLE UserAdmin™. Declining a task sets the state of the task to *Closed*, with a reason of *Declined*.
- **Edit** – All users can add notes. A user to whom a task is assigned can also edit the due date. A user who created a task can edit the title, category, and description.
- **Send to Queue** – Available only when a task is assigned to the user. Sending a task back to the queue moves the task back into an unassigned state within its queue.
- **Reassign** – Available only if a user has permission to reassign, as determined by privileges in EXAMPLE UserAdmin™. This action allows the user to select a new queue and (if desired) a new assignee.
- **Delete the task** – Available only if a user has permission to delete the task, as determined by privileges in EXAMPLE UserAdmin™. This action marks the task for deletion. The task is not deleted until the associated policy is purged from the database.

In addition, users who have attachment privileges and have created or been assigned a task can add and delete attachments from that task.

The Task Details page appears as follows:



The Task Details page is controlled by the Task Detail ManuScript. If desired, carriers can use the out-of-the-box Task Detail ManuScript, or they can modify their implementation to use a custom Task Detail ManuScript.

For instructions on using a custom Task Detail ManuScript or customizing the out-of-the-box Task Detail ManuScript, see *Customizing the Task Detail ManuScript*. For information about system ManuScripts in general, see *System ManuScripts*.

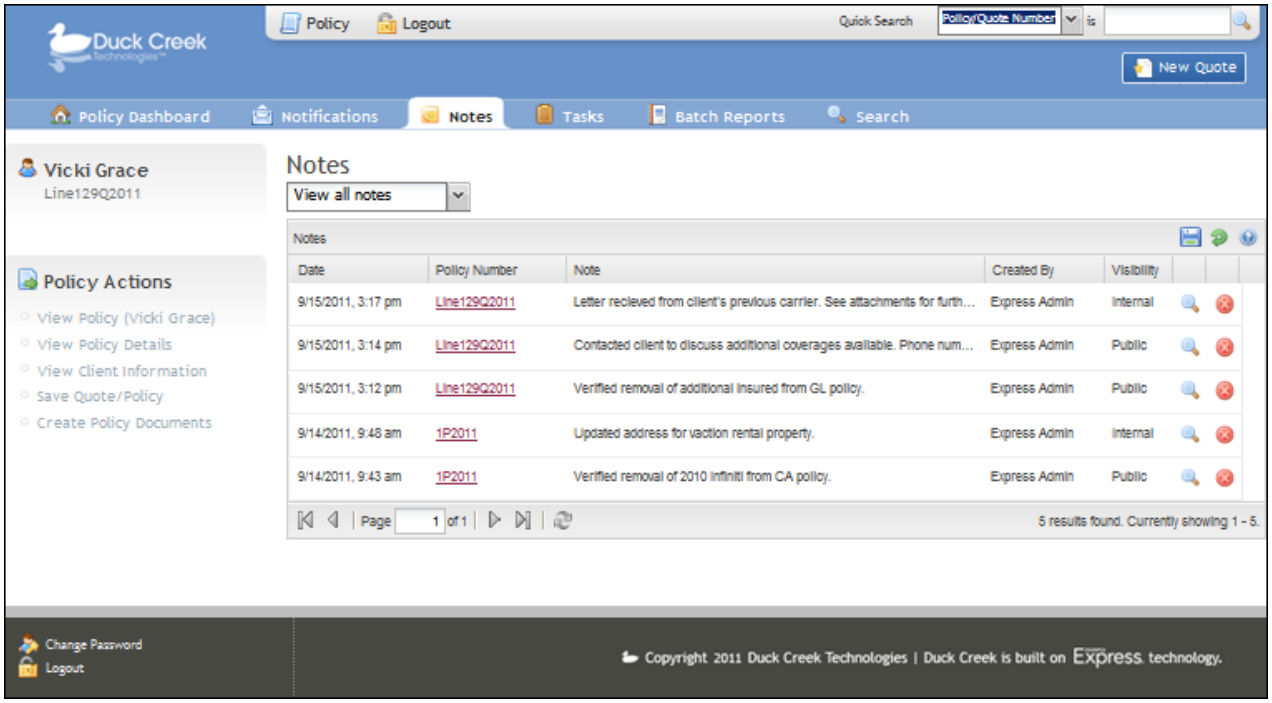
Notes

The Notes page allows users to view all notes associated with a given policy or quote. Notes are linked to the history image of a given policy. For a group of notes from particular transactions or notes created while the policy was in the quoting phase, users can view the following:

- The order in which the notes were entered based on creation date
- The state of the policy when the note was entered
- The transactions the notes apply to

This allows users to locate a specific transaction or point in the policy lifecycle and see which notes applied. Users can also view a specific endorsement and the applicable notes.

Out of the box, the Notes page appears as follows:



From the Notes page, users can:

Function	Description
Filter notes view	In the View notes dropdown on the Notes page or on the Policy Details page, users have the following options to filter the notes they wish to view: • View all notes (default) • View my notes • View deleted notes
View note	In the list of notes for a given quote or policy, click **View**  . When a user views an existing note, the note text and Internal Only checkbox appear as read-only. Once saved, a note cannot be edited. Users must have the Security Level CarrierAdmin (SecLevel_CarrierAdmin) or Security Level CarrierUser (SecLevel_Underwriter) security level with the entity associated to the policy to view Internal Only notes.

Delete	In the list of notes for a given quote or policy, click **Delete**  . This will remove the note from the users list and set the status to Mark for Delete in the database. Deleting a note will not remove the it from the database. Users must have the Security Level CarrierAdmin (SecLevel_CarrierAdmin) security level to delete a note. The delete icon will not display for other users.

In addition to viewing notes from the Notes page, users can add and view notes associated with a specific policy from the Policy Details page. For information about additional functionality Policy Details page and its notes section, see *Policy Details*. For general information about customizing EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Batch Reports

The Batch Reports page provides the ability to view results of batch processes performed by the system. Users can drill down into individual batch reports and view the policies or notifications involved in a particular batch process, and whether the actions performed on each policy or notification succeeded or failed. From this page, EXAMPLE Express® users can open any policy or notification listed and perform any necessary actions.

When viewing a batch report for policies, the user can only see those policies for which he or she has the *ViewQuotes* privilege.

The Batch Reports page can be customized. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Search

EXAMPLE Express® provides two types of searches for clients, quotes, and policies: a Quick Search feature available from anywhere in the system, and a more advanced Search page.

Both the Quick Search feature and the Search page can be customized. For information, see *Customizing Search Options*. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Quick Search

The Quick Search feature is available from anywhere in the system and allows users to search for policies and quotes by policy or quote number, or for clients by client name or phone number. When searching by policy or quote number or by client name, users can enter any part of the policy/quote number or client name. When searching by phone number, users must enter the entire number.

When searching for a policy or quote, the search results will include policies and quotes from all entities where the user has the *ViewQuotes* privilege. When searching for a client, the search results will include all clients from all entities where the user has the *ViewClients* privilege and where the client has a quote for an entity to which the user belongs.

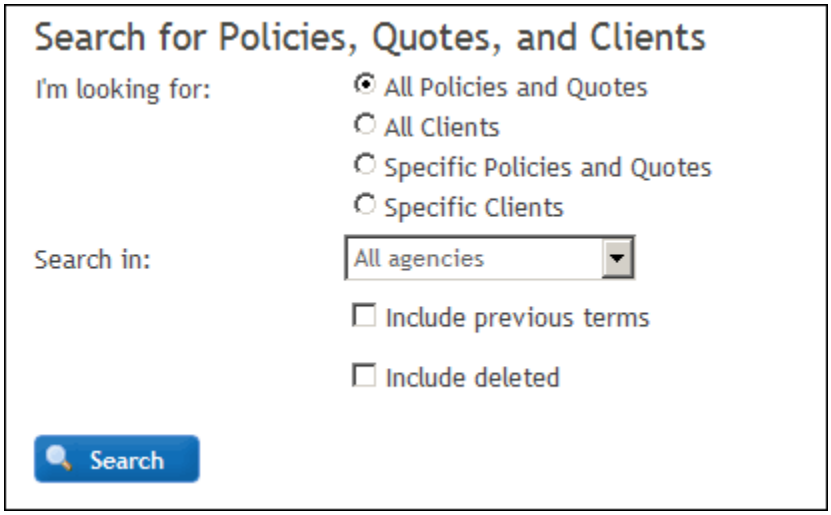
The Quick Search feature appears as follows:

Search Page

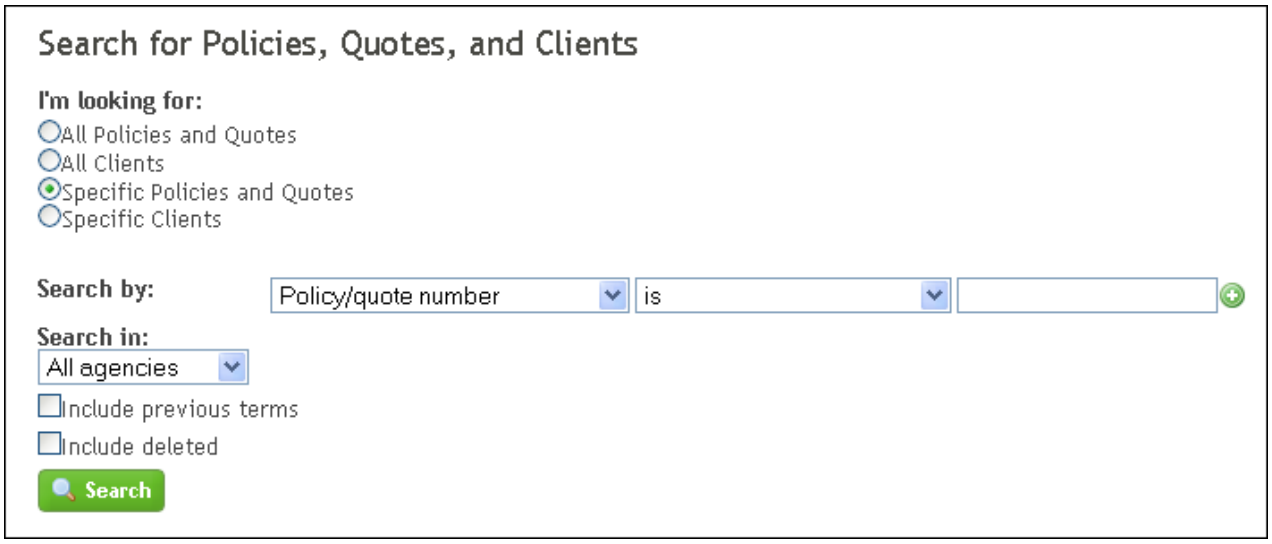
The Search page provides the ability to search for clients, policies, and quotes using a variety of search options.

When searching for a policy or quote, the search results will include policies and quotes from all entities where the user has the *ViewQuotes* privilege. When searching for a client, the search results will include all clients from all entities where the user has the *ViewClients* privilege and where the client has a quote for an entity to which the user belongs.

Out of the box, the Search page appears as follows:



If the **Specific Policies and Quotes** option or **Specific Clients** option is selected, the search page expands to allow the user to enter specific search criteria.



Out of the box, the search page contains the following options.

Item	Description
I'm looking for	Specifies whether to search for policies and quotes or for clients, and whether to list all policies/quotes or clients, or whether to return specific policies/quotes or clients.
I want to search by	Specifies search criteria. Appears only if searching for specified policies, quotes, or clients.
Search in (optional)	Entity to search. This list contains only entities at which the user has the *ViewQuotes* privilege (when searching for quotes and policies) or the *ViewClients* privilege (when searching for clients). If an entity is specified, the search results will include only quotes/ policies or clients for that entity. Appears only if searching for specified policies, quotes, or clients. Out of the box, users with access to 15 or fewer agencies can select the desired agency from a list. For users with access to more than 15 agencies, users can click the **Select an Agency** link to view the list of agencies in a paginated table the user can filter and sort. You can customize the number of agencies required for the list to be displayed in a table by modifying the *SelectAgencyLimit* configuration setting in the Express\Web.config file. For more information about this configuration setting, see *SelectAgencyLimit*.
Include previous terms (optional)	(Policies and quotes only) Indicates whether to search previous policy terms when searching for policies and quotes by policy/quote number. Appears only if searching for specified policies or quotes.

Include deleted (optional)	Indicates whether to search deleted policies and quotes or clients. Appears only if searching for specified policies, quotes, or clients.

Out of the box, the *Policy/quote number* option searches the *PolicyNumber* column in the Quote table.

When both *and* and *or* operators are used in a search, *and* operators take precedence over *or* operators. For example, if using the following criteria to search for a policy:
Client Name contains Smith AND
Address is 4659 W. Brookside OR
Address is 1544 Birmingham Dr.
The system will return the following policies:
1. All policies with a client name that contains the text *Smith* and an address of *4659 W. Brookside.*
2. All policies with an address of *1544 Birmingham Dr.*

Policy Pages

Users can view and work with individual quotes and policies through a series of quote and policy pages in EXAMPLE Express®. Users can access these pages if they have the *ViewQuotes* privilege assigned at the entity with which a given policy is associated. These pages include:

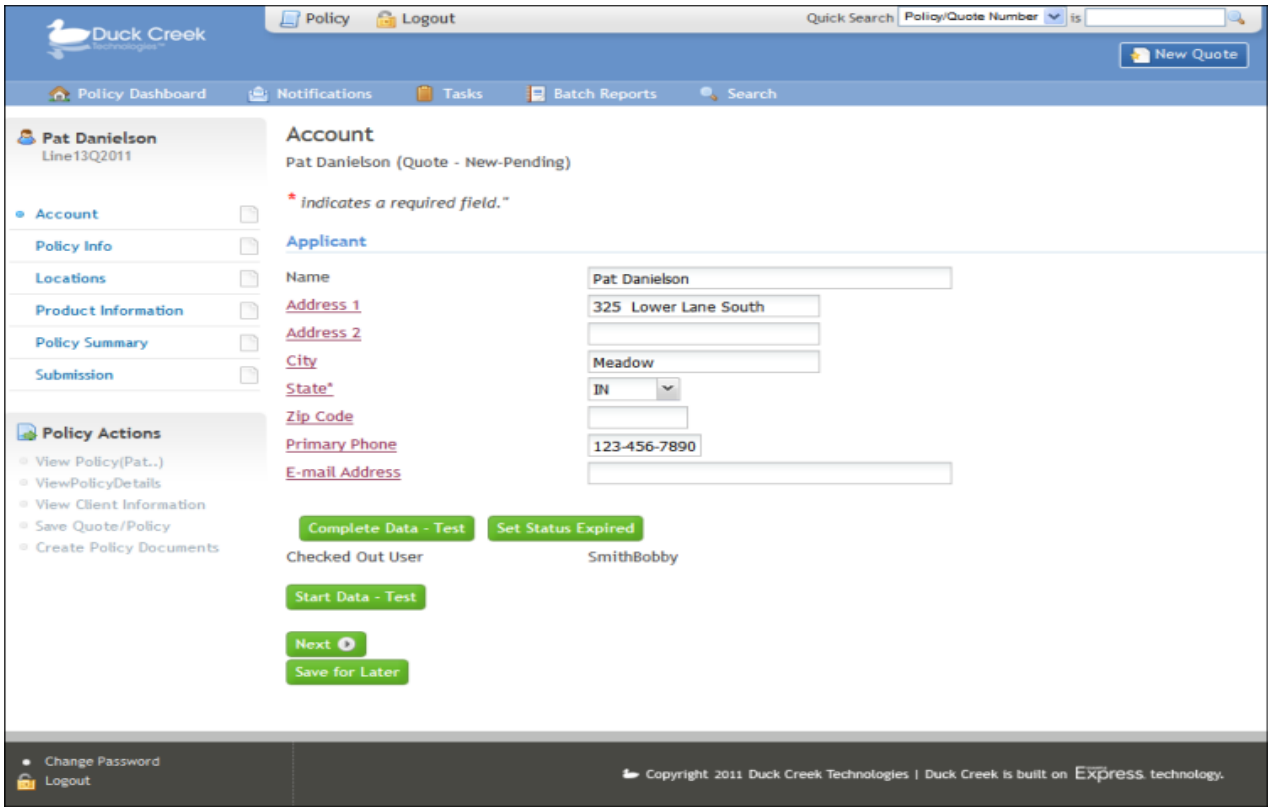
- **Policy data pages** – Users can view and work with quote and policy data through pages defined by product ManuScripts created in EXAMPLE Author®. For more information about these pages, see *Policy Data*.
- **Policy Details page** – Users can view general details about a quote or policy, including client information, effective and expiration dates, agency and producer information, tasks and notifications associated with the quote or policy, and any documents attached to the quote or policy. For more information about the Policy Details page, see *Policy Details*.
- **Create Policy Documents page** – Users can generate and print new quote or policy documents from this page. For more information about the Policy Documents page, see *Create Policy Documents*.

With EXAMPLE TransACT®, users can also view a page that displays a policy's transaction history, including any transactions that have been completed or are pending on the policy. Users can also view details of existing transactions and perform new transactions on the policy. For more information about this page and others in EXAMPLE TransACT®, see the *EXAMPLE TransACT® 2.1 Overview*.

Policy Data

Users can view and work with quote and policy data through pages defined by product ManuScripts created in EXAMPLE Author®. These pages are controlled by ManuScript logic but are viewed through EXAMPLE Express®.

The following is an example of a policy data page.



Quote and policy pages are accessed by:

- Clicking the quote or policy number for a given quote or policy in the **Recently Accessed** list on the Policy Administration Dashboard.
- Clicking the **View Policy** button  for a given quote or policy on the Client Information page for a given client or on the Search page.
- Clicking **View Policy** in the left panel any time a quote or policy is open.

The content and appearance of the product pages through which policies are viewed is completely customizable through EXAMPLE ManuScripts™.

Policy Details

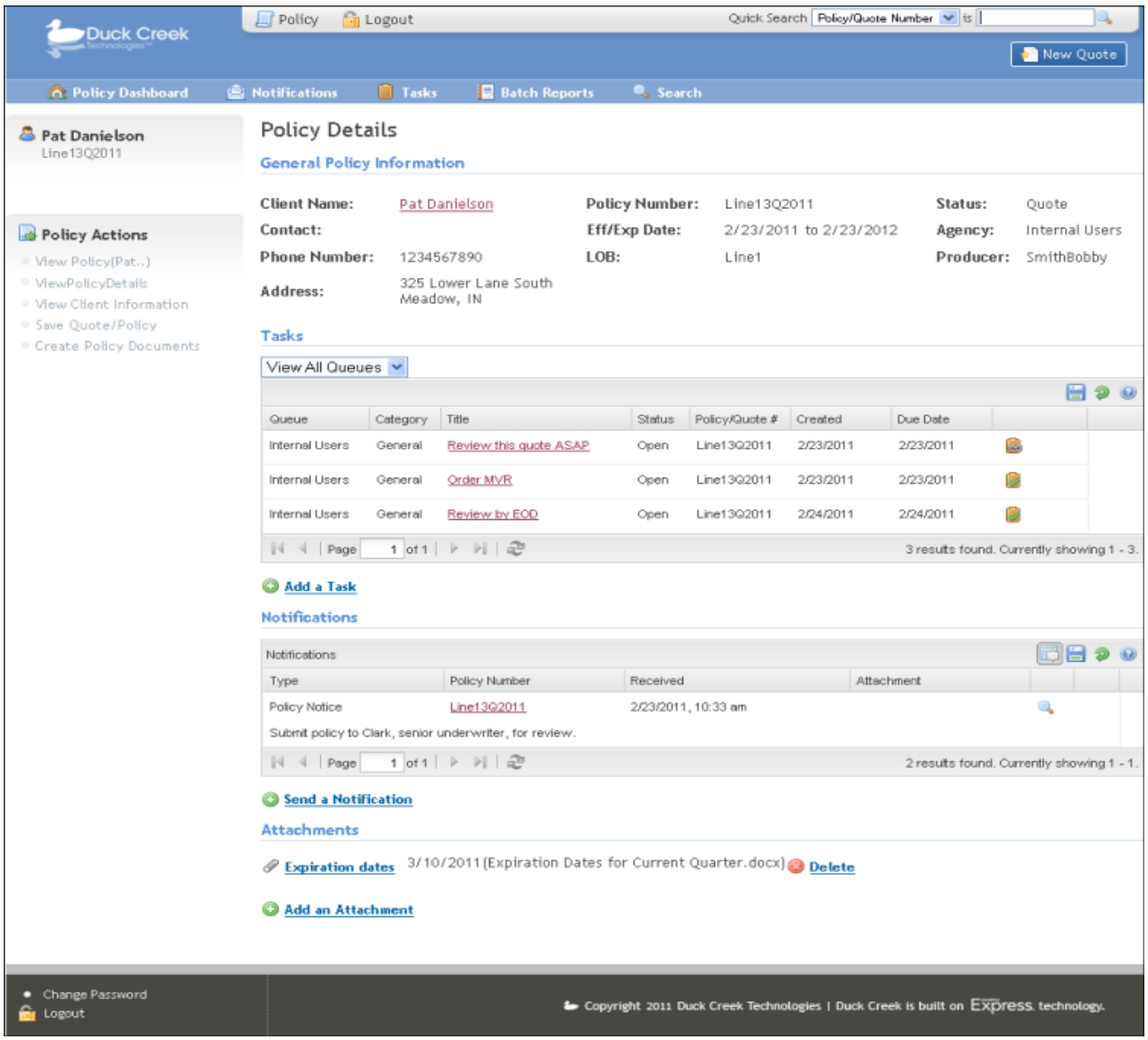
On the Policy Details page, users assigned viewing privileges at the associated entity can view general details about a quote or policy, including:

- Client information (name, phone number, address)
- Policy information (policy number, line of business, effective and expiration dates)
- Agency and producer names

In addition, the Policy Details page includes:

- **Tasks** – Users can only see tasks assigned to them or to their task queues.
- **Notifications** – Users can see notifications if they have the *ViewNotification* privilege at the entity with which the policy is associated.
- **Documents** – All users who can view a policy's details can view documents attached to that policy.

Out of the box, the Policy Details page appears as follows:



The Policy Details page for a given quote or policy can be accessed by:

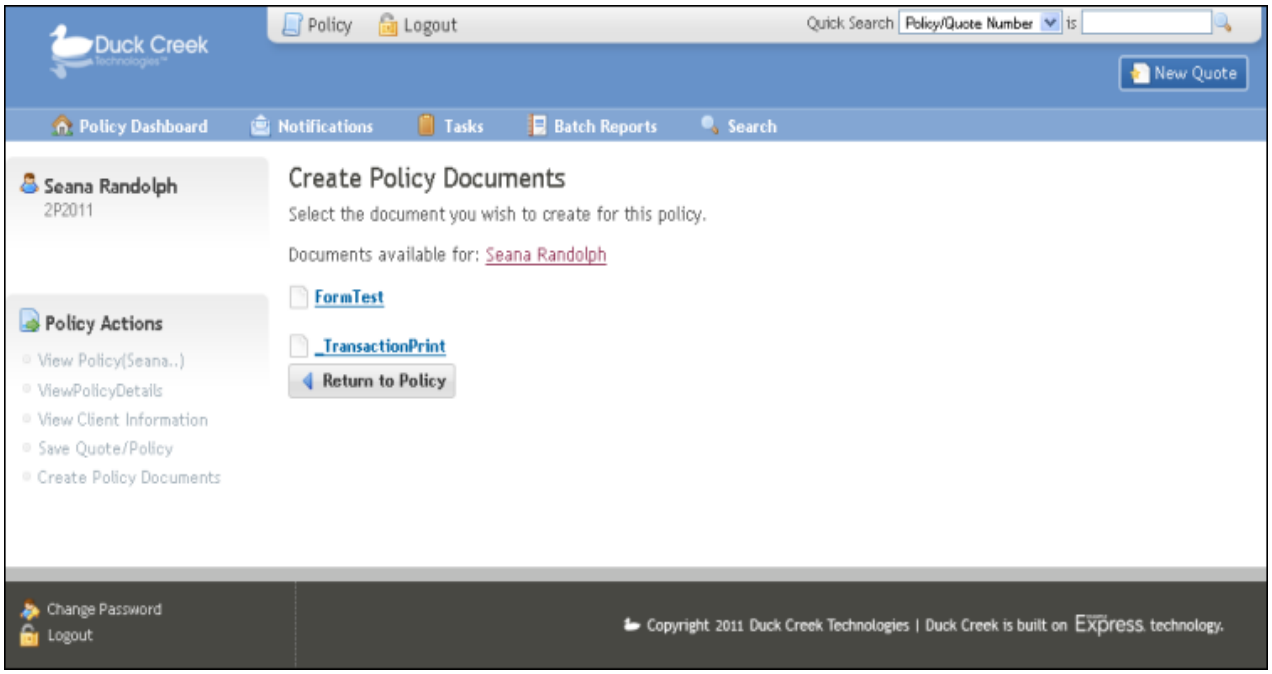
- Clicking the **View Policy Details** button  for that quote or policy on the Client Information page for a given client or on the Search page.
- Clicking **View Policy Details** in the left panel any time a quote or policy is open.

The Policy Details page can be customized. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Create Policy Documents

On the Create Policy Documents page, users can generate quote or policy documents. They can print or save the documents, and—if desired—attach them to the quote or policy on the Policy Details page.

Out of the box, the Create Policy Documents page appears as follows:



The Create Policy Documents page for a given quote or policy can be accessed by clicking **Create Policy Documents** in the left panel any time a quote or policy is open. The Create Policy Documents page can be customized. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Client Pages

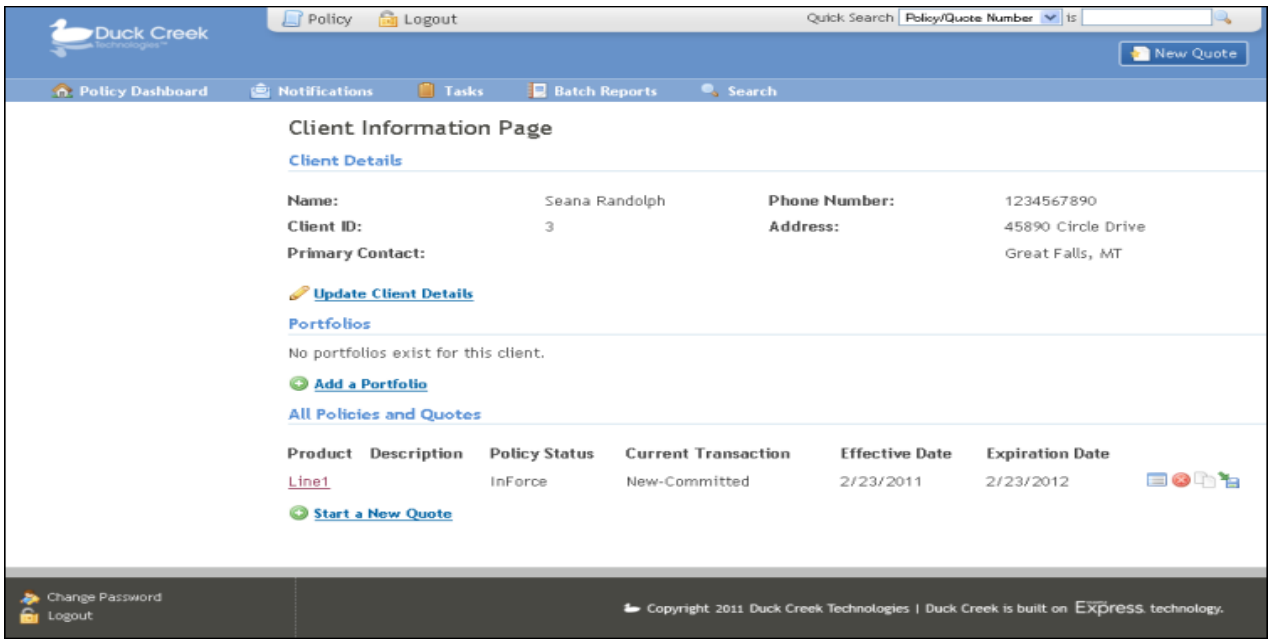
Users can view and work with individual clients through a series of client pages in EXAMPLE Express®. Users can view and access these pages if they have the *ViewClients* privilege assigned at the entity with which the client is associated. These pages include:

- **Client Information page** – Users can view client information, including contact information and a list of the client's portfolios, policies, and quotes. Users can also access the client's policies and quotes from this page. For more information about the Client Information page, see *Client Information*.

- **Client Details page** – Users can drill down into additional client details. As a ManuScript-driven page, this page can be easily customized. For more information about the Client Details page, see *Client Details*.

Client Information

Users can view client information and access a client's policies and quotes from the Client Information page. This page will display all quotes and policies from all entities where the user has the *ViewQuotes* privilege. Out of the box, the Client Information page appears as follows:



The Client Information page for a given client can be accessed by clicking the client name on the Search page, or by clicking the client name in the Recently Accessed section of the Policy Administration Dashboard. This page can be customized. For general information on customizing out-of-the-box EXAMPLE Express® pages, see *Customizing EXAMPLE Express® Skins*.

Client Details

From the Client Information page, users can drill down into additional client details. The Client Details page is a ManuScript-driven page that can collect and display any number of client details. Using a customized Client Detail ManuScript, Carriers can define advanced editing and business rules.

This ManuScript can be easily customized. For instructions on customizing the Client Detail ManuScript, see *Configuring a Client Detail ManuScript*. For information about system ManuScripts in general, see *System ManuScripts*.

New Quote

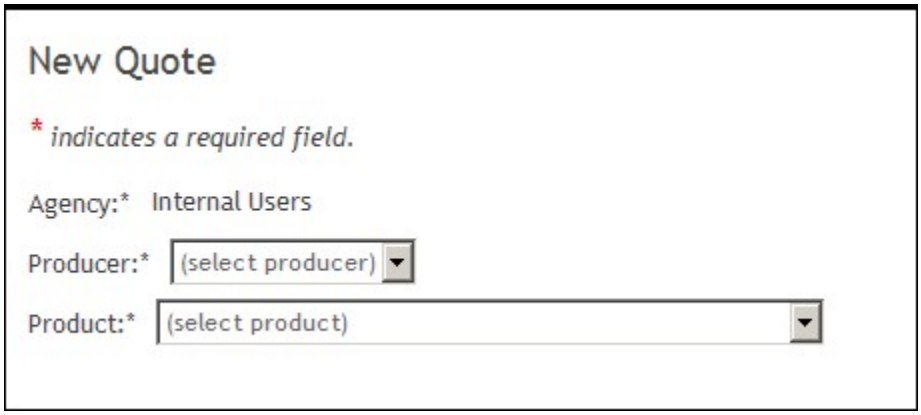
The EXAMPLE Express® policy administration interface provides the following methods for starting a new quote:

- **Clicking** *New Quote* **in the upper right corner of the application** – This method allows users to easily start a new quote from anywhere in the application.

- **Using the New Quote home page module** – This method allows new and infrequent users to easily see where and how to start a new quote.
- **Starting a new quote from the Client Information page** – This method allows users to start a new quote for an existing client, without having to re-enter certain client information.
- **Duplicating an existing quote or policy** – This method allows users to create a quote very quickly by duplicating information in another quote and then making the desired changes.

When starting a new quote from the **New Quote** action in the main navigation, from the home page module, or from the Client Information page, users are presented with the ability to select the appropriate product before starting the quote. Out of the box, EXAMPLE Express® uses a Duck Creek Technologies, Inc. New Quote Selection ManuScript for this process. The out-of-the-box New Quote Selection ManuScript presents two unique experiences for starting a quote: a *basic* process for agents, and an *extended* process for carrier users such as underwriters and managers. (Out of the box, these two processes are identical, but carriers can customize each process as desired to meet their unique needs.)

Out of the box, the New Quote Selection ManuScript appears as follows.



Out of the box, the New Quote page contains the following fields.

Field	Description
Agency	Specifies the entity with which to associate the quote. This list only includes entities that are not Task Queue entities and at which the user has the *CreateQuotes* privilege.
Producer	Specifies the producer with which to associate the quote. This list only includes producers at the specified agency.
Product	Specifies which product to quote. This list includes all lines of business specified in EXAMPLE UserAdmin™ for the selected agency.

If desired, carriers can use the out-of-the-box New Quote Selection ManuScript, or they can modify their implementation to use a custom New Quote Selection ManuScript. If desired, carriers can also use an EXAMPLE Express® system page (or a set of system pages) to help users start the new quote process.

For instructions on customizing EXAMPLE Express® to use a custom ManuScript for this page, see *Configuring a New Quote Selection ManuScript*. For instructions on customizing EXAMPLE Express® to use system pages for this page,

see *Using New Quote Selection System Pages*. For more information about system ManuScripts in general, see *System ManuScripts*.

Pitney Bowes Address Verification

Pitney Bowes Address Verification allows users to send a single address to the Pitney Bowes data service for verification and to receive a response that includes latitude, longitude, validated address, percentage of confidence in the match, and a match code.

All requests receive a date and time stamp, and the user has the option to accept the Pitney Bowes' verified address or reject it and use the original address for quotation purposes. Optionally, Pitney Bowes Address Verification can be set up as a requirement for all addresses, so that only verified addresses can be used when binding a policy.

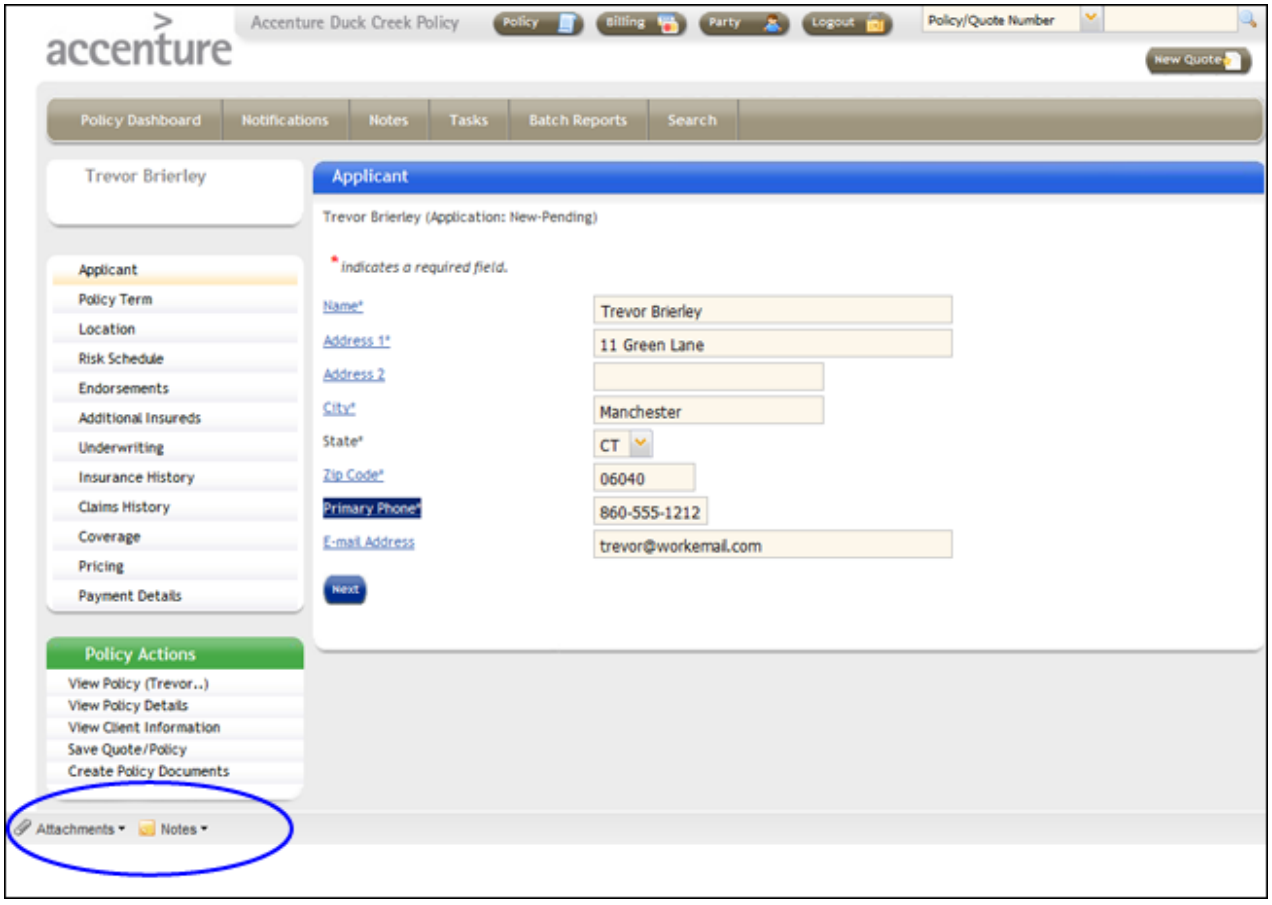
For more information, see:

- *Setting Up Pitney Bowes Address Verification*
- *Working with Addresses*

Underwriter's Workbench Toolbar

Underwriter's Workbench Toolbar is available in EXAMPLE Express® to allow users with appropriate permissions to view, create, or delete notes and attachments related to the policy or quote currently displayed in the main window. The toolbar appears attached to the bottom of your browser window on top of the EXAMPLE Express® screen.

Previously, the user had to navigate away from their current page in EXAMPLE Express® and view the Policy Details page to access this information. Now, the Underwriter's Workbench Toolbar provides a toolbar at the bottom of the browser window that allows users to access Attachments and Notes (to which they have permission) without leaving the page they are on.



For more information, see:

- *Setting up Underwriter's Workbench Toolbar*
- *Working with Notes*
- *Working with Attachments*

Consumer Access Interface

EXAMPLE Express® provides an out-of-the-box consumer access screen that allows consumers to access their policies and quotes online. Carriers can integrate this screen with custom ManuScripts, their own billing system, and their own web site to create a complete Consumer Access experience.

The account summary page can be used for policyholders or for prospective customers who have multiple quotes in the system. This page is controlled by the Consumer Dashboard ManuScript. If desired, carriers can use the out-of-the-box Consumer Dashboard ManuScript, or they can modify their implementation to use a custom Consumer Dashboard ManuScript.

From the account summary page, policyholders can:

- View and access their quotes and policies
- Start new quotes
- Update account information (username and password)
- Update billing and contact information
- Update policy information (perform endorsements)
- View and print quote and policy documents

From the account summary page, prospective customers can:

- View and access their quotes
- Start new quotes
- View and print quote documents

For more information about the Consumer Access feature, see *Consumer Access in the EXAMPLE Platform®*.

Database Management Tools

The quotes and policies that users create and work with in EXAMPLE Express® are stored in the ExampleData database. System administrators can perform database maintenance related to these quotes and policies using EXAMPLE UserAdmin™. In EXAMPLE UserAdmin™, administrators can:

- **Manage quote and policy number blocks** — With the Number Blocks tool, administrators can add, delete, and update quote and policy number blocks in the system.
- **Delete quotes** —With the Old Quotes Deletion tool, administrators can select quotes to delete by date range. Quotes that have been marked for deletion remain in the system until erased with the Purge tool.
- **Purge database records** — With the Purge tool, administrators can permanently remove quotes and policies that have been marked for deletion from the database.
- **Move policies between agencies** — With the Move Policies tool, administrators can reassign policies from one entity to another.

For more information about the database management tools in EXAMPLE UserAdmin™, see the *EXAMPLE UserAdmin™ User Guide*.

Portfolio

The Portfolio feature offers agents and underwriters the ability to easily view, update, and perform simultaneous transactions on a client's quotes and policies. Designed to work with product ManuScripts, portfolios present a consolidated view of a customer's associated quotes and policies. Functionally, a portfolio enables transaction events and business rules to be applied to sets of associated information.

Using Portfolio, users may:

- View an at-a-glance presentation of all quotes and policies for a customer's account
- Easily enter or update information shared across multiple quotes and policies
- Perform simultaneous transactions on multiple policies
- Print portfolio policy packages with a consolidated billing page for all coverages
- Apply discount credits for multiple policies

For more information about the Portfolio feature, see *Portfolio in the EXAMPLE Platform®*.

Security

EXAMPLE Express® security is controlled through authentication (provided through any of five authentication methods available for the EXAMPLE Platform®) and authorization (provided through EXAMPLE UserAdmin™).

Authentication

Authentication is the process that determines whether a user can access the EXAMPLE Platform®. The EXAMPLE Express® application does not authenticate users itself, but instead can use any of the following five authentication methods:

- EXAMPLE Authentication Server™
- Database authentication
- Windows authentication
- Mixed mode authentication (combines database and Windows authentication)
- Third party authentication

EXAMPLE Express® can be configured to accommodate single sign-on through some of these methods. For more information about EXAMPLE Platform® authentication, single sign-on options, and how to set up each authentication method, see the *EXAMPLE Authentication Server™ User Guide*.

Authorization

Authorization is the process that determines what actions a user can perform in the EXAMPLE Platform®. EXAMPLE Express® authorization is controlled through EXAMPLE UserAdmin™, a powerful, easy-to-use tool that allows carriers to organize access permissions according to their distinct organizational needs. EXAMPLE UserAdmin™ provides role-based security for EXAMPLE Express® by allowing administrators to assign system privileges and product permissions to users based on their association with one or more entities within a carrier's corporate hierarchy. Administrators can grant system and custom privileges to users through the assignment of specified roles (underwriter, agent, administrator, etc.) that are specific to each entity.

EXAMPLE Express® permissions that can be controlled by EXAMPLE UserAdmin™ include:

- Which items (policies and quotes, clients, etc.) users can see in the system
- Which actions users can perform in the system
- Which products an entity's users can access

EXAMPLE UserAdmin™ also provides the ability to specify custom privileges that can be used by EXAMPLE ManuScripts™ to control properties of individual fields (visibility, limits, defaults, options, workflow, etc.) according to the identity of the current user.

For more information about the security provided through EXAMPLE UserAdmin™, see the *EXAMPLE UserAdmin™ Overview* or the *EXAMPLE UserAdmin™ User Guide*.

Though EXAMPLE UserAdmin™ privileges control the actions users can perform on the EXAMPLE Platform®, the pages and features that actually display for each user are controlled through EXAMPLE Express® configuration and skins files. When configuring the EXAMPLE Platform®, determine what the EXAMPLE Express® display needs are for your users, then modify the EXAMPLE Express® configuration files and skins to display those features as appropriate for each role you have defined in EXAMPLE UserAdmin™.

For instructions on modifying EXAMPLE Express® for each role, see *Customizing the EXAMPLE Express® Experience*.

Out-of-the-Box Roles

EXAMPLE Express® is shipped with four sample roles. Each role has a unique set of permissions and dashboard configurations. The following table provides a description of each role's privileges in EXAMPLE Express® (defined in EXAMPLE UserAdmin™) and a description of the Policy Administration Dashboard for each role (defined in the ApplicationLayoutMap.xml file in the Global Root Directory\Express\App_Data directory).

Role	EXAMPLE Express® Privileges	Policy Administration Dashboard
Agent	• View and update quotes and policies, and create quotes • View and update client information, and create clients • View and update portfolio information, and create portfolios • View and create notifications • Create and work with tasks, and close and delete tasks created or owned by the agent • Add and remove attachments	• **New Quote module** — Agents can select the appropriate product and start a quote. • **Last Accessed module** — Contains the last five quotes or policies accessed. • **Notifications module** — Contains the last five notifications received. • **Tasks module** — Contains assigned tasks due this week.
Underwriter Assistant	• View and update quotes and policies, and create quotes • View and update client information, and create clients • View and update portfolio information, and create portfolios • View and create notifications • Create and work with tasks, and close and delete tasks created or owned by the underwriter assistant • Add and remove attachments	• **New Quote module** — Assistants can select the appropriate agency, producer, and product and start a quote. • **Last Accessed module** — Contains the last five quotes or policies accessed. • **Notifications module** — Contains the last five notifications received. • **Tasks module** — Contains assigned tasks due today.

Underwriter	• View and update quotes and policies, and create quotes • View and update client information, and create clients • View and update portfolio information, and create portfolios • View and create notifications • Create and work with tasks, and close and delete tasks created or owned by the underwriter • Add and remove attachments	• **Last Accessed module** — Contains the last five quotes or policies accessed. • **Notifications module** — Contains the last five notifications received. • **Tasks module** — Contains assigned tasks due today.
Underwriter Manager	• View and update quotes and policies, and create quotes • View and update client information, and create clients • View and update portfolio information, and create portfolios • View and create notifications • Create and work with tasks, assign tasks, and close and delete any task • Add and remove attachments	• **Last Accessed module** — Contains the last five quotes or policies accessed. • **Notifications module** — Contains the last five notifications received. • **Tasks module** — Contains assigned tasks due today.

OnDemand Framework

EXAMPLE Express® uses the OnDemand Framework to facilitate extensions to the application layer. This framework allows customers to extend and customize EXAMPLE Express® in a way that facilitates easy adoption of Duck Creek Technologies, Inc. updates without overwriting their extensions and customizations.

The OnDemand Framework uses a factory model to specify the objects that handle various parts of the system. This model allows users to extend the framework without changing original framework code. Using the assembly loading facilities of the Microsoft .NET framework and reflection, customers can extend the ContentMap.xml, PostActionMap.xml, and ApplicationLayoutMap.xml files as desired.

Three mapping files define the out-of-the-box content of the OnDemand Framework:

- **ContentMap.xml** — Defines content pages.
- **PostActionMap.xml** — Defines Express Actions that users can perform.
- **ApplicationLayoutMap.xml** — Defines navigation elements and dashboard pages for different types of users.

The flow of the OnDemand Framework—which begins when a user does anything in EXAMPLE Express® that triggers a post to the OnDemand Framework—is as follows:

1. The system authenticates the request.

2. The system processes the requested Express Action (if any). If an Express Action is specified, the _submitAction HTML form field must contain the name of the action to process. If the _submitAction field is blank or not found, no Express Action is performed.
3. The system develops the requested content. The _targetPage HTML form field must contain the desired content page. At times, the _submitAction may override the _targetPage. If no _targetPage is specified, the last visited _targetPage is used.
4. The system builds navigation elements based on the requested content.
5. The system uses an XSLT file to transform the requested content into HTML for output to the browser.

Steps 2, 3, and 4 use the map files listed above. EXAMPLE Express® uses these map files to determine which code to use to process each request. The configured class is instantiated so the request can process the activities.

In addition to the out-of-the-box map files, customers can use their own custom map files. To support custom map definitions, any .xml file in the directories specified by the *ContentFactory*, *PostActionFactory*, and *ApplicationLayoutFactory* configuration settings in the Express\Web.config file with a name that includes the word *custom* and the text also specified in those settings (out of the box, this is *ContentMap*, *PostActionMap, or ApplicationLayoutMap*) will load into EXAMPLE Express® as map files. (For example: *CustomContentMap.xml*.)

Custom map files should adhere to the following naming convention: *Custom[MapFile].xml*, where [MapFile] is the text specified in the appropriate Express\Web.config setting (e.g., ContentMap, PostActionMap, ApplicationLayoutMap). The *[MapFile]*.xml* convention is reserved for Duck Creek Technologies, Inc. use only. If you have migrated your custom map files from a previous version of the EXAMPLE Platform® to EXAMPLE Platform® 4.0, Duck Creek Technologies, Inc. recommends that you apply the *Custom[MapFile].xml* naming convention to your custom map files.

The system loads map files in the following order:

1. [MapFile].xml
2. [MapFile]*.xml
3. Custom[MapFile].xml

For more information on customizing the OnDemand Framework, see the following topics:

- **ContentMap.xml** — *Adding Custom Pages*
- **PostActionMap.xml** — *Adding Custom Express Actions*
- **ApplicationLayoutMap.xml** — *Customizing the EXAMPLE Express® Experience*

ContentMap.xml

The ContentMap.xml file contains a list of all content pages that EXAMPLE Express® may process. Each <content> entry is identified by a unique name that specifies the content to be processed when that content is requested. The OnDemand Framework creates a new instance of the specified class for the requested content item. Each implementation may provide a unique assembly and class for each named content and add any additional content needed. The custom content classes must inherit from the DuckCreek.OnDemand.ContentPage class found in the DuckCreek.OnDemandLib assembly and must override the Process() method:

```
public override void Process(PageState state,
System.Collections.Specialized.NameValueCollection requestForm)
```

Each content page must have an .xsl file of the same name in the *Global Root Directory*\Express\Skins*CustomSkin*\xsl directory. The XSLT uses this file to build the content page.

The following is an excerpt from a ContentMap.xml file:

```
<contentMap>
  <content name="admin" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentAdmin" menu="" useXmlPage="false" loadCache="1"/>
  <content name="adminDetail" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentAdminDetailEdit" menu="" useXmlPage="false"
loadCache="1"/>
  <content name="messages" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentMessages" menu="" useXmlPage="false"
loadCache="1"/>
  <content name="message" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentMessage" menu="" useXmlPage="false"
loadCache="1"/>
  <content name="messageNew" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentMessageNew" menu="" useXmlPage="false"
loadCache="1"/>
  <content name="messageUserList" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentMessageUserList" menu="" useXmlPage="false"
responseType="text/xml" loadCache="1"/>
  <content name="messageDetail" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentMessage" menu="" useXmlPage="false"
loadCache="1"/>
  <content name="client" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentClient" menu="" useXmlPage="false" loadCache="1"/>
  <content name="clientInfo" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.ContentClient" menu="" useXmlPage="false" loadCache="1"/>
</contentMap>
```

Any .xml file in the directory specified by the *ContentFactory* configuration setting in the Express\Web.config file with a name that includes the word *custom* and the text also specified in that setting (out of the box, this is *ContentMap*) will load into EXAMPLE Express® as a map file containing content functions. For more information, see *Adding Custom Pages*.

For information about customizing the ContentMap.xml file, see *Adding Custom Pages*.

PostActionMap.xml

The PostActionMap.xml file contains a list of all Express Actions available in EXAMPLE Express®. Each <action> entry is identified by a unique name that specifies the action to be processed when that action is requested. The OnDemand Framework creates a new instance of the specified class for the requested action item. Each implementation may provide a unique assembly and class for each named action and add any additional actions necessary. The custom action classes must inherit from the DuckCreek.OnDemand.PostAction class found in the DuckCreek.OnDemandLib assembly and must override the Process() method:

```
public override void Process(PageState state,
System.Collections.Specialized.NameValueCollection requestForm)
```

The following is an excerpt from a PostActionMap.xml file:

```
<actionMap>
  <action name="login" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.PostLoginGuest" />
```

```
  <action name="loginGuest" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.PostLoginGuest" />
  <action name="switchUser" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.PostSwitchUser" />
  <action name="load" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.PostLoadQuote" />
  <action name="newQuote" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.PostNewQuote" />
  <action name="save" assembly="DuckCreek.dctPASLibrary.DLL"
typeName="DuckCreek.OnDemand.PostSaveQuote" />
</actionMap>
```

Any .xml file in the directory specified by the *PostActionFactory* configuration setting in the Express\Web.config file with a name that includes the word *custom* and the text also specified in that setting (out of the box, this is *PostActionMap*) will load into EXAMPLE Express® as a map file containing Express Actions. For more information, see *Adding Custom Express Actions*.

For information about customizing the PostActionMap.xml file, see *Adding Custom Express Actions*.

ApplicationLayoutMap.xml

The ApplicationLayoutMap.xml file contains all navigation elements and dashboard pages for each type of user.

Out of the box, the ApplicationLayoutMap.xml file defines navigation elements and dashboard pages for the following groups of users:

- Agent
- Underwriter Assistant
- Underwriter/Underwriter Manager

The following is an excerpt from an ApplicationLayoutMap.xml file:

```
<applicationLayoutRoot>
  <applicationLayout id="AgentPAS" caption="Policy" roles="Agent" image="Nav/
topNavPolicy.gif" application="PAS" default="1">
    <menu name="PAS" startPage="dashboardHome">
      <menuItem id="Home" caption="Policy Dashboard" content="dashboardHome"
action="" java="submitPage" location="header" consumer="0" guest="0" login="1"
edit="0" security="15" roleBased="1" multiAgency="0" hideInPortfolio="0"/>
      <menuItem id="Diary" caption="Notifications" content="messages" action=""
java="submitPage" location="header" consumer="0" guest="0" login="1" edit="0"
security="15" roleBased="1" multiAgency="0" hideInPortfolio="0"/>
      <menuItem id="TaskMngmt" caption="Tasks" content="taskInfo" action=""
java="submitPage" location="header" consumer="0" guest="0" login="1" edit="0"
security="15" roleBased="1" multiAgency="0" hideInPortfolio="0">
      </menuItem>
      <menuItem id="Batch" caption="Batch Reports" content="batchProcess" action=""
java="submitPage" location="header" consumer="0" guest="0" login="1" edit="0"
security="15" roleBased="1" multiAgency="0" hideInPortfolio="0">
        <menu name="Default" portal="PAS">
          <menuItem id="BatchPolicy" caption="Batch Policy" content="batchPolicy"
action="" java="batchSelectTab" location="header" consumer="0" guest="0" login="1"
```

```
edit="0" security="15" roleBased="1" multiAgency="0"/>
          <menuItem id="BatchMessage" caption="Batch Notification"
content="batchMessage" action="" java="batchSelectTab" location="header" consumer="0"
guest="0" login="1" edit="0" security="15" roleBased="1" multiAgency="0"/>
        </menu>
      </menuItem>
      <menuItem id="Search" caption="Search" content="search" action=""
java="submitPage" location="header" consumer="0" guest="0" login="1" edit="0"
security="15" roleBased="1" multiAgency="0" hideInPortfolio="0"/>
      <menuItem id="Save" caption="Save Quote/Policy" content="" action="save"
java="submitAction" location="actions" consumer="1" guest="0" login="1" edit="2"
security="15" roleBased="1" multiAgency="0" hideInPortfolio="0"/>
      <menuItem id="Print" caption="Create Policy Documents" content="print"
action="" java="submitPage" location="actions" consumer="0" guest="0" login="1"
edit="2" security="15" roleBased="1" multiAgency="0" hideInPortfolio="1"/>
      <menuItem id="New" caption="New Quote" content="newBasic" height="400"
width="400" action="" java="submitPage" location="special" consumer="0" guest="0"
login="1" edit="0" security="15" roleBased="1" multiAgency="0" hideInPortfolio="0"/>
      <menuItem id="LogOut" image="blank.gif" caption="Logout" content="logout"
action="" java="submitPage" location="product" consumer="1" guest="1" login="1"
edit="0" security="0" roleBased="1" multiAgency="0" hideInPortfolio="0"/>
      <menuItem id="Password" caption="Change Password" content="admin" action=""
java="submitPage" location="logonOptions" consumer="1" guest="0" login="1" edit="0"
security="15" roleBased="2" multiAgency="0" hideInPortfolio="0"/>
    </menu>
    <pageLayout id="dashboardHome">
      <zone>
        <column width=".31">
          <module id="newQuote" title="New Quote" content="new" collapsed="0">
            <attribute name="_ManuScriptPageSet" value="Basic"/>
            <attribute name="_ManuScriptPage" value="Basic"/>
          </module>
          <module id="mruQuote" title="Recently Accessed" content="mru" collapsed="1">
            <attribute name="pageSize" value="10"/>
          </module>
        </column>
        <column width=".69">
          <module id="notifications" content="messages" title="Notifications"
collapsed="0">
            <attribute name="_displayCount" value="5"/>
            <attribute name="_showBody" value="1"/>
            <attribute name="_callbackForList" value="0"/>
          </module>
          <module id="taskQueue" title="Tasks" content="tasks" collapsed="1">
            <attribute name="_queryFilterSubset" value="QueueAssigned"/>
            <attribute name="_keynameQueueAssigned" value="TaskStatusCode"/>
            <attribute name="_keyvalueQueueAssigned" value="OPN"/>
            <attribute name="_keyopQueueAssigned" value="eq"/>
            <attribute name="_keyconnectorQueueAssigned" value=""/>
            <attribute name="_showFilter" value="DueToday"/>
            <attribute name="start" value="0"/>
```

```
            <attribute name="limit" value="100"/>
            <attribute name="_entityId" value=""/>
          </module>
        </column>
      </zone>
    </pageLayout>
  </applicationLayout>
</applicationLayoutRoot>
```

Any .xml file in the directory specified by the *ApplicationLayoutFactory* configuration setting in the Express\Web.config file with a name that includes the word *custom* and the text also specified in that setting (out of the box, this is *ApplicationLayoutMap*) will load into EXAMPLE Express® as a map file containing application layout information. For more information, see *Customizing ApplicationLayoutMap.xml*.

For information about customizing the ApplicationLayoutMap.xml file, see *Customizing ApplicationLayoutMap.xml*.

EXAMPLE Express® Skins

The content, behavior, and look and feel of EXAMPLE Express® can be customized through EXAMPLE Express® skins. EXAMPLE Express® skins are composed of .xsl, .css, .xml, custom .js files, and open source JavaScript frameworks. Carriers can customize the appearance and behavior of EXAMPLE Express® by overriding files in the DCTCore (Avant Garde skins) and DCTBase (Avant Garde 2 skins) directories. Customizations can include:

- Changing the look and feel of EXAMPLE Express®
- Adding custom pages
- Overriding core behavior
- Adding additional JavaScript
- Changing certain HTML features, such as the browser title bar, meta tags, and HTML footer text

For instructions on customizing EXAMPLE Express® skins, see *Customizing EXAMPLE Express® Skins*.

Avant Garde

The Avant Garde skins delivered with EXAMPLE Platform® 4.0.0 provide users with some great new features in EXAMPLE Express® including:

- A new task system
- Dashboard home page capabilities
- Enhanced search capabilities
- Improved New Quote process
- Notifications

The table below describes the major differences that exist between Lila-based (3.x line) and AvantGarde-based (4.x line) skins.

Difference	Description
Look and feel	Out-of-the-box Lila-based skins provide a different look and feel than out-of-the-box AvantGarde-based skins, with different color schemes, images, layouts, and text.

Policy Administration Dashboard	Out-of-the-box Lila-based skins do not support a role-based dashboard. Instead, Lila-based skins include a Home page that displays the five most recently accessed quotes and policies.
Task system	Out-of-the-box Lila-based skins do not support the task system available with AvantGarde-based skins.
Notification system (message system)	Out-of-the-box Lila-based skins use a message and diary system instead of a notification system. Out of the box, Lila-based skins provide the ability to send, receive, and reply to messages, and to create messages (diary items) that can be attached to clients or policies.
Search functionality	Out-of-the-box Lila-based skins do not incorporate the Quick Search feature available with AvantGarde-based skins. In addition, Lila-based skins use an Open page and a Client page to view and search for policies and clients (respectively), rather than a single search page.

For more information, see *Customizing Skins*.

Directory Structure

The Avant Garde skin includes an entirely new directory structure, designed to make managing your skin files easier than ever.

The following table illustrates the directory structure of the Avant Garde skin.

Directory	Description
DCTCore	
scripts	JavaScript to be maintained and updated by Duck Creek Technologies, Inc. This directory contains core functionality that should not be customized.
xsl	XSL to be maintained and updated by Duck Creek Technologies, Inc. This directory contains core functionality that should not be customized.
Customizable Directory	
css	Customizable .css files. Files preceded by *x_* indicate override files to be used for CSS styles that apply to a single page only.
images	Customizable image files.
scripts	Customizable JavaScript files.
Source	Obsolete as of EXAMPLE Platform® 4.0.
xsl	Customizable .xsl files.

Avant Garde 2

EXAMPLE Platform® 4.2.2 includes the Avant Garde 2 skin, built on the AvantGarde 1 foundation. Complete with a new directory structure, new ExtJs controls, and more, Avant Garde 2 delivers extreme ease of use and a world-class look and feel.

The Avant Garde 2 skin includes the following significant ehnhancements:

- **New directory structure** – A new core set of skin files has been created to allow for easy upgrades. For more information about the new directory structure, see *Directory Structure*.
- **Updated ExtJs framework** – Avant Garde 2 is built on the latest version of the ExtJs framework. It provides increased performance, new controls, enhanced stability, and a complete set of entry fields, combo boxes, radio buttons, and check boxes that has been converted to the new ExtJs framework to take full advantage of new features such as validation edits, visual cues, and auto-filtering. EXAMPLE Platform® users can now implement these new controls in their system pages. For more information, see *The ExtJs Framework*.
- **Minified JavaScript files** – The EXAMPLE Platform® now includes compressed, minified versions of the JavaScript files to assist with performance during runtime. For more information about minified JavaScript files, see *Minified JavaScript Files*.
- **One-to-one override structure** – The new one-to-one file structure allows for much easier customizations. By maintaining base and customized versions with the same file name in different directories, users can easily compare them each time an EXAMPLE Platform® upgrade is performed and then make any necessary changes to the customizable files. For more information, see *Customizing EXAMPLE Express® Skins*.

  At no time is it necessary to modify any files in the Base directory to change the way a skin function or layout is processed.

- **Refactored JavaScript** – All JavaScript has been reorganized based on function in the DCTBase\Core\Devscripts directories, and all unused JavaScript has been removed. All EXAMPLE Platform® core JavaScript code is stored in the DCTBase\Core\Devscripts directories.
- **Updated user input fields** – User input fields have now been converted to ExtJs controls to allow user to take advantage of visual cues such as required field indicators, validation edits, auto-filtering combo boxes, and quicktips for dropdown lists containing content that exceeds the width of the dropdown list.
- **New EXAMPLE Express® themes** – Avant Garde 2 introduces a new theme system designed to assist in applying CSS style customizations. For more information, see *Themes*.
- **Updates to EXAMPLE TransACT® pages** – A new Version 2 pages version of each EXAMPLE TransACT® page has been created for use with the Avant Garde 2 skin. When using the Avant Garde 2 skin, the ManuScript catalog should reference the TransACTMaster2_1_0_PagesV2 ManuScript.

Directory Structure

The Avant Garde 2 skin includes an entirely new directory structure, designed to make managing your skin files easier than ever. Much like the DCTCore directory and the Avant Garde skin, Avant Garde 2 is associated with a new DCTBase directory. This directory contains all base JavaScript and XSL files required to make EXAMPLE Express® skins function correctly.

When customizing EXAMPLE Express® skins, do not modify any files in the DCTBase or AvantGarde2 directories. All files in these directories are overwritten during upgrades. Instead, to override these files, you

must create a custom skins directory (copied from the AvantGarde2 directory) and modify the files in this directory. For instructions on creating a custom skins directory and overriding DCTBase files, see *Creating a Custom Skins Directory* and *Overriding DCTBase Files*.

The following table illustrates the directory structure of the Avant Garde 2 skin.

Directory	Description
DCTBase (Non-Customizable)	
css	CSS to be maintained and updated by Duck Creek Technologies, Inc. This directory contains core functionality that should not be customized.
images	Images to be maintained and updated by Duck Creek Technologies, Inc. This directory contains core functionality that should not be customized.
scripts	JavaScript to be maintained and updated by Duck Creek Technologies, Inc. This directory contains core functionality that should not be customized.
devscripts	Devscripts to be maintained and updated by Duck Creek Technologies, Inc. Used to reference functions that need to be overridden. This directory contains core functionality that should not be customized.
themes	Themes to be maintained and updated by Duck Creek Technologies, Inc. This directory contains core functionality that should not be customized.
xsl	XSL to be maintained and updated by Duck Creek Technologies, Inc. Used to reference files and templates that need to be overridden.This directory contains core functionality that should not be customized.
AvantGarde2 (Customizable)	
css	Customizable .css files. Filenames beginning with *x_* indicate override files to be used for CSS styles that apply to a single page only. By default, this directory includes the following files: • **dctExtJsOverrides.css** – Used to override layouts for HTML elements instead of altering the Ext-all.css file. • **dctExt-xtheme-blue.css** – Blue theme for any of the EXAMPLE Platform® ExtJs extensions. • **dctExt-xtheme-gray.css** – Gray theme for any of the EXAMPLE Platform® ExtJs extensions. • **ext-xtheme-blue.css** – Blue theme for any of the ExtJs controls. • **ext-xtheme-gray.css** – Gray theme for any of the ExtJs controls. If a different theme is desired, you must copy one of each of the color themes and name them accordingly.
images	Customizable image files. By default, this directory includes image files used with the gray and blue themes added during installation.
scripts	Customizable JavaScript files.

themes	Customizable theme files.
xsl	Customizable .xsl files. Content and Common sub-directories are created during installation. Users must create copies of any DCTBase .xsl files they wish to customize and add them to one of these directories. All base page .xsl files are considered core files.

The ExtJs Framework

ExtJs is an open source JavaScript framework that provides rich browser controls such as calendar controls, sliders, accordions, and grids. Duck Creek Technologies, Inc. maintains a commercial license for the framework, allowing it to be packaged and installed with the EXAMPLE Platform®.

For API documentation for ExtJs controls, see one of the following:

- *http://www.sencha.com* – Browse to this page and then navigate to the API documentation for ExtJs.
- *http://dev.sencha.com/deploy/dev/docs* – Browse to this page to access API documentation for the latest version of the ExtJs framework.

For archived versions of API documentation, browse to *http://www.sencha.com/learn/Ext_Version_Archives*.

ExtJs Controls

The following ExtJs controls are supported by the EXAMPLE Platform®:

ExtJs Control	Description
Dashboard portal widgets	*Portal pages* are the dashboard pages for Duck Creek Billing™ and EXAMPLE Express®. They are the main home pages that users are directed to when they initially log in. *Portlets* are the small sections built on each page to give the user quick access to information for the product.

Grid widgets	The following grids and grid features are available for use with the Avant Garde 2 skin: • **Array** – Array grids do not have any paging function. All of the data for the grid appears when the page initially displays. • **Paging** – Paging grids display a portion of the overall data to limit how much the user can view at one time. By default, most pages only display 20 items at a time. The user can view additional information by selecting the next page or specifying the page they would like to view. Paging grids go back and retrieve data once the initial page displays. • **Editable** – The EXAMPLE Platform® uses editable grids for interview pages to allows the user to click a cell within the row to modify the data. Once the data is modified, the updated information is stored. • **Column Filtering** – Column filters are used primarily by Duck Creek Billing™ and are only available for paging grids. They allow users to filter lists of data based on the column. To enable this feature, the user must click the header for the column and then select column filtering. Column filtering can be based on strings, dates, or lists of items. • **Grouping** – Groupings are used with Duck Creek Billing™ to allow grids to group similar rows together and organize the data for the user. For example, a user may wish to group the changes to a policy term together.
Column tree widgets	Column tree widgets are only used within Duck Creek Billing™ to display an installment schedule. This widget looks like a directory structure to help organize information for the user.
Popup widgets	Popup widgets are used throughout the EXAMPLE Platform®. A couple of uses for popup widgets include: • Modal popups used within the interview pages. • Popups to capture user information, such as notes for Duck Creek Billing™.
Accordion panels	Accordion panels allow multiple sections to be grouped together. With accordion panels, only one section can be open at a time.
Collapsible panels	Collapsible panels are used in: • Duck Creek Billing™ in the navigation area to display high-level information for the account. They allow sections to be open or closed based on the user preference. • EXAMPLE Express® to make sections within the interview pages collapsible.
Sliders	Sliders are widgets that allow the user to slide a bar back and forth to increase or decrease some value.
QuickTips	QuickTips are small popup windows (usually attached to headers and lables) that display helpful information about an item on the page. Annotation hover help is an example of a QuickTip.
Charts	Chart widgets allow data to be displayed in various types of charts. Currently, the EXAMPLE Platform® supports the use of pie charts, line charts, and bar charts.

Minified JavaScript Files

Avant Garde 2 skins include new compressed and minified JavaScript files to help boost performance during runtime.

Two versions of each JavaScript file are added during installation:

- **Minified** – Compressed version of the JavaScript file used at runtime (for example, Dct-base.js).
- **Debug** – Debug version of the JavaScript file that allows skin developers to debug a given JavaScript file (for example, Dct-base-debug.js). Debug files are created with the files located in the DCTBase\Core\devscript directory.

To enable debug mode, ensure that the value of the *debugMode* configuration setting in the Express\Web.config file is set to a positive integer.

Custom Styles and Format Masks

In general, the look and feel of EXAMPLE Express® should be governed by general style rules defined in the EXAMPLE Express® skins. At times, however, the appearance of specific page elements may depend upon criteria specified in a ManuScript. To accommodate this, ManuScripts allow you to apply custom styles and format masks to page elements. These custom styles and format masks can be configured in the EXAMPLE Express® skins to provide the desired appearance and behavior.

Custom Styles

In EXAMPLE Express® skins, carriers can define styles across the entire EXAMPLE Express® application or across only parts of it. If desired, carriers can also apply custom CSS styles to specific page elements in a ManuScript. These styles are defined in a single file in the skins directory, and can be applied to individual fields and page elements in EXAMPLE Author®.

For information about customizing CSS styles, including custom styles for ManuScript pages, see *Customizing CSS*.

Format Masks

In EXAMPLE Author®, users can apply format masks to fields. Format masks can apply to user input or display output. To function properly, format masks must be supported in the EXAMPLE Express® skins.

With out-of-the-box EXAMPLE Express® skins, you can apply the following format masks to fields. In most cases, only one format mask can be applied to any given field. If the text entered does not match the corresponding format, an alert box informs the user that the value is invalid, and the focus is set to the offending field. All format masks are case-sensitive.

Ensure that a format mask is supported by EXAMPLE Express® skins before applying it to a field. A format mask must be supported by each installation of EXAMPLE Express® in order to function properly. For additional information, see *Format Masks* in the *EXAMPLE Express® Reference Guide*.

Labeled Format Masks

Labeled format masks must be supported by each installation of EXAMPLE Express® in order to function properly. The following labeled format masks are supported by EXAMPLE Express® out of the box.

String

Format Mask	Output
email	user@domain.tld
phone	###-###-####
time	HH:MM:SS AM/PM (Note: seconds are optional.)
time24	HH:MM:SS (military time)
textarea:rows:4:width:100%: textarea:rows:4:width:100:	4 and 100 represent values for the number of rows (height) and the percentage of the window or the width in percentage or pixels.  Any non-numeric characters other than the percent sign (%) entered after the width will be removed and replaced with "px" to denote that they are pixels.
zipcode	##### or #####-####
ip	??#.??#.??#.??#
ssn	###-##-####
hide:	Always masks characters (displays an asterisk (*)) on entry and on display. An additional format mask can be added after the colon. (For example, hide:ssn.)  This format mask only works with interview pages.
hideOnEntry:	Masks characters (displays an asterisk (*)) as they are entered. An additional format mask can be added after the colon. (For example, hideOnEntry:ssn.)  This format mask only works with interview pages and does not hide characters displayed as read-only.
hideOnDisplay:	Always masks characters (displays an asterisk (*)) only on display, not on entry. An additional format mask can be added after the colon. (For example, hideOnDisplay:ssn.)  This format mask only works with interview pages.

Float

Format Mask	Output
C*n*	Field value with culture-appropriate (as determined in ManuScript properties) currency symbol and numeric separators, to *n* number of decimal places. With the base culture in a ManuScript set to *en-US*, the following are examples of output for the value 11.87: • C0 — $12 • C1 — $11.9 • C2 — $11.87 The rounding that is applied when using this format mask occurs at the display level and is completely separate from any rounding defined in a ManuScript field.
hide	Always masks characters (displays an asterisk (*)) on entry and on display. An additional format mask can be added after the colon. (For example, hide:ssn.)  This format mask only works with interview pages.
hideOnEntry	Masks characters (displays an asterisk (*)) as they are entered. An additional format mask can be added after the colon. (For example, hideOnEntry:ssn.)  This format mask only works with interview pages and does not hide characters displayed as read-only.
hideOnDisplay	Always masks characters (displays an asterisk (*)) only on display, not on entry. An additional format mask can be added after the colon. (For example, hideOnDisplay:ssn.)  This format mask only works with interview pages.

Date

Date field display (with no format mask applied) defaults to the short date format appropriate to the culture set in Web browser options. Format masks placed on Date fields will override this default. Input fields of a Date data type will assume dates are entered in the short date order appropriate to the browser settings and will accept hyphens (-), periods (.), or slashes (/) as separators.

 Applying format masks to date input fields can result in incorrect data collection and other errors. To avoid confusion, Duck Creek Technologies recommends indicating in the user interface which format the end user should use.

Format Mask	Output

mm-dd-yyyy	month-day-year format
dd-mm-yyyy	day-month-year format
yyyy-mm-dd	year-month-day format
MMMM d, yyyy	Long date format such as *January 1, 2007*.
d	Short date format (such as *01-01-2007*) appropriate to the culture as determined by browser settings.
D	Long date format (such as *Monday, January 1, 2007*) appropriate to the culture as determined by browser settings. This mask should only be applied to read-only fields.

Boolean

The default display for a Boolean field is a check box.

Format Mask	Output
radio	Displays a Boolean field as side-by-side Yes and No option buttons.
radio:1	Displays a Boolean field as above/below Yes and No option buttons.

Other (with options)

The default display for defined options is a combo box.

Format Mask	Output
radio	Displays each as an option button in a single row.
blankOption:(select)	*(select)* will appear as a default in a drop-down list of options, and may be replaced in the mask with any text.

Custom Format Masks

There are two types of custom format syntax: numeric and character. The first character of the format indicates the type of formatting:

- Numeric format is indicated by a first character of # or 0.
- Character formatting is indicated by the @ character. The edit mask string immediately follows the @ character, and consists of three fields separated by semicolons. The first part of the mask is the mask itself. The second part is the character that determines whether the literal characters of the mask are

matched to characters in the Value parameter or are inserted into the Value string. The third part of the mask is the character used to represent missing characters in the mask.

The following are the special characters used in the first field of the mask.

Character	Meaning in mask
>	If a greater than symbol appears in the mask, all *a* format characters that follow are in upper case until the end of the mask or until a < character is encountered.
<	If a less than symbol appears in the mask, all *a* format characters that follow are in lower case until the end of the mask or until a > character is encountered.
<>	If the less than and greater than symbols appear together in a mask, no case checking is done. This turns off the > or <, while > or < will turn it back on. (sequentially)
a	Requires an alphabetical character in this position, and the case is based upon the presence of one of the case indicators (>,<,<>) in the mask previous to the 'a' format character.
L	Requires an alphabetic character in this position.
A	Requires an alphanumeric character in this position.
C	Requires an upper-case alphabetic character in this position.
c	Requires a lower-case alphabetic character in this position.
0	Allows a numeric character in this position.
9	Requires a numeric character or a positive or negative sign (+ or -) in this position.
#	Requires a numeric character or a positive or negative sign (+ or -) in this position.
-	Inserts spaces into the returned string.
*	Requires a character in this position in a format mask. This can be any character allowed by the interface.
^	The character that follows a caret is a literal character. Use this character before any mask characters used as a literal. For example, in the mask `@LL^xAA` the third character of the entry must be *x*.

Any character that does not appear in the preceding table can appear in the first part of the mask as a literal character. Literal characters are inserted automatically if the second field of the mask is 0, or matched to characters in the Value parameter if the second field is any other value. The special mask characters can also appear as literal characters if preceded by a back slash character (\).

Using Regular Expressions (Regexes) to Create Custom Format Masks

Regular expressions (regexes) can be used to create custom format masks. Using a regex, a user can create custom format masks that will display a message for invalid field values. The most common regex structure for this type of format mask is:

regex:[regular expression]:[message (opt'l)]:[attributes (opt'l)]–The most common attribute value for this type of regex is *i* for case sensitive.

The following are examples of additional regex format masks:

- **regex:^\\d+$:Please enter a number** – This forces the user to use only digits, or the message *Please enter a number* is displayed. The *d,* written as \\d, specifies that only digits can be entered. The + specifies one or more digits, and the ^ and $ symbols specify the beginning and end of the string (this will prevent other characters from being entered before or after the digits).

Escape backslashes by using double backslashes (\\) instead of a single backslash(\).

- **regex:^\\d{5}$:Please enter five digits** – This forces the user to enter exactly 5 digits, or they receive the message, *Please enter five digits.*
- **regex:^(\\$)?\\d+$:Please enter the cost** – Any number of digits, with or without a leading dollar sign. The dollar sign should be "escaped" by putting backslashes in front of it because without them, the *$* character signifies the end of the line. The *?* character means that the dollar sign is optional.
- **regex:^\\d{0,5}\\.\\d{1,3}$:Please enter a number with a decimal** – A number with 0-5 digits followed by a decimal and 1-3 digits must be entered. A double backslash is necessary in front of the decimal because a period in regexes is a special wildcard character that will allow any character to be used.
- **regex:^((?!(p\\.?o\\.? box)).)*$:Cannot enter a PO box:i** – This would prevent a user from entering *P.O. Box* in any combination of uppercase or lowercase characters, with or without periods, in the field.

A colon (:) cannot be used in the regular expression or message. Colons are used as delimiters of format mask sections.

Format masks do not run a check on empty fields, so a user is not forced to enter a value. Instead use the **required** attribute.

Ajax Refreshes

EXAMPLE Express® leverages Ajax refresh techniques to provide a more streamlined user experience. Using Ajax, communication between the browser and EXAMPLE Server® can often be performed asynchronously, enabling the partial refreshes of EXAMPLE Express® pages instead of entire page refreshes.

Even with the use of Ajax, full page refreshes will occur in EXAMPLE Express® under some circumstances. For example, full refreshes will occur when:

- An item is added to the page
- Buttons are added or removed (or are shown/hidden)
- A required item is removed from the page
- Rating messages exist on the page
- A widget exists on the page
- The header or footer changes

- The page navigation changes
- The *Required* property of a field in a table layout changes

Passthrough.config File

In some cases, EXAMPLE Server® requests need to be passed to EXAMPLE Server® through EXAMPLE Express®. This may occur, for example, when a PreEvent field on an action is executed, or when a collapsible section on a page is expanded.

To accommodate the need for EXAMPLE Express® to pass requests to EXAMPLE Server®, a *Passthrough.config* file in the Duck Creek Technologies\Express directory defines the specific requests that can be passed through EXAMPLE Express® to EXAMPLE Server®. This file limits the requests that can be sent through EXAMPLE Express® to a small number of specified requests. Limiting the requests that can be passed through EXAMPLE Express® limits the security exposure that could result from all requests being available for execution through EXAMPLE Express®.

The Out-of-the-Box Passthrough.config File

The Passthrough.config file that is installed with EXAMPLE Express® includes the following EXAMPLE Server® requests that are commonly used by EXAMPLE Express® and that represent a very low security risk. Out of the box, this file appears as follows:

```
<rulesets>
    <ruleset>
        <rule type="RequestName">Session.resumeRq</rule>
    </ruleset>
    <ruleset>
        <rule type="RequestName">Session.storeDataRq</rule>
    </ruleset>
    <ruleset>
        <rule type="RequestName">ManuScript.getValueRq</rule>
    </ruleset>
    <ruleset>
        <rule type="RequestName">ManuScript.getPageRq</rule>
    </ruleset>
</rulesets>
```

Customizing the Passthrough.config File

If your EXAMPLE Express® skins or ManuScripts must send additional requests through EXAMPLE Express®, you can customize the Passthrough.config file. If an additional request is required, add the request to this file, and then bounce EXAMPLE Express® to see your changes.

Adding Parameters to EXAMPLE Server® Requests

To increase the security of server requests that are added or included in the Passthrough.config file, you can add Request Parameter rules. Request Parameter rules can be used to further restrict EXAMPLE Server® requests through attributes. The following is an example of a Request Parameter ruleset for <ManuScript.getValueRq> request:

```
<rulesets>
   <ruleset>
      <rule type=”RequestName”> ManuScript.getValueRq</rule>
      <rule type=”RequestParameter” name=”manuscript”>BaseProduct</rule>
      <rule type=”RequestParameter”
```

```
name="field">Bind.PreEventField,Billing.SetupBillingPreEventField</rule>
   </ruleset>
   <ruleset>
      <rule type="RequestName">ManuScript.getPageRq</rule>
    </ruleset>
</rulesets>
```

If more than one parameter is needed for a particular request, additional parameters can be added by separating them with commas, as shown above in the **Bind.PreEventField** and **Billing.SetupBillingPreEventField** fields. Rulesets are processed in the order they are placed in the Passthrough.config file. The request will pass for the first ruleset that it matches and will not continue searching through additional rulesets once a match is found.

When adding requests to the Passthrough.config file, be aware that adding additional requests may increase security risks. Evaluate the risk of each request and consider alternate strategies for achieving the desired result.

System ManuScripts

Carriers can customize the content, appearance, and behavior of certain EXAMPLE Express® system pages by using customizable EXAMPLE ManuScripts™. The following EXAMPLE Express® system pages can use a ManuScript:

- **New Quote Selection page** – This page appears when a user selects to start a new quote. It is used to provide a user-friendly process for selecting the appropriate product ManuScript for a quote. This ManuScript can also be displayed as a module on the EXAMPLE Express® home page. The New Quote Selection ManuScript replaces the XSL versions of the New Quote Selection page (new.xsl, newBasic.xsl, and newExtended.xsl).
- **Client Details page** – This page appears when a user selects to add a new client or update existing client information. This ManuScript gathers and displays all details about a client. There is no XSL version of the Client Details page.
- **Send a Notification/Notification Details pages** – These pages appear when a user selects to view or create a notification. These pages gather and display (respectively) all details of a notification. The Message Detail ManuScript replaces the XSL versions of the Send a Notification and the Notification Details pages (messageNew.xsl and message.xsl). When these pages are driven by the Message Detail ManuScript, the system uses both the Message Detail ManuScript and the messageDetail.xsl page to render the page content.
- **New Task/Task Details pages** – These pages appear when a user selects to view or create a task. These pages gather and display (respectively) all details of a task. There is no XSL version of the New Task and the Task Details pages, but the system uses both the Task Detail ManuScript and the taskDetail.xsl page to render the page content.

For instructions on using system ManuScripts, see *Using EXAMPLE Express® System ManuScripts*.

Express Actions

Users can execute specific EXAMPLE Express® actions from EXAMPLE ManuScripts™ using Express Actions. Express Actions are system-defined actions (defined in the PostActionMap.xml file) that are applied to ManuScript pages in Page Designer. Specifically, Express Actions are applied to page actions (buttons and links). Users can specify the Express Action to take when the button or link is clicked using the *Express Action* property of the action.

For a list of all Express Actions available in the out-of-the-box PostActionMap.xml file or for details about each Express Action, see the appendix *Express Actions*.

Express Action Syntax

Express Actions are applied to page actions (buttons and links) in Page Designer using the *Express Action* property. The *Express Action* property syntax is as follows:

```
postActionCommand:targetPage
```

The `postActionCommand` portion of the *Express Action* syntax is the action to perform, and the `targetPage` portion is the EXAMPLE Express® system page to load. If no `targetPage` is specified, the system continues with the ManuScript-defined interview.

If desired, you can also instruct the system to save the active policy before taking action by adding an optional `save` element before the `postActionCommand`. This syntax appears as follows:

```
save,postActionCommand:targetPage
```

Express Action Processing

For each post-back to EXAMPLE Express®, the following actions are performed on the HTML form. The interview action may also be peeked into if specified in the field referenced by the _action form field.

When an Express Action requires HTML form fields, the HTML form from which the action is executed must contain those fields. For details about each Express Action and any HTML form fields required, see the appendix *Express Actions*.

1. EXAMPLE Express® loads the two primary action fields, *_submitAction* and *_targetPage*. *_submitAction* contains the specified Express Action, and *_targetPage* contains the name of the EXAMPLE Express® system page to load. If _submitAction is empty, the system will take no action unless an action is specified in the interview action below. If _targetPage is blank, the system does not load an EXAMPLE Express® system page, but continues with the interview content.
2. EXAMPLE Express® looks for the interview action name in the form field *_action*.
3. EXAMPLE Express® looks at the *_fieldChanged* form field. If _fieldChanged="1", then the data fields on the form have been modified by the user and the session must be updated. Also, if auto-save is turned on, any field that does not begin with an underscore (_) is stored using the <Session.storeDataRq> request. If fields have changed and auto-save is on, then the quote or policy is stored using the <OnlineData.storePolicyRq> request.

   Auto-save is also executed when the interview action has an add, delete, mark-for-delete, or duplicate command in the interview action, even if no fields have been modified.

4. EXAMPLE Express® peeks into the <ManuScript.getPageRq> request of the interview action for certain parameter conditions. There are two activities performed on the interview action: *peek* and *extract*. *Peek* looks for the value of the interview action, while *extract* pulls the attribute from the <ManuScript.getPageRq> request attribute list, if it exists. The interview action has a request that may have some of the following attributes:

   ```
   <ManuScript.getPageRq manuscript="" postAction="" eventField=""
   preCommand="", etc>
   ```

   - Peek for *postAction* attribute, and set it as the post action to be performed before the interview content is rendered. This attribute will override the Express Action set by the _submitAction HTML form field, though it will preserve the save if specified previously.
   - Peek for *resetManuScriptID* and *resetLOB*. If either exists, perform them.

- Peek for *eventField*. If *eventField=""*, extract *preCommand=""* from the request and treat it as the *eventField.*

This scenario will cause the *preCommand* to execute before the page is rendered and then remove that action from the <ManuScript.getPageRq> request. This allows some actions to execute completely before the page content is rendered.

- If an *eventField* is specified, peek for the *manuscript* attribute and execute the field through the <ManuScript.getValueRq> request in the specified ManuScript of the interview action. Also, peek for the *contextPath* attribute (if specified) to use as the context path for the <ManuScript.getValueRq> request.
- Process any postAction specified in the _postAction form field or the *postAction* attribute of the <ManuScript.getPageRq> request.
- Render the content page, using the newly modified interview action request.

Express Action Commands

Express Action	Description	
activatePolicy	Creates a consumer account; returns a clientID, quoteID, and LOB; and then loads the specified quote or policy.	
addTaskAttachment	Adds an attachment to a task.	
admin	Updates administration information.	
assignTask	Updates the queue and/or user to which a task is assigned.	
commit, commit2, audit	Processes an out-of-sequencetransaction and commits all changes to the database.	
consumerConvertToPolicy	Moves a quote from the DC_POL_ConsumerQuote table to the Quote and History tables.	
consumerCreateAccount	Creates a user account for the EXAMPLE Platform®.	
consumerLoadQuote	Loads a quote from the DC_POL_ConsumerQuote table.	
consumerLogin	Logs a user in under the proxy account used for consumer access.	
consumerNewQuote	Creates a new quote in the DC_POL_ConsumerQuote table.	
consumerReLogin	From within EXAMPLE Express®, logs a consumer user out of the proxy account and logs them in to an existing full user account.	
delete	Marks or un-marks the specified quote or policy for deletion from the database. The specified item is not removed from the database until a purge is performed.	
deleteClient	Marks the specified client for deletion from the database. The client is not removed from the database until a purge is performed.	

deleteMessage	Deletes the specified notification from the database.	
deleteTaskAttachment	Removes an attachment from a task.	
details	Updates the details of a quote or policy in the database.	
download	Sends a copy of the XML image of the specified quote or policy to the browser.	
duplicate	Duplicates the specified quote or policy in the database.	
initAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *1*. Loads the Policy Forms page.	
initPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request with the *forceInit* attribute set to *1*. Loads the Policy Forms page.	
issue	Saves the active quote or policy to the database using the <OnlineData.storeTransactionRq> request.	
load	Loads the specified quote or policy into the session and invokes the interview process of the associated ManuScript.	
loadHistoryTx2	Loads the specified history image of a quote or policy into the session and invokes the interview process of the associated ManuScript.	
loadPortfolio	Loads the specified Portfolio into the session and invokes the interview process of the associated ManuScript.	
login	Logs the user into EXAMPLE Server® and begins a new EXAMPLE Express® session.	
messageDetail	Updates the active notification in the database using the interview content of a Message Detail ManuScript.	
newMessage	Displays a New Message page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action. Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.	
newNotification	Displays a Send a Notification page (messageNew.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action.	
newQuote	Creates a new quote in the active session, invoking the interview process of the specified ManuScript. Performs no database action.	
newTask	Adds a new task based on the task record located in the session at the path _TaskData/TaskRecord.	

previewAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *0* or *3*. Loads the Policy Forms page.	
previewPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request with the *forceInit* attribute set to *0* or *3*. Loads the Policy Forms page.	
readMessage	Displays the Notification Details page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl) for the specified notification.	
refreshAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *3*. Loads the Policy Forms page and executes the <FormsEngine.setPrintJobItemsRq> request with an *autoMark* attribute set to *markDefault*.	
rescind	Reverts the current policy image back to the state of the specified history image, retaining records of any transactions that are removed.	
resetManuScriptID	Resets the associated ManuScript for the active quote.	
resetSession	Clears all session information.	
rollbackHistory	Rolls back the transaction records of a policy to the specified history image, removing any subsequent transactions and making the specified history image the current image. This action then loads the (new) current policy image into the active session and invokes the interview process of the associated ManuScript.	
runRulesAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *2*. Loads the Policy Forms page.	
runRulesPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request with the *forceInit* attribute set to *2*. Loads the Policy Forms page.	
save	Saves the active quote or policy to the database using the <OnlineData.storePolicyRq> request.	
saveMessage	Updates the specified message in the database from the HTML form fields on the New Message page (message.xsl). Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.	
saveNotification	Updates the specified notification in the database from the HTML form fields on the New Message page (messageNew.xsl).	
setActiveAgency	Updates the active agency for the logged in user in the EXAMPLE Express® <state> node.	

setLanguage	Sets the language property in the session and the EXAMPLE Express® <state> node to the specified language.	
setPrintManuScript	Sets the ManuScriptID to use when the **Create Policy Documents** link is clicked. This action should be used when a different ManuScript than the loaded interview ManuScript (state.ManuScriptID) is needed.	
setTaskComplete	Sets the status of a task to *Complete*.	
setTaskDeclined	Sets the status of a task to *Closed*, with a reason of *Declined*.	
setTaskDeleted	Marks a task for delete. (Sets the status of the task to *Deleted*.)	
setTaskStatus	Sets the status of a task.	
signup	Creates an agency signup request and sends an e-mail to the recipient specified by the *MailTo* configuration setting in the Express\Web.config file.	
submit	Saves the active quote or policy to the database using the <OnlineData.storeTransactionRq> request.	
switchUser	Moves the active quote or policy to the specified agency and logs in as a different user to load the active policy.	
updateTask	Updates a task in the database based on the task record located in the session at the path _TaskData/TaskRecord.	
upload	Uploads a file from a ManuScript to the directory specified in the *UploadDir* configuration setting in the Express\Web.config file.	
validatePrintJob	Verifies that no required fields on selected forms are empty.	

Configuring EXAMPLE Express®

To configure EXAMPLE Express®, you must:

- Set up EXAMPLE Express® security
- Set up event logging

In addition, you can:

- Set up the Exit Interview Field
- Set up an EXAMPLE Express® proxy server
- Secure IIS with a self-signed SSL certificate
- Set up dynamic skin selection
- Set up portfolio
- Set up consumer access
- Set up mass tasks
- Set up Pitney Bowes address verification

- Set up underwriter's workbench toolbar

Setting Up EXAMPLE Express® Security

EXAMPLE Express® security is controlled through authentication (provided through any of five authentication methods available for the EXAMPLE Platform®) and authorization (provided through EXAMPLE UserAdmin™). For instructions on setting up security for EXAMPLE Express®, see the following documents:

- **Authentication** — *EXAMPLE Authentication Server™ User Guide*
- **Authorization** — *EXAMPLE UserAdmin™ User Guide*

Setting Up Event Logging

You can use the Microsoft Windows Application Event Log to log exception conditions and notifications in the EXAMPLE Platform®.

To set up event logging:

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
3. Under **Mode**, click **Form**.
4. On the **Web Config** tab, in the list of settings, click **EventLog**.
5. In the **value** text box, click one of the following options:
   - **Errors** — Logs errors that occur in the EXAMPLE Express® system.
   - **All** — Logs both errors and notifications.
6. Click **Save**.
7. Click **Close**.

Using the *EventLog* configuration setting requires an addition to the machine registry for the configured identity of the EXAMPLE Express® application. Usually, this identity is ASPNET local user, unless a configured identity is used for EXAMPLE Express®.

There may be multiple instances of these processes on the machine, and each instance may execute as a unique user.

1. Add the key name "Express" under HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Eventlog\Application.
2. Grant write access to the EXAMPLE Express® entry (HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Eventlog\Application\Express) for the user associated with the ASPNET_WP or WPW3 process (usually ASPNET).
   This key is optional and must be added manually.

Setting Up the Exit Interview Field

When a user completes an interview process, EXAMPLE Express® executes an Exit Interview field. This field can modify session data or execute additional ManuScript logic when the user leaves the final page of the interview. It can

change session values, store data to the database, or operate conditionally based on the final page of the interview process for a particular user.

To define an Exit Interview field, you must define the field in EXAMPLE Author®, and then specify the field in the *ExitInterviewField* setting in the Express\Web.config file.

If an Exit Interview field does not exist in a ManuScript that contains the final page of an interview, EXAMPLE Express® will continue processing as normal. However, any value specified in the *ExitInterviewField* configuration setting will cause an additional round trip request to EXAMPLE Server® at the end of each interview process.

Defining the Field in EXAMPLE Author®

To define the Exit Interview field in EXAMPLE Author®, create a field that executes the desired logic. In some cases, an Exit Interview field may need to execute conditionally based on which EXAMPLE Express® system page a user has requested when leaving an interview. To accommodate this situation, EXAMPLE Express® sets a value in the session properties that contains the name of the page the user has requested. If desired, a ManuScript can use this value to build conditional logic by using the <Session.getPropertyRq> request as follows:

```
<Session.getPropertyRq name="ExitInterviewTarget" />
```

The value of the *ExitInterviewTarget* property can be the name of any valid content page defined in the ContentMap.xml file (or any custom ContentMap.xml file).

Identifying the Field in Express\Web.config

To define the Exit Interview field in the Express\Web.config file:

1. In each ManuScript, add a field that specifies which page ends the interview process. This page can be in EXAMPLE TransACT®, in the product ManuScript, or in any other ManuScript called during the interview process. Give the field the same name in each ManuScript using the same EXAMPLE Express® configuration.
2. Open the Edit Configuration Utility.
3. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
4. Under **Mode**, click **Form**.
5. On the **Web Config** tab, in the list of settings, click **ExitInterviewField**.
6. In the **value** text box, enter the name of the ManuScript field defined in Step 1 .
7. Click **Save**.
8. Click **Close**.

Setting Up an EXAMPLE Express® Proxy Server

If you wish to add an additional layer of security to EXAMPLE Express®, you can configure an EXAMPLE Express® proxy server. To set up an EXAMPLE Express® proxy server, you must:

- Install EXAMPLE Express®
- Set up a proxy directory
- Set up a virtual directory
- Configure the Express\Web.config file

- Verify configuration

Installing EXAMPLE Express®

For instructions on installing EXAMPLE Express®, see *EXAMPLE Express® Installation*.

Setting Up a Proxy Directory

To set up the proxy directory:

1. In the desired location, create a new directory. Name the directory as desired.
2. In the *Global Root Directory*\Shared\Bin directory, copy the **Proxy.aspx** file, and paste it into the newly created directory.
3. Rename the Proxy.aspx file to `Default.aspx`.
4. Copy the Default.aspx file.
5. Paste the copied file into the same directory, and rename the file `CheckSemaphoreStatus.aspx`.
6. In the Default.aspx file, change the value of the *string url* variable to the appropriate EXAMPLE Express® URL (such as `http://localhost/Express/Default.aspx`).
7. Save and close the Default.aspx file.
8. In the CheckSemaphoreStatus.aspx file, change the value of the *string url* variable to the appropriate EXAMPLE Express® URL, changing the file name at the end of the URL from `Default.aspx` to `CheckSemaphoreStatus.aspx`.
9. Save and close the CheckSemaphoreStatus.aspx file.
10. Navigate to the *Global Root Directory*\Express\Skins directory.
11. Perform one of the following actions:
    - Copy your skins files to the proxy directory, placing them in a sub-directory named *Skins*.

      If your skins use the Next Generation Web Design, you must copy the DCTCore directory as well as your customized skin directory to the proxy directory.

    - Copy the entire Skins directory and paste it into the proxy directory.

Setting Up a Virtual Directory

In IIS, create a virtual directory for the EXAMPLE Express® proxy. Name the directory as desired, and point it to the proxy directory.

Configuring Express\Web.config

To configure the Express\Web.config file:

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
3. Under **Mode**, click **Form**.
4. On the **Web Config** tab, in the list of settings, click **ProxyMode**.
5. In the **value** text box, enter `1`.
6. Click **Save**.

7. Click **Close**.

Verifying Configuration

To verify EXAMPLE Express® proxy configuration:

- In a Web browser, navigate to EXAMPLE Express® using the proxy (for example, `http://localhost/ExpressProxy`).
  The EXAMPLE Express® login page should appear.

Securing IIS with a Self-Signed SSL Certificate

You can secure your IIS configuration using the *SelfSSL.exe* file to generate and install a self-signed SSL certificate. However, because this tool generates a self-signed certificate that does not come from a trusted source, Duck Creek Technologies, Inc. recommends that it only be used in the following situations:

- When creating a security-enhanced private channel from a server to a limited, known user group
- When troubleshooting third-party certificate issues

To generate a self-signed SSL certificate:

1. Download the SelfSSL.exe file from a trusted site such as *www.Microsoft.com*, or acquire it from the IIS resource kit.
2. Because there will likely be more than one Web site on the IIS machine, execute the following command in a command prompt to determine the site ID:
   `cscript iisweb.vbs/query`
3. Run the SelfSSL.exe file to generate a self-signed certificate, and put it in the personal store on the server using the following command:
   `Selfssl.exe/s:3` (where 3 is the site ID)

To secure your IIS configuration with a self-signed SSL certificate:

1. In IIS, right-click the EXAMPLE Express® Web site, and then click **Properties**.
2. On the **Directory Security** tab, click **Edit**.
3. Ensure that the **Require Secure Channel** check box is selected.

Setting Up Dynamic Skin Selection

If desired, you can override the *XSLTDir* configuration setting in the Express\Web.config file and determine which EXAMPLE Express® skin to use at runtime.

When overriding the *XSLTDir* configuration setting, the following skin selection priority rules are used:

- If there is an “xslt” option on the URL request string or if an HTML form field named *xslt* exists, select that skin.

- If there was previously a skin selected by a request string override since the user last logged in, maintain that same skin override.
- Any time the currently selected application layout has an “xsltDir” override specified, use that skin.

  This rule does not apply after switching from a layout with an override specified. If that is the case, proceed with the next rule.

- The skin directory defined in the Express web.config’s “XSLTDir” setting will be used to select the current skin.

Setting Up Portfolio

For information on setting up the Portfolio feature, see *Portfolio in the EXAMPLE Platform®*.

Setting Up Consumer Access

For information on setting up the EXAMPLE Platform® Consumer Access feature, see *Consumer Access in the EXAMPLE Platform®*.

Setting Up Mass Tasks

Mass Tasks functionality can be enabled by performing an upgrade of an existing installation or by installing a new instance of EXAMPLE Server®, EXAMPLE Express®, and Duck Creek Billing™.

When Mass Task functionality is installed, the following are also added or modified:

- Stored procedure: DC_SP_PLT_ProcessAssignWorkflowTasksFromXML
- TaskDetails ManuScript.
- Skins

Additionally, the following are added:

- Mass Reassign page (see *Mass Reassign page*)
- TaskRecords group with these fields: EntityId, TaskId, TaskLockingTS, and UserId
- ModifyTasks is assigned to those users who have access to ViewTasks (see *EXAMPLE UserAdmin™ Privileges*)
- ViewTasksOnly is added

For more information about Mass Tasks, see *Tasks*.

Setting Up Pitney Bowes Address Verification

About the Installation

Pitney Bowes Address Verification requires the following be installed:

- EXAMPLE Server®
- EXAMPLE Express®
- EXAMPLE TransACT®

The installation utility sets up the following for Pitney Bowes Address Verification:

- Copies the DuckCreek.PitneyBowes.dll file to the Shared\Bin directory – Determines how to send and receive messages from Pitney Bowes.
- Updates the IntegrationCatalog.xml file in the Shared directory – Instructs the system to use the PitneyBowes_GeoCodeReport.xml file.
- Links the GeoCodeManuScript.xml file to Shared\ManuScriptCatalog.xml – Links the ManuScript to the system.
- Copies the PitneyBowes_GeoCodeReport.xml file to the Reports\Pitney-Bowes\Geocoding directory – Contains mappings to get messages from the .dll file into the session XML.
- Copies the GeoCodeManuScript.xml file to the Reports\Pitney-Bowes\Geocoding directory
- Provides a sample ManuScript that contains two pages that call PBBI and scrub an address. This installs an inherited ManuScript (Sample Product – PBBI) on top of the Sample Product ManuScript that verifies addresses entered on the Account and Location pages.
- Adds the following XML to theWeb\Web.config file:

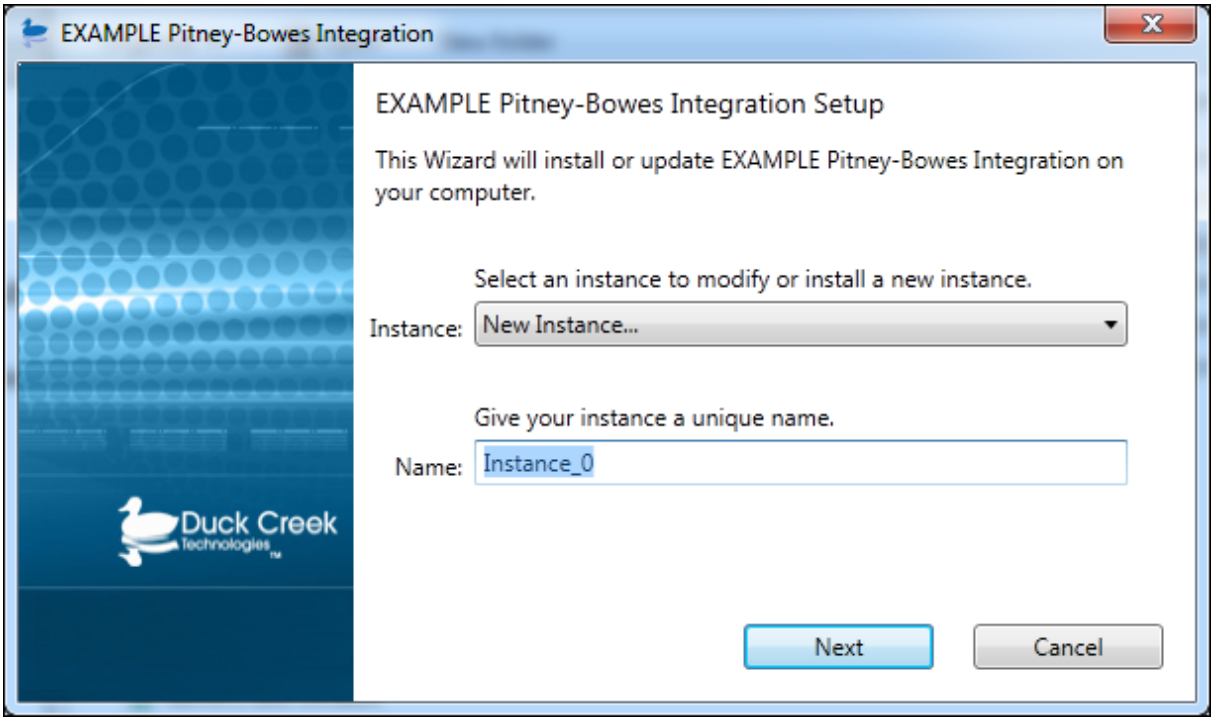
```
<!-- This file shows the XML configuration changes necessary to use the Pitney Bowes
report(s).     The installer should make these changes automatically, but details are
provided for reference.     Changes should be made to the %DCT Root%\web\web.config
file. -->
        <configSections>
            <section name="PitneyBowes_DCT"
type="System.Configuration.NameValueSectionHandler" />
        </configSections>
        <PitneyBowes_DCT>
            <add key="AccountID" value="user123" />
            <add key="AccountPassword" value="pass123" />
        </PitneyBowes_DCT>
        <system.serviceModel>
            <bindings>
                <basicHttpBinding>
                <binding name="GeocodeUSAddressSoapBinding" closeTimeout="00:01:00"
openTimeout="00:01:00" receiveTimeout="00:10:00" sendTimeout="00:01:00"
allowCookies="false" bypassProxyOnLocal="false"
hostNameComparisonMode="StrongWildcard" maxBufferSize="65536"
maxBufferPoolSize="524288" maxReceivedMessageSize="65536" messageEncoding="Text"
textEncoding="utf-8" transferMode="Buffered"
useDefaultWebProxy="true">
                <readerQuotas maxDepth="32" maxStringContentLength="8192"
maxArrayLength="16384" maxBytesPerRead="4096" maxNameTableCharCount="16384"
/>
                <security mode="None">
                <transport clientCredentialType="None" proxyCredentialType="None"
realm="" />
                <message clientCredentialType="UserName" algorithmSuite="Default"
/>
                </security>
                </binding>
                </basicHttpBinding>
            </bindings>
            <client>
                <endpoint address="http://staging.g1.com:8080/services/
GeocodeUSAddress" binding="basicHttpBinding"
```

```
bindingConfiguration="GeocodeUSAddressSoapBinding" contract="GeocodeUSAddress"
name="GeocodeUSAddressPort" />
            </client>
        </system.serviceModel>
    </configuration>
```

Installing Pitney Bowes Address Verification

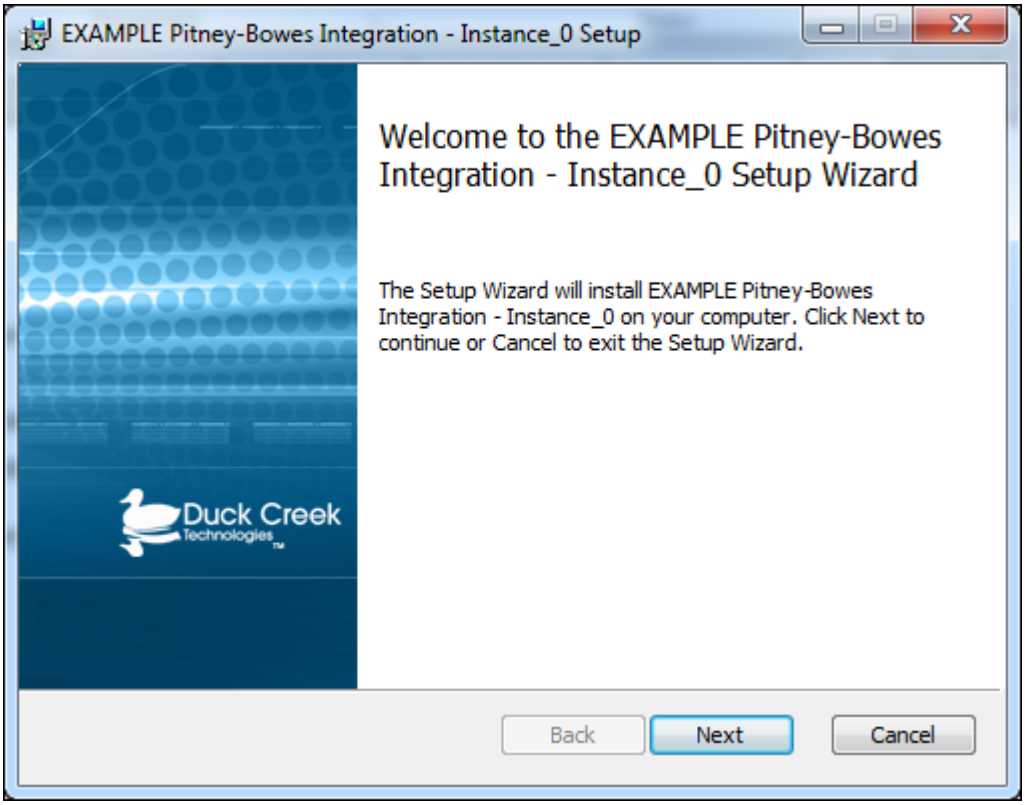
To install Pitney Bowes Address Verification:

1. Make sure EXAMPLE Server® and EXAMPLE Express® are installed before installing Pitney Bowes Address Verification.
2. Download the appropriate installation file from the Duck Creek Technologies, Inc. Customer Support Site.
3. Double-click the installation file.
4. Select an existing installation instance to modify it, or choose to install a new instance.
5. Name the instance of Pitney Bowes Address Verification.



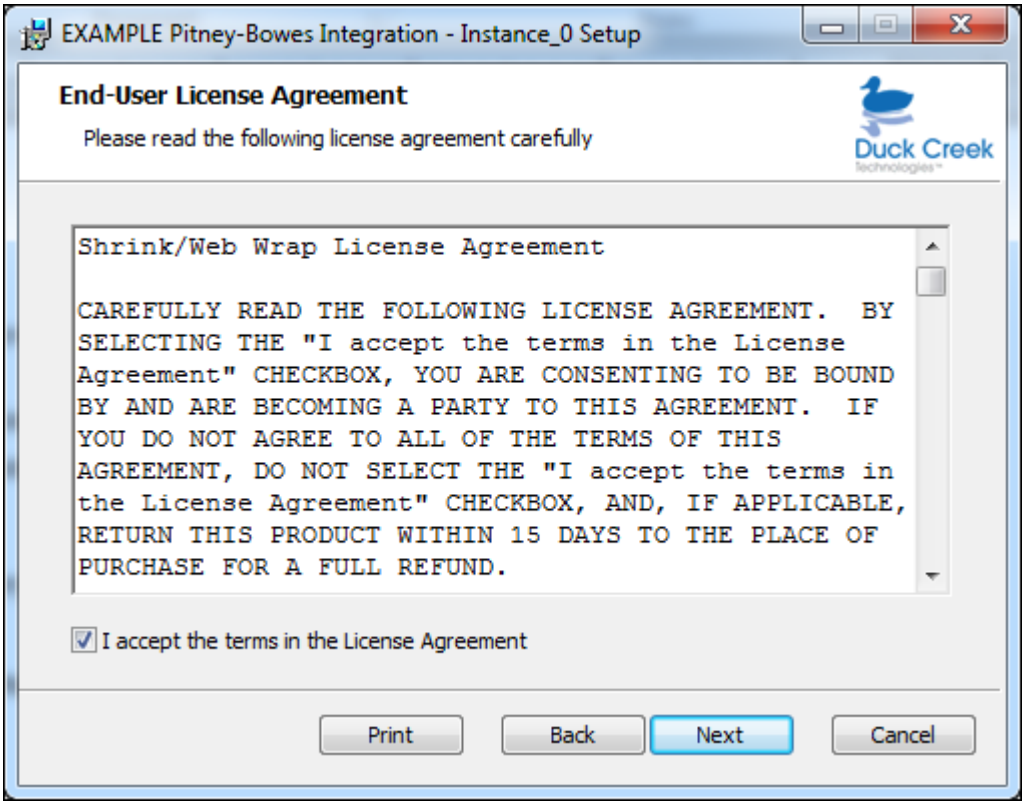
6. Click **Next**.

7. When prompted by the Setup Wizard, click **Next**.

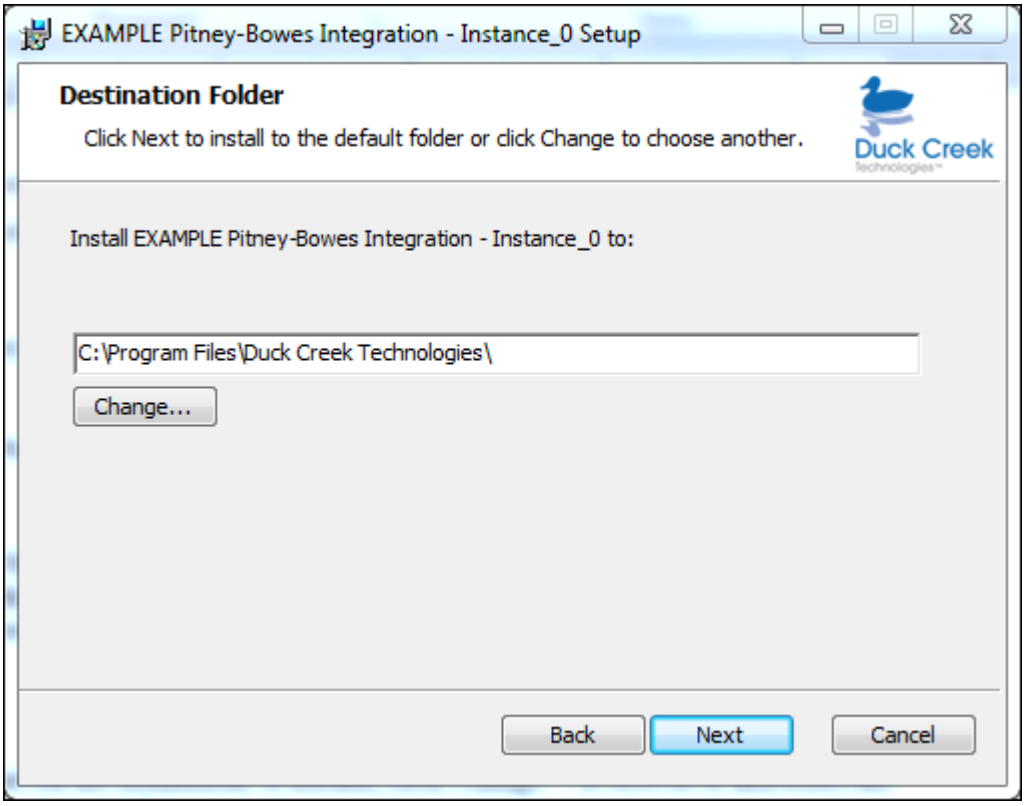


8. Scroll to read the terms of the End-User License Agreement. After reading the agreement, enable the **I accept the terms in the license agreement** checkbox, and then click **Next**. (The Next button remains disabled until the

user selects the checkbox.)

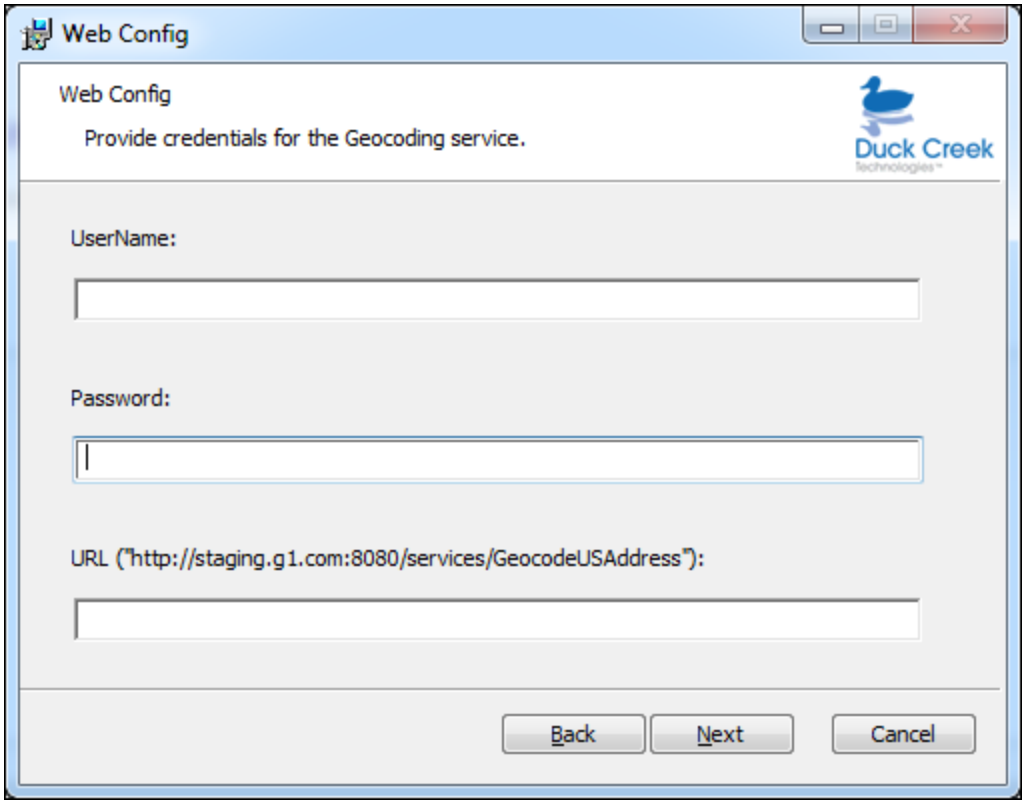


9. Select the destination folder for the installation. If needed, click **Change…** to browse to and select a different destination directory.



10. Click **Next**.
11. Enter the Web config settings, and then click **Next**. These settings allow the connection to the Pitney Bowes service.

Verify that the correct user name, password, and URL are entered, as this data is not verified when it is entered. A system error displays at the time the user attempts to use Pitney Bowes Address Verification if the wrong credentials have been entered here.



12. When prompted, click **Install**.
13. Click **Finish**.

Uninstalling Pitney Bowes Address Verification

To uninstall Pitney Bowes Address Verification:

1. On the Windows taskbar, click **Start**, and then click **Control Panel**.
2. Click **Uninstall a program**.
3. Click the instance of EXAMPLE Pitney-Bowes Integration to uninstall, and then click **Uninstall**.
4. The Programs and Features dialog box appears and asks if users are sure they want to uninstall the program. Click **Yes** to complete the uninstallation process.

Enabling Pitney Bowes Address Verification

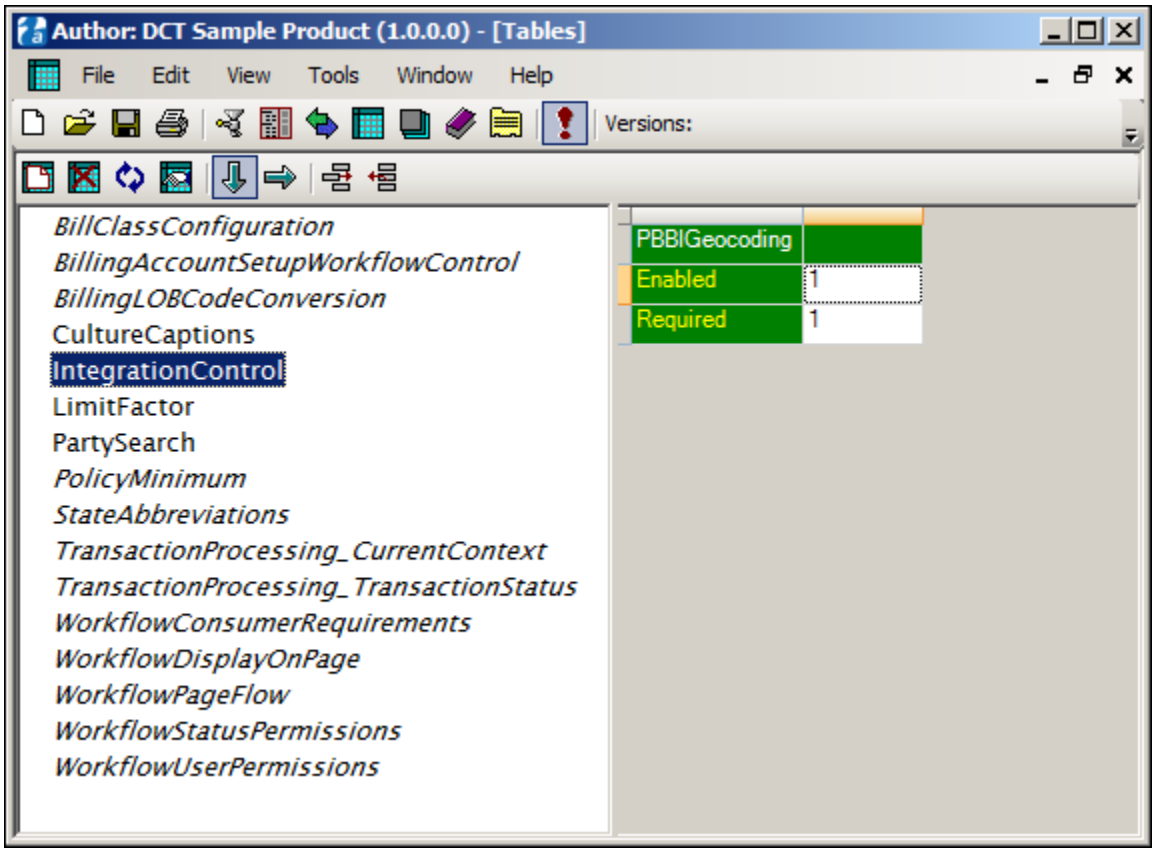
Pitney Bowes Address Verification can be implemented in several ways, depending on the user's customizations. For example, it could be added to a ManuScript as shown here:

1. In Tables View in EXAMPLE Author®, click **IntegrationControl** to access the PBBIGeocoding fields.
2. To enable Pitney Bowes address verification, in the PBBIGeocoding field, set the value of the **Enabled** setting to *1*.
   To disable the functionality, set the value of the **Enabled** setting to *0*.

3. To make Pitney Bowes Address Verification required to complete an application, set the value of **Required** to *1*.

To allow applications that use unverified addresses, set the value of **Required** to *0*.

4. Save the changes.



Address verification cannot be used for addresses stored under a parent group (such as, data). exportRq moves an address from a certain place in the session into a temporary session document for address verification, and exportRq cannot map to a ManuScript group that doesn't have a parent.

For more information about address verification:

- See *Working with Addresses*
- See *Pitney Bowes Address Verification*

Setting Up Underwriter's Workbench Toolbar

Underwriter's Workbench Toolbar lets users view, create, and delete Notes and Attachments.

New Installations and Upgrades

Underwriter's Workbench Toolbar installs as part of an EXAMPLE Express® upgrade or new installation without manual configuration.

Underwriter's Workbench Toolbar is supported only for AvantGarde 2.1 and later skins.

Configuring Underwriter's Workbench Toolbar

Users can enable and disable Underwriter's Workbench Toolbar and configure it to match current skins and localization.

The Underwriter's Workbench Toolbar is built on top of the new MVC Extension Framework. As a new technology, this framework is expected to experience significant growth and change in upcoming releases as the feature set expands and the implementation matures. As such, supported customization options for the Workbench Toolbar are limited to localization based label changes and styling changes using theme based overrides.

It is recommended that items in the Assets directory are not changed, as these will be overridden by future installations and updates.

Enabling and Disabling Underwriter's Workbench Toolbar

By default, Underwriter's Workbench Toolbar is fully enabled. However, the user can disable or configure Underwriter's Workbench Toolbar by editing the skin.config file in the main Skins directory (such as, C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2).

To configure skin.config:

1. Open skin.config in a text editor.
2. Within the <controls> section, locate the section, <workbench toolbar enabled="*1*" />
3. By default, workbench toolbar is enabled and all items are present, including Notes and Attachments. Set enabled="*0*" to turn off workbench toolbar, and then save your changes.

Customizing Underwriter's Workbench Toolbar

The toolbar can be configured to match your skins and localization.

- For style changes, see *Working with CSS Override Styles*.
- For localization changes, see *Using the Localization Framework*.

Out-of-the-box, Underwriter's toolbar does not manipulate data on the main page, default.aspx.

For more information, see:

- *Working With Underwriter's Workbench Toolbar*
- *Underwriter's Workbench Toolbar*

Customizing EXAMPLE Express®

EXAMPLE Express® can be customized extensively to meet each carrier's unique needs. Carriers can customize EXAMPLE Express® by:

- **Customizing skins** — Carriers can customize the content, look and feel, and behavior of EXAMPLE Express® pages through XSL, JavaScript, and CSS changes. For general instructions on customizing these files, see *Customizing EXAMPLE Express Skins*.
- **Using EXAMPLE Express® system ManuScripts** — Carriers can customize the content, appearance, and behavior of certain system pages by replacing XSL pages with customized EXAMPLE ManuScripts™. For information about which pages can be replaced with system ManuScripts and instructions on using them, see *Using EXAMPLE Express® System ManuScripts*.
- **Customizing the appearance of interview (ManuScript) pages** — Carriers can control the appearance of interview (ManuScript) pages by defining the content and layout of each page in EXAMPLE Author®. In addition, carriers can further control the appearance of these pages through custom styles and format masks. For instructions on customizing the appearance of ManuScript pages through custom styles and format masks, see *Customizing Interview (ManuScript) Pages*.
- **Adding custom pages** — Carriers can add custom pages to EXAMPLE Express® through the ContentMap.xml file available with the OnDemand Framework. For instructions on adding custom pages, see *Adding Custom Pages*.
- **Adding or customizing modules** — Carriers can customize the out-of-the-box modules available for the Policy Administration Dashboard (or any other page) or create their own. For instructions on adding custom modules, see *Adding or Customizing Modules*.
- **Adding custom Express Actions** — Carriers can define custom Express Actions that can be used in EXAMPLE Express® pages or interview (ManuScript) pages using the PostActionMap.xml file available with the OnDemand Framework. For instructions on adding custom Express Actions, see *Adding Custom Express Actions*.
- **Customizing the EXAMPLE Express® experience** — Carriers can customize the total EXAMPLE Express® experience by customizing which pages are available in EXAMPLE Express® and what each page contains using the ApplicationLayoutMap.xml file available with the OnDemand Framework. If desired, through the OnDemand Framework and EXAMPLE Express® skins files, carriers can specify a unique EXAMPLE Express® experience for each type of user. For instructions on customizing the total EXAMPLE Express® experience, see *Customizing the EXAMPLE Express® Experience*.
- **Adding external content** — Carriers can load and embed content from external URLs in EXAMPLE Express® pages by modifying any of the following page elements or files to further customize their EXAMPLE Express® experience:
  - Sections in EXAMPLE Author®
  - Left panels in the skin.config file
  - Dashboard modules through the ApplicationLayoutMap.xml file

For more information, see *Using External Content in EXAMPLE Express®*.

- **Customizing specific EXAMPLE Express® features** — Carriers can customize a number of specific features and functions in EXAMPLE Express®, including:
  - *Customizing the Policy Administration Dashboard*
  - *Customizing Tasks*
  - *Customizing Notifications*
  - *Customizing Search Options*

- *Using New Quote Selection System Pages*
- *Customizing the System Loading Message*
- *Enabling Top Navigation*
- *Enabling Help Link Tab Indexes*
- *Setting Up Navigation Classes*

Customizing EXAMPLE Express Skins

The content, behavior, and look and feel of EXAMPLE Express® can be customized through EXAMPLE Express® skins (composed of XSL, JavaScript, and CSS files). EXAMPLE Express® skins isolate the customizable look and feel of EXAMPLE Express® from core EXAMPLE Express® functionality. Static .xsl and. js files that define core EXAMPLE Express® behavior are stored in either the DCTCore or DCTBase (depending on which skin you are using) directories in the Duck Creek root directory (for example, `C:\Program Files\Duck Creek Technologies\`). These files handle rendering of the HTML layout structure and the main JavaScript processing. Files in the Avant Garde, AvantGarde2, AvantGarde 2.1, DCTCore, and DCTBase directories are overwritten during Duck Creek Technologies, Inc. installations and upgrades, and should NOT be altered.

By contrast, customizable .xsl, .js, and .css files that define the look and feel of EXAMPLE Express® are stored in the *`Global Root Directory`*`\Express\Skins\`*`CustomSkin`*`\` directory (where *CustomSkin* is the name of a customized skins directory). These files can be customized. For information about creating a custom skins directory, see *Creating a Custom Skins Directory*.

When customizing EXAMPLE Express® skins, do not modify any files in the DCTCore, DCTBase, AvantGarde (all versions) directories. All files in these directories are overwritten during upgrades. Instead, to override these files, you must create a custom skins directory (copied from the appropriate AvantGarde directory) and modify the files in this directory. For instructions on creating a custom skins directory and overriding DCTCore or DCTBase files, see *Creating a Custom Skins Directory*, *Overriding DCTCore Files*, or *Overriding DCTBase Files*.

All EXAMPLE Express® skins use the DCTCore or DCTBase directories (depending on the skin being used) in conjunction with a customizable directory that defines the unique look and feel of EXAMPLE Express®.

Out of the box, all skins directories and files are located in the *`Global Root Directory`*`\Express\Skins directory\`. EXAMPLE Express® is shipped with the DCTCore and DCTBase directories and a sample directory (AvantGarde, AvantGarde2 or AvantGarde2.1) that defines the out-of-the-box EXAMPLE Express® look and feel. During upgrade installations, the DCTCore or DCTBase and AvantGarde or AvantGarde2 directories are overwritten. Files in a carrier's custom skins directory, however, will NOT be overwritten during an upgrade installation and may be customized extensively.

Follow the general instructions in this section to customize EXAMPLE Express® skins.

Customizing Avant Garde Skins

By *Creating a Custom Skins Directory* and *Overriding DCTCore Files*, you can modify the XSL, CSS, and JavaScript files to customize the Avant Garde skin to best suit your needs.

Creating a Custom Skins Directory

When customizing EXAMPLE Express® skins, do not modify any files in the DCTCore or AvantGarde directories. All files in these directories are overwritten during upgrades. Instead, you must create a custom skins directory (copied from the AvantGarde directory) and modify the files in this directory.

To create a custom skins directory:

1. In the *Global Root Directory*\Express\Skins directory, copy the **AvantGarde** directory and paste it in the desired location within the Express directory.
2. Rename the directory.
3. Make any desired customizations in the new directory. Use the information in this section to help you customize the system.

Once you have created a custom skins directory, you must configure EXAMPLE Express® to run using your customized skin.

To configure EXAMPLE Express® to use your customized skin:

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
3. Under **Mode**, click **Form**.
4. On the **Web Config** tab, in the list of settings, click **XSLTDir**.
5. In the **value** text box, enter the relative path to the custom directory from the *Global Root Directory*\Express directory (for example, `Skins\MySkins`).
6. Click **Save**.
7. Click **Close**.

Overriding DCTCore Files

Customizing skins sometimes requires that you override files in the DCTCore directory. To do this, you must create a custom skins directory, copy the desired templates or functions from the DCTCore files to that directory, and then make your customizations. Files in the custom skins directory are called after files in the DCTCore directory and therefore take precedence. For information about creating a custom skins directory, see *Creating a Custom Skins Directory*.

When customizing EXAMPLE Express® skins, do not modify any files in the DCTCore or AvantGarde directories. All files in these directories are overwritten during upgrades. Instead, you must create a custom skins directory (copied from the AvantGarde directory) and modify the files within it.

Overriding XSL Files

To override DCTCore .xsl files:

1. Create a custom skins directory. For information about creating a custom skins directory, see *Creating a Custom Skins Directory*.
2. Copy the templates you wish to change from DCTCore to the appropriate files in the custom skins directory.
3. Make customizations to the templates in the custom skins directory.

The following are considered best practices with regard to overriding .xsl files:

- When copying templates from DCTCore files to the custom skins directory, copy entire templates, not parts of them.
    - To help distinguish between custom and base skin code, create a comment each time you change the skins.
        - For more information about overriding DCTBase files, see *Overriding DCTBase Files* in the *EXAMPLE Express® Reference Guide*.
        - For more information about overriding DCTCore files, see *Overriding DCTCore Files* in the *EXAMPLE Express® Reference Guide*.
- Each overridden template should be placed in the correct .xsl file in the custom skins directory. For example:
    - *Home* screen changes should be copied to the home.xsl file.
    - *Open* screen changes should be copied to open.xsl.
    - Page Version 1 interview screen changes should be copied to the interview.xsl file; Page Version 2 interview screen changes should be copied to the interviewNew.xsl file.

To make it easier for the necessary files to call the correct overrides, you can also maintain all overrides in a single template file called custom.xsl. Create a custom.xsl file by copying the dctCommonControls.xsl file into a new file and removing all templates and comments so that only the opening and closing tags for <xsl:stylesheet...> remain.

- Often, Page Version 1 and Page Version 2 pages with the same name exist. Ensure that overridden templates specific to Page Version 2 are not used in Version 1 pages. Failure to check for this will result in broken pages. To prevent this from occurring, ensure that the custom.xsl file is called by the following files:
    - dctCommonControls.xsl—Because no import statements exist in this file, the import statement for the custom.xsl file should be added at the top of the file, immediately after the <xsl:stylesheet> opening tag. The dctCommonControls.xsl file is called by most other .xsl files. Any files that call the dctCommonControls.xsl file do not need to call the custom.xsl file.
    - documentation.xsl (for PV2 annotations)—The import statement for the custom.xsl file should be added after all pre-existing import statements.
    - interview.xsl (for PV1)—This file should only call the custom.xsl file if dctCommonControls.xsl is not imported by this file.
    - interviewNew.xsl (for PV2)—This file should only call the custom.xsl file if dctCommonControls.xsl is not imported by this file.

As of the EXAMPLE Platform 4.0.0 release, the interview.xsl and interviewNew.xsl files now import the dctCommonControls.xsl file.

- Re-order any existing import statements at the beginning of the following files so that the dctShared.xsl file is the last import statement:
    - interview.xsl
    - editPrintJob.xsl

If the following are all true, you must place your overrides in the interviewNew.xsl file:

- You must override a template that uses **match** instead of a name for PV2 pages. For example, <xsl:template match="action"> instead of <xsl:template name="questionInput">.
- The element you are matching may be present in the debugContent.xml for Version 1 or Version 2 pages. For example, "action" occurs in both page versions, but "section" does not.
- A corresponding match template for Version 1 pages (usually found in the dctShared.xsl or interview.xsl files) is not present.

Overriding JavaScript Files

To override DCTCore .js files:

1. In the *Global Root Directory*\Express\Skins directory, open the **custom.js** file.
2. Copy all functions you wish to override to the custom.js file.

Follow the best practices below when overriding DCTCore .js files:

- Some functions from DCTCore .js files and DCTBase.js files may be overridden. When overriding .js files in the DCTCore/DCTBase directory (depending upon your version of EXAMPLE Platform®), Duck Creek Technologies, Inc. recommends copying and overriding an entire function, rather than only a part of it. For additional information, see *Overriding JavaScript Files* in the *EXAMPLE Express® Reference Guide*.
- To help distinguish between custom and base skin code, create a comment each time you change the skins.
    - For more information about overriding DCTBase files, see *Overriding DCTBase Files* in the *EXAMPLE Express® Reference Guide*.
    - For more information about overriding DCTCore files, see *Overriding DCTCore Files* in the *EXAMPLE Express® Reference Guide*.

Customizing XSL

When customizing XSL files, follow the guidelines below.

Interview Pages

When customizing interview (ManuScript-driven) pages:

- Customize Version 2 interview pages by modifying the *InterviewNew.xsl* file. For additional information, see *Customizing XSL* in *EXAMPLE Express® Reference Guide*.
- If working with Version 1 pages (not recommended), modify the *interview.xsl* file.

System Pages

When customizing or creating system pages, follow the sequence of template calls below for building a page.

1. Include a template that matches the root of the content for the page.
2. Within this template, start the process of building the container by calling the *buildContainer* template in the dctContainer.xsl file (located in the *Global Root Directory*\Express\Skins\DCTCore\xsl directory).
3. Set up the following templates. (These templates will be called by the process begun in Step 2.)
    - **processPageHeader** — Controls the header content for the page.
    - **buildLeftPanelContent** — Controls the content of the left panel area of the container.
    - **buildPageContent** — Controls the main page content.
4. (Optional) In the *buildLeftPanelConten*t template, call any of the following component templates:

- **buildPolicyLeftNavHeader** — Controls the header of the left panel.
- **createSubMenuArea** — Renders the left navigation (such as the sub navigation for certain pages).
- **buildPolicyRelatedTasks** — Controls the Policy Actions in the left panel.
- **buildPolicyLogonItems** — Controls the Login Options in the left panel.

Customizing CSS

You can customize CSS for EXAMPLE Express® by:

- Customizing a set of global .css files that apply across the entire EXAMPLE Express® application.
- Customizing individual override files (indicated by the filename *x_pagename.css*) that apply to single pages or portions of the EXAMPLE Express® application.
- Applying custom CSS styles to specific page elements in a ManuScript. These styles are defined in a single override file (*x_interview.css*), and are applied to individual fields and page elements in EXAMPLE Author®. For more information about creating and applying custom CSS styles in EXAMPLE Author®, see *Working with CSS Override Styles* in the *Guide to EXAMPLE Author®*.

The x_interview.css file is automatically modified by EXAMPLE Author® any time styles are edited using the CSS Manager in Page Designer. This file does not need to be edited by hand.

The table below describes the customizable .css files in the *Global Root Directory*\Express\Skins*CustomSkin*\css directory. Use this table to determine which files to modify when customizing CSS.

File	Description
buttons.css	Contains styles for all links and buttons used in EXAMPLE Express®.
ext-all.css	Contains styles controlling the layout of the EXT JavaScript framework used throughout EXAMPLE Express®.
forms.css	Contains styles that define how EXAMPLE Express® forms (interview pages) are displayed in the browser. Two standard sets of form layouts exist in the base installation of EXAMPLE Express®: vertical and horizontal (across and down).
generic.css	Obsolete as of EXAMPLE Platform® 4.0.
leftNav.css	Contains styles controlling left navigation for ManuScript pages. This file is the default file used to control ManuScript page navigation.
leftNavFloating.css	Contains styles controlling the floating navigation panel. This file is only used if floating navigation is enabled.
legacySkins.css	Contains styles released in versions of EXAMPLE Express® prior to 3.1. These styles are only loaded for ManuScripts containing version 1 pages.
nav.css	Contains styles controlling application-level navigation.
popUp.css	Contains styles controlling the layout of pop-up window contents.

reset.css	Contains style definitions that override those defined by the browser vendor for common HTML elements. By resetting these definitions to a common baseline, fewer cross-browser incompatibility issues occur. This file should not be modified.
style.css	Contains styles for visual elements that appear on pages (banners, footers, tabs, font sizes).
topNav.css	Contains styles controlling top navigation for ManuScript pages. This file is used only if specified for a PageSet in EXAMPLE Author®.
x_*pagename*.css	Contains page-specific styles that override the global styles assigned in the above files. To allow for extensive layout customization, each system page can have a corresponding x_*pagename*.css file (for example, x_login.css.) If an x_*pagename*.css file exists for a given page, the file will be loaded when that page is loaded. Out of the box, EXAMPLE Express® skins include an x_*pagename*.css file for commonly customized pages. If desired, you can add additional x_*pagename*.css files for any other pages you wish to customize.
x_interview.css	Contains styles to be applied to individual fields or page elements in EXAMPLE ManuScripts™. These styles are assigned in EXAMPLE Author®.

HTML Div Structure

By default, EXAMPLE Express® pages include the following container divs:

- **wrapper** — Controls the overall page width
- **banner** — Contains branding and top navigation
  - **logo** — Contains the logo
  - **productNavigationDiv** — Contains product-level navigation
  - **quickSearch** — Contains the quick search
  - **actionGroup** —Contains application-level navigation
  - **specialactions** — Contains the New Quote feature
- **formContent** — Controls page content between the banner and footer
  - **leftPanelArea** — Contains content of the left panel
    - **policyActiveSection** — Contains information on the active policy, if an active policy exists
    - **subActionGroup** — Contains the sub navigation for pages (if in interview, contains navigation among ManuScript pages)
    - **relatedActions** — Contains policy-related actions if a policy is active
    - **logonActions** — Contains login options (**Change Password**)
  - **main** — Contains content of the page area between banner and footer to the right of the left panel
    - **header** — Contains the page header
    - **pageTop** — Contains page title (pageTitle) and instructional text (pageInst)

- **footer** — Contains the page footer

Customizing JavaScript

When customizing JavaScript for EXAMPLE Express®, it is important to understand the JavaScript frameworks and processing functions used in EXAMPLE Express® skins.

JavaScript Frameworks in EXAMPLE Express®

The following open source JavaScript frameworks are used in EXAMPLE Express® skins:

- **Ext JS** (*http://extjs.com/*) — This framework is used for the following controls:
  - Grid controls
  - Calendar control
  - Accordion control
  - Progress bar control
  - Tool tips
  - Alert messages, confirmation messages, Yes/No save messages, and modal pop-ups
- **Scriptaculous** (*http://script.aculo.us/*) — This framework is used for section toggling.
- **Prototype.js** (*http://www.prototypejs.org/*) — This framework is used for AJAX processing and HTML snippet manipulation (dynamically inserting or removing HTML).

Customizing ManuScript Processing

To customize system behavior during ManuScript processing, modify the following variables and functions in the *custom.js* file in the *Global Root Directory*\Express\Skins*CustomSkin*\scripts directory.

Customize the system behavior that occurs during ManuScript processing using the *custom.js* file in the *Global Root Directory*\Express\Skins*Custom Skin*\scripts directory.

Item	Description
Variables	
_loadingWaitTime	Time to elapse (in milliseconds) before displaying a "loading" message when performing system actions.
Functions	
customOnload()	Executed when pages are displayed or when any AJAX process causes the container (the area containing the header, footer, and page tabs) to be redrawn.
skipRequiredChecking()	Executed before the doCheck event is called. This function must return a value of *true* to skip required checking.
customOnSubmit()	Executed before the page or AJAX process is called.
customFieldFormat()	Executed before the onBlur and prevent functions. This function allows for field format changes before submitting a page.

customOnBlur(field)	Executed when an onBlur event is executed.
customOnKeyPress(field)	Executed when an onKeyPress event is executed (for multi-line text boxes).
customOnClick(field)	Executed when an onClick event is executed.
customOnChange(field)	Executed when an onChange event is executed.
customOnKeyUp(field)	Executed when an onKeyUp event is executed (for multi-line text boxes).

Customizing Out-of-the-Box Pages

To customize the look and feel of out-of-the-box system pages, modify the appropriate .xsl and .css files in the *Global Root Directory*\Express\Skins*CustomSkin*\xsl and *Global Root Directory*\Express\Skins*CustomSkin*\css directories. Use the table below to determine which files to modify.

Page	Files to Modify	Notes
Login		
Login	login.xsl, x_login.css	N/A
Agency Signup	signup.xsl, x_signup.css	N/A
Policy Administration Dashboard		
Policy Administration Dashboard	dashboardHome.xsl, x_dashboardHome.css	N/A
Clients		
Client Information	clientInfo.xsl, x_clientInfo.css	N/A
Client Details	• **ManuScript** — Client Detail ManuScript • **System pages** — clientDetail.xsl, x_clientDetail.css	This page can be driven by the Client Detail ManuScript. To customize this page, modify this ManuScript.
Policies		
Interview	interviewNew.xsl, interview.xsl, x_interview.css	The interviewNew.xsl file is used for version 2 pages, and the interview.xsl file is used for version 1 pages. Both version 1 and version 2 pages use x_interview.css.

New Quote	• **ManuScript** — New Quote Selection ManuScript • **System pages** — new.xsl, x_new.css newBasic.xsl, x_newBasic.css, newExtended.xsl, x_newExtended.css	Out of the box, this page is driven by the New Quote Selection ManuScript. To customize this page, modify this ManuScript. If using the system screens instead, the out-of-the-box newBasic.xsl and newExtended.xsl files call new.xsl. If desired, these pages can be customized to contain unique content.
Policy Details	policy.xsl, x_policy.css	N/A
Policy Documents	print.xsl, x_print.css	N/A
Policy Forms	printjob.xsl, x_printjob.css	N/A
Edit Form	editPrintJobNew.xsl, editPrintJob.xsl, x_editPrintJob.css	The editPrintJobNew.xsl file is used for version 2 pages, and the editPrintJob.xsl file is used for version 1 pages. This page is ManuScript-driven and is accessed from the **Policy Forms** page.
Notifications		
Notifications	messages.xsl, x_messages.css	N/A
Send a Notification	• **ManuScript** — Message Detail ManuScript, MessageDetail.xsl • **System pages** — messageNew.xsl, x_messageNew.css	This page can be driven by the Message Detail ManuScript. When driven by the Message Detail ManuScript, the system uses MessageDetail.xsl.
Notification Details	• **ManuScript** — Message Detail ManuScript, MessageDetail.xsl • **System pages** — message.xsl, x_message.css	This page can be driven by the Message Detail ManuScript. When driven by the Message Detail ManuScript, the system uses MessageDetail.xsl.
Tasks		
My Tasks	taskInfo.xsl	N/A

Task Management	taskManagement.xsl	N/A
New Task	Task Detail ManuScript, taskDetail.xsl	The system uses both the Task Detail ManuScript and taskDetail.xsl.
Task Details	Task Detail ManuScript, taskDetail.xsl	The system uses both the Task Detail ManuScript and taskDetail.xsl.
Other Screens		
Search	search.xsl	N/A
Batch Reports	batchProcess.xsl, x_batchProcess.css	N/A
Error screen	errors.xsl, x_errors.css	N/A
Help (annotation pop-ups)	documentation.xsl	N/A
Account summary screen for consumers	Consumer Dashboard ManuScript, consumerDashboard.xsl	This page is used for Consumer Access and is driven by the Consumer Dashboard ManuScript. The system also uses consumerDashboard.xsl. For more information about the Consumer Access feature, see *Consumer Access in the EXAMPLE Platform®*.

Customizing Shared Components

To customize components shared by multiple system pages, modify the following .xsl files (in the *Global Root Directory*\Express\Skins*CustomSkin*\xsl directory).

dctCommonControls.xsl

The dctCommonControls.xsl file contains the following templates.

Template	Description
buildActiveSessionSection	Controls the active session information that displays in page headers.
buildPageHeaderSection	This deprecated template is included for backwards compatibility.
processPageHeader	Controls page header content.
buildCurrentFiltersTable	This deprecated template is included for backwards compatibility.
buildOperationsDropdown	This deprecated template is included for backwards compatibility.
bulletinListWidget	This deprecated template is included for backwards compatibility.

selectAgencyControl	Controls the agency drop-down lists on the new.xsl and search.xsl pages.
buildPortfolioList	Controls the portfolio list on the clientInfo.xsl page. This template calls the portfolioPolicyList template to build the table rows.
policyList (matched template)	Controls the header row of the HTML table for the quote/policy list on the clientInfo.xsl page. This template calls the policyList template to build the quote/policy list HTML table.
portfolio	Controls the header row of the HTML table for portfolio data.
buildPolicyGridHeader	Controls the header row of the HTML table for the policy list on the search.xsl and clientInfo.xsl pages.
buildPolicyActions	Controls the action icons that appear in the policy list on the clientInfo.xsl page.
portfolioPolicyList	This deprecated template is included for backwards compatibility.
policyList	Controls the quote/policy list on the clientInfo.xsl page.
buildListPagingSection	Controls the paging links on the print.xsl page.
messagesListWidget	Controls the Notifications lists on the messages.xsl and policy.xsl pages.
buildMessagesListQueryBuilderDialog	This deprecated template is included for backwards compatibility.
clientDetailWidget	Controls the Client Detail table on the clientInfo.xsl page. This template calls the buildClientDetailTable template.
buildClientDetailTable	Controls the Client Detail table on the clientInfo.xsl page.
buildClientDetailAdditionalActionsSection	Controls the action buttons available in the Client Details section of the clientInfo.xml page.
buildCommonLayout	Controls the rendering of ManuScript-driven system screens (clientDetail.xsl, editPrintJob.xsl/editPrintJobNew.xsl, and messageDetail.xsl).

Other XSL Files

The table below describes all other .xsl files that can be modified.

File	Description
dctDocMode.xsl	Controls the documentation pop-ups (annotations) for interview pages rendered with the *interview.xsl* file.
dctGenericProcessFormat.xsl	Controls display formatting for field values rendered during AJAX processing. This template calls the dctGenericSetFieldFormat.xsl file.

dctGenericSetFieldFormat.xsl	Controls display formatting for field values rendered by a full screen change or refresh.
dctGenericSetFieldMask.xsl	Displays ID string for HTML form fields. Facilitates custom format mask validation by allowing the user to modify the *fieldItems* attribute of the appropriate HTML form field. Custom format mask validation will also require the appropriate custom JavaScript be added to the custom.js file in the *Global Root Directory*\Express\Skins*CustomSkin*\scripts directory.
dctGenericSetMaxLength.xsl	Controls the max length for input fields (for example, for custom format masks).
dctShared.xsl	Controls HTML element generation for version 1 interview pages rendered with the *interview.xsl* file.
htmlFooter.xsl	Controls the EXAMPLE Express® footer.
htmlMeta.xsl	Controls meta tags for EXAMPLE Express®.
htmlTitle.xsl	Controls the title that appears in the browser title bar for EXAMPLE Express®.

Customizing Avant Garde 2 Skins

By *Creating a Custom Skins Directory* and *Overriding DCTBase Files*, you can modify the XSL, CSS, and JavaScript files to customize the Avant Garde 2 skin to best suit your needs.

Creating a Custom Skins Directory

When customizing EXAMPLE Express® skins, do not modify any files in the DCTBase or AvantGarde (all versions) directories. All files in these directories are overwritten during upgrades. Instead, you must create a custom skins directory (copied from your version of the AvantGarde directory) and modify the files in this directory.

To create a custom skins directory:

1. In the *Global Root Directory*\Express\Skins directory, copy the correct version of the **AvantGarde** directory and paste it in the desired location within the Express directory.
2. Rename the directory.
3. Make any desired customizations in the new directory. Use the information in this section to help you customize the system.

Once you have created a custom skins directory, you must configure EXAMPLE Express® to run using your customized skin.

To configure EXAMPLE Express® to use your customized skin:

1. Open the Edit Configuration Utility.

2. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
3. Under **Mode**, click **Form**.
4. On the **Web Config** tab, in the list of settings, click **XSLTDir**.
5. In the **value** text box, enter the relative path to the custom directory from the *Global Root Directory*\Express directory (for example, `Skins\MySkins`).
6. Click **Save**.
7. Click **Close**.

Overriding DCTBase Files

Customizing skins sometimes requires that you override files in the DCTBase directory. To do this, you must create a custom skins directory, copy the desired templates or functions from the DCTBase files to that directory, and then make your customizations. Files in the custom skins directory are called after files in the DCTBase directory and therefore take precedence. For information about creating a custom skins directory, see *Creating a Custom Skins Directory*.

When customizing EXAMPLE Express® skins, do not modify any files in the DCTBase or AvantGarde2 directories. All files in these directories are overwritten during upgrades. Instead, you must create a custom skins directory (copied from the AvantGarde2 directory) and modify the files within it.

Overriding XSL Files

To override DCTBase .xsl files:

1. In the Content or Common folders in the `C:Program Files\Duck Creek Technologies\Skins\DCTBase\Core\xsl\` directory, locate the file containing the template you wish to override.
2. Copy the file containing the template you wish to override to the xsl\common or xsl\content folders in the custom skins directory, and ensure that its name remains the same as it is in the DCTBase directory.
3. In the custom skins file (the one you copied in Step 2), remove all templates you do not wish to override as well as all non-template elements (other than the `<xsl:stylesheet>`) tags, such as `<xsl:import>`, `<xsl:output>`, `<xsl:variable>`, etc.
4. Customize the templates you wish to override.

Overriding JavaScript Files

To override DCTBase .js files:

1. Locate the file containing the JavaScript function you wish to override in one of the following directories:
   - DCTBase\Core\devscripts\Billing
   - DCTBase\Core\devscripts\Policy
   - DCTBase\Core\devscripts\common
2. Copy the file containing the function you wish to override to the scripts folder in the custom skins directory and give it the same name as in the DCTBase directory.
3. In the custom skin file, remove all functions you do not wish to override.
4. Customize the function you wish to override.

Customizing XSL

When customizing XSL files, follow the guidelines below.

Interview Pages

When customizing interview (ManuScript-driven) pages:

- Modify the **interview.xsl** file and follow the guidelines found in *Overriding DCTBase Files*.

System Pages

When customizing or creating system pages, follow the sequence of template calls below for building a page.

1. Include a template that matches the root of the content for the page.
2. Within this template, start the process of building the container by calling the *buildContainer* template in the dctContainer.xsl file (located in the `Global Root Directory\Express\Skins\DCTBase\xsl\` directory).
3. Set up the following templates. (These templates will be called by the process begun in Step 2.)
   - **processPageHeader** – Controls the header content for the page.
   - **buildLeftPanelContent** – Controls the content of the left panel area of the container.
   - **buildPageContent** – Controls the main page content.
4. (Optional) In the *buildLeftPanelConten*t template, call any of the following component templates:
   - **buildPolicyLeftNavHeader** – Controls the header of the left panel.
   - **createSubMenuArea** – Renders the left navigation (such as the sub navigation for certain pages).
   - **buildPolicyRelatedTasks** – Controls the Policy Actions in the left panel.
   - **buildPolicyLogonItems** – Controls the Login Options in the left panel.

Customizing CSS

You can customize CSS for EXAMPLE Express® by:

- Customizing a set of global .css files that apply across the entire EXAMPLE Express® application.
- Customizing individual override files (indicated by the filename *x_pagename.css*) that apply to single pages or portions of the EXAMPLE Express® application.
- Applying custom CSS styles to specific page elements in a ManuScript. These styles are defined in a single override file (*x_interview.css*), and are applied to individual fields and page elements in EXAMPLE Author®. For more information about creating and applying custom CSS styles in EXAMPLE Author®, see *Working with CSS Override Styles* in the *Guide to EXAMPLE Author®*.

The x_interview.css file is automatically modified by EXAMPLE Author® any time styles are edited using the CSS Manager in Page Designer. This file does not need to be edited by hand.

The table below describes the customizable .css files in the *Global Root Directory*\Express\Skins*CustomSkin*\css directory. Use this table to determine which files to modify when customizing CSS.

File	Description
buttonDefaults.css	Contains default styles for buttons. These can be overwritten by defining specific button styles in buttons.css.

buttons.css	Contains styles for all links and buttons used in EXAMPLE Express®.
customContainers.css	Contains style definitions for 7 built-in custom containers.
dctExtJsExtensions.css	Contains extensions to the Ext JS components, such as the slider bar widget.
dctExtJsOverrides.css	Contains styles to override those used by Ext JS.
dctExt-xtheme-blue.css	Contains style definitions created to override styles in the blue Ext JS theme.
dctExt-xtheme-gray.css	Contains style definitions created to override styles in the gray Ext JS theme.
ext-all-notheme.css	Contains default styles for Ext JS components.
ext-xtheme-blue.css	Contains style definitions for Ext JS components with a blue color theme.
ext-xtheme-gray.css	Contains style definitions for Ext JS components with a gray color theme.
forms.css	Contains styles that define how EXAMPLE Express®(interview pages)are displayed in the browser. Two standard sets of form layouts exist in the base installation of EXAMPLE Express®: vertical and horizontal (across and down).
hyperlinks.css	Contains default styles for hyperlinks.
leftNav.css	Contains styles controlling left navigation for ManuScript pages. This file is the default file used to control ManuScript page navigation.
leftNavFloating.css	Contains styles controlling the floating navigation panel. This file is only used if floating navigation is enabled.
main.css	Contains styles for many basic HTML elements, such as the body, tables, inputs, etc.
nav.css	Contains styles controlling application-level navigation.
popUp.css	Contains styles controlling the layout of pop-up window contents.
reset.css	Contains style definitions that override those defined by the browser vendor for common HTML elements. By resetting these definitions to a common baseline, fewer cross-browser incompatibility issues occur.  This file should not be modified.
style.css	Contains styles for visual elements that appear on pages (banners, footers, tabs, font sizes).
topNav.css	Contains styles controlling top navigation for ManuScript pages. This file is used only if specified for a PageSet in EXAMPLE Author®.

x_[pageName].css	Contains styles for the page using the [pageName].xsl file. To determine which XSL file a page is using, click the "Current Information" button on the developer toolbar and look for the "Active XSL" page name. This information can also be found by searching for the JavaScript variable "sCurrentPage".
x_interview.css	Contains styles applied to a ManuScript page through EXAMPLE Author®. This file is intended to be modified by ManuScript developers.
x_interview_main.css	Contains styles applied to a ManuScript page through the XSL. This file is intended to be modified by skins developers.

HTML Div Structure

By default, EXAMPLE Express® pages include the following container divs:

- **wrapper** – Controls the overall page width
- **banner** – Contains branding and top navigation
  - **logo** – Contains the logo
  - **productNavigationDiv** – Contains product-level navigation
  - **quickSearch** – Contains the quick search
  - **actionGroup** – Contains application-level navigation
  - **specialactions** – Contains the New Quote feature
- **formContent** – Controls page content between the banner and footer
  - **leftPanelArea** – Contains content of the left panel
    - **policyActiveSection** – Contains information on the active policy, if an active policy exists
    - **subActionGroup** – Contains the sub navigation for pages (if in interview, contains navigation among ManuScript pages)
    - **relatedActions** – Contains policy-related actions if a policy is active
    - **logonActions** – Contains login options (**Change Password**)
  - **main** – Contains content of the page area between banner and footer to the right of the left panel
    - **header** – Contains the page header
    - **pageTop** – Contains page title (pageTitle) and instructional text (pageInst)
- **footer** — Contains the page footer

Customizing JavaScript

When customizing JavaScript for EXAMPLE Express®, it is important to understand the JavaScript framework and processing functions used in EXAMPLE Express® skins.

JavaScript Frameworks in EXAMPLE Express®

The EXAMPLE Platform® uses the ExtJs framework with EXAMPLE Express® skins. This framework includes, but is not limited to, the following controls:

- Grid controls
- Calendar control

- Accordion control
- Collapsible panels
- Popups
- Sliders
- Progress bar control
- Quick tips
- Charts
- Alert messages, confirmation messages, Yes/No save messages, and modal pop-ups

Customizing ManuScript Processing

To customize system behavior during ManuScript processing, modify the following variables and functions in the *custom.js* file in the *Global Root Directory*\Express\Skins*CustomSkin*\scripts directory.

Item	Description
Variables	
_loadingWaitTime	Time to elapse (in milliseconds) before displaying a "loading" message when performing system actions.
Functions	
customOnload()	Executed when pages are displayed or when any AJAX process causes the container (the area containing the header, footer, and page tabs) to be redrawn.
skipRequiredChecking()	Executed before the doCheck event is called. This function must return a value of *true* to skip required checking.
customOnSubmit()	Executed before the page or AJAX process is called.
customFieldFormat()	Executed before the onBlur and prevent functions. This function allows for field format changes before submitting a page.
customOnBlur(field)	Executed when an onBlur event is executed.
customOnKeyPress(field)	Executed when an onKeyPress event is executed (for multi-line text boxes).
customOnClick(field)	Executed when an onClick event is executed.
customOnChange(field)	Executed when an onChange event is executed.
customOnKeyUp(field)	Executed when an onKeyUp event is executed (for multi-line text boxes).

Interview Page Styles

Avant Garde 2 skins control interview page styling using the following two CSS files:

- **x_interview** – Contains styles applied to a page through EXAMPLE Author®.
- **x_interview_main** – Contains styles applied to interview pages through XSL.

Using the Localization Framework

Using the localization framework in the EXAMPLE Platform®, you can configure the EXAMPLE Platform® for use with multiple languages. Depending on your installation of the EXAMPLE Platform®, you may have to localize any of the following by using lookup tables:

- *XSLT*
- *JavaScript*
- *Images*

In these sources, lookups should be performed instead of hard-coding the strings so that the system can react to the current language.

XSLT

To assist in looking up a string, you can use the dctLocalizedDirectory.xsl template found in one of the following directories (depending on the skin you are using):

- `Skins\DCTCore\xsl\` (AvantGarde)
- `Skins\DCTBase\Core\xsl\common\` (AvantGarde2)

For example, to use one of the lookup templates:

- In the dashboardHome.xsl file in the `Skins\DCTBase\Core\xsl\content\` directory, locate the XSLT code that resembles the following code sample, and adjust accordingly:

```
<div id="pageInstruction">
      <xsl:text> Welcome to the Policy Administration Dashboard, </xsl:text>
      <xsl:value-of select="/page/state/user/fullName"/>
      <xsl:text>. </xsl:text>
</div>
```

All base .xsl files in the EXAMPLE Platform® (including the dashboardHome.xsl file) have been localized. Files mentioned on this page are referred to as examples only.

Localizing Custom XSL

If you wish to use a lookup template with your customized XSL, you must convert the code from a hard-coded string to a localized lookup.

To convert a hard-coded XSL string to a localized lookup (using the example from above):

1. Register the string in a globalization dictionary.
    a. Edit the DefaultDictionary.resources file in the `Skins\AvantGarde2\xsl\localization\` directory. For information about editing resource files, see *Editing Resource Files*.
    b. Add a new string named WELCOME_PAS_DB and whose value is *Welcome to the Policy Administration Dashboard*.
2. Configure the skins to use the new string in the globalization dictionary.
3. Replace the xsl:text XML that contains “Welcome to the Policy Administration Dashboard,” with a lookup. For example:

4. Verify that the EXAMPLE Express® home page says “Welcome to the Policy Administration Dashboard, Express Admin”.
5. In the `Skins\AvantGarde2\xsl\localization\` directory, create a copy of the DefaultDictionary.resources file and name it DefaultDictionary.fr.resources.
6. Edit the DefaultDictionary.fr.resources file you just created as desired. For information about editing resource files, see *Editing Resource Files*.
7. Add or edit the WELCOME_PAS_DB string, and give it a unique value such as “Welcome to the Policy Administration Dashboard,” in French.
8. Modify your browser language settings and verify that the Policy Administration dashboard now reads, "Welcome to the Policy Administration Dashboard," in French. For information about changing your browser language, see *Changing Browser Languages*.

JavaScript

JavaScript code uses the same resource files as XSLT code, but those resource files are available in a format that is natural to the JavaScript language. All strings stored in the resource file are converted into globally available string variables. For example, the DefaultDictionary.resources file gets converted at runtime to these lines of JavaScript code:

```
DCT.Strings.TEST = "Test";
DCT.Strings.WELCOME_PAS_DB = "Welcome to the Policy Administration Dashboard,";
```

As a result, hard-coded strings throughout the JavaScript code can be replaced with references to these string variables to achieve localization. The skins will automatically set the variable appropriately based on the language specified in the browser.

To make accessing string variables easier, you can use the T() function. This function, declared in EXAMPLE Express® skins, translates values by finding its value in the DCT.String namespace. For example:

```
T('TEST')
T('WELCOME PAS DB')
```

One benefit of using the T() function is that if the key is not present in the DCT.Strings namespace, it is returned as if it were the value.

The following table displays some examples of keys and the values returned by each.

Key	Value Returned
T('TEST') //	Test
DCT.Strings.TEST	Test
T('MISSING KEY') //	MISSING KEY
DCT.Strings.MISSING KEY	Undefined

To use these strings:

- Use the DCT.Strings namespace or the T() function as desired in any JavaScript in the skin.

The Avant Garde 2 skins are configured to make these string variables available on any page.

Localizing Custom JavaScript

The following example illustrates how to successfully localize your custom JavaScript.

Example:

Assume that a customer-specific business requirement requires that an alert check box which states, "Hello world!", displays in the appropriate language each time the user presses TAB to navigate to the next field in EXAMPLE Express®.

To implement this change:

1. In the Skins\AvantGarde2\xsl\localization\ directory, modify the DefaultDictionary.resources file by adding a new BLUR_EVENT_FIRED string with a value of *Hello World*. For more information about editing resource files, see *Editing Resource Files*.
2. (optional) Register the string in a second resource file for another language by creating a new DefaultDictionary.es.resources file (copied from the DefaultDictionary.resources file) and then editing the BLUR_EVENT_FIRED string with a value of *Hola mundo!*
3. Configure EXAMPLE Express® skins to use the newly-created string in the globalization dictionary by adding the following line of code to the customOnBlur function:
   alert(T('BLUR_EVENT_FIRED'));

To test the change above:

1. In EXAMPLE Express®, load a quote, leave a required field blank, and attempt to navigate to the next field by pressing TAB.
2. Verify that a message saying, "Hello world!"" or "Hola mundo!" (depeinding on the current browser culture) appears.
   When no resource file is present, the language defaults to the invariant culture.

Testing EXAMPLE Express® with Another Language

To test EXAMPLE Express® with another language:

1. In the `Skins\AvantGarde2\xsl\localization\` directory, copy the DefaultDictionary.fr.resources file.
2. Edit the copy of the DefaultDictionary.resources file you just created. For information about editing resource files, see Editing Resource Files.
3. Add or edit the REQD_FIELD_NOT_BLANK string, and give it a unique value such as “Ceci est un champ obligatoire et ne peut pas être blanc." in French.
4. Modify your browser language settings and verify that the dashboard now reads “Ceci est un champ obligatoire et ne peut pas être blanc.” in French. For information about changing browser languages, see *Changing Browser Languages*,

Images

Some images in the EXAMPLE Express® skins contain text, and will need to be localized if being used with a language other than English. For example, assume that a business requirement exists to display a different logout icon for French users in EXAMPLE Express®.

Using the example above, adhere to the following steps to replace the padlock image with another image:

1. Localize the image found at `Skins/AvantGarde2/images/icons/lock_open.png` for the French culture (fr-FR).

2. Locate the desired image file, and save it as lock_open.png in the `Skins/AvantGarde2/images/icons/fr-FR/` directory.

Localized image files can be saved as .jpg, .gif, and .png files.

3. Change your browser language settings and then verify in EXAMPLE Express® that the changes have been implemented. For information about changing browser language settings, see *Changing Browser Languages*.

Changing Browser Languages

Localizing EXAMPLE Express® skins requires that you change your browser language settings. You can change your browser language settings by:

- Changing the language in your Web browser. For more information, see *Changing Your Browser Language in a Web Browser*.
- Modifying the value of the appSettings configuration setting in the Express\web.config file. For more information, see *Changing Your Browser Language Using a Configuration Setting*.

Using a configuration setting to modify your browser language settings is recommended when changing the browser language on multiple machines at once.

Changing Your Browser Language in a Web Browser

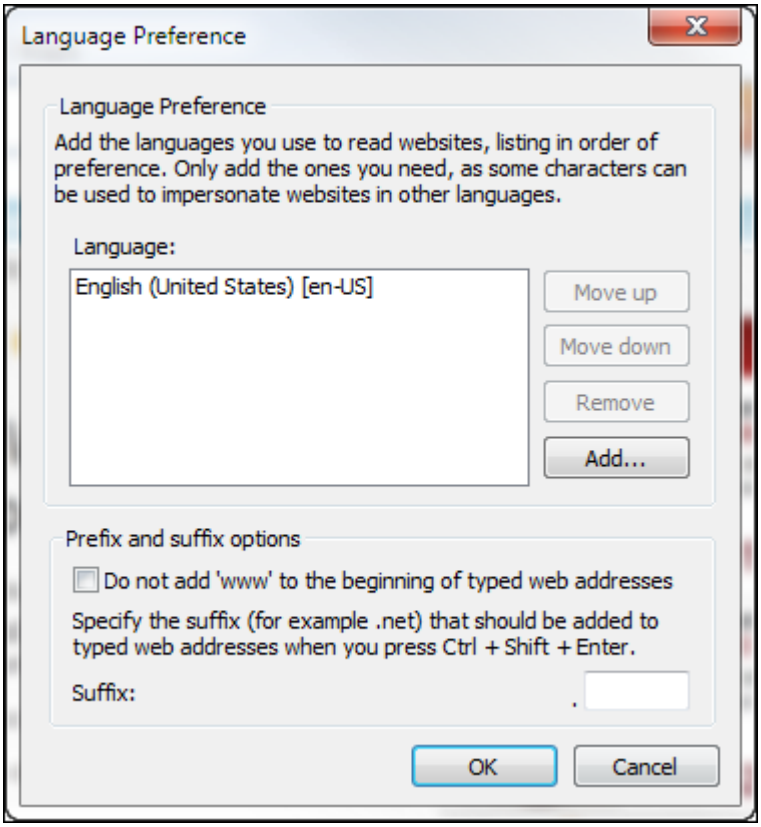
Using a Web browser, you can change the language to be used by the EXAMPLE Platform®.

Changing Your Browser Language in Internet Explorer

To change your browser language in Internet Explorer:

1. On the **Tools** menu, click **Internet Options**.

2. On the **General Tab**, click **Languages**.
   The Languages dialog box appears.



3. In the Language Preference dialog box, click **Add**.
   The Add Language dialog box appears.
4. In the Add Language dialog box, select a language from the list, and then click **OK**.

> You can change the order of preference of the browser languages by selecting a language and then clicking **Move Up** or **Move Down**. To remove a language from the list, select the language you wish to remove, and then click **Remove**.

5. Repeat steps four and five until you have added all of the languages you wish to use.
6. Click **OK** twice.

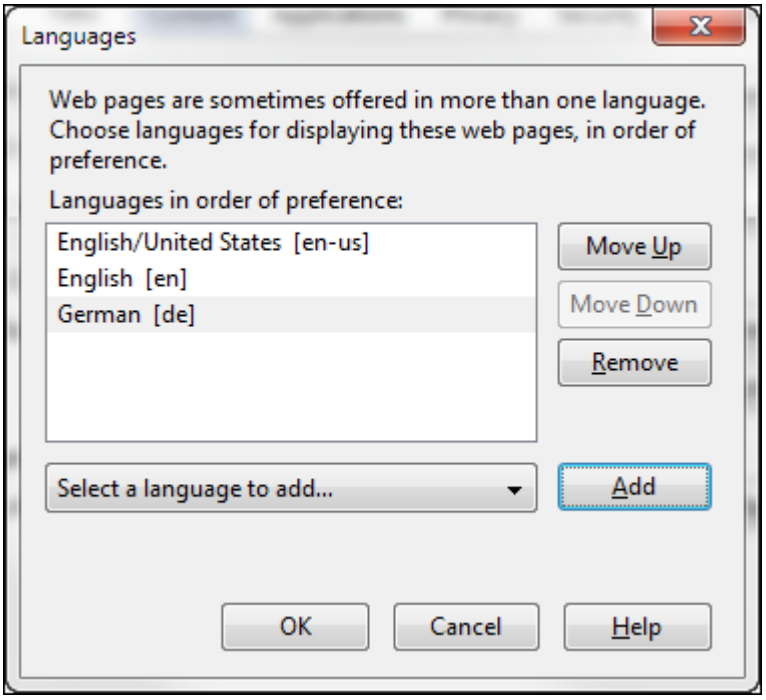
> The expected behavior of the platform is to use the first language listed. If dictionaries are not available for that language, the system will default back to English (the invariant culture).

Changing Your Browser Language in Mozilla Firefox

To change your browser language in Mozilla Firefox:

1. On the **Tools** menu, click **Options**.
   The Options dialog box appears.

2. On the **Content** tab, in the **Languages** section, click **Choose**.
   The Languages dialog box appears.



3. Select the language you wish to use from the **Select a langauge to add...** drop-down list, ant then click **Add**.
   The language is added to the list of preferred languages.

> You can change the order of preference of the browser languages by selecting a language and then clicking **Move Up** or **Move Down**. To remove a language from the list, select the language you wish to remove, and then click **Remove**.

4. Repeat step three until you have added all of the languages you wish to use.
5. Click **OK** twice.

> The expected behavior of the EXAMPLE Platform® is to use the first language listed. If dictionaries are not available for that language, the system will default back to English (the invariant culture).

Changing Your Browser Language Settings Using a Configuration Setting

Using a configuration setting, you can override the language of the captions currently displayed on the screens of the EXAMPLE Platform®.

- In the Express\Web.config settings, change the value of the **OverrideLanguageForScreens** attribute in the appSettings section to the desired language. For example, <add key="OverrideLanguageForScreens" value="en-US">.

This attribute will intially be blank to maintain compatibility with previous versions of the EXAMPLE Platform®. When the attribute is blank, the EXAMPLE Platform® will generate the screen captions and dates to the language setting of the browser. This means that different users may see the same screen in different languages depending on the browser settings. However, if the EXAMPLE Express® Web.config language attribute is set to a valid language setting, the screen captions and dates will be formatted for that language for all users.

Resource Files

With regard to the EXAMPLE Platform®, lookup tables refer to resource files to determine the appropriate text for a given culture or language. As of EXAMPLE Platform® 4.2.2, EXAMPLE Express® skins now ship with an `xsl\localization\` directory that contains resource files. These files serve as dictionaries of strings that are accessed by the XSLT through the templates in the dctLocalizedDictionary.xsl file. The .NET framework currently supports the following three culture types for use with localization:

Culture Type	File Name	Purpose
Invariant Culture	DefaultDictionary.resources	Default text for all strings
Neutral Culture	• DefaultDictionary.en.resources • DefaultDictionary.fr.resources	• English overrides • French overrides
Specific Culture	• DefaultDictionary.en-US.resources • DefaultDictionary.fr-FR.resources	• US specific English overrides • French specific overrides

Editing Resource Files

Using the EXAMPLE Platform®, you can edit resource files to control the text to be used for a specific culture or language.

Because .resx files are XML-based, .resources files should be converted to .resx files when editing them. To do this, use a tool such as ResGen.exe, which ships with the Microsoft Windows SDK.

Resource files can also be easily converted between the two formats from a command line. For example:

- Convert DefaultDictionary.resources to DefaultDictionary.resx (for editing)
- ResGen.exe DefaultDictionary.resources DefaultDictionary.resx
- Convert DefaultDictionary.resx back to DefaultDictionary.resources (for runtime use)
- ResGen.exe DefaultDictionary.resx DefaultDictionary.resources

Because it is overwritten during installation, It is recommended that you do not modify the DefaultDictionary.resources file (or any other file shipped in the AvantGarde or AvantGarde2

directories). Instead, it is recommended that you create a custom skin and modify its DefaultDictionary.resources file.

.resx files must be converted back to .resources format prior to being used in a runtime environment.

Buttons

You can create custom buttons for use with the Avant Garde 2 skin using the following files:

- **buttonDefaults.css** – Sets the default styles for buttons. For more information, see *buttonDefaults.css*.
- **buttons.css** – Contains style definitions for 15 built-in button types. For more information, see *buttons.css*.
- **custom-buttons.css** – File where all custom buttons are defined. For more information, see *custom-buttons.css*.
- **globalButton.psd** – Pre-divided template for creating buttons. For more information, see *globalButton.psd*.

buttonDefaults.css

The buttonDefaults.css file sets default styles for buttons. These default styles can be overwritten in the buttons.css and custom-buttons.css files.

The following table list some notable classes you can use with the buttonDefaults.css file.

Class	Description
g-btn-bar	Forces multiple buttons contained in a to align side-by-side instead of stacking vertically.
g-btn	Sets default button height to 24px and right margin to 10px. Applied to every button file.
g-btn-l, g-btn-m, g-btn-r	Corresponds to the left, middle, and right sections of a button. Background images are set to g-btn-slice.png. By default, the background position is set to the position of the green button in g-btn-slice.png.
g-btn-left, g-btn-right	Determines the position of icons.
g-btn-text	Sets the default font size to *0.9em*, font weight to *bold*, and font color to *#333333*.

buttons.css

The buttons.css file contains style definitions for 15 built-in button types. The name of each button can be specified as the value of the button type parameter in EXAMPLE Author®.

This file imports the defaults from the buttonDefaults.css file.

custom-buttons.css

The custom-buttons.css file is used to define custom buttons. To effectively use the custom-buttons.css file:

1. Create a custom theme. For information about creating a custom theme, see *Themes*.

You must create a custom theme before defining custom buttons.

2. In the **css** folder in your custom theme directory, open the **custom-buttons.css** file.
3. In the **buttons.css** file, copy the code for an existing button. The following is an example of a button definition:

```
.g-myButton .g-btn-text{ color:
#ffffff;}
.g-myButton .g-btn-l{ background-position: 0px 0px; }
.g-myButton .g-btn-m{ background-position: 0px -24px; }
.g-myButton .g-btn-r{ background-position: 0px -48px; }
.g-myButton .g-btn-img-left{ }
.g-myButton .g-btn-img-right{ }
```

4. Paste the code into the **custom-buttons.css** file.
5. Replace the button name (myButton in this case) with a new name for your custom button.
6. Modify your button as desired.
   The following table lists some notable classes that you can use with the custom-button.css file.

Class	Description
g-btn-text	Changes the color of the button text.
g-btn-l, g-btn-m, g-btn-r	Adjusts the background position for each section of the button in the g-btn-slice.png file.
g-btn-img-left, g-btn-image-right	Adjusts the icon poosition by adding padding to the left or to the right.

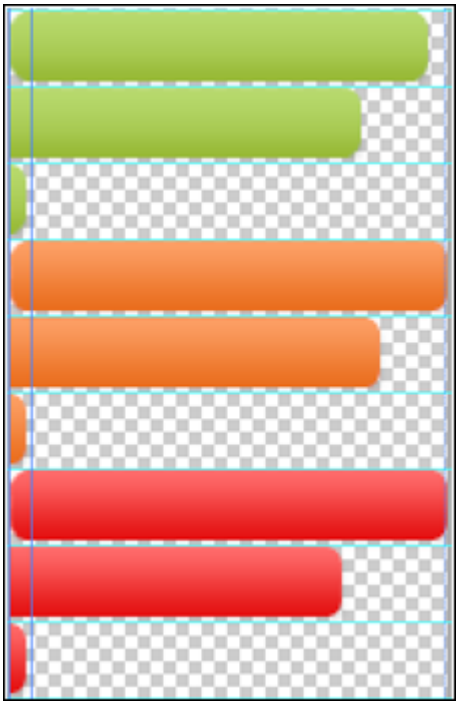
globalButton.psd

The globalButton.psd file (located in the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\Photoshop Source` directory) is a pre-divided template for creating new custom buttons.

To use the globalButton.psd file:

1. Add your custom button design under the existing images.

2. Use the 7-pixel wide slice on the left side of the image as a guide for creating the left, middle, and right slices of the button.



3. On the **File** menu, click **Save for Web and Devices**.
   The Save dialog box appears.
4. Navigate to the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\images\` directory, name the file **g-btn-slice.png**, and then click **Save**.
   The slice will now be used in the buttons.css file as the background image for your new button style.

Custom Containers

You can create custom containers (elements containing other HTML elements) for use with the Avant Garde 2 skin using the following files:

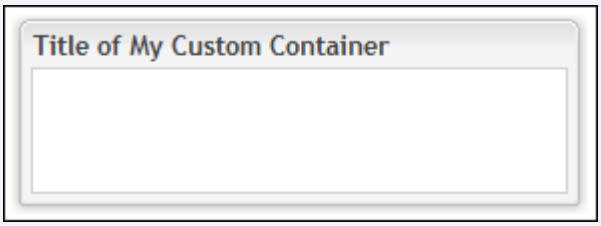
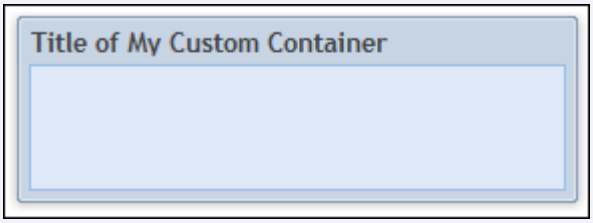
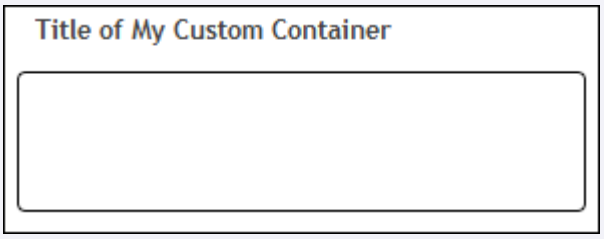
- **customContainers.css** – Contains style definitions for seven built-in custom containers. For more information, see *customContainers.css*.
- **ccTemplate.psd** – Pre-divided template for creating a custom container. For more information, see *ccTemplate.psd*.
- **custom-cc.css** – Location where custom containers are defined. For more information, see *custom-cc.css*.

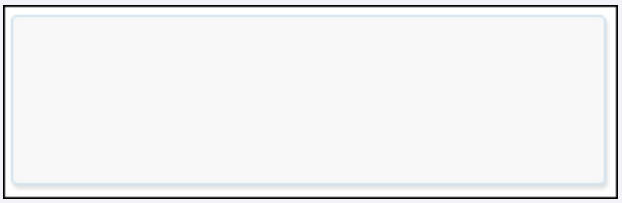
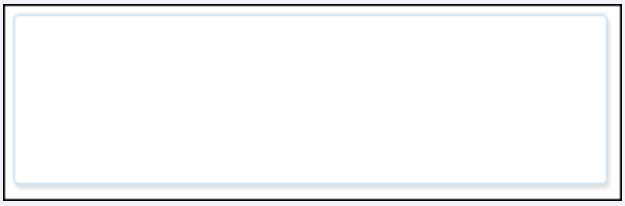
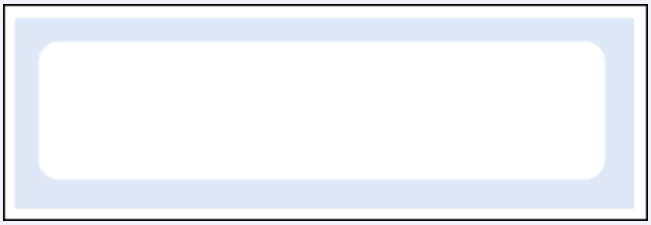
customContainers.css

The customContainers.css file contains style definitions for the following built-in custom containers.

The name of each custom container can be specified as the value of the *Custom Container* parameter in EXAMPLE Author®.

The following table displays the seven custom containers stored in the customContainers.css file.

Custom Container	Image
GrayPanel	
LightPanel	
BluePanel	
BlackBorder	

MinimalLightGray	
MinimalWhite	
PlainWhite	

ccTemplate.psd

The ccTemplate.psd file (located in the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\Photoshop Source` directory) is a pre-divided template that can be used to create a custom container.

To use the ccTemplate.psd file:

1. In Adobe Photoshop, replace the sample container with your own design.
2. On the **File** menu, click **Save for Web and Devices**.
   The Save dialog box appears.
3. Navigate to the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\images\cc\` directory, create a new folder, and then click **Save**.

By default, the sections are saved as cc-t.png, cc-tl.png, cc-tr.png, etc.

custom-cc.css

The custom-cc.css file is where all custom containers are defined. To use this file:

1. Create a custom theme. For information about creating a custom theme, see *Themes*.

You must create a custom theme before defining custom containers.

2. In the **css** folder in your custom theme directory, open the custom-cc.css file.
3. In the **customContainers.css** file, copy the code for an existing container. The following is an example of a container definition:

```
.cc_CustomExample > .cc-bubble-tl {background: transparent url(../images/cc/
CustomExample/cc-tl.png) no-repeat left top;}
.cc_CustomExample > div > .cc-bubble-tr {background: transparent url(../
images/cc/CustomExample/cc-tr.png) no-repeat right top;}
.cc_CustomExample > div > div > .cc-bubble-tc {background: transparent
url(../images/cc/CustomExample/cc-t.png) repeat-x 0 top; height:15px;}
.cc_CustomExample > div > div > div > .cc-bubble-title
{padding-top:0px;padding-bottom:12px;}
.cc_CustomExample > div > div > div > .cc-bubble-title > span
{font-weight:bold;}
.cc_CustomExample > .cc-bubble-bwrap {background: transparent;}
.cc_CustomExample > div > .cc-bubble-ml {background: transparent url(../
images/cc/CustomExample/cc-l.png) repeat-y 0 0;}
.cc_CustomExample > div > .cc-bubble-ml {padding-left:10px;zoom:1;}
.cc_CustomExample > div > div > .cc-bubble-mr {background: transparent
url(../images/cc/CustomExample/cc-r.png) repeat-y right 0;}
.cc_CustomExample > div > div > .cc-bubble-mr {padding-right:10px;zoom:1;}
.cc_CustomExample > div > div > div > .cc-bubble-mc {background: #fff;}
.cc_CustomExample > div > div > div > div > .cc-bubble-body {background:
transparent;}
.cc_CustomExample > div > .cc-bubble-bl {background: transparent url(../
images/cc/CustomExample/cc-bl.png) no-repeat 0 bottom;}
.cc_CustomExample > div > div > .cc-bubble-br {background: transparent
url(../images/cc/CustomExample/cc-br.png) no-repeat right bottom;}
.cc_CustomExample > div > div > div > .cc-bubble-bc {background:
transparent url(../images/cc/CustomExample/cc-b.png) repeat-x 0 bottom;}
```

4. In the **custom-cc.css** file, paste the code.
5. Replace the the container name (CustomExample in this case) with the name of the folder where your custom container images are stored.
6. Modify the value of the bottom padding in the following line of code from the example above:

```
.cc_CustomExample > div > div > div > .cc-bubble-title
{padding-top:0px;padding-bottom:12px;}
```

To adjust the horizontal position of a container:

- In EXAMPLE Author®, add an *alignHeader* custom parameter and enter *left*, *center*, or *right* as the value.

By default, all container titles align to the left.

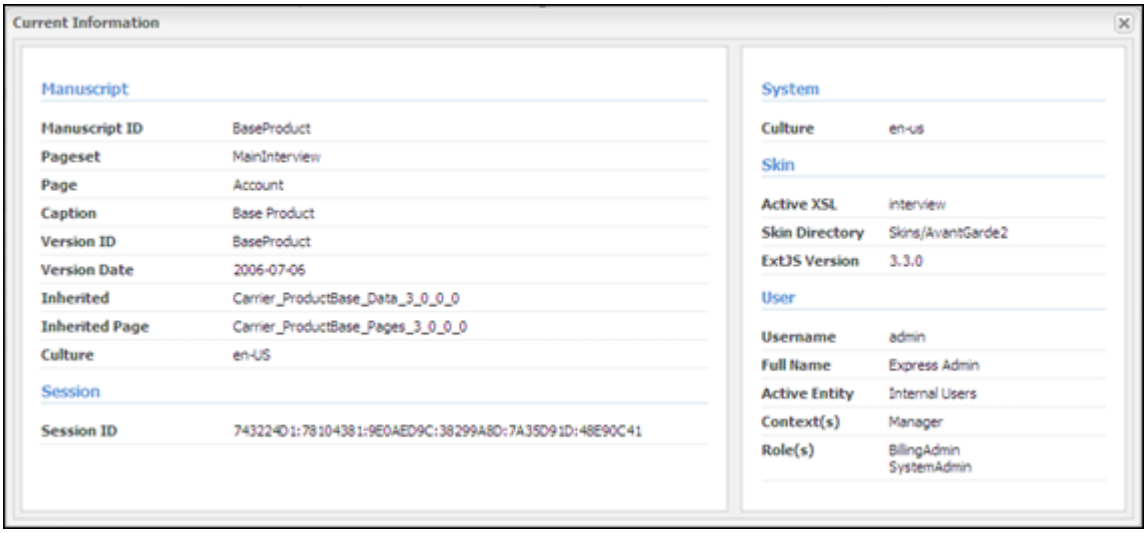
Developer Toolbar

The Duck Creek Developer Toolbar is designed to provide easy access to information and actions that are helpful for debugging and development.

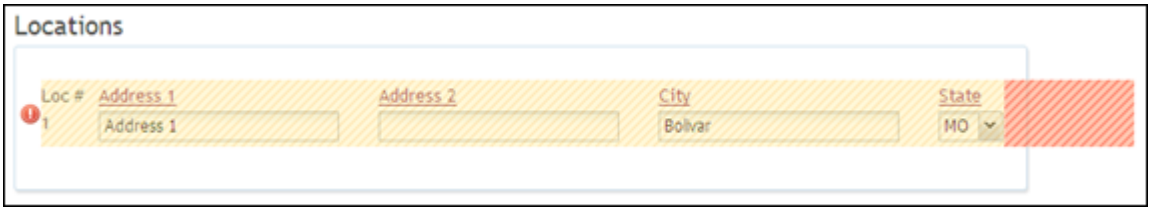
Duck Creek Developer Toolbar Tools

The Duck Creek Developer Toolbar includes the following tools:

- **Current Information** – Displays useful ManuScript, session, system, skin, and user information.

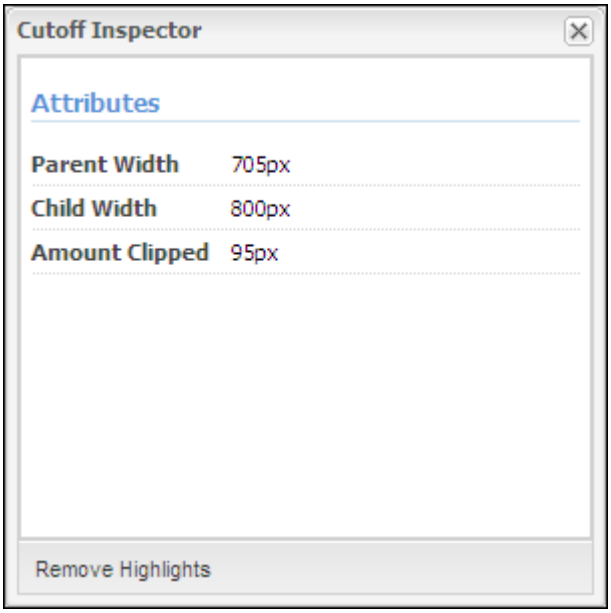


- **Cutoff Inspector** – When a parent element is not wide enough, the Cutoff Inspector identifies the child elements that are cut off. For example, the ZIP code field has been cut off in the following image:



To view additional details about the child elements being cut off as well as their parent element, click the problem indicator button  .

The Cutoff Inspector dialog box appears.

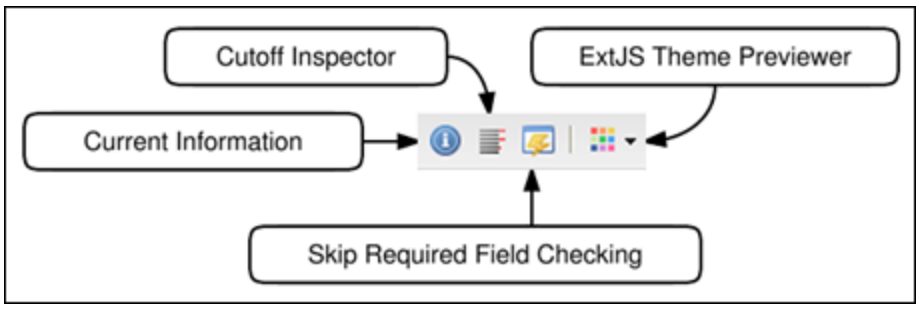


- **Skip Required Field Checking** – Allows a developer to proceed to the next page without filling out required fields.
- **ExtJs Theme Previewer** – Allows a developer to preview an ExtJs theme.
- **ManuScript ID** – Displays the active ManuScript ID. Additional details about the ManuScript can be seen by using the **Current Information** tool.
- **Execute JavaScript** – Allows a developer to execute JavaScript quickly and easily.
- **Ext.query Counter** – Returns the length of the array returned by `Ext.query("INPUT SELECTOR");`. The following table provides examples of some information returned by `Ext.query("INPUT SELECTOR:);`.

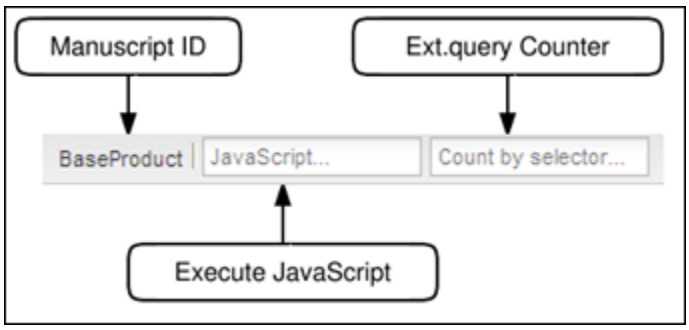
Input	Description
div	Returns a count of divs on a page.
input[type=text]	Returns a count of text inputs on a page.
#main *[class^=x_]	Returns a count of elements in main area with styles generated in EXAMPLE Author® applied.

The Duck Creek Developer Toolbar is comprised of two groups of tools.

The following tools can be found on the left side of the toolbar:



The following tools can be found on the right side of the toolbar:



Enabling the Duck Creek Developer Toolbar

To enable the Duck Creek Developer Toolbar:

- In the Express\Web.config file, set the value of the *ShowDeveloperTools* configuration setting to *1*. The toolbar displays at the bottom of the browser.

Themes

Avant Garde 2 skins include a theme system designed to provide an additional layer to which users can apply style customizations.

Creating a Custom Theme

To create a custom theme:

1. In the `C:\Program Files\Duck Creek Technologies\Express\Skins\[CUSTOM SKIN]\themes\` directory, create the following files in the following folders:

File name	Description
images	
cc	Used for custom container images.
css	

theme.css	Imports the custom-cc.css, custom-buttons.css, and custom-ext.css files.
custom-cc.css	Used for custom container definitions.
custom-buttons.css	Used for custom button definitions.
custom-ext.css	Used for theme-specific ExtJs CSS overrides.

2. In the **theme.css** file in the `C:\Program Files\Duck Creek Technologies\Express\Skins\[CUSTOM SKIN]\themes\[CUSTOM THEME]\css\` directory, add the following import statements to the beginning of the file:

```
@import url(custom-ext.css);
@import url(custom-cc.css);
@import url(custom-buttons.css);
```

3. Save the file.
   If the skin.config file is configured to use a custom theme, the theme.css file will automatically be used.

The custom-cc.css, custom-buttons.css, and custom-ext.css files must be imported at the beginning of the theme.css file.

4. In the **skin.config** file for the custom skin, change the value of the <Theme name> setting to the name of your custom theme.
5. Save the file.
   The custom theme is enabled, custom style definitions can be applied to it, and custom images can be added to the custom theme's Images directory.

Customizing Avant Garde 2.1 Skins

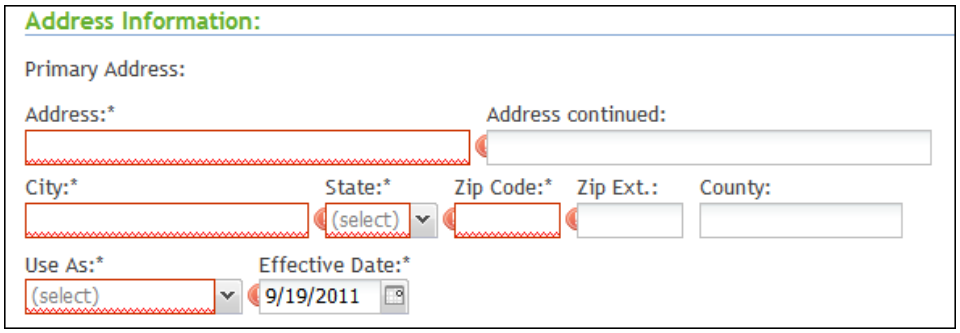
EXAMPLE Express® 5.0 includes Avant Garde 2.1 skins. Avant Garde 2.1 skins continue Accenture Duck Creek Technologies, Inc. effort to move toward an object oriented model using the Ext JS 3.4 Framework. Moving towards an object oriented model, the following enhancements have been made:

- All Ext JS features are availble for customizations
- Customizations are easier through overrides and extensions
- Maintenance is easier
- Code duplication is eliminated
- Page rendering is more efficient

Avant Garde 2.1 skins configure controls as objects based on class attributes (DCT.InputIntField or DCT.InputFloatField). Each control is built based on its type. Each DCT class individually validates, formats, sets focus, and handles events based on its class. Classes can be globally extended or overridden at the base class or specific child input class for any property or method. All Ext JS methods or properties may be available to be extended or overridden. For Avant Garde 2.1 skin customizations that do not affect JavaScript controls, see *Customizing Avant Garde 2 Skins*.

Behavior Changes

Out of the box, Avant Garde 2.1 controls validate input fields as data is entered. For example, a field such as a telephone number input field will not accept alphabetical characters. Alert messages were removed and replaced with visual cues and indicators. The following shows input fields with indicators of required fields. Unless the fields are completed, the user will not be able to navigate to the next page.



Customizing Avant Garde 2.1 Skins

Customizing skins has been made easier with Avant Garde 2.1. Each control is in its own file, and extending or overriding a control is done at the child class for individual fields or at the base class for all field types. Once a property has been added or removed, it is defined in the data-config section of the XSLT file for the control that was changed. Changes are made in one place (XSLT, child class, or base class) depending on the extent of the change.

Making Changes in the XSLT Files

To add or remove properties from controls, make changes to the data-config attribute in the XSLT file. Helper templates are provided to add properties to the data-config attribute. To add a new property, call the template called "addConfigProperty" and customize any of three parameters that it will accept.

name — Name of the property

value — Value for the property

type — Type of property (string, number, Boolean, object, array, regEx)

The following is an example of the template:

```
<xsl:template name="addConfigProperty">
  <xsl:param name="name"/>newpropertyname</xsl:with-param>-->
  <xsl:param name="value"/>newpropertyvalue</xsl:with-param>-->
  <xsl:param name="type"/>string</xsl:with-param>-->
```

Making Changes in the ExtJs Files

Each control in Avant Garde 2.1 has its own source file that makes finding and customizing the control easier. All the source files are located in the `C:\Program Files\Duck Creek Technologies\Express\Skins\DCTBase\Core\devscripts\common` folder. To make changes to specific fields, customizations can be done at the child class. The following is a code sample of the child class available for customizations in the Ext JS controls.

```
DCT.InputStringField = Ext.extend(DCT.InputField, {
    inputXType: 'dctinputstringfield',
```

```
  setValidator: function(config) {
    config.validator = this.checkstring;
},
  setFormatter: function(config) {
      switch (config.dct.JSFormatter)

          {
    case: 'upper':
       config.formatter = this.formatUpper;
    break;
    case 'lower' :
       config.formatter = this.formatLower;
    break;
    case 'string':
       config.formatter = this.formatString;
    break;
     }
}
```

To make changes across fields, customize the Ext JS at the base class. The following is a code sample of the base class available for customizations in the Ext JS controls.

```
Ext.override (Ext.BoxComponent,   {
   skipRequiredChecking: false,
  customOnBlur:  function ()   {
},
  customOnChange: function ()    {
},
  customFieldFormat: function ()   {
},
fieldChanged: function ()  }
   fieldChanged.set () ;
    this.customOnChange () ;
},
```

The following is a list of source files and features that are available.

Control	Source File	Class Name	Features
Accordian panels	dctAccordians.js	DCT.Accordian DCT.AccordianItem	• Handles creating accordian parent and accordian panel items • Sets the accordian items to the control and assigns the panel that should be expanded • Assigns listeners to handle the Ajax processing before the primary panel is expanded

Checkboxes	dctCheckboxes.js	DCT.CheckboxGroup DCT.CheckboxField	• Establishes the checkbox group and checkbox items • Sets debug information if Web.config key is turned on • Creates the checkbox group as vertical or horizontal based on the configuration passed • Sets event listeners for check, focus, and afterrender • Fires field processing if the field is checked or unchecked • Notifies focus is set if the field is selected
Dates	dctInterviewDates.js	DCT.InterviewDate	• Setting whether **Today** button is available based on minimum or maximum dates • Sets debug information if Web.config key is turned on • Sets maxlength if passed • Assigns event listeners for blur, change, focus, and afterrender • Sets date format • Handles U.K. and U.S. formats • Validation is done by the ExtJs control
Display (read only)	dctDisplayFields.js	DCT.DisplayField	• Sets debug information if the Web.config key is turned on

Dropdowns/ Combos	dctComboField.js	DCT.ComboField	• Auto filtering to allow a user to filter a list • Selection of an item will fire selection event for processing • Hover Help for items in a list • Hover Help for a selected item • Handles focus notification • Sets debug information if the **Web.config** key is turned on for list and input fields • Handles optimum sizing based on the items in the list
Editor grids	dctBaseGrids.js	DCT.EditorGridPanel	• Defines the datastore information to process the list of data that displays ◦ Includes defining the record structure and the URL proxy to use to obtain the data. If processing is through an Ajax call, the data will be displayed based on the data returned for the page instead of retrieving it again. • Defines and assigns the selection model to use for highlighting and selecting rows • Defines the grid view to use for the grid • Creates the paging toolbar if there are more items to display than the user can view • Defines and assigns the column model definition for the column headers • Assigns the listeners for row clicks, sorting change, after editing, and before editing ◦ Before editing event will handle creating the specific form field control for editing data ◦ After editing will handle processing the Ajax call for the entered data and assignment of data to a hidden field on the page

Email	dctInputEmailFields.js	DCT.InputEmailField	• Assigns the validation routine to validate email values • Sets the validation type process to mask the field for correct characters and validation • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus
Extended dropdowns	dctExtendedDropDown.js	DCT.ExtendedComboField	• Extension of a regular dropdown • No Auto-filtering • Selection of an item will fire the selection event for processing • Hover Help for items in the list • Hover Help for a selected item • Handles focus notification • Sets debug information if the Web.config key is turned on for the list and input field • Handles optimum sizing based on items in the list • Defines a template layout for styling the data for the user

File upload	dctInputFileFields.js	DCT.InputFileField	• Assigns the validation routine for required checking • Sets debug information if the Web.config key is turned on • Sets event listeners for blur, change, focus, keypress, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus • Creates a control that looks the same across all browsers
Floats	dctInputFloatFields.js	DCT.InputFloatField	• Assigns the validation routine to validate float values • Assigns formatter to format the field with commas and currency symbol once the user leaves the field • Assigns culture symbols to be used during formatting and validation • Assigns getter method for validation routine to strip culture symbols before validation checking • Sets debug information if Web.config key is turned on • Sets masking routine to mask invalid characters for the float field • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus

Grid editor controls	dctEditorComboFields.js dctEditorInputFields.js dctEditorNumberFields.js dctEdiorCheckBoxes.js	DCT.EditorComboField DCT.EditorInputField DCT.EditorNumberField DCT.EditorCheckBoxField	• Extended from the base controls to process data within editable grids • Available only at the time a user edits data in a row or column field in an editable grid ◦ Controls will have limited functionality since they are temporary ◦ Combos are only selectable ◦ Input fields will have validation but not formatting • Controls are destroyed once the user leaves the field
Integers	dctInputIntFields.js	DCT.InputIntField	• Assigns the validation routine to validate integer value • Assigns formatter to format the number with commas once the user leaves the field • Assigns culture symbols to be used during formatting and validation • Assigns getter method for validation routine to strip culture symbols before validation checking • Sets debug information if Web.config key is on • Sets masking routine to mask invalid character for integer fields • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field recieves focus

IP Address	dctInputIPFields.js	DCT.InputIPField	• Sets the validation type process to mask the field for correct characters and validation ◦ Checks for numbers and periods • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus
Phone	dct.InputPhoneFields.js	DCT.InputPhoneField	• Assigns the validation routine to validate and mask phone fields • Sets the formatter to handle formatting if formatmask has been set ◦ Formats for U.S. format • Sets the validation type process to mask the field for correct characters and validation ◦ Checks for numbers, parentheses, and dashes • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus
Progress Bar	dctProgressBar.js	DCT.ProgressBar	• Handles creating progress bar control • Sets width of control • Sets the text displayed within the control • Assigns control to the progress bar collection to help manage multiple controls

Radio buttons	dctRadiobuttons.js	DCT.RadioGroup DCT.RadioField	• Establishes the radio group and radio items • Sets debug information if the Web.config key is turned on • Creates the radio group as vertical or horizontal based on the configuration passed • Calculates widths of each radio button to span the group width correctly • Sets event listeners for check, focus, and afterrender • Fires field processing if the field is checked or unchecked • Notifies focus is set if the field is selected • Radio field is reused for Extended radios
Slider	dctSliders.js	DCT.ComparisonSlider	• Handles creating slider control • Sets width of the control • Sets the maximum value for the control to calculate the tick values • Sets the listeners to handle changecomplete and change events • Assigns initial value for the display • Creates the datastore object for available values • Displays comparison values above the slider

SSN	dctInputSSNFields.js	DCT.InputSSNField	• Assigns the validation routine to validate and mask SSN fields • Sets the formatter to handle formatting if formatmask has been set ◦ Formats for U.S. format • Sets the validation type process to mask the field for correct characters and validation ◦ Checks for numbers and dashes • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus
String	dctInputStringFields.js	DCT.InputStringField	• Assigns the validation routine to validate string values • Sets the formatter to handle formatting if formatmask has been set ◦ Formats for uppercase, lowercase, specific string layout, or no formatting • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus

Tab panels	dctTabs.js	DCT.TabPanel	• Handles creating tabs per section that is identified through EXAMPLE Author® • Sets the tab panel items to the control and assigns the panel that should be shown first • Assigns a listener to handle the before tab change
Text area	dctTextAreaFields.js	DCT.TextAreaField	• Assigns required, regular expression, and max length validation checking • Sets debug information if the Web.config key is turned on • Sets event listeners for blur, change, focus, keypress, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus • Styles the width and height based on the formatmask configuration
Time	dctInputTimeFields.js	DCT.InputTimeField	• Assigns the validation routine to validate and mask time fields • Sets the validation type process to mask the field for correct characters and validation ◦ Checks for military time or standard time format • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus

ZIP code	dctInputZipCodeFields.js	DCT.InputZipCodeField	• Assigns the validation routine to validate ZIP code values • Sets the validation type process to mask the field for correct characters and validation • Sets debug information if Web.config key is turned on • Sets event listeners for blur, change, focus, and afterrender • Fires field processing if the field loses focus • Notifies focus is set if the field receives focus

Buttons

You can create custom buttons for use with the Avant Garde 2 skin using the following files:

- **buttonDefaults.css** – Sets the default styles for buttons. For more information, see *buttonDefaults.css*.
- **buttons.css** – Contains style definitions for 15 built-in button types. For more information, see *buttons.css*.
- **custom-buttons.css** – File where all custom buttons are defined. For more information, see *custom-buttons.css*.
- **globalButton.psd** – Pre-divided template for creating buttons. For more information, see *globalButton.psd*.

buttonDefaults.css

The buttonDefaults.css file sets default styles for buttons. These default styles can be overwritten in the buttons.css and custom-buttons.css files.

The following table list some notable classes you can use with the buttonDefaults.css file.

Class	Description
g-btn-bar	Forces multiple buttons contained in a to align side-by-side instead of stacking vertically.
g-btn	Sets default button height to 24px and right margin to 10px. Applied to every button file.
g-btn-l, g-btn-m, g-btn-r	Corresponds to the left, middle, and right sections of a button. Background images are set to g-btn-slice.png. By default, the background position is set to the position of the green button in g-btn-slice.png.
g-btn-left, g-btn-right	Determines the position of icons.
g-btn-text	Sets the default font size to *0.9 em*, font weight to *bold*, and font color to *#333333*.

buttons.css

The buttons.css file contains style definitions for 15 built-in button types. The name of each button can be specified as the value of the button type parameter in EXAMPLE Author®.

This file imports the defaults from the buttonDefaults.css file.

custom-buttons.css

The custom-buttons.css file is used to define custom buttons. To effectively use the custom-buttons.css file:

1. Create a custom theme. For information about creating a custom theme, see *Themes*.

   You must create a custom theme before defining custom buttons.

2. In the **css** folder in your custom theme directory, open the **custom-buttons.css** file.
3. In the **buttons.css** file, copy the code for an existing button. The following is an example of a button definition:

```
.g-myButton .g-btn-text{ color:
#ffffff;}
.g-myButton .g-btn-l{ background-position: 0px 0px; }
.g-myButton .g-btn-m{ background-position: 0px -24px; }
.g-myButton .g-btn-r{ background-position: 0px -48px; }
.g-myButton .g-btn-img-left{ }
.g-myButton .g-btn-img-right{ }
```

4. Paste the code into the **custom-buttons.css** file.
5. Replace the button name (myButton in this case) with a new name for your custom button.
6. Modify your button as desired.
   The following table lists some notable classes that you can use with the custom-button.css file.

Class	Description
g-btn-text	Changes the color of the button text.
g-btn-l, g-btn-m, g-btn-r	Adjusts the background position for each section of the button in the g-btn-slice.png file.
g-btn-img-left, g-btn-image-right	Adjusts the icon poosition by adding padding to the left or to the right.

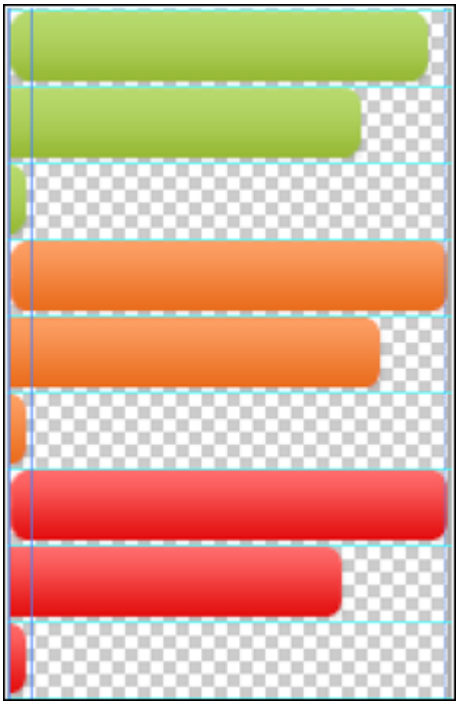
globalButton.psd

The globalButton.psd file (located in the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\Photoshop Source` directory) is a pre-divided template for creating new custom buttons.

To use the globalButton.psd file:

1. Add your custom button design under the existing images.

2. Use the 7-pixel wide slice on the left side of the image as a guide for creating the left, middle, and right slices of the button.



3. On the **File** menu, click **Save for Web and Devices**.
   The Save dialog box appears.
4. Navigate to the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\images\` directory, name the file **g-btn-slice.png**, and then click **Save**.
   The slice will now be used in the buttons.css file as the background image for your new button style.

Creating a Custom Skins Directory

When customizing EXAMPLE Express® skins, do not modify any files in the DCTBase or AvantGarde2.1 directories. All files in these directories are overwritten during upgrades. Instead, you must create a custom skins directory (copied from the AvantGarde2 directory) and modify the files in this directory.

To create a custom skins directory:

1. In the *Global Root Directory*\Express\Skins directory, copy the **AvantGarde2.1** directory and paste it in the desired location within the Express directory.
2. Rename the directory.
3. Make any desired customizations in the new directory. Use the information in this section to help you customize the system.

Once you have created a custom skins directory, you must configure EXAMPLE Express® to run using your customized skin.

To configure EXAMPLE Express® to use your customized skin:

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
3. Under **Mode**, click **Form**.
4. On the **Web Config** tab, in the list of settings, click **XSLTDir**.
5. In the **value** text box, enter the relative path to the custom directory from the *Global Root Directory*\Express directory (for example, `Skins\MySkins`).
6. Click **Save**.
7. Click **Close**.

Custom Containers

You can create custom containers (elements containing other HTML elements) for use with the Avant Garde 2 skin using the following files:

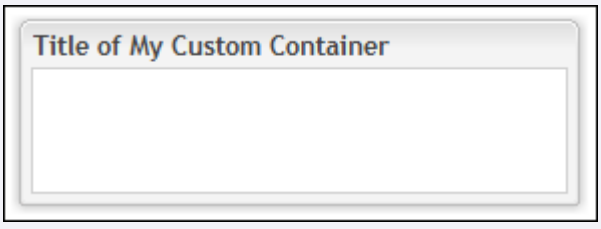
- **customContainers.css** – Contains style definitions for seven built-in custom containers. For more information, see *customContainers.css*.
- **ccTemplate.psd** – Pre-divided template for creating a custom container. For more information, see *ccTemplate.psd*.
- **custom-cc.css** – Location where custom containers are defined. For more information, see *custom-cc.css*.

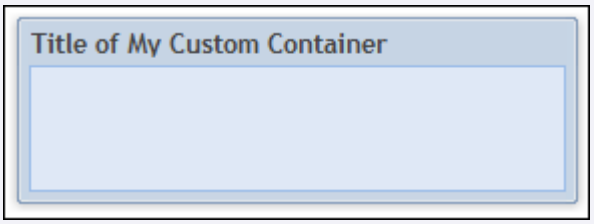
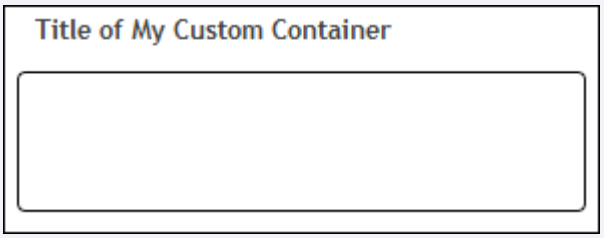
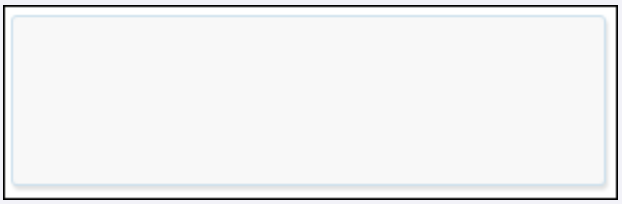
customContainers.css

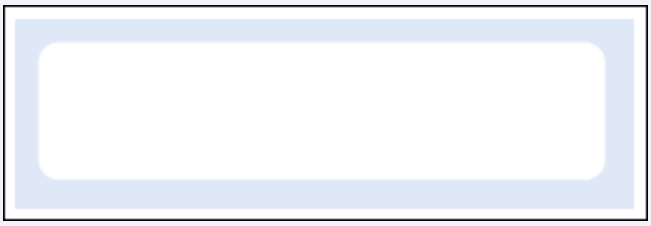
The customContainers.css file contains style definitions for the following built-in custom containers.

The name of each custom container can be specified as the value of the *Custom Container* parameter in EXAMPLE Author®.

The following table displays the seven custom containers stored in the customContainers.css file.

Custom Container	Image
GrayPanel	

LightPanel	
BluePanel	
BlackBorder	
MinimalLightGray	
MinimalWhite	

PlainWhite	

ccTemplate.psd

The ccTemplate.psd file (located in the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\Photoshop Source` directory) is a pre-divided template that can be used to create a custom container.

To use the ccTemplate.psd file:

1. In Adobe Photoshop, replace the sample container with your own design.
2. On the **File** menu, click **Save for Web and Devices**.
   The Save dialog box appears.
3. Navigate to the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\images\cc\` directory, create a new folder, and then click **Save**.

> By default, the sections are saved as cc-t.png, cc-tl.png, cc-tr.png, etc.

custom-cc.css

The custom-cc.css file is where all custom containers are defined. To use this file:

1. Create a custom theme. For information about creating a custom theme, see *Themes*.

> You must create a custom theme before defining custom containers.

2. In the **css** folder in your custom theme directory, open the custom-cc.css file.
3. In the **customContainers.css** file, copy the code for an existing container. The following is an example of a container definition:

```
.cc_CustomExample > .cc-bubble-tl {background: transparent url(../images/cc/
CustomExample/cc-tl.png) no-repeat left top;}
.cc_CustomExample > div > .cc-bubble-tr {background: transparent url(../
images/cc/CustomExample/cc-tr.png) no-repeat right top;}
.cc_CustomExample > div > div > .cc-bubble-tc {background: transparent
url(../images/cc/CustomExample/cc-t.png) repeat-x 0 top; height:15px;}
.cc_CustomExample > div > div > div > .cc-bubble-title
{padding-top:0px;padding-bottom:12px;}
.cc_CustomExample > div > div > div > .cc-bubble-title > span
{font-weight:bold;}
```

```
.cc_CustomExample > .cc-bubble-bwrap {background: transparent;}
.cc_CustomExample > div > .cc-bubble-ml {background: transparent url(../
images/cc/CustomExample/cc-l.png) repeat-y 0 0;}
.cc_CustomExample > div > .cc-bubble-ml {padding-left:10px;zoom:1;}
.cc_CustomExample > div > div > .cc-bubble-mr {background: transparent
url(../images/cc/CustomExample/cc-r.png) repeat-y right 0;}
.cc_CustomExample > div > div > .cc-bubble-mr {padding-right:10px;zoom:1;}
.cc_CustomExample > div > div > div > .cc-bubble-mc {background: #fff;}
.cc_CustomExample > div > div > div > div > .cc-bubble-body {background:
transparent;}
.cc_CustomExample > div > .cc-bubble-bl {background: transparent url(../
images/cc/CustomExample/cc-bl.png) no-repeat 0 bottom;}
.cc_CustomExample > div > div > .cc-bubble-br {background: transparent
url(../images/cc/CustomExample/cc-br.png) no-repeat right bottom;}
.cc_CustomExample > div > div > div > .cc-bubble-bc {background:
transparent url(../images/cc/CustomExample/cc-b.png) repeat-x 0 bottom;}
```

4. In the **custom-cc.css** file, paste the code.
5. Replace the the container name (CustomExample in this case) with the name of the folder where your custom container images are stored.
6. Modify the value of the bottom padding in the following line of code from the example above:

```
.cc_CustomExample > div > div > div > .cc-bubble-title
{padding-top:0px;padding-bottom:12px;}
```

To adjust the horizontal position of a container:

- In EXAMPLE Author®, add an *alignHeader* custom parameter and enter *left*, *center*, or *right* as the value.

By default, all container titles align to the left.

Customizing CSS

You can customize CSS for EXAMPLE Express® by:

- Customizing a set of global .css files that apply across the entire EXAMPLE Express® application.
- Customizing individual override files (indicated by the filename *x_pagename.css*) that apply to single pages or portions of the EXAMPLE Express® application.
- Applying custom CSS styles to specific page elements in a ManuScript. These styles are defined in a single override file (*x_interview.css*), and are applied to individual fields and page elements in EXAMPLE Author®. For more information about creating and applying custom CSS styles in EXAMPLE Author®, see *Working with CSS Override Styles* in the *Guide to EXAMPLE Author®*.

The x_interview.css file is automatically modified by EXAMPLE Author® any time styles are edited using the CSS Manager in Page Designer. This file does not need to be edited by hand.

The table below describes the customizable .css files in the *Global Root Directory*\Express\Skins*CustomSkin*\css directory. Use this table to determine which files to modify when customizing CSS.

File	Description

buttonDefaults.css	Contains default styles for buttons. These can be overwritten by defining specific button styles in buttons.css.
buttons.css	Contains styles for all links and buttons used in EXAMPLE Express®.
customContainers.css	Contains style definitions for 7 built-in custom containers.
dctExtJsExtensions.css	Contains extensions to the Ext JS components, such as the slider bar widget.
dctExtJsOverrides.css	Contains styles to override those used by Ext JS.
dctExt-xtheme-blue.css	Contains style definitions created to override styles in the blue Ext JS theme.
dctExt-xtheme-gray.css	Contains style definitions created to override styles in the gray Ext JS theme.
ext-all-notheme.css	Contains default styles for Ext JS components.
ext-xtheme-blue.css	Contains style definitions for Ext JS components with a blue color theme.
ext-xtheme-gray.css	Contains style definitions for Ext JS components with a gray color theme.
forms.css	Contains styles that define how EXAMPLE Express®(interview pages)are displayed in the browser. Two standard sets of form layouts exist in the base installation of EXAMPLE Express®: vertical and horizontal (across and down).
hyperlinks.css	Contains default styles for hyperlinks.
leftNav.css	Contains styles controlling left navigation for ManuScript pages. This file is the default file used to control ManuScript page navigation.
leftNavFloating.css	Contains styles controlling the floating navigation panel. This file is only used if floating navigation is enabled.
main.css	Contains styles for many basic HTML elements, such as the body, tables, inputs, etc.
nav.css	Contains styles controlling application-level navigation.
popUp.css	Contains styles controlling the layout of pop-up window contents.
reset.css	Contains style definitions that override those defined by the browser vendor for common HTML elements. By resetting these definitions to a common baseline, fewer cross-browser incompatibility issues occur.  This file should not be modified.
style.css	Contains styles for visual elements that appear on pages (banners, footers, tabs, font sizes).
topNav.css	Contains styles controlling top navigation for ManuScript pages. This file is used only if specified for a PageSet in EXAMPLE Author®.

x_[pageName].css	Contains styles for the page using the [pageName].xsl file. To determine which XSL file a page is using, click the "Current Information" button on the developer toolbar and look for the "Active XSL" page name. This information can also be found by searching for the JavaScript variable "sCurrentPage".
x_interview.css	Contains styles applied to a ManuScript page through EXAMPLE Author®. This file is intended to be modified by ManuScript developers.
x_interview_main.css	Contains styles applied to a ManuScript page through the XSL. This file is intended to be modified by skins developers.

HTML Div Structure

By default, EXAMPLE Express® pages include the following container divs:

- **wrapper** – Controls the overall page width
- **banner** – Contains branding and top navigation
  - **logo** – Contains the logo
  - **productNavigationDiv** – Contains product-level navigation
  - **quickSearch** – Contains the quick search
  - **actionGroup** – Contains application-level navigation
  - **specialactions** – Contains the New Quote feature
- **formContent** – Controls page content between the banner and footer
  - **leftPanelArea** – Contains content of the left panel
    - **policyActiveSection** – Contains information on the active policy, if an active policy exists
    - **subActionGroup** – Contains the sub navigation for pages (if in interview, contains navigation among ManuScript pages)
    - **relatedActions** – Contains policy-related actions if a policy is active
    - **logonActions** – Contains login options (**Change Password**)
  - **main** – Contains content of the page area between banner and footer to the right of the left panel
    - **header** – Contains the page header
    - **pageTop** – Contains page title (pageTitle) and instructional text (pageInst)
- **footer** — Contains the page footer

Customizing XSL

When customizing XSL files, follow the guidelines below.

Interview Pages

When customizing interview (ManuScript-driven) pages:

- Modify the **interview.xsl** file and follow the guidelines found in *Overriding DCTBase Files*.

System Pages

When customizing or creating system pages, follow the sequence of template calls below for building a page.

1. Include a template that matches the root of the content for the page.
2. Within this template, start the process of building the container by calling the *buildContainer* template in the dctContainer.xsl file (located in the *Global Root Directory*\Express\Skins\DCTBase\xsl\ directory).
3. Set up the following templates. (These templates will be called by the process begun in Step 2.)
   - **processPageHeader** – Controls the header content for the page.
   - **buildLeftPanelContent** – Controls the content of the left panel area of the container.
   - **buildPageContent** – Controls the main page content.
4. (Optional) In the *buildLeftPanelConten*t template, call any of the following component templates:
   - **buildPolicyLeftNavHeader** – Controls the header of the left panel.
   - **createSubMenuArea** – Renders the left navigation (such as the sub navigation for certain pages).
   - **buildPolicyRelatedTasks** – Controls the Policy Actions in the left panel.
   - **buildPolicyLogonItems** – Controls the Login Options in the left panel.

Interview Page Styles

Avant Garde 2.1 skins control interview page styling using the following two CSS files:

- **x_interview** – Contains styles applied to a page through EXAMPLE Author®.
- **x_interview_main** – Contains styles applied to interview pages through XSL.

Overriding DCTBase Files

Customizing skins sometimes requires that you override files in the DCTBase directory. To do this, you must create a custom skins directory, copy the desired templates or functions from the DCTBase files to that directory, and then make your customizations. Files in the custom skins directory are called after files in the DCTCore directory and therefore take precedence. For information about creating a custom skins directory, see *Creating a Custom Skins Directory*.

When customizing EXAMPLE Express® skins, do not modify any files in the DCTBase or AvantGarde2 directories. All files in these directories are overwritten during upgrades. Instead, you must create a custom skins directory (copied from the AvantGarde2 directory) and modify the files within it.

Overriding XSL Files

To override DCTBase .xsl files:

1. In the Content or Common folders in the `C:Program Files\Duck Creek Technologies\Skins\DCTBase\Core\xsl\` directory, locate the file containing the template you wish to override.
2. Copy the file containing the template you wish to override to the xsl\common or xsl\content folders in the custom skins directory, and ensure that its name remains the same as it is in the DCTBase directory.
3. In the custom skins file (the one you copied in Step 2), remove all templates you do not wish to override as well as all non-template elements (other than the `<xsl:stylesheet>`) tags, such as `<xsl:import>`, `<xsl:output>`, `<xsl:variable>`, etc.
4. Customize the templates you wish to override.

Overriding JavaScript Files

To override DCTBase .js files:

1. Locate the file containing the JavaScript function you wish to override in one of the following directories:
   - DCTBase\Core\devscripts\Billing
   - DCTBase\Core\devscripts\Policy
   - DCTBase\Core\devscripts\common
2. Copy the file containing the function you wish to override to the scripts folder in the custom skins directory and give it the same name as in the DCTBase directory.
3. In the custom skin file, remove all functions you do not wish to override.
4. Customize the function you wish to override.

Using the Localization Framework

Using the localization framework in the EXAMPLE Platform®, you can configure the EXAMPLE Platform® for use with multiple languages. Depending on your installation of the EXAMPLE Platform®, you may have to localize any of the following by using lookup tables:

- *XSLT*
- *JavaScript*
- *Images*

In these sources, lookups should be performed instead of hard-coding the strings so that the system can react to the current language.

XSLT

To assist in looking up a string, you can use the dctLocalizedDirectory.xsl template found in one of the following directories (depending on the skin you are using):

- `Skins\DCTCore\xsl\` (AvantGarde)
- `Skins\DCTBase\Core\xsl\common\` (AvantGarde2)

For example, to use one of the lookup templates:

- In the dashboardHome.xsl file in the `Skins\DCTBase\Core\xsl\content\` directory, locate the XSLT code that resembles the following code sample, and adjust accordingly:

```
<div id="pageInstruction">
      <xsl:text> Welcome to the Policy Administration Dashboard, </xsl:text>
      <xsl:value-of select="/page/state/user/fullName"/>
      <xsl:text>. </xsl:text>
</div>
```

All base .xsl files in the EXAMPLE Platform® (including the dashboardHome.xsl file) have been localized. Files mentioned on this page are referred to as examples only.

Localizing Custom XSL

If you wish to use a lookup template with your customized XSL, you must convert the code from a hard-coded string to a localized lookup.

To convert a hard-coded XSL string to a localized lookup (using the example from above):

1. Register the string in a globalization dictionary.
    a. Edit the DefaultDictionary.resources file in the `Skins\AvantGarde2\xsl\localization\` directory. For information about editing resource files, see *Editing Resource Files*.
    b. Add a new string named WELCOME_PAS_DB and whose value is *Welcome to the Policy Administration Dashboard*.
2. Configure the skins to use the new string in the globalization dictionary.
3. Replace the xsl:text XML that contains “Welcome to the Policy Administration Dashboard,” with a lookup. For example:
4. Verify that the EXAMPLE Express® home page says “Welcome to the Policy Administration Dashboard, Express Admin”.
5. In the `Skins\AvantGarde2\xsl\localization\` directory, create a copy of the DefaultDictionary.resources file and name it DefaultDictionary.fr.resources.
6. Edit the DefaultDictionary.fr.resources file you just created as desired. For information about editing resource files, see *Editing Resource Files*.
7. Add or edit the WELCOME_PAS_DB string, and give it a unique value such as “Welcome to the Policy Administration Dashboard,” in French.
8. Modify your browser language settings and verify that the Policy Administration dashboard now reads, "Welcome to the Policy Administration Dashboard," in French. For information about changing your browser language, see *Changing Browser Languages*.

JavaScript

JavaScript code uses the same resource files as XSLT code, but those resource files are available in a format that is natural to the JavaScript language. All strings stored in the resource file are converted into globally available string variables. For example, the DefaultDictionary.resources file gets converted at runtime to these lines of JavaScript code:

```
DCT.Strings.TEST = "Test";
DCT.Strings.WELCOME_PAS_DB = "Welcome to the Policy Administration Dashboard,";
```

As a result, hard-coded strings throughout the JavaScript code can be replaced with references to these string variables to achieve localization. The skins will automatically set the variable appropriately based on the language specified in the browser.

To make accessing string variables easier, you can use the T() function. This function, declared in EXAMPLE Express® skins, translates values by finding its value in the DCT.String namespace. For example:

```
T('TEST')
T('WELCOME PAS DB')
```

One benefit of using the T() function is that if the key is not present in the DCT.Strings namespace, it is returned as if it were the value.

The following table displays some examples of keys and the values returned by each.

Key	Value Returned
T('TEST') //	Test
DCT.Strings.TEST	Test
T('MISSING KEY') //	MISSING KEY
DCT.Strings.MISSING KEY	Undefined

To use these strings:

- Use the DCT.Strings namespace or the T() function as desired in any JavaScript in the skin.

The Avant Garde 2 skins are configured to make these string variables available on any page.

Localizing Custom JavaScript

The following example illustrates how to successfully localize your custom JavaScript.

Example:

Assume that a customer-specific business requirement requires that an alert check box which states, "Hello world!", displays in the appropriate language each time the user presses TAB to navigate to the next field in EXAMPLE Express®.

To implement this change:

1. In the Skins\AvantGarde2\xsl\localization\ directory, modify the DefaultDictionary.resources file by adding a new BLUR_EVENT_FIRED string with a value of *Hello World*. For more information about editing resource files, see *Editing Resource Files*.
2. (optional) Register the string in a second resource file for another language by creating a new DefaultDictionary.es.resources file (copied from the DefaultDictionary.resources file) and then editing the BLUR_EVENT_FIRED string with a value of *Hola mundo!*
3. Configure EXAMPLE Express® skins to use the newly-created string in the globalization dictionary by adding the following line of code to the customOnBlur function:
   alert(T('BLUR_EVENT_FIRED'));

To test the change above:

1. In EXAMPLE Express®, load a quote, leave a required field blank, and attempt to navigate to the next field by pressing TAB.
2. Verify that a message saying, "Hello world!"" or "Hola mundo!" (depeinding on the current browser culture) appears.
   When no resource file is present, the language defaults to the invariant culture.

Testing EXAMPLE Express® with Another Language

To test EXAMPLE Express® with another language:

1. In the `Skins\AvantGarde2\xsl\localization\` directory, copy the DefaultDictionary.fr.resources file.

2. Edit the copy of the DefaultDictionary.resources file you just created. For information about editing resource files, see Editing Resource Files.
3. Add or edit the REQD_FIELD_NOT_BLANK string, and give it a unique value such as “Ceci est un champ obligatoire et ne peut pas être blanc." in French.
4. Modify your browser language settings and verify that the dashboard now reads “Ceci est un champ obligatoire et ne peut pas être blanc.” in French. For information about changing browser languages, see *Changing Browser Languages*.

Images

Some images in the EXAMPLE Express® skins contain text, and will need to be localized if being used with a language other than English. For example, assume that a business requirement exists to display a different logout icon for French users in EXAMPLE Express®.

Using the example above, adhere to the following steps to replace the padlock image with another image:

1. Localize the image found at `Skins/AvantGarde2/images/icons/lock_open.png` for the French culture (fr-FR).
2. Locate the desired image file, and save it as lock_open.png in the `Skins/AvantGarde2/images/icons/fr-FR/` directory.

   Localized image files can be saved as .jpg, .gif, and .png files.

3. Change your browser language settings and then verify in EXAMPLE Express® that the changes have been implemented. For information about changing browser language settings, see *Changing Browser Languages*.

Changing Browser Languages

Localizing EXAMPLE Express® skins requires that you change your browser language settings. You can change your browser language settings by:

- Changing the language in your Web browser. For more information, see *Changing Your Browser Language in a Web Browser*.
- Modifying the value of the appSettings configuration setting in the Express\web.config file. For more information, see *Changing Your Browser Language Using a Configuration Setting*.

  Using a configuration setting to modify your browser language settings is recommended when changing the browser language on multiple machines at once.

Changing Your Browser Language in a Web Browser

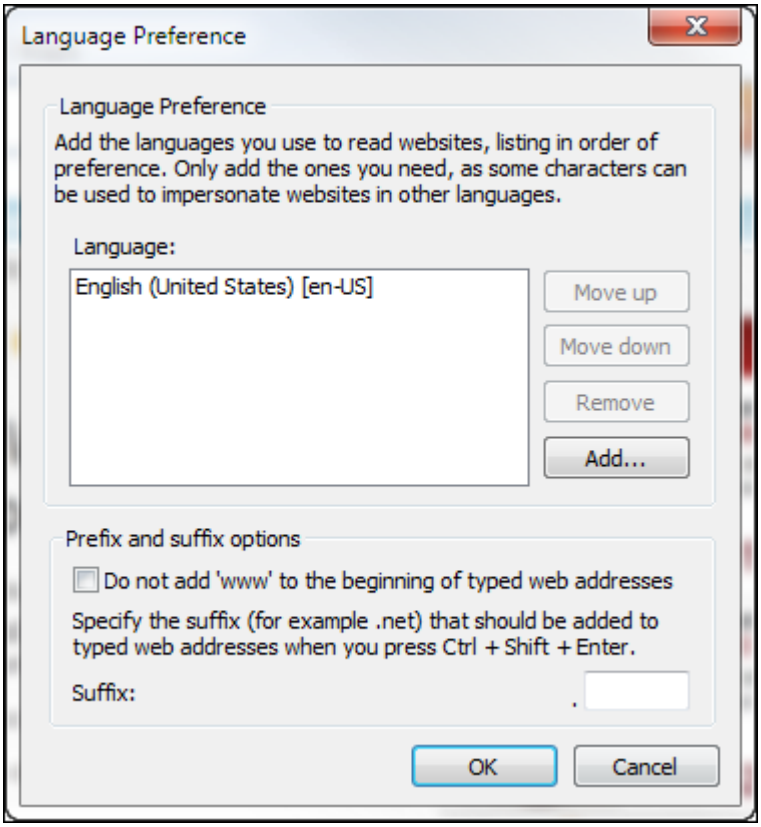
Using a Web browser, you can change the language to be used by the EXAMPLE Platform®.

Changing Your Browser Language in Internet Explorer

To change your browser language in Internet Explorer:

1. On the **Tools** menu, click **Internet Options**.

2. On the **General Tab**, click **Languages**.
   The Languages dialog box appears.



3. In the Language Preference dialog box, click **Add**.
   The Add Language dialog box appears.
4. In the Add Language dialog box, select a language from the list, and then click **OK**.

> You can change the order of preference of the browser languages by selecting a language and then clicking **Move Up** or **Move Down**. To remove a language from the list, select the language you wish to remove, and then click **Remove**.

5. Repeat steps four and five until you have added all of the languages you wish to use.
6. Click **OK** twice.

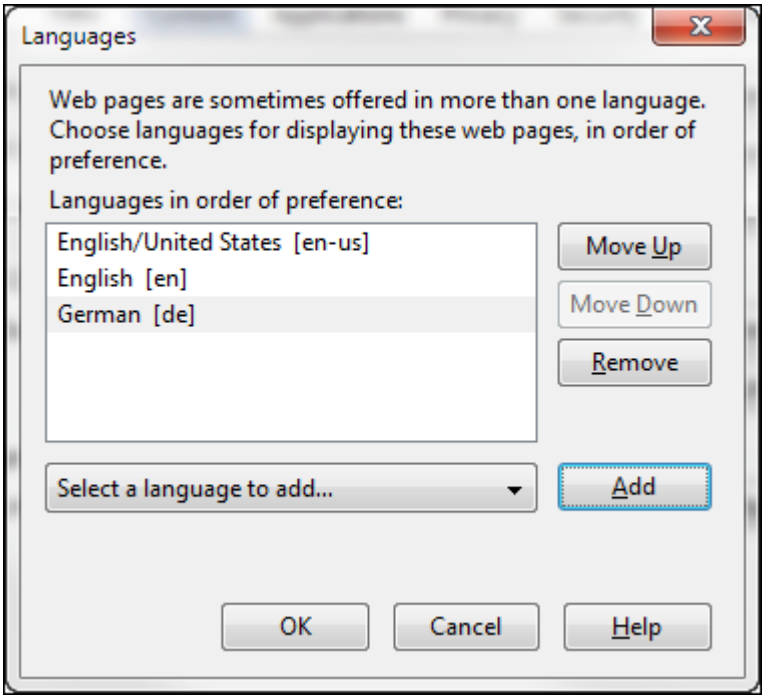
> The expected behavior of the platform is to use the first language listed. If dictionaries are not available for that language, the system will default back to English (the invariant culture).

Changing Your Browser Language in Mozilla Firefox

To change your browser language in Mozilla Firefox:

1. On the **Tools** menu, click **Options**.
   The Options dialog box appears.

2. On the **Content** tab, in the **Languages** section, click **Choose**.
   The Languages dialog box appears.



3. Select the language you wish to use from the **Select a langauge to add...** drop-down list, ant then click **Add**.
   The language is added to the list of preferred languages.

> You can change the order of preference of the browser languages by selecting a language and then clicking **Move Up** or **Move Down**. To remove a language from the list, select the language you wish to remove, and then click **Remove**.

4. Repeat step three until you have added all of the languages you wish to use.
5. Click **OK** twice.

> The expected behavior of the EXAMPLE Platform® is to use the first language listed. If dictionaries are not available for that language, the system will default back to English (the invariant culture).

Changing Your Browser Language Settings Using a Configuration Setting

Using a configuration setting, you can change the language to be used by the EXAMPLE Platform®.

To change the browser language to be used by the EXAMPLE Platform® using a configuration setting:

- In the Express\Web.config file, change the value of the DefaultCulture attribute for the appSettings configuration setting to the desired language. For example:
  `<add key="DefaultCulture" value="en-US"/>.`

Resource Files

With regard to the EXAMPLE Platform®, lookup tables refer to resource files to determine the appropriate text for a given culture or language. As of EXAMPLE Platform® 4.2.2, EXAMPLE Express® skins now ship with an `xsl\localization\` directory that contains resource files. These files serve as dictionaries of strings that are accessed by the XSLT through the templates in the dctLocalizedDictionary.xsl file. The .NET framework currently supports the following three culture types for use with localization:

Culture Type	File Name	Purpose
Invariant Culture	DefaultDictionary.resources	Default text for all strings
Neutral Culture	• DefaultDictionary.en.resources • DefaultDictionary.fr.resources	• English overrides • French overrides
Specific Culture	• DefaultDictionary.en-US.resources • DefaultDictionary.fr-FR.resources	• US specific English overrides • French specific overrides

Editing Resource Files

Using the EXAMPLE Platform®, you can edit resource files to control the text to be used for a specific culture or language.

Because .resx files are XML-based, .resources files should be converted to .resx files when editing them. To do this, use a tool such as ResGen.exe, which ships with the Microsoft Windows SDK.

Resource files can also be easily converted between the two formats from a command line. For example:

- Convert DefaultDictionary.resources to DefaultDictionary.resx (for editing)
- ResGen.exe DefaultDictionary.resources DefaultDictionary.resx
- Convert DefaultDictionary.resx back to DefaultDictionary.resources (for runtime use)
- ResGen.exe DefaultDictionary.resx DefaultDictionary.resources

Because it is overwritten during installation, It is recommended that you do not modify the DefaultDictionary.resources file (or any other file shipped in the AvantGarde or AvantGarde2 directories). Instead, it is recommended that you create a custom skin and modify its DefaultDictionary.resources file.

.resx files must be converted back to .resources format prior to being used in a runtime environment.

Using EXAMPLE Express® System ManuScripts

The content, appearance, and behavior of certain system pages can be customized by using customized EXAMPLE ManuScripts™. The following EXAMPLE Express® system pages can use a customized ManuScript:

- **New Quote Selection** – Replaces the the XSL versions of the New Quote Selection page (new.xsl, newBasic.xsl, and newExtended.xsl). See *Configuring a New Quote Selection ManuScript*.
- **Client Details** – No XSL equivalent. See *Configuring a Client Detail ManuScript*.
- **Send a Notification/Notification Details** – Replaces the XSL versions of the Send a Notification and the Notification Details pages (messageNew.xsl and message.xsl). When using the Message Detail ManuScript, the system also uses the messageDetail.xsl page to render part of the page content. See *Configuring a Message Detail ManuScript*.
- **New Task/Task Details** – No XSL equivalent. Uses the Task Detail ManuScript shipped with EXAMPLE Server® and the taskDetail.xsl page. See *Customizing the Task Detail ManuScript*.

For a description of each of the above system ManuScripts, see *System ManuScripts*.

Configuring a New Quote Selection ManuScript

To configure EXAMPLE Express® to use a custom ManuScript for the New Quote Selection page:

1. Use EXAMPLE Author® to create the ManuScript you wish to use for the New Quote Selection page. For instructions on creating ManuScripts in EXAMPLE Author®, see the *Guide to EXAMPLE Author®*.

When customizing the New Quote Selection ManuScript pages, do not modify the DuckCreekTech_NewQuoteSelectionManuScript (located in the *Global Root Directory*\ManuScripts\Admin\US directory). This ManuScript will be overwritten during upgrade installations. Modifications should be made at the carrier-level.

An out-of-the-box New Quote Selection ManuScript (NewQuoteSelectionPortfolio.xml) is located in the *Global Root Directory*\ManuScripts\Admin\NewQuote directory. If desired, you can modify this ManuScript to meet your individual needs for working at the carrier-level.

2. Open the Edit Configuration Utility.
3. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
4. Under **Mode**, click **Form**.
5. On the **Web Config** tab, in the list of settings, click the **NewQuoteManuScriptID**.
6. In the **value** text box, enter the ManuScript ID of the ManuScript to use.
7. Click **Save**.
8. Click **Close**.

If desired, you can configure unique New Quote Selection pages for different types of users. To do so:

1. In the New Quote Selection ManuScript, create a unique PageSet and page for each New Quote Selection page desired.

2. In the ContentMap.xml file (or any custom ContentMap file), add a <content> entry for each desired New Quote Selection page. Perform one of the following actions:
   - Within the class for each <content> entry, specify the PageSet and page to use for that content page.
   - Configure the system to post the following HTML form fields to the New content page:
     - **_ManuscriptPageSet** — PageSet containing the page to use for the New Quote Selection page.
     - **_ManuscriptPage** — Page to use for the New Quote Selection page.

   If you do not specify a PageSet and page, the system will automatically use the first PageSet and page in the specified New Quote Selection ManuScript.

3. In the ApplicationLayoutMap.xml file (or any custom ApplicationLayoutMap files), specify the desired new quote content page in each <applicationLayout> element.

The out-of-the-box New Quote Selection ManuScript offers a *basic* and an *extended* page. Though these pages are identical out of the box, you can customize these pages if the New Quote page needs to function differently for different roles.

Configuring a Client Detail ManuScript

To configure EXAMPLE Express® to use a custom ManuScript for the Client Details page:

1. Use EXAMPLE Author® to create the ManuScript you wish to use for the Client Details page. For instructions on creating ManuScripts in EXAMPLE Author®, see the *Guide to EXAMPLE Author®*.
2. Open the Edit Configuration Utility.
3. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
4. Under **Mode**, click **Form**.
5. On the **Web Config** tab, in the list of settings, click the **ClientDetailsManuScriptID**.
6. In the **value** text box, enter the ManuScript ID of the ManuScript to use.
7. Click **Save**.
8. Click **Close**.

Configuring a Message Detail ManuScript

To configure EXAMPLE Express® to use a customized ManuScript for the Send a Notification and Notification Details pages:

1. Use EXAMPLE Author® to create the ManuScript you wish to use for the Send a Notification and Notification Details pages. For instructions on creating ManuScripts in EXAMPLE Author®, see the *Guide to EXAMPLE Author®*.
2. Open the Edit Configuration Utility.
3. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
4. Under **Mode**, click **Form**.
5. On the **Web Config** tab, in the list of settings, click the **MessageDetailsManuScriptID**.
6. In the **value** text box, enter the ManuScript ID of the ManuScript to use.

7. Click **Save**.
8. Click **Close**.

When using the same Message Detail ManuScript with multiple product ManuScripts, the Message Detail ManuScript and all product ManuScripts must share the same structure. Make sure that all product ManuScripts share a common mapping for any information used by the Message Detail ManuScript. There is no system-defined schema for this shared information, but each implementation must use the same schema paths for the data to be accessed.

Customizing the Task Detail ManuScript

When customizing the New Task and Task Details pages, you can customize the out-of-the-box Task Detail ManuScript or configure EXAMPLE Express® to use a custom Task Detail ManuScript.

If customizing the New Task and Task Details pages, do not modify the DuckCreekTech_TaskDetailManuScript (located in the *Global Root Directory*\ManuScripts\Admin\US directory). Changes made in this ManuScript will be overwritten during upgrade installations. Modify the Carrer_TaskDetailManuScript (located in the *Global Root Directory*\ManuScripts\CarrierAdmin\US directory) instead. For additional information, see *Customizing the Task Detail ManuScript* in *EXAMPLE Express® Reference Guide*.

Customizing the Out-of-the-Box Task Detail ManuScript

To customize the out-of-the-box Task Detail ManuScript:

- In EXAMPLE Author® customize the **Carrier_TaskDetailManuScript** as desired. For instructions on creating ManuScripts in EXAMPLE Author®, see the *Guide to EXAMPLE Author®*.

Using a Custom Task Detail ManuScript

To configure EXAMPLE Express® to use a custom Task Detail ManuScript:

1. Use EXAMPLE Author® to create the ManuScript you wish to use for the New Task and Task Details pages. For instructions on creating ManuScripts in EXAMPLE Author®, see the *Guide to EXAMPLE Author®*.

An out-of-the-box Carrier Task Detail ManuScript (Carrier_TaskDetailManuScript.xml) is located in the *Global Root Directory*\ManuScripts\CarrierAdmin\US directory. If desired, you can modify this ManuScript to contain the desired logic.

2. Open the Edit Configuration Utility.
3. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
4. Under **Mode**, click **Form**.
5. On the **Web Config** tab, in the list of settings, click the **TaskDetailManuScriptID**.
6. In the **value** text box, enter the ManuScript ID of the ManuScript to use.
7. Click **Save**.
8. Click **Close**.

Customizing Interview (ManuScript) Pages

Carriers can control the appearance of interview (ManuScript) pages by defining the content and layout of each page in EXAMPLE Author®. In addition, carriers can further control the appearance of these pages through custom styles and format masks.

For instructions on creating and using custom styles in ManuScript pages, see *Customizing CSS*, and see *Working with CSS Override Styles* in the *Guide to EXAMPLE Author®*.

For instructions on using format masks, see *Format Masks* and *Setting Field Format Masks*.

Adding Custom Pages

Custom content pages can be added to EXAMPLE Express® by using the Edit Configuration Utility to extend the ContentMap.xml file.

Do not customize the shipped ContentMap files. Instead, make customizations in custom ContentMap files to prevent changes from being lost during the upgrade process.

You must reset EXAMPLE Express® (run DCTReset.exe) for any customizations to OnDemand files (PostActionMap.xml, ContentMap.xml, and ApplicationLayoutMap.xml) to appear.

Creating Custom ContentMap Files

The ContentMap.xml file is configured so that a particular implementation of EXAMPLE Express® can have multiple ContentMap.xml files at the path specified by the *ContentFactory* configuration setting in the Express\Web.config file. If desired, you can create a custom ContentMap file in addition to ContentMap.xml. At runtime, the system will use the following files at the path specified by the *ContentFactory* configuration setting. (For each of the following files, [MapFile] is the file name specified in the *ContentFactory* configuration setting.)

- [MapFile].xml
- [MapFile]*.xml
- Custom[MapFile].xml

Custom map files should adhere to the following naming convention: *Custom[MapFile].xml*, where [MapFile] is the text specified in the appropriate Express\Web.config setting (e.g., ContentMap, PostActionMap, ApplicationLayoutMap). The *[MapFile]*.xml* convention is reserved for Duck Creek Technologies, Inc. use only. If you have migrated your custom map files from a previous version of the EXAMPLE Platform® , Duck Creek Technologies, Inc. recommends that you apply the *Custom[MapFile].xml* naming convention to your custom files.

Because the EXAMPLE Platform® combines all files that use the above naming conventions, keep the following tips in mind when using a custom ContentMap file:

- Be sure no conflicts exist among any items defined in any ContentMap file.

- If you create backup copies of ContentMap files, move those files to a different directory, or do not include the specified text in the file name.
- The system loads map files in the following order:
  - [MapFile].xml
  - [MapFile]*.xml
  - Custom[MapFile].xml

Customizing ContentMap Files

To create a new content page, you must:

1. Add the new class to a class library to build the custom XML content. For more information about adding a new class, see *Adding a New Class* (below).
2. In the Global Root Directory\Express\Skins\CustomSkin\xsl directory, create an .xsl file to handle the content of the page.
3. Publish the new content to the ContentMap.xml file. For a description of the attributes of each <content> entry, see *<content> Attributes* (below).
4. (Optional) Extend the ApplicationLayoutMap.xml file to contain the new content item. For information about customizing this file, see *Customizing ApplicationLayoutMap.xml*.

Adding a New Class

The base class for any content page is based on the following class. Each derived content page overrides the Process() method to build the XML content for the page:

```
namespace DuckCreek.OnDemand
{
  /// <summary>
  /// Base Class for each content page in the OnDemand framework
  /// </summary>
  public class ContentPage
  {
    public ContentPage()
    {
    }
    public virtual void Process(PageState state,
System.Collections.Specialized.NameValueCollection requestForm)
    {
    }
    public virtual string MenuName(ContentFactory factory, string ContentName)
    {
      // default will get menu name from the factory map file, derived classes may
override for special behavior.
      XmlElement Elem = factory.GetEntry(ContentName);
      return (Elem == null)? "" : Elem.GetAttribute("menu");
    }
  }
}
```

The following is an example of a new class in a class library. The class should be derived from the ContentPage and implement an override to the Process() method that creates the XML content for the page. In this example, the

assembly is the DLL file name of the class library in which it resides. The type name of the above example would be *typeName="MyCompany.Express.SpecialContent"*.

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Xml;
using DuckCreek.OnDemand;

// namespace is used to publish the class name to the content factory.
namespace MyCompany.Express
{
  // class name is used to publish the class name to the content factory.
  public class SpecialContent: ContentPage
  {
  public override void Process(PageState state,
System.Collections.Specialized.NameValueCollection requestForm)
  {
    // get <content> node from ResponseDoc (XmlDocument) to place XML on this page
    XmlNode content = state.ResponseDoc.DocumentElement.SelectSingleNode("content");
    // TODO: build your custom content here
    XmlDocument MyContent = new XmlDocument();
    MyContent.LoadXml("<myContent><todo>Build your content here</todo></myContent>");
    // insert into the <content> for this page (may be repeated if more content
needed)
    content.AppendChild(content.OwnerDocument.ImportNode(MyContent.DocumentElement,
true));
    }
  }
}
```

<content> Attributes

The following table describes the attributes of each <content> entry in the ContentMap.xml file.

Node	Description
name	Unique name for the content page. This name should also be the name of an .xsl file in the *Global Root Directory*\Express\Skins*CustomSkin*\xsl folder.
assembly	Assembly DLL name. The assembly should reside in the directory defined in the *runtimeRoot* configuration setting in the Server.Config file.
typeName	Class name in the assembly that is the ContentPage-based class for the provided overridden Process() method.
menu	ID of the menu in the ApplicationLayoutMap.xml file (or any custom ApplicationLayoutMap file) that defines which menu to display with the content page. If the value of this attribute is null, the system will use the menu defined by the active portal.

Adding or Customizing Modules

You can customize the out-of-the-box modules available for the Policy Administration Dashboard (or any other page), or create new ones.

Customizing an Out-of-the-Box Module

You can customize an out-of-the-box module by overriding it with a customized version of the module in the dctCustomModules.xsl file.

To override an out-of-the-box module:

1. In the *Global Root Directory*\Express\Skins\DCTCore\xsl directory, open **dctDashboard.xsl**.
2. Copy the XSL template with the *match* attribute that matches the ID of the module you wish to override:
   - **New quote module** — newQuote
   - **Recently Accessed module** — mruQuote
   - **Notification module** — notifications
   - **Task module** — taskQueue
3. In the *Global Root Directory*\Express\Skins*CustomSkin*\xsl directory, open **dctCustomModules.xsl.**
4. Paste the copied template into **dctCustomModules.xsl.**
5. Make any desired changes to the module.
6. Save and close **dctCustomModules.xsl.**

Creating a New Module

To create a custom module:

1. In the *Global Root Directory*\Express\Skins*CustomSkin*\xsl directory, open **dctCustomModules.xsl**.
2. Add an XSL template with a *match* attribute that looks for a specific <module> element by ID. Specify a unique ID. (This ID will be assigned to the module during its creation in Step 4.)
   For example: `match="module[@id='myID']"`
3. Set the *mode* attribute on the template to *content*.
4. Add a element as a child of the XSL template, and give it an *id* attribute with the value of the ID specified in Step 2.
5. Add a *class* attribute with a value of *x-hide-display*.
   (This allows the ExtJS objects in the dctDashboard.js file to move the content to the correct location based on the page layout definition without requiring it to initially load elsewhere.)
6. Add custom XSL code to the element to create the module content.
7. Save and close **dctCustomModules.xsl.**
8. In the *Global Root Directory*\Express\App_Data directory, open **ApplicationLayoutMap.xml**.
9. Configure any desired <pageLayout> elements to specify the new module by ID. Give the module specified a *content* attribute with the identifier for a content page that will load the required content.

Using the ApplicationLayoutMap.xml file, you can add external content to a module. For information about using external content with EXAMPLE Express®, see *Using External Content in EXAMPLE Express®*.

10. Save and close **ApplicationLayoutMap.xml**.

Adding Custom Express Actions

You can add custom Express Actions to EXAMPLE Express®by using the Edit Configuration Utility to extend the PostActionMap.xml file.

Do not customize the shipped PostActionMap files. Instead, make customizations in custom PostActionMap files to prevent changes from being lost during the upgrade process.

You must reset EXAMPLE Express® (run DCTReset.exe) for any customizations to OnDemand files (PostActionMap.xml, ContentMap.xml, and ApplicationLayoutMap.xml) to appear.

Creating Custom PostActionMap Files

The PostActionMap.xml file is configured so that a particular implementation of EXAMPLE Express® can have multiple PostActionMap.xml files at the path specified by the *PostActionFactory* configuration setting in the Express\Web.config file. If desired, you can create a custom PostActionMap file in addition to PostActionMap.xml. At runtime, the system will use the following files at the path specified by the *PostActionFactory* configuration setting. (For each of the following files, [MapFile] is the file name specified in the *PostActionFactory* configuration setting.)

- [MapFile].xml
- [MapFile]*.xml
- Custom[MapFile].xml

Custom map files should adhere to the following naming convention: *Custom[MapFile].xml*, where [MapFile] is the text specified in the appropriate Express\Web.config setting (e.g., ContentMap, PostActionMap, ApplicationLayoutMap). The *[MapFile]*.xml* convention is reserved for Duck Creek Technologies, Inc. use only. If you have migrated your custom map files from a previous version of the EXAMPLE Platform® , Duck Creek Technologies, Inc. recommends that you apply the *Custom[MapFile].xml* naming convention to your custom files.

Because the EXAMPLE Platform® combines all files that use the above naming conventions, keep the following tips in mind when using a custom PostActionMap file:

- Be sure no conflicts exist among any items defined in any PostActionMap file.
- If you create backup copies of PostActionMap files, move those files to a different directory, or do not include the specified text in the file name.
- The system loads map files in the following order:

- [MapFile].xml
- [MapFile]*.xml
- Custom[MapFile].xml

Customizing PostActionMap Files

To create a new Express Action, you must:

1. Add the new class to a class library that performs the action. For more information about adding a new class, see *Adding a New Class* (below).
2. (Optional) Modify your custom post action to force navigation to a specific content page.
3. Publish the new post action to the PostActionMap.xml file. For a description of the attributes of each <action> entry, see *<action> Attributes* (below).
4. (Optional) If the action requires a file download, configure or override the *ResponseCode*, and override the three download methods. For more information, see *Configuring File Download* (below).

Adding a New Class

The base class for any Express Action is based on the following class. Each derived action overrides the Process() method to build the XML content for the page.

```
namespace DuckCreek.OnDemand
{
  /// <summary>
  /// Each post-back action will have a PostAction-derived class for the activities
  /// </summary>
  public class PostAction
  public const string
    POSTACTIONRESPONSE_NORMAL = "",
    POSTACTIONRESPONSE_DOWNLOAD = "download";
    public PostAction()
    {
    }
    public virtual void Process(PageState state,
System.Collections.Specialized.NameValueCollection requestForm)
    {
    }
    public virtual string ResponseCode(PostActionFactory factory, string ActionName)
    {
      // pull response code from the map file, derived classes may override
      XmlElement Elem = factory.GetEntry(ActionName);
      return (Elem == null) ? "" : Elem.GetAttribute("response");
    }
    public virtual string DownloadSourceFile
    {
      get { return ""; }
    }
    public virtual bool DownloadTempFile
    {
      // indicates to the OnDemand framework that the download file should be deleted.
      get { return true; }
```

```
    }
    public virtual string DownloadBaseFileName
    {
      get { return ""; }
    }
  }
}
```

<action> Attributes

The following table describes the attributes of each <action> entry in the PostActionMap.xml file.

Node	Description
name	Unique name for the action.
assembly	Assembly DLL name. The assembly should reside in the directory defined in the *runtimeRoot* configuration setting in the Server.Config file.
typeName	Class name in the assembly that is the PostAction-based class that provided the overridden Process() method.
response	Non-blank defines the enumeration of HTTP content-type and content-disposition ("", "download").

Configuring File Download

The following class methods can be overridden to create a content page that works with downloads by adding a new class to the OnDemand Framework.

Class Method	Description
ResponseCode	Allows the action to change the HTTP response (download).
DownloadSourceFile	URL to the file to be sent from the Web Server. If blank (""), then the entire state.ResponseDoc.OuterXml is sent as the file.
DownloadTempFile	Boolean value indicating that the DownloadSourceFile is a temporary file and should be deleted after the content is sent to the browser.
DownloadBaseFileName	Filename to send in HTTP response as default 'Save As' filename.

The following is an example of a new post action class that downloads a static file:

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Xml;
using DuckCreek.OnDemand;
namespace ABC.Express
{
  class DownloadDocument: PostAction // "ABCdownload" action processing
  {
    private string m_SourceURL;
```

```
    private string m_BaseFileName;
    public override void Process(PageState state,
System.Collections.Specialized.NameValueCollection requestForm)
    {
      try
      {
        // may need to access form fields to know what has been selected in Browser.
        // this example accesses a form field named _selectedFileID, but use
anything.
        string FileID = requestForm["_selectedFileID"];

        // just put sample data here for now
        m_SourceURL = "c:\\Temp\\SomeFile.txt";
        m_BaseFileName = "SomeOutpuFile.txt"; // can be any filename
      }
      catch (Exception e)
      {
        throw new Exception("Could not download attachment.", e);
      }
    }
    public override string DownloadSourceFile
    {
      // defines to the OnDemand framework where to find the file to send to the
browser
      get { return m_SourceURL; }
    }
    public override bool DownloadTempFile
    {
      // indicates to the OnDemand framework to delete file after download.
      get { return true; }
    }
    public override string DownloadBaseFileName
    {
      // indicates the filename to use when user is prompted for filename
      get { return m_BaseFileName; }
    }
  }
}
```

Customizing the EXAMPLE Express® Experience

The total EXAMPLE Express® experience can be customized by specifying which pages should be available in EXAMPLE Express® and what each page should contain using the ApplicationLayoutMap.xml file available with the OnDemand Framework. If desired, through the OnDemand Framework and EXAMPLE Express® skins files, a unique EXAMPLE Express® experience can be defined for each type of user that works in EXAMPLE Express®.

Customizing the EXAMPLE Express® experience requires modifications to the following items:

- **The ApplicationLayoutMap.xml file** — This file controls which navigation elements appear in EXAMPLE Express® for each type of user and what content appears on each page. If desired, this information can be

defined for each type of user in the system. For instructions on customizing the ApplicationLayoutMap.xml file, see *Customizing ApplicationLayoutMap.xml*.

- **EXAMPLE Express® Skins** — While security permissions defined in EXAMPLE UserAdmin™ control the EXAMPLE Server® permissions available to each type of user (e.g., whether a particular role can work with quotes and policies, and which quotes and policies that role can work with), the display of certain features in EXAMPLE Express® (e.g., the grid that displays the list of quotes and policies) is controlled through EXAMPLE Express® skins files. While roles and privileges in EXAMPLE UserAdmin™ control server-level permissions, EXAMPLE Express® skins control what interface elements appear on the screen. When customizing EXAMPLE Express®, you must modify EXAMPLE Express® skins to hide or display features as appropriate for each type of user. For instructions on customizing EXAMPLE Express® skins to display the desired features for each type of user, see *Customizing Skins Per Role*.

Customizing ApplicationLayoutMap.xml

The ApplicationLayoutMap.xml file controls which navigation elements appear in EXAMPLE Express® for each type of user and what content appears on each page. You can define this information for each type of user in the system by creating a custom ApplicationLayoutMap.xml file using the Edit Configuration Utility.

Any changes made to the shipped ApplicationLayoutMap.xml files are overridden during the upgrade process. All changes should be made in the appropriate custom ApplicationLayoutMap files.

You must reset EXAMPLE Express® (run DCTReset.exe) for any changes to appear in EXAMPLE Express®.

Creating Custom ApplicationLayoutMap Files

The ApplicationLayoutMap.xml file is configured so that a particular implementation of EXAMPLE Express® can have multiple ApplicationLayoutMap.xml files at the path specified by the *ApplicationLayoutFactory* configuration setting in the Express\Web.config file. If desired, you can create a custom ApplicationLayoutMap file in addition to ApplicationLayoutMap.xml. At runtime, the system will use the following files at the path specified by the *ApplicationLayoutFactory* configuration setting. (For each of the following files, [MapFile] is the file name specified in the *ApplicationLayoutFactory* configuration setting.)

- [MapFile].xml
- [MapFile]*.xml
- Custom[MapFile].xml

Custom map files should adhere to the following naming convention: *Custom[MapFile].xml*, where [MapFile] is the text specified in the appropriate Express\Web.config setting (e.g., ContentMap, PostActionMap, ApplicationLayoutMap). The *[MapFile]*.xml* convention is reserved for Duck Creek Technologies, Inc. use only. If you have migrated your custom map files from a previous version of the EXAMPLE Platform® , Duck Creek Technologies, Inc. recommends that you apply the *Custom[MapFile].xml* naming convention to your custom files.

Because the EXAMPLE Platform® combines all files that use the above naming conventions, keep the following tips in mind when using a custom ApplicationLayoutMap file:

- Be sure no conflicts exist among any items defined in any ApplicationLayoutMap file.

- For any <applicationLayout> elements in the ApplicationLayoutMap.xml file that you do not wish to use, set the *default* attribute to a value of *0*.
- If you create backup copies of ApplicationLayoutMap files, move those files to a different directory, or do not include the specified text in the file name.
- The system loads map files in the following order:
  - [MapFile].xml
  - [MapFile]*.xml
  - Custom[MapFile].xml

Customizing ApplicationLayoutMap Files

The ApplicationLayoutMap.xml file contains <applicationLayout> elements that define unique implementations of the EXAMPLE Express® interface. These <applicationLayout> elements contain the following child elements:

- **<menu> and <menuItem> elements** — These elements control the navigation elements that appear in EXAMPLE Express® and which content page each element loads.
- **<pageLayout> elements** — These elements control the content of dashboard pages in EXAMPLE Express®, such as the Policy Administration dashboard.

When customizing <applicationLayout> elements, you must define a separate <applicationLayout> element for each role or group of roles that will have a distinct EXAMPLE Express® configuration. To define these <applicationLayout> elements, you can modify the out-of-the-box <applicationLayout> elements in the ApplicationLayoutMap.xml file and—if desired—create new ones.

To modify or create an <applicationLayout> element, you must:

Step	Description
1	Define the attributes of the <applicationLayout> element.
2	Define the <menu> and <menuItem> elements for the <applicationLayout>.
3	Define any desired <pageLayout> elements.

For instructions on these three procedures, see:

- *Defining the Attributes of an <applicationLayout> Element*
- *Defining the <menu> and <menuItem> Elements*
- *Defining <pageLayout> Elements*

Custom XML can be defined for each application layout using the <customLayout> XML element. Whn debug mode is enabled (greater than 0), the custom XML is returned in the debugContent.xml file. For example:

```
</module>
     </pageLayout>
     <customLayout>
       <myElem>Hello World</myElem>
     </customLayout>
   </applicationLayout>
```

Defining the Attributes of an <applicationLayout> Element

Each <applicationLayout> element should reside under the <applicationLayoutRoot> element. When a user logs in, the system loads ApplicationLayoutMap files in the order specified above, combining the content of those files. The system then uses the last <applicationLayout> that has a default attribute of 1 and a role attribute containing any of the roles or contexts assigned to that user's home entity. If no matching <applicationLayout> has a default attribute of 1, the system uses the last <applicationLayout> with a role attribute containing any of the roles or contexts assigned to that user's home entity. Using the last matching <applicationLayout> enables custom <applicationLayout> elements to override out-of-the-box <applicationLayout> elements.

Use the table below to help you customize existing <applicationLayout> elements or create new ones.

When modifying the ApplicationLayoutMap.xml file or any custom ApplicationLayoutMap file, be sure that no two <applicationLayout> elements for the same application in any ApplicationLayoutMap file are applicable for the same role. If more than one <applicationLayout> element (for the same application) is defined for a single role, the system will not be able to determine which <applicationLayout> to use.

The <applicationLayout> element uses the following attributes.

Attribute	Required/ Optional	Description
id	Required	Unique ID for the <applicationLayout>.
caption	Optional	Caption of the <applicationLayout>.
application	Optional	Identity of the target application. Options are: • PAS • Billing • Party If *application="PAS"*, the autoSave function will occur for the policy.
roles	Required (if *excluded roles* is not specified)	Comma-delimited list of the roles and contexts (custom privileges) that should use the <applicationLayout>. Any user who has one or more of the specified roles or contexts will use this layout. Use an asterisk (*) to indicate all roles and contexts.
excluded roles	Required (if *roles* is not specified)	Comma-delimited list of the roles and contexts (custom privileges) that can NOT use the <applicationLayout>. Any user who does NOT have any of the specified roles or contexts will use this layout. If a *roles* attribute exists, this attribute's values will be ignored.

default	Optional	Boolean value indicating whether the <applicationLayout> should be the default application when the user first logs in. This attribute is applicable when a particular role has access to multiple <applicationLayout> elements (for different applications). A value of *1* will cause the system to default to this application when the user logs in.
image	Optional	Path to the image to use to denote the specific application in the system-level navigation. This path is relative to the skins directory. This attribute also specifies the image to be used as an icon on the left side of the menu item. Image icons can be found in the icons folder in the C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\images directory.
xsltDir	Optional	Relative path to the skins directory to use for the <applicationLayout>.
culture	Optional	Culture-code identifier that can be used to identify the intended culture of the given <applicationLayout>.

Defining the <menu> and <menuItem> Elements

Each <applicationLayout> element must contain a <menu> element that defines the <menuItems> that should appear in EXAMPLE Express®. Use the table below to help you customize existing <menu> and <menuItem> elements or create new ones.

You can configure EXAMPLE Express® to use multi-level navigation by nesting <menu> and <menuItem> elements within each other.

The <menu> Element

The <menu> element uses the following attributes.

Attribute	Required/ Optional	Description
name	Required	Unique ID for the <menu>.
startPage	Optional	Default start page. If not specified, the first <menuItem> element will be used.
logoutPage	Optional	Page to be displayed after logging out. If not specified, *contentLogin* will be used.
logoutURL	Optional	URL to be displayed after logging out. If specified, this attribute will override the *logoutPage* specified.

The <menuItem> Element

The <menuItem> element uses the following attributes.

Attribute	Required/ Optional	Description
id	Required	Unique ID for the <menuItem>.

caption	Optional	Text to display in EXAMPLE Express® for the menu item. If extending the caption of the New Quote <menuItem> beyond the size of the existing button image, you may need to customize the buttons.css and associated image files.
content	Optional	Name of the <pageLayout> or—if the specified <pageLayout> does not exist—the ContentMap.xml item to call. If blank, the current page will be used unless an action calls a new page.
action	Optional	Name of the PostActionMap.xml item to execute when the <menuItem> is selected.
java	Required	Name of the JavaScript method or action to perform (usually, *submitPage*). This attribute is used by the skin to determine what code to execute on the client/ JavaScript. The following parameters will be passed to the method: 1. **content** —The value of the *content* attribute (above) 2. **action** — The value of the *action* attribute (above)
location	Optional	Location for the menu item. Defaults to *header*. Options are: • **header** — In the header (where the application-level navigation appears in the out-of-the-box screens) • **special** — Right edge of the header (where the New Quote action appears in the out-of-the-box screens) • **extendedLeftNav** — In the Policy Actions list • **product** — Top edge of the application (where the system navigation appears in the out-of-the-box screens) • **logonOptions** — In the Login Options list Menu location can be changed by modifying the *location* attirbute only. You do not need to recreate CSS definitions and images.
consumer	Optional	Value indicating how the item should be treated if the user is using consumer access mode. Options are: • **0** — Hide if user is a consumer • **1** — Show to all users • **2** — Show only if user is a consumer
guest	Optional	Value indicating how the item should be treated if the user is using guest access mode. Options are: • **0** — Hide if user is a guest • **1** — Show to all users • **2** — Show only if user is a guest

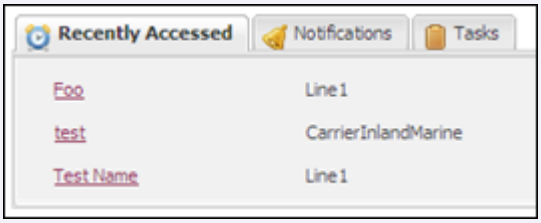
login	Optional	Boolean value indicating whether the menu item is valid only if the user is logged in. A value of *1* indicates the menu item is valid only if logged in.
edit	Optional	Value indicating whether to display the menu item based on session content. Options are: • **0** — Ignore session content • **1** — Hide when no session • **2** — Show when session exists • **3** — Show if session is "dirty"
security	Optional	Bit-mask (additive) value of the security levels for which the menu item is applicable. • **0** — All users • **1** — Agent • **2** — Underwriter • **4** — Carrier • **8** — Admin
multiAgency	Optional	Boolean value indicating whether the menu item is valid only when the user belongs to multiple agencies. A value of *1* indicates the menu item is valid only when the user belongs to multiple agencies.
hideInPortfolio	Optional	Boolean value indicating whether the menu item should be hidden when in a portfolio. A value of *1* indicates the menu item should be hidden when in a portfolio.

Defining <pageLayout> Elements

If desired, the <applicationLayout> element can contain <pageLayout> elements that define dashboard pages. These dashboard pages can be called by the *content* attribute of a <menuItem> element. Use the table below to help you customize existing <pageLayout> elements or create new ones.

The <pageLayout> element uses the following elements and attributes.

Attribute	Required/ Optional	Description
id	Required	Unique ID of the <pageLayout>.
<zone> (child of <pageLayout>)		
<column> (child of <zone>)		
width	Required	Decimal value from 0 to 1 that specifies the percentage of the page width that the column should consume.
<module> (child of <column>)		

id	Required	Unique ID of the module. Out of the box, options are: • newQuote • mruQuote • notifications • taskQueue Custom modules can be created by adding templates to the dctCustomModules.xsl file. For instructions on adding custom modules, see *Adding or Customizing Modules*.
title	Optional	Text that will display as the module title in EXAMPLE Express®.
content	Optional	Name of the content item to call. Out of the box, options are: • new • mru • messages • tasks
tabPanel	Optional	Displays modules in a tab panel. For example:  Valid values include: • **0** – Do not display the module in a tab panel. • **1** – Display the module in a tab panel.
simple	Optional	Renders a module without a frame or background. Valid values include: • **0** – Do not use a frame or background with the module. • **1** – Use a frame or background with the module.
collapsed	Optional	Renders a module collapsed when the page is loaded. Valid values include: • **0** – Do not collapse the module when the page is loaded. • **1** – Collapse the module when the page is loaded.
iconCls	Optional	Specifies the class to use to load the icon that will be displayed on a tab panel. Out of the box, the EXAMPLE Platform® includes some panel icon classes, defined at the bottom of the style.css file.
<attribute> (child of <module>)		
name	Required	Name of the form variable to be applied to the module. Varies per module. (See table below for more detail.)

value	Required	Value of the form variable to be applied to the module. Varies per module. (See table below for more detail.)

The following form variables are available for the out-of-the-box modules.

Variable	Description	Valid Values
newQuote		
_ManuScriptPageSet	PageSet containing the page to use for the New Quote module.	Name of a PageSet in the ManuScript specified by the *NewQuoteManuScriptID* setting in the Express\Web.config file.
_ManuScriptPage	Page to use for the New Quote module.	Name of a page in the PageSet defined by the _ManuScriptPageSet variable.
mruQuote		
pageSize	Number of quotes/policies to display.	Integer
notifications		
displayCount	Number of notifications to display.	Integer
showBody	Boolean value indicating whether to show the body of the notification in the grid control.	• **0** — Do not show body text • **1** — Show body text
callbackForList	Boolean value indicating whether to generate list items through an AJAX callback.	• **0** — Do not generate list through an AJAX callback • **1** — Generate list through an AJAX callback
taskQueue		
_queryFilterSubset	Subset prefix name for the keyValue, keyName, keyConnector, and keyOp form fields that control the filtering of list content. Also specifies the SearchType.	Options are: • QueueAssigned • QueueUnassigned • QueueAdmin
start	Record number to start the list from. Usually *1*.	Integer
limit	Maximum number of records to display.	Integer

_keynameQueueAssigned	Comma-delimited list of key fields to filter by.	Comma-delimited list NOTE: Only certain fields are valid. To determine which are valid, consult the comments in the ApplicationLayoutMap.xml file.
_keyvalueQueueAssigned	Comma-delimited list of values to filter by.	Comma-delimited list
_keyopQueueAssigned	Comma-delimited list of operators for the filter.	Comma-delimited list. Options are: • contains • equals • greater than • less than NOTE: Only certain combinations of fields and operators are valid. To determine which are valid, consult the comments in the ApplicationLayoutMap.xml file.
_keyconnectorQueueAssigned	Comma-delimited list of connectors.	Comma-delimited list. Options are: • and • or
_showFilter	Filter to use.	Options are: • Open • New • DueToday • DueThisWeek • Overdue
_entityID	ID of the entity by which to filter the list. The null value of this variable is used to clear the entityID and prevent the list from filtering by entity.	null
_QuoteID	ID of the quote or policy by which to filter the list. The null value of this variable is used to clear the quoteID and prevent the list from filtering by quote or policy.	null

Customizing Skins Per Role

While roles and privileges in EXAMPLE UserAdmin™ control server-level permissions, when customizing EXAMPLE Express® to display or hide interface elements for each user, you must modify EXAMPLE Express® skins.

In EXAMPLE Express® skins, the system privileges available in EXAMPLE UserAdmin™ are mapped to XSLT variables defined in the dctSecurity.xsl file (located in the *Global Root Directory*\Express\Skins\DCTCore directory). This file is imported into the dctContainer.xsl file (also located in the DCTCore directory), where the variables are available to any EXAMPLE Express® page.

To modify EXAMPLE Express® skins to hide or display features as appropriate for each type of user:

- Using the variables specified in the dctSecurity.xsl file, modify each XSL page (located in the *Global Root Directory*\Express\Skins*CustomSkin*\xsl directory) as desired to specify the privileges required for each feature to display by referencing those privileges in an xpath expression.

For example, to conditionally render an element based on the *canViewQuotes* privilege, wrap its rendering XSL statements in an `xsl:if` statement, such as the following:

```
<xsl:if test="$canViewQuotes">
  ...xsl statements that render the page element …
</xsl:if>
```

To conditionally change the CSS class defined on an element based on a privilege, use an `xsl:choose` statement, such as the following:

```
<div id="exampleElement">
  <xsl:attribute name="class">
  <xsl:choose>
    <xsl:when test="$canUpdateClient">
      <xsl:text>elementEnabled</xsl:text>
    </xsl:when>
    <xsl:otherwise>
      <xsl:text>elementDisabled</xsl:text>
    </xsl:otherwise>
  </xsl:choose>
  </xsl:attribute>
</div>
```

Duck Creek Technologies, Inc. reserves the right to change privilege names and how they map to variables in future versions. To facilitate future migration processes, use variables to identify the logical right to perform the described action instead of literal privilege names.

Using External Content in EXAMPLE Express®

Using EXAMPLE Author®, you can embed content from external URLs in EXAMPLE Express® to display useful information such as:

- Advertisements on EXAMPLE Express® pages. Once on the page, the user can make modifications to those advertisements without having access to the EXAMPLE Express® server.

- Terms and Conditions documents on an interview page.
- Various company facts or achievements on the left side of the page during the application process.

Methods of Loading External Content

Before embedding external content in EXAMPLE Express®, it is important to understand the two methods of doing so:

- **HTML** – Use the HTML method only for retrieving content that originates from the same domain as EXAMPLE Express® (for example, loading the URL "content/TermsOfUse.html" which resides at *http://your-site/Express/content/TermsOfUse.html*).

  Loading content(using the HTML method)from external domains can be insecure, and each browser handles it differently (security warnings, preventing content from loading, etc.).

  Since the HTML is being loaded directly into the EXAMPLE Express® HTML document, the externally-loaded HTML elements will inherit styling from the EXAMPLE Express® CSS definitions. This is useful when creating an external "Terms of Use" HTML page because the <h2> and <p> elements will be styled the identically to the <h2> and <p> elements throughout the rest of the page.
- **iFrame** – The iFrame method is the recommended method for loading content from external domains. Using this method, the external content will be contained in its own frame without being able to interact with EXAMPLE Express®.

  Since pages in an iFrame are kept separate from EXAMPLE Express®, iFrame content will not be altered by the CSS definitions from EXAMPLE Express®, making them very useful when external content should be displayed as if it had been opened by itself.

Adding External Content

Out of the box, external content can be applied to:

- Sections through EXAMPLE Author®
- The left panel through the skin.config file
- Dashboard modules through the ApplicationLayoutMap.xml file

Adding External Content to Sections

Using EXAMPLE Author®, you can embed external content in EXAMPLE Express®.

To add external content to EXAMPLE Express® through EXAMPLE Author®:

1. In Page Designer, click the section to which you wish to add external content.
2. In the **Advanced** section of the **Properties** pane, select a value for the **Static Content Type** property. Additional properties appear in the **Advanced** section.
3. Enter values for all desired section properties, and then click **Save**.

For more information about defining section properties, see *Sections* in the *Guide to EXAMPLE Author®*.

Adding External Content to the Left Panel

You can embed external content in the left panel by making changes to the Skin.config file.

The following configuration settings are available for use with external content:

- ContentUrl

- ContentTitle (optional)
- ContentBorder (optional)
- ContentFrame (optional)
- ContentCollapsible (optional)
- ContentScrollable (optional)
- ContentHeight (optional)
- ContentName (optional)

For information about these configuration settings, see *Skin.config Settings*.

Adding External Content to Dashboard Modules

You can customize the out-of-the-box modules available for the Policy Administration Dashboard (or any other page), or create new ones. One way to customize these modules is by adding external content to them. For more information about adding or customizing dashboard modules, see *Adding or Customizing Modules*.

Customizing Specific EXAMPLE Express® Features

A number of specific features and functions in EXAMPLE Express® can be customized, including:

- The Policy Administration Dashboard
- Tasks
- Notifications
- Search options
- The new quote process
- The system loading message
- The navigation
- Help link tab indexes
- Navigation classes

Customizing the Policy Administration Dashboard

If desired, you can customize the Policy Administration Dashboard that appears for each type of user. To customize the Dashboard, you must modify <pageLayout> elements in the ApplicationLayoutMap.xml file. For information about customizing the ApplicationLayoutMap.xml file, see *Customizing ApplicationLayoutMap.xml*. For information about <pageLayout> elements, see *Defining <pageLayout> Elements*.

Customizing Tasks

When customizing the EXAMPLE Express® task system, you can customize:

- **Task categories** — If desired, carriers can customize categories for tasks. Out of the box, EXAMPLE Express® comes with a single task category: General. For instructions on customizing task categories, see *Customizing Task Categories*.
- **Reason codes** — If desired, carriers can customize reason codes for closing a task. Out of the box, EXAMPLE Express® comes with a single reason code: Decline. For instructions on customizing reason codes, see *Customizing Task Reason Codes*.

- **Task queues** — Carriers can set up queues that serve as holding places for tasks. Queue members can pick up tasks as time and workloads allow. For information on setting up task queues, see *Setting Up Task Queues*.
- **Task generation/management** — If desired, carriers can use ManuScript logic to generate and manage tasks automatically during policy processing. For instructions on setting up ManuScripts to generate tasks automatically, see *Generating and Managing Tasks Automatically*.
- **Batch processing** — If desired, carriers can set up batch processes to query the task system or execute defined rules or processes. For information about setting up batch processing, see *Tasks and Batch Processing*.

Customizing Task Categories

If desired, you can customize categories for tasks. Out of the box, EXAMPLE Express® comes with a single task category: General. If desired, you can change this category or add new ones.

To customize task categories:

1. Open Microsoft SQL Server Management Studio and navigate to the **ExampleData** database.
2. In the **ExampleData** database, navigate to the **DC_PLT_Support_CodeLists** table.
3. Modify the record for the out-of-the box task category (General), or add a new record for each task category you wish to add. Specify the following fields for each record:

Field	Description	Value
CultureCode	Standard culture code (e.g., en-US). Allows for localized variations of codes.	Enter the desired culture code for the category.
CodeType	Value used to group related records in the table into a logical list. When adding task categories, use PLT_TASKTYPE.	Enter `PLT_TASKTYPE`.
Code	Key value for the entry. This value can be any unique character string, 20 characters or less.	Enter an appropriate code for the task category.
SortSequence	Default sort order of entries in a list of related records. Number each entry 1–x to specify the order in which the items will appear in the list.	Null, or enter the desired number.
Description	Caption that will display in EXAMPLE Express®. This value can be any unique character string, 20 characters or less.	Enter the desired caption.
BaseCodeIndicator	Indicator for whether the record has been customized.	Enter `FALSE` for any record you add or modify.

4. Save and close Management Studio.

Customizing Task Reason Codes

If desired, you can customize reason codes for closing a task. Out of the box, EXAMPLE Express® comes with a single reason code: Declined. If desired, you can change this reason code or add new ones.

To customize reason codes for closing a task:

1. Open Microsoft SQL Server Management Studio and navigate to the **ExampleData** database.
2. In the **ExampleData** database, navigate to the **DC_PLT_Support_CodeLists** table.
3. Modify the record for the out-of-the box reason code (Decline), or add a new record for each reason code you wish to add. Specify the following fields for each record:

Field	Description	Value
CultureCode	Standard culture code (e.g., en-US). Allows for localized variations of codes.	Enter the desired culture code for the reason code.
CodeType	Value used to group related records in the table into a logical list. When adding reason codes for closing a task, use PLT_TASKSTATUSREASON.	Enter `PLT_TASKSTATUSREASON`.
Code	Key value for the entry. This value can be any character string, 20 characters or less.	Enter an appropriate code for the reason code.
SortSequence	Default sort order of entries in a list of related records. Number each entry 1–x to specify the order in which the items will appear in the list.	Null, or enter the desired number.
Description	Caption that will display in EXAMPLE Express®. This value can be any character string, 20 characters or less.	Enter the desired caption.
BaseCodeIndicator	Indicator for whether the record has been customized.	Enter `FALSE` for any record you add or modify.

4. Save and close Management Studio.

Setting Up Task Queues

You can set up task queues in EXAMPLE Express® when you set up entities, roles, and users in EXAMPLE UserAdmin™. Any entity can be a task queue for any or all of its member users. To cause an entity to be a task queue for particular users, you must:

- Assign the entity a role with the desired task privileges.
- Assign that role to the desired users.

For more information about setting up task queues in EXAMPLE UserAdmin™, see the *EXAMPLE UserAdmin™ User Guide*.

Generating and Managing Tasks Automatically

If desired, you can use ManuScript logic to generate and manage tasks automatically during policy processing. To do so, you can use one of two methods:

- EXAMPLE Server® requests
- Express Actions

If using EXAMPLE TransACT®, you can generate and manage tasks automatically using an infrastructure that already exists in the out-of-the-box EXAMPLE TransACT® ManuScripts.

EXAMPLE Server® Requests

You can use the following EXAMPLE Server® requests to execute task logic from within a ManuScript. For more information about each request and their attributes, see the *EXAMPLE Server® Object Reference*.

- <Task.addRq>
- <Task.assignRq>
- <Task.getRq>
- <Task.listRq>
- <Task.listQueuesRq>
- <Task.listQueueUsersRq>
- <Task.setStatusRq>
- <Task.updateRq>

Express Actions

You can use the following Express Actions to execute task logic from within a ManuScript. For more information about each action or about Express Actions in general, see *Express Actions*.

Express Action	Description
addTaskAttachment	Adds an attachment to a task.
assignTask	Updates the queue and/or user to which a task is assigned.
deleteTaskAttachment	Removes an attachment from a task.
newTask	Adds a new task based on the task record located in the session at the path _TaskData/TaskRecord.
setTaskComplete	Sets the status of a task to *Complete*.
setTaskDeclined	Sets the status of a task to *Closed*, with a reason of *Declined*.
setTaskDeleted	Marks a task for delete. (Sets the status of the task to *Deleted*.)
setTaskStatus	Sets the status of a task.
updateTask	Updates a task in the database based on the task record located in the session at the path _TaskData/TaskRecord.

Using EXAMPLE TransACT®

When using EXAMPLE TransACT®, you can use the AttachTaskRule (part of the StatusSet event) to automatically generate and manage tasks. Out of the box, EXAMPLE TransACT® uses this rule to handle task management when a transaction's status changes. This logic works in the following way:

When the StatusSet event is executed, the system checks the **PolicyAdminData.UseDCTTasks** field in the Basic Transaction Rules ManuScript. This boolean field is set by a field in the product ManuScript and indicates whether to

use the out-of-the-box task management infrastructure available on the StatusSet event. If the value of **PolicyAdminData.UseDCTTasks** is *true*, the system executes the AttachTaskRule. (The **PolicyAdminData.UseDCTTasks** field defaults to *false*.)

The AttachTaskRule executes a field in the product ManuScript containing the desired task logic that should execute upon a transaction status change. This logic can include adding a task, reassigning a task, updating a task, or performing any other function available with the various task requests.

If you wish to use the AttachTaskRule:

1. In the desired product ManuScript, add a boolean field pathed to `data/policyAdmin/UseDCTTasks`. Set the desired logic to determine whether the system should use the AttachTaskRule upon transaction status change.
2. Also in the product ManuScript, create an additional field called **StatusChangeTask.Attach**. Set the field to execute the desired task requests that should execute upon transaction status change.

Tasks and Batch Processing

Management of tasks can be automated using EXAMPLE Platform® batch processing. Using the Batch Script Editor (Extended) tool provided with the EXAMPLE Platform®, batch scripts can be created to query the task system or execute defined rules or processes. These scripts can then be set to run at scheduled times or initiated manually using the BatchExecute.exe utility, or initiated manually using the <BatchProcess.processByScriptIdRq> request.

For information about using the Batch Script Editor (Extended) to define batch scripts for tasks, see *Using the Extended Batch Script Editor*.

For information about manually initiating batch scripts using the BatchExecute.exe utility, see *BatchExecute.exe*.

For information about the <BatchProcess.processByScriptIdRq> request, see the *EXAMPLE Server® Object Reference*.

Customizing Notifications

When customizing the EXAMPLE Express® notification system, you can customize:

- **Notification types** — If desired, carriers can customize types of notifications. Out of the box, EXAMPLE Express® comes with five notification types: Cancellation, Invalid Territory, Miscellaneous, Policy Notice, and Renewal Activity. For instructions on adding notification types, see *Customizing Notification Types*.
- **Notification generation/management** — If desired, carriers can use ManuScript logic to generate and manage notifications automatically during policy processing. For instructions on setting up ManuScripts to generate notifications automatically and using policy data, see *Generating and Managing Notifications Automatically*.
- **Use of policy data** — If desired, notifications can access and use policy data for various operations. For instructions on using policy data when creating or modifying notifications, see *Notifications and Policy Data*.
- **Batch processing** — If desired, carriers can set up batch processes to query the notification system or execute defined rules or processes. For information about setting up batch processing, see *Notifications and Batch Processing*.

Customizing Notification Types

If desired, you can customize types of notifications. Out of the box, EXAMPLE Express® comes with five notification types: Cancellation, Invalid Territory, Miscellaneous, Policy Notice, and Renewal Activity. If desired, you can change these types or add new ones.

To customize notification types:

1. Open Microsoft SQL Server Management Studio and navigate to the **ExampleData** database.
2. In the **ExampleData** database, navigate to the **DC_PLT_Support_CodeLists** table.
3. Modify the records for the out-of-the box notification types (Cancellation, Invalid Territory, Miscellaneous, Policy Notice, and Renewal Activity), or add a new record for each type you wish to add. Specify the following fields for each record:

Field	Description	Value
CultureCode	Standard culture code (e.g., en-US). Allows for localized variations of codes.	Enter the desired culture code for the notification type.
CodeType	Value used to group related records in the table into a logical list. When adding notification types, use PLT_MESSAGETYPE.	Enter `PLT_MESSAGETYPE`.
Code	Key value for the entry. This value can be any unique character string, 20 characters or less.	Enter an appropriate code for the notification type.
SortSequence	Default sort order of entries in a list of related records. Number each entry 1–x to specify the order in which the items will appear in the list.	Null, or enter the desired number.
Description	Caption that will display in EXAMPLE Express®. This value can be any unique character string, 20 characters or less.	Enter the desired caption.
BaseCodeIndicator	Indicator for whether the record has been customized.	Enter `FALSE` for any record you add or modify.

4. Save and close Management Studio.

Generating and Managing Notifications Automatically

If desired, you can use ManuScript logic to generate and manage notifications automatically during policy processing. To do so, you can use one of two methods:

- EXAMPLE Server® requests
- Express Actions

EXAMPLE Server® Requests

You can use the following EXAMPLE Server® requests to execute notification logic from within a ManuScript. For more information about each request and their attributes, see the *EXAMPLE Server® Object Reference*.

- <OnlineData.deleteMessageRq>
- <OnlineData.saveMessageRq>
- <OnlineData.setMessageStatusRq>

Express Actions

You can use the following Express Actions to execute notification logic from within a ManuScript. For more information about each action or about Express Actions in general, see *Express Actions*.

Express Action	Description
deleteMessage	Deletes the specified notification from the database.
messageDetail	Updates the active notification in the database using the interview content of a Message Detail ManuScript.
newMessage	Displays a New Message page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action. Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.
newNotification	Displays a Send a Notification page (messageNew.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action.
readMessage	Displays the Notification Details page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl) for the specified notification.
saveMessage	Updates the specified message in the database from the HTML form fields on the New Message page (message.xsl). Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.
saveNotification	Updates the specified notification in the database from the HTML form fields on the New Message page (messageNew.xsl).

Notifications and Policy Data

When notifications associated with a particular policy are edited using a Message Detail ManuScript, the Message Detail ManuScript can access policy data at runtime. This is because the policy data already exists in the session when a user creates the policy-specific notification.

When a user creates a policy-specific notification, the system appends a <messages> element to the policy data. EXAMPLE Express® will update the database with the data in this <messages> element when the notification is saved.

While the Message Detail ManuScript can pull and use policy data from the session, the EXAMPLE Express® system will only update the database with data in the <messages> element.

The table below describes the data contained in the <messages> element. For more information about pathing notification details, see the MessageDetailSchema.xml file available in the *Global Root Directory*\Schemas directory.

Any changes to items marked as read-only in this element are discarded. (Read-only items are extracted when the notification content is retrieved from EXAMPLE Server® and are used for display purposes only.)

XML	Description	Database Column (length)
<session>		[read-only]
<data id="d0">		[read-only]
<!--policy data goes here-->	Associated policy data.	[read-only]
<messages messageID=""	Primary tag for notification details. This tag is used by the <OnlineData.saveMessageRq> request to determine which notification to update.	MessageID (int 4)
quoteID=""	ID of any quotes associated with the notification. Can be null.	[read-only]
clientID="">	ID of any clients associated with the notification. Can be null. The out-of-the-box notifications model does not use this field.	[read-only]
<message messageID=""	Notification details.	[read-only] (same as <messages> above)
locked=""	Indicator of whether the notification should be locked.	Locked (bool)
securityLevel="">	Security level that can access the notification.	SecurityLevel (int 4)
<agency />	Agency to which the sender belongs.	[read-only]
<sender />	Username of the sender.	Sender (50)
<fullname />	Full name of the sender.	[read-only]
<securityLevel />	Security level of the sender.	[read-only]
<email />	E-mail address of the sender.	[read-only]
<subject />	Subject line of the notification.	Subject (255)
<sentOn />	Date the notification was sent.	SentOn (datetime)
<remindOn />	Remind date. The out-of-the-box notifications model does not use this field.	RemindOn (datetime)
<action />	Type of notification.	Action (50)
<notifyEmail />	E-mail address to be notified upon status change or inactivity. The out-of-the-box notifications model does not use this field.	NotifyEmail (50)
<notifyDate />	Date to auto-notify if there is no status change by this date. The out-of-the-box notifications model does not use this field.	NotifyDate (datetime)

<closedBy />	Username of the person who closed the notification. The out-of-the-box notifications model does not use this field.	ClosedBy (50)
<closedDate />	Date the notification was closed. The out-of-the-box notifications model does not use this field.	ClosedDate (datetime)
<notifyClosedDBy/>	Username of the person who closed the notification. The out-of-the-box notifications model does not use this field.	NotifyClosedBy (50)
<notifyClosedDate/>	Date when notification was closed. The out-of-the-box notifications model does not use this field.	NotifyClosedDate (datetime)
<purgeDate/>	Date the notification should be purged.	PurgeDate (datetime)
<dueDate/>	Date the notification is due for action. The out-of-the-box notifications model does not use this field.	DueDate (datetime)
<remindOnHideUntil/>	Indicator: • If *1*, the notification is not visible until the Remind Date. • If omitted or *0*, the notification is visible. The out-of-the-box notifications model does not use this field.	RemindOnHideUntil (bool)
<priority/>	Numeric priority code. The out-of-the-box notifications model does not use this field.	Priority (int 4)
<body />	Body of the notification.	Body (4096)
<misc/>	Extended data.	XMLData (nvarchar)
<attachments>	List of notification attachments.	[read-only]
<attachment attachmentID=""	Attachment ID.	[read-only]
moniker=""	Location where the file is stored on EXAMPLE Server®.	[read-only]
caption=""	Caption of the attachment.	[read-only]
filename=""	Name of the file when attached.	[read-only]
attachDate=""/>	Date when the file was attached.	[read-only]
</attachments>	Closing tag.	[read-only]
<policyID/>	PolicyID of the associated policy.	[read-only]
<policyNumber/>	Policy number of the associated policy. (Can be null.)	[read-only]
<effectiveDate/>	Effective Date of the associated policy. (Can be null.)	[read-only]
<LOB/>	LOB code of the associated policy. (Can be null.)	[read-only]

<message messageID=" " />	(Optional) Repeated sub-messages or comments (replies). The out-of-the-box notifications model does not use this field.	[read-only]
(Custom data here)	Custom elements, if defined.	[custom columns] (updated from inside XMLData area)
</message>	Closing tag.	[read-only]
<recipients id="">	Controlling tag that enumerates recipients.	[read-only]
<recipient	Entry for each recipient of the notification, found in the MessageLinks table. The content of these records resets the links for the notification.	
name=""	Enumeration: global, clientID, quoteID, userName, agencyID, agencyGroupID	(MessageLinks.LinkType)
value="" />	Value of the key based on the name enumeration.	(MessageLinks.LinkValue)
</recipients>	Closing tag.	
</messages>	Closing tag.	
</data>	Closing tag.	
</session>	Closing tag.	

When using the same Message Detail ManuScript with multiple product ManuScripts, the Message Detail ManuScript and all product ManuScripts must share the same structure. Make sure that all product ManuScripts share a common mapping for any information used by the Message Detail ManuScript. There is no system-defined schema for this shared information, but each implementation must use the same schema paths for the data to be accessed.

Notifications and Batch Processing

Management of notifications can be automated using EXAMPLE Platform® batch processing. Using the Batch Script Editor (Extended) tool provided with the EXAMPLE Platform®, batch scripts can be created to query the notification system or execute defined rules or processes. These scripts can then be set to run at scheduled times or initiated manually using the BatchExecute.exe utility, or initiated manually using the <BatchProcess.processByScriptIdRq> request.

For information about using the Batch Script Editor (Extended) to define batch scripts for notifications, see *Using the Extended Batch Script Editor*.

For information about manually initiating batch scripts using the BatchExecute.exe utility, see *BatchExecute.exe*.

For information about the <BatchProcess.processByScriptIdRq> request, see the *EXAMPLE Server® Object Reference*.

Customizing Search Options

When customizing search options, you can customize the Quick Search available on every page, or the advanced search options available on the Search page.

Customizing QuickSearch Options

If desired, you can customize the options available in the Quick Search.

To customize the Quick Search:

1. In the *Global Root Directory*\Express*CustomSkin*\xsl directory, open **dctCommonControls.xsl**.
2. Modify the **buildQuickSearch** template as desired. Be careful to modify the <when> case that is for the *pas* portal type.
3. In the *Global Root Directory*\Express\DCTCore\scripts directory, open **DCTPAS.js**.
4. Copy the **runQuickSearch()** function.
5. Navigate to the *Global Root Directory*\Express*CustomSkin*\scripts directory, and open **custom.js**.
6. Paste the copied function into custom.js.
7. Modify the **runQuickSearch()** function as desired.

Customizing Search Page Options

If desired, you can customize the options available on the Search page by:

- Adding, modifying, or removing the criteria users can search by (e.g., policy number, named insured, phone number, etc.)
- Modifying or removing the operators available to users when defining search criteria (e.g., contains, is, is not, etc.)
- Modifying or removing the options for handling multiple search filters (i.e., and, or)

Adding, Modifying, or Removing Criteria

To add, modify, or remove options for users to search by:

1. Open Microsoft SQL Server Management Studio and navigate to the **ExampleData** database.
2. In the **ExampleData** database, navigate to the **DC_PLT_Support_CodeLists** table.
3. Modify or remove the records for the out-of-the box search options (indicated by the CodeType `POL_SRCHPOLFILTER`), or add a new record for each option you wish to add.

A database column in the Quote table must exist for each record with the CodeType `POL_SRCHPOLFILTER`. For instructions on extending the database to include custom columns, see the *Guide to EXAMPLE Server®*.

Specify the following fields for each record:

Field	Description	Value
CultureCode	Standard culture code (e.g., en-US). Allows for localized variations of codes.	Enter the desired culture code for the search option.
CodeType	Value used to group related records in the table into a logical list. When adding search options, use POL_SRCHPOLFILTER.	Enter `POL_SRCHPOLFILTER`.
Code	Key value for the entry. This value must be a column in the Quote table.	Enter `Quote.[ColumnName]`, where [ColumnName] is the name of the desired column.
SortSequence	Default sort order of entries in a list of related records. Number each entry 1–x to specify the order in which the items will appear in the list.	Null, or enter the desired number.
Description	Caption that will display in EXAMPLE Express®. This value can be any unique character string, 20 characters or less.	Enter the desired caption.
BaseCodeIndicator	Indicator for whether the record has been customized.	Enter `FALSE` for any record you add or modify.

4. Save and close Management Studio.

Modifying or Removing Operators

To modify or remove the operators available to users when defining search criteria:

1. Open Microsoft SQL Server Management Studio and navigate to the **ExampleData** database.
2. In the **ExampleData** database, navigate to the **DC_PLT_Support_CodeLists** table.
3. Modify or remove the records for the out-of-the box operators (indicated by the CodeType `POL_SRCHFILTEROPT`). Specify the following fields for each record:

Field	Description	Value
CultureCode	Standard culture code (e.g., en-US). Allows for localized variations of codes.	Enter the desired culture code for the operator option.
CodeType	Value used to group related records in the table into a logical list. For operator options, the value is POL_SRCHFILTEROPT.	Do not modify this field.
Code	Key value for the entry. Do not modify this field.	Do not modify this field.
SortSequence	Default sort order of entries in a list of related records. Number each entry 1–x to specify the order in which the items will appear in the list.	Null, or enter the desired number.

Description	Caption that will display in EXAMPLE Express®. This value can be any unique character string, 20 characters or less.	Enter the desired caption.
BaseCodeIndicator	Indicator for whether the record has been customized.	Enter `FALSE` for any record you add or modify.

4. Save and close Management Studio.

Modifying or Removing Handling of Multiple Filters

To modify or remove the options for handling multiple search filters:

1. Open Microsoft SQL Server Management Studio and navigate to the **ExampleData** database.
2. In the **ExampleData** database, navigate to the **DC_PLT_Support_CodeLists** table.
3. Modify or remove the records for the out-of-the box filter options (indicated by the CodeType `POL_SRCHFILTERCNT`). Specify the following fields for each record:

Field	Description	Value
CultureCode	Standard culture code (e.g., en-US). Allows for localized variations of codes.	Enter the desired culture code for the filter option.
CodeType	Value used to group related records in the table into a logical list. For filter options, the value is POL_SRCHFILTERCNT.	Do not modify this field.
Code	Key value for the entry. Do not modify this field	Do not modify this field.
SortSequence	Default sort order of entries in a list of related records. Number each entry 1–x to specify the order in which the items will appear in the list.	Null, or enter the desired number.
Description	Caption that will display in EXAMPLE Express®. This value can be any unique character string, 20 characters or less.	Enter the desired caption.
BaseCodeIndicator	Indicator for whether the record has been customized.	Enter `FALSE` for any record you add or modify.

4. Save and close Management Studio.

Using New Quote Selection System Pages

If desired, you can use an EXAMPLE Express® system page (or a set of system pages) to help users start the new quote process.

To use an EXAMPLE Express® system page (or set of pages):

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Express® directory.
3. Under **Mode**, click **Form**.
4. On the **Web Config** tab, in the list of settings, click the **NewQuoteManuScriptID**.
5. In the **value** text box, clear the value.
6. Click **Save**.
7. Click **Close**.

Be sure to define the desired system page for each <applicationLayout> in your ApplicationLayoutMap.xml file. For more information about the ApplicationLayoutMap.xml file, see *Customizing ApplicationLayoutMap.xml*.

Customizing the System Loading Message

EXAMPLE Express® system pages and version 2 interview (ManuScript) pages display a "loading" message during long processes. By default, this message appears after four seconds. This number can be customized by modifying the EXAMPLE Express® skins.

To customize the number of seconds before the EXAMPLE Express® loading message appears:

1. In the *Global Root Directory*\Express\Skins*CustomSkin*\scripts directory, open the **custom.js** file.
2. Modify the **_loadingWaitTime** variable to contain the desired time to elapse (in milliseconds) before displaying the loading message.
3. Save and close **custom.js**.

Enabling Top Navigation

AvantGarde–based skins support the ability to apply a top-navigation layout to ManuScript pages in place of the default left-navigation layout. This type of navigation is ideal for ManuScript pages that require maximum screen real estate.

To enable top-navigation layout:

1. In EXAMPLE Author®, open the ManuScript containing the pages you wish to change.
2. In Page Designer, click the PageSet containing the pages you wish to change.
3. On the **Definition** tab, click **Rule Set**.
4. In the **Rule Set** text box, enter topNav.
5. Save and close the ManuScript.

Enabling Help Link Tab Indexes

In AvantGarde–based skins, by default, when users navigate through interview (ManuScript) pages in EXAMPLE Express® using the **Tab** key, help links are not included in the **Tab** key navigation.

To configure tab navigation to include help links:

1. In the *Global Root Directory*\Express\Skins*CustomSkin*\xsl directory, open the **interviewNew.xsl** file.
2. Change the following code:

```
<xsl:variable name="DisableHelpTabbing">
  <xsl:text>1</xsl:text>
</xsl:variable>
```

to:

```
<xsl:variable name="DisableHelpTabbing">
  <xsl:text>0</xsl:text>
</xsl:variable>
```

3. Save and close **interviewNew.xsl**.

In Lila-based skins, help links are included in the **Tab** key navigation by default.

Setting Up Navigation Classes

EXAMPLE Express® skins contain the ability to assign classes to interview (ManuScript) pages created in EXAMPLE Author® Page Designer. These classes, called *navigation classes*, provide the ability to assign certain styles to pages under certain conditions. Because these assigned classes are exposed in the page definition XML, if desired, custom skin implementations can also define logic that associates certain behaviors with certain classes. These features can be used to implement custom functionality such as stoplight functionality, which can be used to alert users to unfinished or unverified work, or to prevent them from moving forward in a business workflow before completing specified fields or pages.

Out of the box, EXAMPLE Express® includes three preset classes that may be assigned to pages in Page Designer:

- **Complete** — All required fields on the page are complete.
- **Incomplete** — Required fields on the page are not complete.
- **Caution** — All required fields on the page are complete, but verification is recommended before proceeding.

While users define rules in EXAMPLE Author® to determine when a page should be associated with each of the above classes, the system determines at runtime whether an assigned class is being applied to an active or inactive page. As a result, the skin defines a total of six classes: one for each of the above classes when the page is active, and one for each of the above classes when the page is inactive. Thus, the skin contains style definition for the following six classes:

- activeTabComplete
- inactiveTabComplete
- activeTabIncomplete
- inactiveTabIncomplete
- activeTabCaution
- inactiveTabCaution

Out of the box, the following styling is associated with each of the out-of-the-box classes. At runtime, the system determines whether the page is active or inactive.

- **Complete** — Thumbs up sign to the left of the page name on the page tab. ()
- **Incomplete** — Thumbs down sign to the left of the page name on the page tab. ()

- **Caution** — Halt sign to the left of the page name on the page tab. ( )

You can assign page classes in EXAMPLE Author® Page Designer using the *Navigation Class* property available for each page. This property is defined using a ManuScript field, usually containing rules that determine when each class should be assigned. Out of the box, this property will only accept the above three values (Complete, Incomplete, and Caution), but custom skin implementations can be configured to recognize additional classes.

Customizing the Out-of-the-Box Navigation Classes

To customize the styles associated with the out-of-the-box navigation classes:

1. In the *Global Root Directory*\Express\Skins*CustomSkin*\css directory, open the **style.css** file.
2. Modify the following classes as desired:
   - activeTabComplete
   - activeTabCaution
   - activeTabIncomplete
   - inactiveTabComplete
   - inactiveTabCaution
   - inactiveTabIncomplete

Adding Additional Navigation Classes

To add additional navigation classes:

1. In the *Global Root Directory*\Express\Skins*CustomSkin*\css directory, open the **style.css** file.
2. Add two CSS classes for each desired navigation class: `activeTab[NavigationClass]` and `inactiveTab[NavigationClass]`.

Appendixes

This section contains information about:

- Using Lila-Based Skins
- EXAMPLE Express® ManuScript Processing Flow
- Express\Web.config Settings
- Express Actions
- Skin.config File

Using Lila-Based Skins

If desired, the Lila-based skins delivered with EXAMPLE Express® 3.x can be used with EXAMPLE Express® 4.0. However, substantial differences will exist between an implementation of EXAMPLE Express® 4.0 that uses Lila-based skins and an implementation that uses Avant Garde–based skins. The table below describes the major differences that exist between Lila-based and AvantGarde-based skins in EXAMPLE Express® 4.x.

Difference	Description

Look and feel	Out-of-the-box Lila-based skins provide a different look and feel than out-of-the-box AvantGarde-based skins, with different color schemes, images, layouts, and text.
Policy Administration Dashboard	Out-of-the-box Lila-based skins do not support a role-based dashboard. Instead, Lila-based skins include a Home page that displays the five most recently accessed quotes and policies.
Task system	Out-of-the-box Lila-based skins do not support the task system available with AvantGarde-based skins.
Notification system (message system)	Out-of-the-box Lila-based skins use a message and diary system instead of a notification system. Out of the box, Lila-based skins provide the ability to send, receive, and reply to messages, and to create messages (diary items) that can be attached to clients or policies.
Search functionality	Out-of-the-box Lila-based skins do not incorporate the Quick Search feature available with AvantGarde-based skins. In addition, Lila-based skins use an Open page and a Client page to view and search for policies and clients (respectively), rather than a single search page.

With EXAMPLE Express® 4.x, the System Administration section in EXAMPLE Express® has been removed. If using existing Lila-based skins with EXAMPLE Express® 4.x, note that EXAMPLE Express® will no longer support the features in the System Administration section. Instead of performing these actions in EXAMPLE Express®, users can manage entities, roles, users, and the ExampleData database using EXAMPLE UserAdmin™.

EXAMPLE Express® ManuScript Processing Flow

The following EXAMPLE Express® processing occurs when a user executes an action (clicks a button or hyperlink) on a ManuScript page in EXAMPLE Express®:

1. If anything on the ManuScript page has changed, the system stores the fields to the session. If the *AutoSave* configuration setting in the Express\Web.config file is set to *1*, the system saves the session to the database.
2. If a Pre-Action is defined, the system runs the Pre-Action.
3. If an Express Action (also known as a post action) is defined and includes a `save` command, the system saves the session to the database.
4. If an Express Action (also known as a post action) is defined, the system executes the action.
5. If any requests are attached to the action, the system runs the requests.
6. The system loads the requested page, if any, and calls the Post-Action (the post command) from within the <ManuScript.getPageRq> request.

Express\Web.config Settings

The EXAMPLE Express® Web.config file contains a number of EXAMPLE Express® configuration settings. To modify these settings, use the Edit Configuration Utility (located at *Global Root Directory*\Util\EditConfig.exe).

Key	Default Value

AddFieldInfo	This setting reserved for future use.
AddPartyManuScriptID	null
AgencyDetailsManuScriptID	N/A
AllowMultiLanguageScreens	False
ApplicationLayoutFactory	C:\Program Files\Duck Creek Technologies 4.0\Express\App_Data\ApplicationLayoutMap.xml
AutoSave	1
BypassSSLHostAuthentication	0
CarrierName	Demo National Insurance
ClientDetailsManuScriptID	null
ConsumerAccessManuScriptID	null
ConsumerAccessRole	null
ConsumerAgencyID	null
ContentFactory	C:\Program Files\Duck Creek Technologies 4.0\Express\App_Data\ContentMap.xml
DebugMode	0
DebugOutputDir	C:\Program Files\Duck Creek Technologies 4.0\Express
EventLog	Errors
EXAMPLEServerURL	http://localhost:80/DuckCreek40/dctserver.aspx
ExitInterviewField	null
FormsTempPath	C:\Program Files\Duck Creek Technologies 4.0\Express\Temp
GetPageWaitTime	0
GetPolicyAdminManuScriptID	null
GuestPassword	null
GuestUserName	null
HomeBlockListBulletins	0
HomeBlockListMessages	0

HomeBlockListMRU	0
InitVector	N/A
InstantShred	0
LogoutURL	null
MailTo	support@mycompany.com
MenuMapFactory	N/A
MessageDetailsManuScriptID	null
NewQuoteLimitRequests	0
NewQuoteManuScriptID	NewQuoteSelectionPortfolio
OverrideLanguageForScreens	null
PartySearchManuScriptID	null
PassPhrase	N/A
PerformanceLogging	0
PostActionFactory	C:\Program Files\Duck Creek Technologies 4.0\Express\App_Data\PostActionMap.xml
PolicyAdminManuScriptID	null
PolicyBlockListAttachments	0
PolicyBlockListMessages	0
PolicyBlockListTasks	0
PrintJobAsyncMode	2
PrintJobAutoRefresh	1
PrintPreviewDownloadMode	0
ProcessAsyncFormWaitTime	1
ProxyMode	0
RefreshClientOnLoad	Quote
SaltValue	N/A
SecureLogin	0
SelectAgencyLimit	15

ServerConfig	C:\Program Files\Duck Creek Technologies 4.*x*\Express\App_Data\OnDemand.Config
ServerDomain	N/A
SMTPServer	null
SMTPServerPort	25
SSOErrorMsg	You do not have access to Example Express. Please contact your administrator or login.
TaskDetailManuScriptID	Carrier_TaskDetailEditing
TimeOut	360000
UploadDir	C:\Program Files\Duck Creek Technologies 4.0\Express\Upload
UserDetailsManuScriptID	N/A
XmlNamespaceUri	null
XSLTDir	Skins\AvantGarde2

AddPartyManuScriptID

Reserved for future use.

AgencyDetailsManuScriptID

Obsolete for EXAMPLE Platform® 4.0. This functionality is now available for use in EXAMPLE UserAdmin™. See the *AgencyDetailsManuScriptID* configuration setting in the UserAdmin\Web.config file.

AllowMultiLanguageScreens

A setting of **False** will set the language on the first visit of the Web site based on the Example\Web.config setting **OverrideLanguageForScreen**. If the **OverrideLaguageForScreen** setting is not defined, the language will be set by the browser.

A setting of **True** will allow multiple languages to be used. The browser will check for new versions of objects with every page.

Any language used must have a resource file associated with it. If a resource file is not associated with a language setting, the language will be set to the default resource file.

Using one language in a browser will improve performance because objects are cached in the system and will not require frequent refreshes.

ApplicationLayoutFactory

Fully-qualified directory path to the ApplicationLayoutMap.xml file (and any custom ApplicationLayoutMap files). The path specifies the directory to use, and the file name (minus the file extension) specifies the base term to use to determine custom ApplicationLayoutMap files.

Default	Valid
C:\Program Files\Duck Creek Technologies 4.0\Express\App_Data\ApplicationLayoutMap.xml	Valid path, including file name, to the ApplicationLayoutMap.xml file.

AutoSave

Boolean value indicating whether the interview process should automatically save changes upon page change. The system creates a history only when a save action is performed.

AutoSave saves quote information but does not initiate shredding. Shredding occurs only upon user-initiated save. If a user exits a quote without explicitly saving, the quote may be saved to the ExampleData database but not to the shredding database. This can prevent a 1-to-1 correlation between the two databases.

For Duck Creek Templates™ or EXAMPLE TransACT® 2.1 to function properly, AutoSave must be enabled (1).

Default	Valid
1	• **0** — Do not save quote on page change. • **1** — Save quote on page change.

BypassSSLHostAuthentication

Boolean value indicating whether to ignore SSL certificate validation errors between machines running EXAMPLE Express® and EXAMPLE Server®. This setting is only meaningful if the EXAMPLE Server® URL uses https: protocol.

Default	Valid
0	• **0** — Do not ignore certificate validation errors. • **1** — Ignore certificate validation errors.

CarrierName

Descriptive name of the carrier. This value is used in the EXAMPLE Express® browser window title bar, captions, and other locations, depending upon your custom skin.

Default	Valid
Demo National Insurance	Any text string.

ClientDetailsManuScriptID

ID of the ManuScript to use when editing or viewing client details.

Default	Valid
null	Valid ID of a ManuScript in an accessible catalog.

ConsumerAccessManuScriptID

ID of the ManuScript to use for guest and consumer quotes.

Default	Valid
null	Valid ID of a ManuScript in an accessible catalog.

ConsumerAccessRole

ID of the default role assigned to Consumer Access users. These roles must be created manually.

Default	Valid
null	Any valid role ID.

ConsumerAgencyID

ID of the default entity assigned to Consumer Access users. These entities must be created manually.

Default	Valid
null	Any valid Entity ID.

ContentFactory

Fully-qualified directory path to the ContentMap.xml file (and any custom ContentMap files). The path specifies the directory to use, and the file name (minus the file extension) specifies the base term to use to determine custom ContentMap files.

Default	Valid
C:\Program Files\Duck Creek Technologies 4.0\Express\App_Data\ContentMap.xml	Valid path, including file name, to the ContentMap.xml file.

DebugMode

Integer setting (0,1,2,3) used to log the XML content of each request to the EXAMPLE Express® system. Debug is turned on if the value of the setting is greater than 0. Each request is sent to the debugContent.xml file located in the EXAMPLE Express® directory or to the directory indicated by the *DebugOutputDir* configuration setting. As a result, Duck Creek Technologies, Inc. recommends setting this value to 0 on a runtime system.

This setting also determines whether to enable the **Go Back** button on the EXAMPLE Express® error screen.

Default	Valid
0	• **0** — Do not store EXAMPLE Express® debug information. • **1** — Save EXAMPLE Express® debug information to the debugContent.xml file. Enable the **Go Back** button on the EXAMPLE Express® error screen. • **2** — Save EXAMPLE Express® debug information to the debugContent.xml file, appending the <ManuScript.getPageRs> response (with session information). Enable the **Go Back** button on the EXAMPLE Express® error screen. • **3** — Save EXAMPLE Express® debug information to the directory specified by the *DebugOutputDir* configuration setting. Enable the **Go Back** button on the EXAMPLE Express® error screen.

DebugOutputDir

Location for the debug content file.

Default	Valid
C:\Program Files\Duck Creek Technologies 4.0\Express	Any valid file path.

EventLog

Enables use of the Application Event Log in EXAMPLE Express®, which logs exception conditions and registers when the user receives a notification message.

Default	Valid
Errors	• **null** — Do not log errors or notifications. • **Errors** — Log errors thrown in the EXAMPLE Express® system. • **All** — Log errors and notifications.

EXAMPLEServerURL

URL of the EXAMPLE Server® to be used by the EXAMPLE Express® system. Specify the DCTServer.aspx target if requests to the EXAMPLE Server® are to use HTTP.

If the EXAMPLE Express® system resides on the same machine as the EXAMPLE Server®, this setting can be left blank to indicate that the communication with the EXAMPLE Server® is to be local. This increases performance of each request but requires the EXAMPLE Express® installation to co-exist on the same server with the EXAMPLE Server®.

Default	Valid
http://localhost:80/DuckCreek40/dctserver.aspx	URL of an existing EXAMPLE Server®.

ExitInterviewField

Defines a ManuScript field to execute when a user completes an interview process in EXAMPLE Express®. For more information about the Exit Interview Field, see *Setting up the Exit Interview Field* in the *EXAMPLE Express® Reference Guide*.

Default	Valid
null	Name of a ManuScript field. The name must be identical in all ManuScripts that may be accessed.

FormsTempPath

UNC path accessible from both EXAMPLE Server® and EXAMPLE Express® machines that has both read and write permissions. This allows the transfer of temporary data between EXAMPLE Server® and EXAMPLE Express®.

Duck Creek Technologies, Inc. recommends that this path reside on the EXAMPLE Express® machine in an unclustered environment, or in a shared storage location in a clustered environment.

Default	Valid
C:\Program Files\Duck Creek Technologies 4.0\Express\Temp	Any valid path.

GetPageWaitTime

Number of milliseconds for EXAMPLE Express® to wait on asynchronous requests before loading a page. This setting increases the wait time on any request to EXAMPLE Server® for page information whether or not the request is asynchronous, so use with caution.

Default	Valid
0	Any positive integer or 0.

GetPolicyAdminManuScriptID

Boolean value indicating whether to look for a PolicyAdminManuScriptID field in the active ManuScript. If the field contains a valid ManuScript ID, EXAMPLE Express® will use the specified ManuScript for Policy Administration. Otherwise, EXAMPLE Express® will use the ManuScript ID specified by the *PolicyAdminManuScriptID* setting.

Default	Valid
null	• **0** — Use the ManuScript ID found in the *PolicyAdminManuScriptID* setting. • **1** — Use the ManuScript ID found in the specified PolicyAdminManuScriptID field.

GuestPassword

Password for guest access to EXAMPLE Express®.

Default	Valid

null	Text string.

GuestUserName

Username for guest access to EXAMPLE Express®.

Default	Valid
null	Text string.

InitVector

Encryption key used for authentication in EXAMPLE Express® and EXAMPLE Server®. The value of this setting must be identical when used in multiple config files. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default	Valid
N/A	Any ASCII text string. Should be 16 characters long for maximum encryption strength.

InstantShred

Boolean value that indicates whether information entered into EXAMPLE Express® should be immediately distributed.

Default	Valid
0	• **0** — Write information to the queue for later distribution. • **1** — Immediately distribute data to the database (if *UseShredding* is also set to *1*). This option uses the <OnlineData.storePolicyRq> request or the <OnlineData.storeTransactionRq> request to distribute the data.

LogoutURL

This setting determines the page to which EXAMPLE Express® sends a user upon logging out.

Default	Valid
null	• **null** — Send the user to the EXAMPLE Express® Login page. • **Valid URL** — Send the user to the specified URL.

MailTo

E-mail address to which support issues should be directed. If a user selects to send a support e-mail from EXAMPLE Express®, the e-mail is sent to the recipient specified by this setting.

Default	Valid
support@mycompany.com	Any valid e-mail address.

MenuMapFactory

Obsolete as of EXAMPLE Platform® 4.0.

MessageDetailsManuScriptID

ID of the ManuScript to use for editing or viewing message details.

Default	Valid
null	Valid ID of a ManuScript in an accessible catalog.

NewQuoteLimitRequests

Boolean value indicating whether to run listAgencies and associated requests on the New Quote page when the *NewQuoteManuScriptID* setting is not set to *null*. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default	Valid
0	**0** — Run listAgencies and associated requests on the New Quote page. **1** — Do not run listAgencies and associated requests on the New Quote page.

NewQuoteManuScriptID

ID of the New Quote Selection ManuScript to use when starting a new quote. The New Quote Selection ManuScripts enables a user-friendly interface for determining which ManuScript to use for a new quote.

Default	Valid
NewQuoteSelectionPortfolio	Valid ID of a ManuScript in an accessible catalog.

OverrideLanguageForScreens

The default is null to be compatible with previous versions of the EXAMPLE Platform®. Setting this key to a valid language with a resource file will force all users to use a single browser language.

PartySearchManuScriptID

Reserved for future use.

PassPhrase

Encryption key used for authentication in EXAMPLE Express® and EXAMPLE Server®. The value of this setting must be identical when used in multiple config files. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default	Valid
N/A	Any ASCII text string.

PerformanceLogging

Boolean value indicating whether EXAMPLE Express® should log performance data about the rendered page. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default	Valid
0	**0** — Do not render performance data on the page. **1** — Render performance data on the page.

PolicyAdminManuScriptID

This setting is only for use with EXAMPLE TransACT® 1.8. If using EXAMPLE TransACT® 2.1, leave this setting blank.

ID of the TransACT Master ManuScript to use for EXAMPLE TransACT® 1.8.

Default	Valid
null	Valid ID of a ManuScript in an accessible catalog.

PolicyBlockListAttachments

Boolean value that determines whether attachments display on the Policy Details page.

Default	Valid
0	• 0 – Display attachments. • 1 – Do not display attachments.

PolicyBlockListMessages

Boolean value that determines whether notifications display on the Policy Details page.

Default	Valid
0	• 0 – Display notifications. • 1 – Do not display notifications.

PolicyBlockListTasks

Boolean value that determines whether tasks display on the Policy Details page.

Default	Valid
0	• 0 – Display tasks. • 1 – Do not display tasks.

PostActionFactory

Fully-qualified directory path to the PostAction.xml file (and any custom PostActionMap files). The path specifies the directory to use, and the file name (minus the file extension) specifies the base term to use to determine custom PostActionMap files.

Default	Valid
C:\Program Files\Duck Creek Technologies .40\Express\App_Data\PostActionMap.xml	Valid path, including file name, to the PostActionMap.xml file.

PrintJobAsyncMode

Controls the asynchronous behavior for the previewAsyncPrintJob and the initAsyncPrintJob Express Actions.

Default	Valid
2	• **2** — Asynchronous with a semaphore. • **3** — Place the request in the Asynchronous queue. • **4** — Place the request in the Asynchronous queue and flag it to Auto Close so the session closes when the request completes.

PrintJobAutoRefresh

Boolean value indicating whether EXAMPLE Express® should automatically regenerate the forms list (while saving previous user selections) when the user loads the Policy Forms page.

Default	Valid
1	• **0** — Do not automatically regenerate the forms list. • **1**— Automatically regenerate the forms list.

PrintPreviewDownloadMode

This setting determines whether the preview of a form opens a pop-up window with a preview of the form or an Open/Save dialog box.

If the value of this setting is *1*, the EXAMPLE Express® Web site must be in the client browser's list of trusted sites for the form to download properly.

Default	Valid
0	• **0** — Opens the form in a pop-up window. • **1** — Opens an Open/Save dialog box.

ProcessAsyncFormWaitTime

Value specifying whether EXAMPLE Express® should process requests asynchronously and what the delay time should be (in miliseconds) between checks to see if asynchonous requests have been completed.

Default	Valid
1	**0** — Process all requests synchronously. **Any positive integer** — Process requests asynchronously, and wait this number in milliseconds between checks for request completion.

ProxyMode

Boolean value that, if pertaining to an EXAMPLE Express® proxy, allows responses to successfully return to the client.

Default	Valid
0	**0** — EXAMPLE Express® responses may not successfully return to the client. **1** — EXAMPLE Express® responses will successfully return to the client.

RefreshClientOnLoad

Setting that causes the <OnlineData.loadPolicyRq> request to overwrite client information in the policy data with values from the client record.

A setting of 1 or 0 turns the client information refresh on or off globally. The value for this setting can be a comma-delimited list of the policy statuses that refresh the client information on load, usually *Quote*.

Default	Valid
Quote	• **0** — Do not refresh client information. • **1** — Refresh client information on each <OnlineData.loadPolicyRq> request made. • Comma-delimited list of policy statuses that should refresh client information.

SaltValue

Encryption key used for authentication in EXAMPLE Express® and EXAMPLE Server®. The value of this setting must be identical when used in multiple config files. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default	Valid
N/A	Any ASCII text string.

SecureLogin

Boolean value indicating whether EXAMPLE Express® uses the secure socket layer when accessing the login or administration features of the system.

A value of true (1) for this configuration setting will force EXAMPLE Express® to use HTTPS for login and administration functions. All other pages will use HTTP. A value of true (1) requires that the Web server has a valid security certificate for the secure protocal.

Default	Valid
0	• **0** — Use current protocol for all requests. • **1** — Enter HTTPS upon login or administration requests.

SelectAgencyLimit

Specifies the threshold number of agencies required to display the agency list in *large list* mode. Out of the box, an agency list with a number of agencies less than or equal to the number specified for this setting will appear in a drop-down list in EXAMPLE Express®. An agency list with a number of agencies greater than the number specified in this setting will appear in a grid in EXAMPLE Express® and no longer uses a link to redirect to an agency list.

Default	Valid
15	Any positive integer.

ServerConfig

Fully-qualified directory path to the OnDemand.config file.

Default	Valid
C:\Program Files\Duck Creek Technologies 4.0\Express\App_Data\OnDemand.Config	Valid path, including file name, to the OnDemand.config file.

ServerDomain

Domain name for Microsoft Windows domain user Single Sign On. If null, mixed mode authentication will not be used. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default	Valid
N/A	Text string.

SMTPServer

SMTP mail server used by EXAMPLE Express® or EXAMPLE Server® when generating e-mail messages. The SMTP mail service need not be on the same machine as EXAMPLE Express® or EXAMPLE Server®.

Default	Valid
null	• **null** — No e-mail service. • Fully qualified name of the SMTP Server to which to route service when sending an e-mail message.

SMTPServerPort

Port number to use for the SMTP Server during the agency sign-up process. If the setting is incorrect, an error message is displayed. N.o notification is sent to the address found in the *MailTo* setting to notify the administrator that an agency has requested sign up.

Default	Valid
25	Integers between 1 and 65535.

TaskDetailManuScriptID

ID of the ManuScript to use for editing or viewing task details.

Default	Valid
Carrier_TaskDetailEditing	Valid ID of a ManuScript in an accessible catalog.

TimeOut

Number of milliseconds EXAMPLE Express® will wait for a response from EXAMPLE Server® before timing out.

To force EXAMPLE Express® to never time out when making calls to EXAMPLE Server®, set this value to -1.

The *executionTimeout* attribute of the ASP.NET <httpRuntime> element must be greater than or equal to the Express\Web.config *Timeout* value. If the *executionTimeout* attribute of the ASP.NET <httpRuntime> element is less than the Express\Web.config *Timeout* value, ASP.NET could time out before EXAMPLE Express® and not provide the user any indication that this happened. For more information on the <httpRuntime> element, see *http://msdn.microsoft.com/en-us/library/e1f13641.aspx*.

Default	Valid
360000	• **Any positive integer** — Number of milliseconds to wait before timing out of EXAMPLE Express®. • **-1** — Disable EXAMPLE Express® timeout. • **null** — System will use a value of 90000 milliseconds

UploadDir

Global path to the directory where files are uploaded and downloaded. The specified path should be used for all servers in a cluster that are accessible from the EXAMPLE Express® server. It should be a shared or network path that all machines in a configuration can access. The path must have security access rights to allow the Web user to read and write files to the selected directory.

Default	Valid
C:\Program Files\Duck Creek Technologies 4.0\Express\Upload	Any valid file path.

UserDetailsManuScriptID

Obsolete for EXAMPLE Platform® 4.0. This functionality is now available for use in EXAMPLE UserAdmin™. See the *UserDetailsManuScriptID* configuration setting in the UserAdmin\Web.config file.

XmlNamespaceUri

Value to use for transformation. (This setting is not included in the out-of-the-box Express\Web.config file, but it can be added manually.)

Default
null

XSLTDir

Relative path from the *GlobalRootPath*\Express directory to the appropriate custom skins directory (for example, "Skins\Custom").

Default	Valid
Skins\AvantGarde2	Valid relative directory path.

SSOErrorMsg

Error message displayed when a user has been authenticated by a single-sign-on tool but does not have a valid EXAMPLE Server® authorization account.

Default	Valid
You do not have access to Example Express. Please contact your administrator or login.	Any text string

Express Actions

Out of the box, the PostActionMap.xml file contains the following Express Actions.

When an Express Action requires HTML form fields, the HTML form from which the action is executed must contain those fields. For details about each Express Action and any HTML form fields required, see the description of each action (below).

Express Action	Description	
activatePolicy	Creates a consumer account; returns a clientID, quoteID, and LOB; and then loads the specified quote or policy.	
addTaskAttachment	Adds an attachment to a task.	
admin	Updates administration information.	
assignTask	Updates the queue and/or user to which a task is assigned.	

commit, commit2, audit	Processes an out-of-sequencetransaction and commits all changes to the database.	
consumerConvertToPolicy	Moves a quote from the DC_POL_ConsumerQuote table to the Quote and History tables.	
consumerCreateAccount	Creates a user account for the EXAMPLE Platform®.	
consumerLoadQuote	Loads a quote from the DC_POL_ConsumerQuote table.	
consumerLogin	Logs a user in under the proxy account used for consumer access.	
consumerNewQuote	Creates a new quote in the DC_POL_ConsumerQuote table.	
consumerReLogin	From within EXAMPLE Express®, logs a consumer user out of the proxy account and logs them in to an existing full user account.	
delete	Marks or un-marks the specified quote or policy for deletion from the database. The specified item is not removed from the database until a purge is performed.	
deleteClient	Marks the specified client for deletion from the database. The client is not removed from the database until a purge is performed.	
deleteMessage	Deletes the specified notification from the database.	
deleteTaskAttachment	Removes an attachment from a task.	
details	Updates the details of a quote or policy in the database.	
download	Sends a copy of the XML image of the specified quote or policy to the browser.	
duplicate	Duplicates the specified quote or policy in the database.	
initAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *1*. Loads the Policy Forms page.	
initPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request with the *forceInit* attribute set to *1*. Loads the Policy Forms page.	
issue	Saves the active quote or policy to the database using the <OnlineData.storeTransactionRq> request.	
load	Loads the specified quote or policy into the session and invokes the interview process of the associated ManuScript.	
loadHistoryTx2	Loads the specified history image of a quote or policy into the session and invokes the interview process of the associated ManuScript.	

loadPortfolio	Loads the specified Portfolio into the session and invokes the interview process of the associated ManuScript.	
login	Logs the user into EXAMPLE Server® and begins a new EXAMPLE Express® session.	
messageDetail	Updates the active notification in the database using the interview content of a Message Detail ManuScript.	
newMessage	Displays a New Message page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action. Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.	
newNotification	Displays a Send a Notification page (messageNew.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action.	
newQuote	Creates a new quote in the active session, invoking the interview process of the specified ManuScript. Performs no database action.	
newTask	Adds a new task based on the task record located in the session at the path _TaskData/TaskRecord.	
previewAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *0* or *3*. Loads the Policy Forms page.	
previewPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request with the *forceInit* attribute set to *0* or *3*. Loads the Policy Forms page.	
readMessage	Displays the Notification Details page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl) for the specified notification.	
refreshAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *3*. Loads the Policy Forms page and executes the <FormsEngine.setPrintJobItemsRq> request with an *autoMark* attribute set to *markDefault*.	
rescind	Reverts the current policy image back to the state of the specified history image, retaining records of any transactions that are removed.	
resetManuScriptID	Resets the associated ManuScript for the active quote.	
resetSession	Clears all session information.	

rollbackHistory	Rolls back the transaction records of a policy to the specified history image, removing any subsequent transactions and making the specified history image the current image. This action then loads the (new) current policy image into the active session and invokes the interview process of the associated ManuScript.	
runRulesAsyncPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously with the *forceInit* attribute set to *2*. Loads the Policy Forms page.	
runRulesPrintJob	Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request with the *forceInit* attribute set to *2*. Loads the Policy Forms page.	
save	Saves the active quote or policy to the database using the <OnlineData.storePolicyRq> request.	
saveMessage	Updates the specified message in the database from the HTML form fields on the New Message page (message.xsl). Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.	
saveNotification	Updates the specified notification in the database from the HTML form fields on the New Message page (messageNew.xsl).	
setActiveAgency	Updates the active agency for the logged in user in the EXAMPLE Express® <state> node.	
setLanguage	Sets the language property in the session and the EXAMPLE Express® <state> node to the specified language.	
setPrintManuScript	Sets the ManuScriptID to use when the **Create Policy Documents** link is clicked. This action should be used when a different ManuScript than the loaded interview ManuScript (state.ManuScriptID) is needed.	
setTaskComplete	Sets the status of a task to *Complete*.	
setTaskDeclined	Sets the status of a task to *Closed*, with a reason of *Declined*.	
setTaskDeleted	Marks a task for delete. (Sets the status of the task to *Deleted*.)	
setTaskStatus	Sets the status of a task.	
signup	Creates an agency signup request and sends an e-mail to the recipient specified by the *MailTo* configuration setting in the Express\Web.config file.	
submit	Saves the active quote or policy to the database using the <OnlineData.storeTransactionRq> request.	
switchUser	Moves the active quote or policy to the specified agency and logs in as a different user to load the active policy.	

updateTask	Updates a task in the database based on the task record located in the session at the path _TaskData/TaskRecord.	
upload	Uploads a file from a ManuScript to the directory specified in the *UploadDir* configuration setting in the Express\Web.config file.	
validatePrintJob	Verifies that no required fields on selected forms are empty.	

activatePolicy

This action is intended for use with consumer access features available before EXAMPLE Platform® 4.0.

Creates a consumer account; returns a clientID, quoteID, and LOB; and then loads the specified quote or policy.

Interview parameters:

- **userName** — Username to assign to the user.
- **password** — Password to assign to the user.
- **passwordExpires** — Expiration date for the specified password.
- **fullName** — Full name of the user.
- **email** — E-mail address of the user.

HTML Form Fields:

Field	Description
_username	Username to assign to the user. Used if the *userName* interview parameter is not found.
_password	Password to assign to the user. Used if the *password* interview parameter is not found.
_agencyID	ID of the agency with which to associate the quote or policy.

EXAMPLE Server® Requests: OnlineData.createConsumerAccountRq, Session.loginRq, OnlineData.getUserDetailsRq, Security.listAssignedRolesRq, OnlineData.getAgencyDetailsRq

addTaskAttachment

Adds an attachment to a task. This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the task ID, object code (generally the quote ID), and object type code (generally *POL* for quotes/policies) from the paths defined in the schemas for the <Task.getRq> request response. It also obtains the attachment caption from AttachmentRecord/Caption.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description
_taskId	ID of the task to attach the file to.

_caption	Caption for the attachment.
_quoteID	ID of the policy or quote to attach the file to.
File form field	Form field of type "file" that contains a reference to the file to be attached.

EXAMPLE Server® Request: OnlineData.addAttachmentRq

admin

Updates administration information.

HTML Form Fields:

Field	Description
_pageType	Tabbed page of the admin page against which to operate. Valid pages are: • password
_adminAction	Sub-command of the admin process to invoke. Valid sub-commands (described below) are: • change

change

Updates password information.

HTML Form Fields:

Field	Description
_newpassword	New password text.
_confirmpassword	New password text. This text must match _newpassword or the change is rejected.
_oldpassword	Old password used to verify security.

EXAMPLE Server® Request: Session.changePasswordRq

assignTask

Updates the queue and/or user to which a task is assigned. This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the task ID, queue ID, assigned user ID, and task locking code from the paths defined in the schemas for the <Task.getRq> request response.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description

_taskId	ID of the task to modify.
_entityId	ID of the queue to assign the task to.
_auserId	ID of the user to assign the task to.
_taskLockingTS	Encoded binary value that indicates the last time the task data was read and prevents multiple users from modifying a task at the same time. This value is generated from the <Task.getRq> request or the <Task.listRq> request.

EXAMPLE Server® Request: Task.assignRq

commit, commit2, audit

These actions are intended for use with EXAMPLE TransACT® 1.0.

Processes an out-of-sequencetransaction and commits all changes to the database. These actions reconcile all policy changes to subsequent transactions. (The system applies changes to and re-rates each subsequent transaction, recording both original and new premiums and charges.)

- **commit** — Original action used with EXAMPLE TransACT® 1.0. This deprecated action is included for backwards compatibility.
- **commit2** — Commit action recommended for use with EXAMPLE TransACT® 1.0. This action contains additional logic for dealing with newer transaction properties, out-of-sequence renewals, and out-of-sequence cancellations.
- **audit** — Action recommended for audit transactions with EXAMPLE TransACT® 1.0. This action causes the reconcile process to begin with the first transaction in the term rather than the transaction before the Audit transaction. This action causes all transactions in the term to be updated with the audit changes. In addition, the audit transaction will have its HistoryID replaced with a newer copy of the transaction. This newer HistoryID ensures the audit transaction retains a HistoryID greater than all inserted, modified transactions before the audit transaction.

For these commands to work properly, session data must conform to the Policy Administration guidelines. The last element at the transactions path defined in the TransACTInfo.xml file must be the out-of-sequence transaction and be in an *open* state.

These actions do not require any HTML form fields, but use the active session and EXAMPLE Express® state information to determine the clientID and policyID to save.

consumerConvertToPolicy

Moves a quote from the DC_POL_ConsumerQuote table to the Quote and History tables. This action can be executed from a ManuScript or a system page.

Parameters/HTML Form Fields:

Field	Description
_consumerID	ID for the consumer, stored in the DCT_PLT_ConsumerAccount database table.
_consumerQuoteID	(Optional) ID of the quote to move. This ID is stored in the DCT_POL_ConsumerQuote table. If not specified, the system will move the first quote for the specified consumer.

EXAMPLE Server® Request: Consumer.convertToPolicyRq

consumerCreateAccount

Creates a user account for the EXAMPLE Platform®. This action should be used to register a consumer access user. This action can be executed from a ManuScript or a system page.

Parameters/HTML Form Fields:

Field	Description
_username	Username for the account.
_password	Password for the account.
_policyNumber	(Required if _consumerQuoteNumber is not specified) Policy number of the policy to associate with the account. Use this field when creating an account for a consumer who already has a policy in the system.
_consumerQuoteNumber	(Required if _policyNumber is not specified) Number of the quote to associate with the account. Use this field when creating an account for a consumer who has no existing policies in the system.

EXAMPLE Server® Request: Consumer.createAccountRq

consumerLoadQuote

Loads a quote from the DC_POL_ConsumerQuote table. This action can be executed from a ManuScript or a system page.

Parameters/HTML Form Fields:

Field	Description
_consumerQuoteNumber	(Required if _consumerQuoteID is not specified) ManuScript-generated number of the quote to load.
_consumerQuoteID	(Required if _consumerQuoteNumber is not specified) ID of the quote to load. This ID is stored in the DCT_POL_ConsumerQuote table.
_consumerID	ID for the consumer, stored in the DCT_PLT_ConsumerAccount database table.

EXAMPLE Server® Request: Consumer.loadQuoteRq

consumerLogin

Logs a user in under the proxy account used for consumer access. This action can be executed from a system page only.

HTML Form Fields:

Field	Description

_consumerKey	(Required if _consumerPartialKey and _consumerQuoteNumber are not specified) Full consumer key for the desired quote.
_consumerPartialKey	(Required if _consumerKey is not specified) Partial consumer key for the desired quote.
_consumerQuoteNumber	(Required if _consumerKey is not specified) Quote number of the desired quote.

EXAMPLE Server® Request: Consumer.loginRq, Consumer.listConsumerQuotesAndPoliciesRq, Consumer.loadQuoteRq (if only one consumer quote exists and no in-force policies exist in the results of the Consumer.listConsumerQuotesAndPoliciesRq)

consumerNewQuote

Creates a new quote in the DC_POL_ConsumerQuote table. This action can be executed from a system page only.

HTML Form Fields:

Field	Description
_manuscriptLOB	Combined string of the ManuScriptID and LOB code for the new quote. The syntax of this string is *ManuScriptID*|*LOBCode*. The two elements are separated by the pipe (|) character.

EXAMPLE Server® Request: Consumer.newQuoteRq

consumerReLogin

From within EXAMPLE Express®, logs a consumer user out of the proxy account and logs them in to an existing full user account. This action can be executed from a ManuScript or a system page. This action will execute the ExitInterviewField if this setting is defined in the Express\Web.config file.

Duck Creek Technologies, Inc. recommends executing the <TransACT.checkInRq> request to check the policy in from the proxyaccount user and check it out under the newly created account.

Parameters/HTML Form Fields:

Field	Description
_username	Username under which to log in.
_password	Password for the account under which to log in.
_target	A value of *interview* returns the user to the interview process. A blank value or any other value returns the user to the dashboard page.
_canSave	Boolean value that determines whether the policy is editable. A value of *1* indicates that the policy is editable.
_policyID	ID of the policy to returnt the user to. Applicable if the _target value is *interview*.

delete

Marks or un-marks the specified quote or policy for deletion from the database. The specified item is not removed from the database until a purge is performed.

HTML Form Fields:

Field	Description
QuoteID	ID of the quote or policy to delete.
showDeleted	Boolean flag indicating whether the content page is displaying deleted items. • If *true* (that is, the content page *is* displaying deleted items), then the system marks the specified policy as undeleted. • If *false* (that is, the content page is *not* displaying deleted items), the system marks the specified policy as deleted.

deleteClient

Marks the specified client for deletion from the database. The client is not removed from the database until a purge is performed.

HTML Form Fields:

Field	Description
_ClientID	ID of the client to mark for deletion.

deleteMessage

Deletes the specified notification from the database.

HTML Form Fields:

Field	Description
_messageID	ID of the notification to be removed.

deleteTaskAttachment

Removes an attachment from a task. This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the attachment ID from AttachmentRecord/AttachmentId.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description
_attachmentId	ID of the attachment to delete.

EXAMPLE Server® Request: OnlineData.deleteAttachmentRq

details

Updates the details of a quote or policy in the database.

HTML Form Fields:

Field	Description
_SaveReason	User-entered comment in the History table.
_clientName	Data placed at client/name
_contactName	Data placed at client/contactName
_phoneNumber	Data placed at client/phoneNumber
_phoneExtension	Data placed at client/phoneExtension
_status	Data placed at policy/status
_LOB	Data placed at policy/LOB
_policyNumber	Data placed at policy/policyNumber
_effectiveDate	Data placed at policy/effectiveDate
_expirationDate	Data placed at policy/expirationDate
_currentPremium	Data placed at policy/currentPremium
_address1	Data placed at mailingAddress/address1
_address2	Data placed at mailingAddress/address2
_city	Data placed at mailingAddress/city
_state	Data placed at mailingAddress/state
_ZIP	Data placed at mailingAddress/ZIP

EXAMPLE Server® Request: OnlineData.setDetailsRq

download

Sends a copy of the XML image of the specified quote or policy to the browser.

HTML Form Fields:

Field	Description
_QuoteID	ID of the quote or policy to download.

duplicate

Duplicates the specified quote or policy in the database.

HTML Form Fields:

Field	Description
_QuoteID	ID of the quote or policy to duplicate.
_startIndex	Indication of whether the target page is the Open or Client page.

initAsyncPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *1*.

initPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *1*.

issue

Saves the active quote or policy to the database using the <OnlineData.storeTransactionRq> request. This action allows issuance processes to be invoked from the request map. If an active history image exists, the <OnlineData.updateHistoryRq> request is called instead. This action uses the active EXAMPLE Express® state information to determine the clientID and policyID to save.

load

Loads the specified quote or policy into the session and invokes the interview process of the associated ManuScript. If desired, users can specify the PageSet and page to open by including a **ManuScriptEditingID** field in the specified ManuScript. This field must contain a comma-delimited list of *ManuScriptID*, *PageSet*, *Page* that indicates which ManuScript PageSet and page to load.

Interview parameters:

- **quoteID** – ID of the quote or policy to load.
- **lob** – LOB code of the ManuScript to load (as identified in the ManuScriptCatalog.xml file.)

HTML Form Fields:

Field	Description
_QuoteID	ID of the quote or policy to load.
_manuscriptLOB	LOB code of the ManuScript to load (as identified in the ManuScriptCatalog.xml file).

loadHistoryTx2

Loads the specified history image of a quote or policy into the session and invokes the interview process of the associated ManuScript. This action will use the PageSet and page specified on the action in Page Designer, if

specified. If not specified, the system will use the PageSet and page specified by the **ManuScriptEditingID** field, if one exists. If this field does not exist, the system will load the first PageSet and page in the specified ManuScript. This action is intended for use with EXAMPLE TransACT® 2.1.

The following fields in the session are modified based on the history image loaded:

- data/policyAdmin/ActiveTransaction
- data/policyAdmin/OSETransaction
- data/policyAdmin/ValidTransaction
- data/policyAdmin/AdminManuScriptID

Interview parameters:

- **historyID** — ID of the history record to load.
- **historyEdit** — Boolean flag indicating whether the history record is editable.

HTML Form Fields:

Field	Description
_QuoteID	ID of the quote or policy to load.

loadPortfolio

Loads the specified porfolio into the session and invokes the interview process of the associated ManuScripts. If desired, users can specify the ManuscriptID, PageSet and page to open. This field must contain a comma-delimited list of *ManuScriptID*, *PageSet*, *Page* that indicates which ManuScript PageSet and page to load. If no search item is specified, the current page is refreshed.

HTML Form Fields:

Field	Description
_usingPortfolio	ID of the ManuScript to load.
_PortfolioID	clientID of the ManuScript to load.

login

Logs the user into EXAMPLE Server® and begins a new EXAMPLE Express® session.

HTML Form Fields:

Field	Description
_username	Username under which to log in.
_password	Password for the account.

messageDetail

Updates the active notification in the database using the interview content of a Message Detail ManuScript. This action calls the <OnlineData.SaveMessageRq> request with the resulting XML image. The following two items in the notification XML are used:

- **//messages/message** — Updated notification text.
- **//messages/recipients** — Recipients for the message.

newMessage

Displays a New Message page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action. Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.

HTML Form Fields:

Field	Description
_messageID	If the message is a reply, ID of the message to be replied to. If the message is not a reply, no ID is specified.

newNotification

Displays a Send a Notification page (messageNew.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl). Performs no database action. This action sets the message flag in the EXAMPLE Express® <state> node to *new* and opens the Send a Notification page.

HTML Form Fields:

Field	Description
_messageID	If the notification is a reply, ID of the notification to be replied to. If the notification is not a reply, no ID is specified. Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this field.

newQuote

Creates a new quote in the active session, invoking the interview process of the specified ManuScript. Performs no database action. If desired, users can specify the PageSet and page to open by including a **ManuScriptEditingID** field in the specified ManuScript. This field must contain a comma-delimited list of *ManuScriptID*, *PageSet*, *Page* that indicates which ManuScript PageSet and page to load.

HTML Form Fields:

Field	Description
_manuscriptLOB	Combined string of the ManuScriptID and LOB code for the new quote. The syntax of this string is *ManuScriptID*|*LOBCode*. The two elements are separated by the pipe (|) character.
_clientID	(Optional) ID of the client to which to add the new quote. Specifying the _clientID will cause the quote to be pre-populated with any relevant client information.

newTask

Adds a new task based on the task record located in the session at the path _TaskData/TaskRecord.

EXAMPLE Server® Request: Task.addRq

previewAsyncPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *3* if the *PrintJobAutoRefresh* configuration setting in the Express\Web.config file is set to *1*, or with the *forceInit* attribute set to *0* if the *PrintJobAutoRefresh* configuration setting is set to *0*.

previewPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *3* if the *PrintJobAutoRefresh* configuration setting in the Express\Web.config file is set to *1*, or with the *forceInit* attribute set to *0* if the *PrintJobAutoRefresh* configuration setting is set to *0*.

readMessage

Displays the Notification Details page (message.xsl, or—if a Message Detail ManuScript is used—messageDetail.xsl) for the specified notification. This action sets the message flag in the EXAMPLE Express® <state> node to *readOnly* and opens the Notification Details page for the specified message.

refreshAsyncPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *3*. It also executes the <FormsEngine.setPrintJobItemsRq> request with an *autoMark* attribute set to *markDefault*.

rescind

This action is intended for use with EXAMPLE TransACT® 1.0.

Reverts the current policy image back to the state of the specified history image, retaining records of any transactions that are removed. This action copies the specified history image as the current policy or session image, but preserves the transaction records of the (previous) current image in the active session.

Interview parameter:

- **historyID** — ID of the history record to load.

resetManuScriptID

Resets the associated ManuScript for the active quote. This action uses the *resetManuScriptID*attribute or the *manuscript*attribute on the <ManuScript.getPageRq> request initiated by the interview to determine the new ManuScriptID to use. The system looks first at the *resetManuScriptID*attribute. If none is found or if it is empty, the system uses the *manuscript*attribute. If either is defined, EXAMPLE Express® resets the associated ManuScript for the active quote to the specified ManuScript ID. Once the quote or policy is saved, the database is also updated to associate the quote or policy with the specified ManuScriptID.

resetSession

Clears all session information. This action can be used to return the user to the Login page when an error occurs instead of displaying a fatal error.

rollbackHistory

Rolls back the transaction records of a policy to the specified history image, removing any subsequent transactions and making the specified history image the current image. This action then loads the (new) current policy image into the active session and invokes the interview process of the associated ManuScript.

The following fields in the session are modified based on the history image specified:

- data/policyAdmin/ActiveTransaction
- data/policyAdmin/OSETransaction
- data/policyAdmin/ValidTransaction
- data/policyAdmin/AdminManuScriptID

Interview parameters:

- **historyID** — ID of the history record to load.
- **historyEdit** — Boolean value indicating whether the data is editable.

runRulesAsyncPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request asynchronously, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *2*.

runRulesPrintJob

Executes the <Session.getPropertyRq> request and the <FormsEngine.initPrintJobRq> request, and loads the Policy Forms page. This action causes the <FormsEngine.initPrintJobRq> request to be executed with the *forceInit* attribute set to *2*.

save

Saves the active quote or policy to the database using the <OnlineData.storePolicyRq> request. If an active history image exists, the <OnlineData.updateHistoryRq> request is called instead. This action uses the active EXAMPLE Express® state information to determine the clientID and policyID to save.

saveMessage

Updates the specified message in the database from the HTML form fields on the New Message page (message.xsl). Out-of-the-box pages for EXAMPLE Express® 4.0 are not designed to use this action.

HTML Form Fields:

Field	Description
_messageID	ID of the message to be updated (or 0 for a new message).
_sender	Username of the sender.
_body	Body of the message.

_subject	Subject of the message.
_parentMessageID	If non-zero, message to which this message is a reply.
_remind	Reminder date.
_userList or _users	Array field containing list of user recipients.
_agencyList or _agencies	Array field containing list of agency IDs as recipients.
_agencyGroupList or _agencyGroups	Array field containing list of agency group IDs as recipients.
_useGlobal or _global	Flag indicating whether to use global recipients.
_clientID	If non-zero, client to which the message should be attached.
_quoteID	If non-zero, quote or policy to which the message should be attached.
_thisSecurityLevel or _securityLevel	Security level mask for who can view the message (bit field).

saveNotification

Updates the specified notification in the database from the HTML form fields on the New Message page (messageNew.xsl).

HTML Form Fields:

Field	Description
_messageID	ID of the notification to be updated (or 0 for a new notification).
_sender	Username of the sender.
_body	Body of the notification.
_messageType	Type of notification.
_clientID	If non-zero, client to which the notification should be attached.
_quoteID	If non-zero, quote or policy to which the notification should be attached.
_sendToOption	If *all*, indicates to send the notification to all users. Otherwise, system uses the _gridSelectionArray field.
_gridSelectionArray	Comma-delimited list of user IDs, entity IDs, or entity group IDs, depending on the value of the _pageType field.
_pageType	Type of recipient. Options are: • users • entities • entityGroup

setActiveAgency

Updates the active agency for the logged in user in the EXAMPLE Express® <state> node.

HTML Form Fields:

Field	Description
_ddlAgencies	Agency ID to set in the EXAMPLE Express® <state> node.

EXAMPLE Server® Requests: Session.setActiveAgencyRq, OnlineData.getAgencyDetailsRq

setLanguage

Sets the language property in the session and the EXAMPLE Express® <state> node to the specified language.

HTML Form Fields:

Field	Description
_language	Language to set in the session and the EXAMPLE Express® <state> node.

setPrintManuScript

Sets the ManuScriptID to use when the **Create Policy Documents** link is clicked. This action should be used when a different ManuScript than the loaded interview ManuScript (state.ManuScriptID) is needed. (For example, Duck Creek Templates™ CPP policies will have the CPP ManuScript stored as the interview ManuScript. When printing policy documents, however, the system may need to run a specific product ManuScript.)

Interview parameter:

- **printManuScriptID** — ID of the ManuScript to use for the print page.

If the *printManuScriptID* parameter is empty, the system will use the *state.ManuScriptID*.

setTaskComplete

Sets the status of a task to *Complete*. This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the task ID and task locking code from the paths defined in the schemas for the <Task.getRq> request response.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description
_taskId	ID of the task to complete.
_taskLockingTS	Encoded binary value that indicates the last time the task data was read and prevents multiple users from modifying a task at the same time. This value is generated from the <Task.getRq> request or the <Task.listRq> request.

EXAMPLE Server® Request: Task.setStatusRq

setTaskDeclined

Sets the status of a task to *Closed*, with a reason of *Declined*. This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the task ID and task locking code from the paths defined in the schemas for the <Task.getRq> request response.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description
_taskId	ID of the task to decline.
_taskLockingTS	Encoded binary value that indicates the last time the task data was read and prevents multiple users from modifying a task at the same time. This value is generated from the <Task.getRq> request or the <Task.listRq> request.

EXAMPLE Server® Request: Task.setStatusRq

setTaskDeleted

Marks a task for delete. (Sets the status of the task to *Deleted*.) This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the task ID and task locking code from the paths defined in the schemas for the <Task.getRq> request response.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description
_taskId	ID of the task to delete.
_taskLockingTS	Encoded binary value that indicates the last time the task data was read and prevents multiple users from modifying a task at the same time. This value is generated from the <Task.getRq> request or the <Task.listRq> request.

EXAMPLE Server® Request: Task.setStatusRq

setTaskStatus

Sets the status of a task. This action has two modes:

- **When executed from a ManuScript** — The system uses the task record in the session at _TaskData/TaskRecord to obtain the task ID, task status code, task status reason code, percentage complete, and task locking code from the paths defined in the schemas for the <Task.getRq> request response.
- **When executed from a system page** — The system requires the following HTML form fields.

Field	Description
_taskId	ID of the task to modify.

_taskStatusCode	Status to set the task to. Options are: ◦ **OPN** — Open ◦ **COM** — Complete ◦ **CLS** — Closed ◦ **DEL** — Deleted
_taskStatusReasonCode	(Optional) Reason code for the status. This field can be any status listed in the DC_PLT_SupportCodeLists table in the ExampleData database. Out of the box, the only valid option is *declined* (DEC).
_percentageComplete	(Optional) Integer from 0 to 100 describing the percentage of the task that is complete.
_taskLockingTS	Encoded binary value that indicates the last time the task data was read and prevents multiple users from modifying a task at the same time. This value is generated from the <Task.getRq> request or the <Task.listRq> request.

EXAMPLE Server® Request: Task.setStatusRq

signup

Creates an agency signup request and sends an e-mail to the recipient specified by the *MailTo* configuration setting in the Express\Web.config file.

HTML Form Fields:

Field	Description
_email	E-mail address of the person requesting the account.
_userName	Username for the account.
_password	Initial password for the account.
_fullName _email _name _contact _phone _extension _address1 _address2 _city _state _zip	Agency sign-up XML record.

EXAMPLE Server® Request: OnlineData.addAgencySignupRq

submit

Saves the active quote or policy to the database using the <OnlineData.storeTransactionRq> request. This action allows issuance processes to be invoked from the request map. If an active history image exists, the <OnlineData.updateHistoryRq> request is called instead. This action uses the active EXAMPLE Express® state information to determine the clientID and policyID to save.

switchUser

Moves the active quote or policy to the specified agency and logs in as a different user to load the active policy.

Interview parameters:

- **username** — Username of the account to log in under.
- **password** — Password for the account.
- **agencyid**— ID of the agency with which to associate the policy. If not specified, the system uses the <OnlineData.getUserDetailsRq> request to get the agency ID of the user.

HTML Form Fields:

Field	Description
_username	Username of the account to log in under. Used if the *username* interview parameter is not specified.
_password	Password for the account. Used if the *password* interview parameter is not specified.

updateTask

Updates a task in the database based on the task record located in the session at the path _TaskData/TaskRecord.

EXAMPLE Server® Request: Task.updateRq

upload

Uploads a file from a ManuScript to the directory specified in the *UploadDir* configuration setting in the Express\Web.config file. If executing this action from a ManuScript, the filepath of the uploaded file will be saved to the session using the <OnlineData.autoSaveDataRq> request or the <Session.storeDataRq> request.

HTML Form Fields:

Field	Description
File form field	Form field of type "file" that contains a reference to the file to upload.

validatePrintJob

Verifies that no required fields on selected forms are empty. This action executes the <FormsEngine.validatePrintJobRq> request, returning *valid* or *invalid*. If *invalid* is returned, a list of forms with missing fields is returned.

Skin.config File

The skin.config file is used with the Avant Garde 2 skin. The contents of this file are attached to the internal content XML document for the page XSL to reference during XSLT transforms.

This file must be present for any skin based on Avant Garde 2 skins. Absence of this file forces the code to use the Avant Garde skins.

The following code sample illustrates the basic structure of the skin.config file:

```
<settings>
    <CoreDirectory>Skins\DCTBase\Core</CoreDirectory>
  <Themes>
        <Theme name="DuckCreek"/>
        <ExtJsTheme cssFile="ext-xtheme-gray"/>
        <DCTExtJsExtensionTheme cssFile="dctExt-xtheme-gray"/>
  </Themes>
  <Panels>
        <RatingMessages showBar="1"/>
        <TogglePanelButton showButton="0"/>
  </Panels>
  <Buttons>
    <PreviousButtonDefaultType></PreviousButtonDefaultType>
    <NextButtonDefaultType></NextButtonDefaultType>
  </Buttons>
  <StaticPanelContent location="leftPanel"></StaticPanelContent>
  <Controls>
        <ComboBoxes maxSize="200"/>
  </Controls>
        <custom></custom>
</settings>
```

Skin.config Settings

This section contains descriptions, defaults, and valid values of all the skin.config configuration settings. You can use the Edit Configuration Utility in the Duck Creek root directory (by default `C:\Program Files\Duck Creek Technologies\UtiEditConfig.exe`) to manually alter these settings.

You can only use the Edit Configuration Utility in Text Mode to alter these settings.

Improperly adjusting skin.config settings can lead to unintended behaviors or can prevent EXAMPLE Express® skins from displaying correctly. Use caution when changing these settings.

Key	Default Value
CoreDirectory	Skins\DCTBase\Core
Theme name	DuckCreek
ExtJsTheme	ext-xtheme-gray
DCTExtJsExtensionTheme cssFile	dctExt-xtheme-gray
RatingMessage showBar	1

TogglePanelButton showButton	0
PreviousButtonDefaultType	N/A
NextButtonDefaultType	N/A
StaticPanelContent	leftPanel
ComboBoxes maxSize	200
custom	N/A
ContentUrl	N/A
ContentTitle	blank
ContentBorder	Hide
ContentFrame	Hide
ContentCollapsible	False
ContentScrollable	False
ContentHeight	N/A
ContentName	N/A
IgnoreIECheck	0

ComboBoxes maxSize

Defines the maximum width to use for all combo boxes.

Default	Valid
200	Any positive integer.

If the value of this setting is left blank, the default value of *200* will be used.

ContentUrl

The URL of the external content you wish to use in EXAMPLE Express®.

Default	Valid
blank	Any valid URL or path relative to EXAMPLE Express®.

ContentTitle

If specified, a title bar containing the specified string will appear in the title bar at the top of the tab section.

Default	Valid
blank	Any valid string.

ContentBorder

Displays or hides the section border.

Default	Valid
Hide	• **Show** – Displays the panel border. • **Hide** – Hides the panel border.

ContentCollapsible

Displays or hides the ability to collapse a section.

Default	Valid
False	• **True** – Allows the panel to be collapsed. • **False** – Prevents the panel from being collapsed.

ContentFrame

Displays or hides the section frame.

Default	Valid
Hide	• **Show** – Displays the panel border. • **Hide** – Hides the panel border.

ContentHeight

Height of a panel in pixels.

ContentName

Applies a name to an iFrame, if specified.

ContentScrollable

Enables content scrolling within a section.

Default	Valid
False	• **True** – Enables content scrolling. • **False** – Disables content scrolling.

CoreDirectory

Points to the core being used. This allows EXAMPLE Express® to know if the system is using the old or new core.

When using the Avant Garde 2 skin, the core should point to the Core folder in the `\Duck Creek Technologies\Express\Skins\DCTBase\` directory.

Default	Valid
Skins\DCTBase\Core	Any valid location.

Custom

Contains skin customizations.The value of this setting appears in the content XML document, which the XSL then transforms into page HTML. Skin developers can add any XML they wish to the <custom> element.

New skin.config content should only be added in the <custom> section. Duck Creek will always add new supported skin configuration setting above the <custom> section, and will always ship an empty <custom> section.

Default	Valid
N/A	Any valid XML customization information.

DCTExtJsExtensionTheme

Duck Creek modifications to the ExtJs CSS framework have been moved to their own layer. This configuration setting Defines the theme to be used for the ExtJs EXAMPLE Platform® extensions.

Default	Valid
dctExt-xtheme-gray	Any valid theme.

ExtJsTheme

Defines the base ExtJs theme CSS file to load.

Default	Valid
ext-xtheme-gray	Any valid theme.

IgnoreIECheck

Ignores the browser to check to allow controls on the page to be converted to ExtJs controls when using Internet Explorer 8 or higher. Enabling this setting could result in the increase of memory consumption by the Avant Garde 2 skin.

Default	Valid

0	0 - Ignore the Web browser check. 1 - Check the browser to determine whether to convert all controls to ExtJs.

NextButtonDefaultType

Defines the type of button to be used for the **Next** button.

Default	Valid
N/A	• Button • Hyperlink

The following table lists some examples of default buttons available for use with EXAMPLE Express®.

Configuration Settings	Buttons

`<Buttons>`
`<PreviousButtonDefaultType></PreviousButtonDefaultType>`
`<NextButtonDefaultType></NextButtonDefaultType>`
`</Buttons>`
	

`<Buttons>`
`<PreviousButtonDefaultType>white</PreviousButtonDefaultType>`
`<NextButtonDefaultType>blue</NextButtonDefaultType>`
`</Buttons>`
	

`<Buttons>`
`<PreviousButtonDefaultType>hyperlink</PreviousButtonDefaultType>`
`<NextButtonDefaultType>orange</NextButtonDefaultType>`
`</Buttons>`
	

PreviousButtonDefaultType

Defines the type of button to be used for the **Previous** button.

Default	Valid
N/A	• Button • Hyperlink

This configuration setting allows developers to specify a default button type for the previous buttons throughout the interview screens.

The following table lists some examples of default buttons available for use with EXAMPLE Express®.

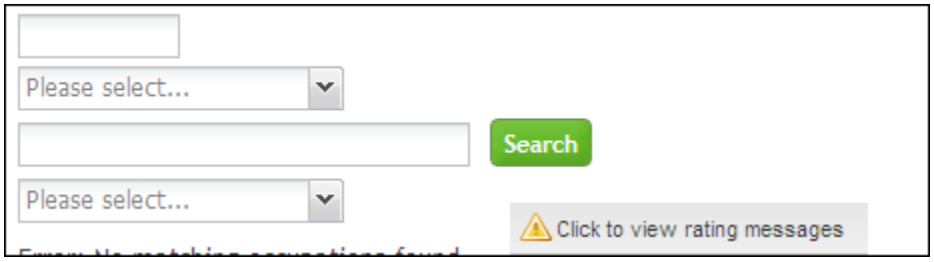
Configuration Settings	Buttons

<Buttons> <PreviousButtonDefaultType></PreviousButtonDefaultType> <NextButtonDefaultType></NextButtonDefaultType> </Buttons>	
<Buttons> <PreviousButtonDefaultType>white</PreviousButtonDefaultType> <NextButtonDefaultType>blue</NextButtonDefaultType> </Buttons>	
<Buttons> <PreviousButtonDefaultType>hyperlink</PreviousButtonDefaultType> <NextButtonDefaultType>orange</NextButtonDefaultType> </Buttons>	

RatingMessage showBar

Defines whether to display a rating message bar at the bottom of the browser that can be used to display a popup with the rating messages displayed.

The following image illustrates the dynamic rating message indicator.



Default	Valid
1	• **1** – Displays the rating message. • **0** – Does not display the rating message.

StaticPanelContent

Defines the location(s) where external content should be added.

Default	Valid
leftPanel	Any valid page panel.

Theme name

Defines the theme to be used with EXAMPLE Express® pages. By default, the the corresponding DuckCreek theme stylesheet will be automatically loaded from the `C:\Program Files\Duck Creek Technologies\Express\Skins\AvantGarde2\themes\DuckCreek\css\` directory.

Default	Valid
DuckCreek	Any valid theme.

TogglePanelButton showButton

Defines whether a button displays at the top of the page to open or close all collapsible sections on the page.

Default	Valid
0	• **0** – Hides the button that collapses all collapsible sections on the page. • **1** – Displays the button that collapses all collapsible sections on the page.

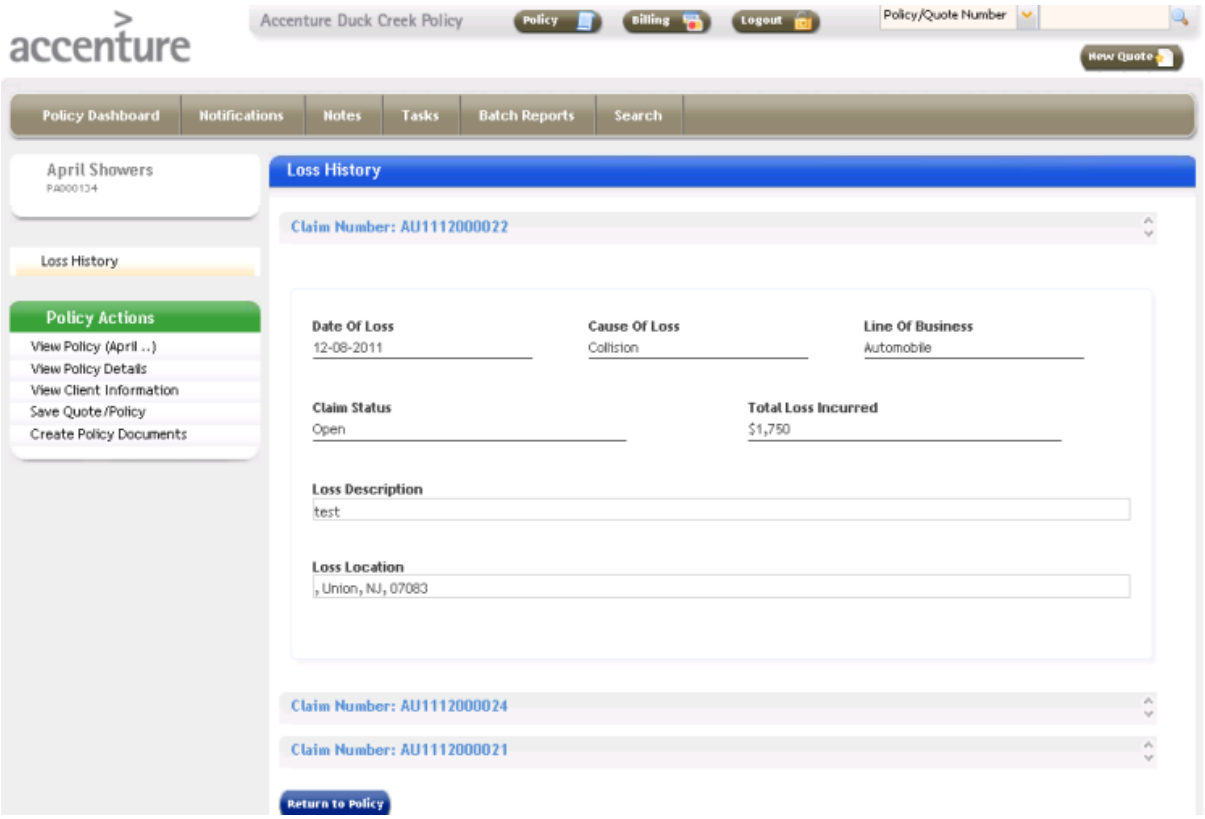
Accenture Duck Creek Claims Integration

The Accenture Duck Creek EXAMPLE Platform® offers the ability to attach a policy to a claim when the Accenture Claim Components is installed.

When the first notice of loss (FNOL) is reported, a loss representative can access, verify, and attach policies to the Accenture Claim Components by accessing Accenture Duck Creek Policy Administration. Policies with claims can be filtered to be automatically sent for underwriter review. When an open claim exists on a policy, the pricing page in EXAMPLE Express® will display a notification that the policy is in underwriter review.

Through the renewal process, underwriters can receive notices on policies with claims. Claim details can be reviewed through the Loss History page of Policy Administration. The following details about the claim are displayed to the underwriter at the claim level on the Loss History page:

- Line of business
- Cause of loss
- Description of loss
- Date of loss
- Claim status
- Total loss incurred
- Loss location



Installing the Accenture Duck Creek Claim Components

With the 4.3.2 release, the EXAMPLE Platform® integration with Accenture Claim Components covers Commercial Property, Personal Auto, Workers Comp, and General Liability. Other lines of business will be included with future releases.

When the EXAMPLE Platform® is installed to use the Claim Components, the installation is done in two steps. The first is an EXAMPLE Server® install that contains the Claim Components Requestmap, DLL files, <Get.LossHistoryRq> request, and Web.config entries added for the call to the Claim Components . The second is a content install that will include the necessary ManuScripts and XSL transforms to generate the Loss History page in EXAMPLE Express®. For information on installing the EXAMPLE Server® Claim Components integration, see *EXAMPLE Server® Installation*. For information about obtaining and installing the Claim Components content files, please contact your Account Manager. Information about the Accenture Claim Components can be found at *http://www.accenture.com/us-en/Pages/service-insurance-claim-components-summary.aspx*.